

spintent

SPANISH PARSE INTENTS

V0.99 2026-10-10*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.4	El comando <code>\spshort</code>	8
1.1	Carga del paquete	1	3.4.1	Llaves para <code>\spshort</code>	8
1.2	El paquete <code>babel</code>	2	3.4.2	Argumentos para <code>\spshort</code>	9
1.3	Fuentes tipográficas	2	4	Utilidades generales	10
1.4	El paquete <code>hyperref</code>	2	5	Utilidades para símbolos	11
2	El comando <code>\setspintent</code>	3	6	Utilidades para números	13
3	The big four	3	7	Utilidades para geometría	19
3.1	El comando <code>\spnum</code>	3	8	Utilidades para álgebra	34
3.1.1	Llaves para <code>\spnum</code>	3	9	Utilidades para estadística	37
3.2	El comando <code>\spunit</code>	4	10	Referencias	43
3.2.1	Llaves para <code>\spunit</code>	4	11	Change history	44
3.2.2	Argumentos para <code>\spunit</code>	4	12	Índice de la Documentación	45
3.2.3	El comando <code>\spnewunit</code>	6	13	Implementation	49
3.2.4	El comando <code>\spunitalias</code>	6	14	Índice de la Implementación	220
3.3	El comando <code>\spmoney</code>	6			
3.3.1	Llaves para <code>\spmoney</code>	7			
3.3.2	Monedas y alias para <code>\spmoney</code>	7			

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza `NVDA`[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con `NVDA/MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar `NVDA/MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de `WTPDF`[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, `Lua $\text{\LaTeX}$` ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-10-10.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/lualatex/spintent/
spintent.lua  ⇒ TDS:tex/lualatex/spintent/
spintent.pdf  ⇒ TDS:doc/lualatex/spintent/
README.md    ⇒ TDS:doc/lualatex/spintent/
spintent.dtx  ⇒ TDS:source/lualatex/spintent/
spintent.ins  ⇒ TDS:source/lualatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves (*keys*) aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{<comando>}{<key-uno = valor, key-dos = valor, key-tres = valor, ...>}`

El comando `\setspintent` establece las llaves (*<keys>*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*<keys>*) definidas en el *argumento opcional* [*<key = val>*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*<key = val>*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*<key = val>*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[<key = val>]{<número,número;número>}` `\spnum[<key = val>]{<número,número;número e exponente>}`
`\spnum[<key = val>]{<número.número;número>}` `\spnum[<key = val>]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*<key = val>*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {<true | false>}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\', entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {<terse | clear>}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {<coma | punto>}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «punto». Por ejemplo $\$ \backslash \text{spnum}\{3;4\}$ imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma cuatro periódico» y $\$ \backslash \text{spnum}[\text{read-period-sep=punto}]\{3;4\}$ imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «tres punto cuatro periódico»

- Esta llave es necesaria SOLO cuando se utilizan «decimales periódicos puros», es decir, cuando no se ha incluido en el $\{ \langle \text{argumento} \rangle \}$ una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = { \langle times \rangle | \langle cdot \rangle }`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «notación científica» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «por»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por». Por ejemplo $\$ \backslash \text{spnum}\{3,14\text{e}3\}$ imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo» y $\$ \backslash \text{spnum}[\text{notation-sym=times}]\{3,14\text{e}3\}$ imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo».

- Decir «notación científica» aquí es un poco exagerado, en realidad es «multiplicación por potencias de 10», no se verifica si la entrada cumple o no con la condición de estar escrita en «notación científica».

3.2 El comando `\spunit`

<code>\spunit</code>	$\backslash \text{spunit}\{ \langle \text{unidad} \rangle \}$	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{unidad} \rangle \}$	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \rangle \}$
	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \rangle \langle \text{unidad} \rangle \}$		$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \rangle \langle \text{unidad} \rangle \}$
	$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número unidad} \rangle \langle \text{unidad} \rangle \langle \text{unidad} \rangle \dots \}$		$\backslash \text{spunit}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{número e exponente unidad} \dots \rangle \}$

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{unidad} \rangle \}$, el cual puede contener una «parte numérica» opcional la cual se procesa internamente por el comando `\spnum` al inicio del $\{ \langle \text{argumento} \rangle \}$ seguido de una $\{ \langle \text{unidad} \rangle \}$ o un $\{ \langle \text{alias} \rangle \}$ valido descritos en §3.2.2 o una $\{ \langle \text{unidad} \rangle \}$ creada por el comando `\spnewunit` (§3.2.3) o un $\{ \langle \text{alias} \rangle \}$ creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spunit}\{23 \text{ m}\}$ imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros».

La multiplicación entre las $\{ \langle \text{unidades} \rangle \}$ se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^ $\{ \langle \text{exponente} \rangle \}$ ’. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m}^* \text{s}^{\wedge}\{2\}\}$ se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros segundos al cuadrado».

Se pueden usar *fracciones lineales* con las $\{ \langle \text{unidades} \rangle \}$ utilizando una «única barra» ‘/’ para la separación entre las $\{ \langle \text{unidades} \rangle \}$ del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo: $\$ \backslash \text{spunit}\{23 \text{ m} / \text{s}^{\wedge}\{2\}\}$ imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros por segundos al cuadrado».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El $\{ \langle \text{argumento} \rangle \}$ puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo, NO puede contener más de una «única barra» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para `\spunit`

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de $\{ \langle \text{unidades} \rangle \}$. Si se establece en `true`, se imprimirá ‘·’ entre las $\{ \langle \text{unidades} \rangle \}$ en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las $\{ \langle \text{unidades} \rangle \}$ en la salida PDF.

`read-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de $\{ \langle \text{unidades} \rangle \}$. Si se establece en `true`, NVDA/MathCAT verbalizará «por» entre las $\{ \langle \text{unidades} \rangle \}$; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las $\{ \langle \text{unidades} \rangle \}$.

`read-slash = { \langle por \rangle | \langle partido-por \rangle }`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «barra» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «partido por».

3.2.2 Argumentos para `\spunit`

Resumen de $\{ \langle \text{unidades} \rangle \}$ y $\{ \langle \text{alias} \rangle \}$ soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1				
Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	«Lux»
cm	-	Centímetro	cm	«Centímetro»
min	-	Minuto (Tiempo)	min	«Minuto (Tiempo)»
mm	-	Milímetro	mm	«Milímetro»
mol	-	Mol	mol	«Mol»
pc	-	Pársec	pc	«Pársec»
rad	-	Radián	rad	«Radián»
rpm	-	Rev. por minuto	rpm	«Rev. por minuto»
sr	-	Estereorradián	sr	«Estereorradián»
t	-	Tonelada	t	«Tonelada»
a	-	Año / Área (are)	a	«Año / Área (are)»
y	-	Año (Year)	y	«Año (Year)»
°	-	Grado (Arco)	°	«Grado (Arco)»

3.2.3 El comando \spnewunit

\spnewunit

`\spnewunit{<nueva unidad>}{<verbalización NVDA>}`

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nueva unidad>}` que establece lo que se imprimirá en la salida PDF y `{<verbalización NVDA>}` que establece como NVDA/MathCAT verbalizará la `{<nueva unidad>}`.

La `{<nueva unidad>}` no debe estar registrada como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2. Si la `{<nueva unidad>}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la `{<nueva unidad>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<unidad>}` valida para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la `{<nueva unidad>}` ‘px’ y cuando se utilice `$\spunit{100 px}$` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

\spunitalias

`\spunitalias{<nuevo alias>}{<expresión de unidades>}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nuevo alias>}` que establece es el nuevo `{<alias>}` de `{<argumento>}` para `\spunit` y `{<expresión de unidades>}` que puede contener multiplicación o división de `{<unidades>}` ya registradas que se imprimirán en la salida PDF.

El `{<nuevo alias>}` no debe estar registrado como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2 o estar definido como `{<unidad>}` por medio del comando `\spnewunit`. Si el `{<nuevo alias>}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{<nuevo alias>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<alias>}` valido para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `{<alias>}` ‘caudal’ y cuando se utilice `$\spunit{50 caudal}$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

\spmoney

`\spmoney{<cifra>}`
`\spmoney{<cifra símbolo moneda>}`
`\spmoney{<cifra alias moneda>}`

`\spmoney[<key = val>]{<cifra>}`
`\spmoney[<key = val>]{<cifra símbolo moneda>}`
`\spmoney[<key = val>]{<cifra alias moneda>}`

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<cifra>}` que contiene la «cifra numérica» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «símbolo moneda» o «alias moneda» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmoney{500}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «símbolo moneda» o «alias moneda» en el *argumento obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dólares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos dólares» aunque la moneda por defecto sean pesos.

Si se pasa un «alias moneda» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}$` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

©2026 by Pablo González L

7 / 230

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT. Los «*alias moneda*» se pueden pasar con o sin tilde, por ejemplo dolares o dólares.

- Los «*alias moneda*» se deben escribir en minúsculas y los códigos ISO en minúsculas o todo en mayúsculas. Cualquier otra escritura, por ejemplo PESO o Usd, devolverá un “error” y se detendrá la compilación del documento.
- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando (o del *modo texto* si se establece `money-math=false`), se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, con la única excepción de `\$`. La coma ‘,’ o el punto ‘.’ como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la «*meta-key*» `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

- Si la «*cifra numérica*» no tiene parte decimal, la llave `sub-units` no tiene efecto. Si la moneda en curso no admite *subunidades* se emite una advertencia y la llave tampoco tiene efecto.

`money-math = {(true | false)}` default: true

Establece si el *símbolo moneda* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir el *símbolo moneda* en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir el *símbolo moneda* en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
dolar	dolares, dólar, dólares	dólar	\$	« <i>dólar / dólares</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
yuan	yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₱	rublo, rublos	rublo	₱	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »
Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	—	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort{<abreviaturas | acrónimos | números ordinales>}
\spshort[<key = val>]{<abreviaturas | acrónimos | números ordinales>}
\spshort[<read-arg={<verbalización NVDA>}]{<comandos LATEX>}
```

El comando `\spshort` trabaja solo en modo texto y recibe un argumento obligatorio del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por spintent dentro del módulo Lua spintent.lua descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del argumento obligatorio que puede contener `{<comandos LATEX>}` en modo texto los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del modo texto está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para `shorthands` y el comando `\sptext` provistos por el paquete `babel` en español compatible con `tagged` PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja solo en modo texto. Internamente utiliza la llave `E` (expansion text) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado

Establece la verbalización del argumento {<comandos LATEX>} de la nueva {<abreviatura>}, {<acrónimo>}
o {<número ordinal>} para NVDA. El valor de esta llave siempre debe pasarse entre ‘{ }’. Por ejemplo:
\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}} creará una nueva {<abreviatura>}
que imprimirá ‘8.avo’ en la salida PDF y NVDA verbalizará «octavo».
```

```
small-caps = {<true | false>} default: false
```

Establece si se usarán versalitas para imprimir `{<acrónimos>}` y para la «letra volada» en `{<abreviaturas>}` y `{<números ordinales>}` respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas

con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^A’ en la salida PDF y NVDA verbalizará «*doctora*».

3.4.2 Argumentos para `\spshort`

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	« <i>doctora</i> »
dr.as	—	Dr. ^{as}	« <i>doctoras</i> »
prof.a	—	Prof. ^a	« <i>profesora</i> »
d.a	d. ^a	D. ^a	« <i>doña</i> »
m.a	—	M. ^a	« <i>maría</i> »
n.o	n. ^o	N. ^o	« <i>número</i> »
n.os	—	N. ^{os}	« <i>números</i> »
c.ia	—	C. ^{ia}	« <i>compañía</i> »
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	« <i>página</i> »
vol.	—	vol.	« <i>volumen</i> »
etc.	—	etc.	« <i>etcétera</i> »
et al.	et. al.	et al.	« <i>y otros</i> »
ibíd.	ibid.	ibíd.	« <i>ibídem</i> »
op. cit.	op.cit.	op. cit.	« <i>obra citada</i> »
loc. cit.	loc.cit.	loc. cit.	« <i>lugar citado</i> »
v. gr.	v.gr.	v. gr.	« <i>verbigracia</i> »
i. e.	i.e.	i. e.	« <i>esto es</i> »
e. g.	e.g.	e. g.	« <i>por ejemplo</i> »
p. ej.	p.ej.	p. ej.	« <i>por ejemplo</i> »
Títulos, profesiones y cargos			
sr.	—	Sr.	« <i>señor</i> »
sra.	—	Sra.	« <i>señora</i> »
srta.	—	Srta.	« <i>señorita</i> »
dr.	—	Dr.	« <i>doctor</i> »
dra.	—	Dra.	« <i>doctora</i> »
dres.	—	Dres.	« <i>doctores</i> »
dras.	—	Dras.	« <i>doctoras</i> »
prof.	—	Prof.	« <i>profesor</i> »
profa.	—	Profa.	« <i>profesora</i> »
profs.	—	Profs.	« <i>profesores</i> »
ing.	—	Ing.	« <i>ingeniero</i> »
ings.	—	Ings.	« <i>ingenieros</i> »
lic.	—	Lic.	« <i>licenciado</i> »
v. b.	v.b.	V. B.	« <i>visto bueno</i> »
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	« <i>sociedad anónima</i> »
ee. uu.	ee.uu.	EE. UU.	« <i>estados unidos</i> »
dd. hh.	dd.hh.	DD. HH.	« <i>derechos humanos</i> »
jj. oo.	jj.oo.	JJ. OO.	« <i>juegos olímpicos</i> »
ff. aa.	ff.aa.	FF. AA.	« <i>fuerzas armadas</i> »
rr. ee.	rr.ee.	RR. EE.	« <i>relaciones exteriores</i> »
cc. aa.	cc.aa.	CC. AA.	« <i>comunidades autónomas</i> »
p. d.	p.d.	P. D.	« <i>posdata</i> »
a. c.	a.c.	a. C.	« <i>antes de Cristo</i> »
d. c.	d.c.	d. C.	« <i>después de Cristo</i> »
d. n. i.	d.n.i.	D. N. I.	« <i>documento nacional de identidad</i> »
r. u. t.	r.u.t.	R. U. T.	« <i>rol único tributario</i> »
r. u. n.	r.u.n.	R. U. N.	« <i>rol único nacional</i> »

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	—	ONG	«organización no gubernamental»
ongs	—	ONGs	«organizaciones no gubernamentales»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

Los números ordinales se verbalizan con palabras desde el 1 hasta el 99. Desde el 100 el ordinal no se convierte en palabras y se entrega a NVDA/MathCAT tal como se imprime, por ejemplo 100. o como «100^o».

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>} \spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un número arábigo ‘21’ o un número romano ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio $\{\langle siglo \rangle\}$ «*siempre*» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

print-era = $\{\langle ac \mid dc \mid none \rangle\}$ default: none

Establece si se *imprime* o no la abreviatura después de imprimir el argumento $\{\langle siglo \rangle\}$. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xxi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno antes de cristo*»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xxi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno después de cristo*»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno*».

El comando `\sptime`

`\sptime` `\sptime{\langle horas : minutos \rangle}` `\sptime{\langle horas : minutos : segundos \rangle}`

El comando `\sptime` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle horas : minutos \rangle\}$ o $\{\langle horas : minutos : segundos \rangle\}$. Por ejemplo: `\sptime{23:45}` imprimirá en «*modo texto*» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «*las veintitrés y cuarenta y cinco minutos*».

El comando `\spdate`

`\spdate` `\spdate{\langle DD/MM/YYYY \rangle}` `\spdate{\langle YYYY/MM/DD \rangle}` `\spdate{\langle YYYY-MM-DD \rangle}` `\spdate{\langle DD-MM-YYYY \rangle}`

El comando `\spdate` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle DD/MM/YYYY \rangle\}$, $\{\langle DD-MM-YYYY \rangle\}$ o $\{\langle YYYY-MM-DD \rangle\}$. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en «*modo texto*» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «*dos de enero de mil novecientos ochenta y uno*».

- Si se pasa el argumento $\{\langle YYYY/MM/DD \rangle\}$, internamente se convertirá en $\{\langle DD/MM/YYYY \rangle\}$ y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «*símbolos matemáticos*» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «*símbolo*» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «*símbolos matemáticos*», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

`\spnewread` `\spnewread{\langle símbolo \rangle}{\langle verbalización NVDA \rangle}`

El comando `\spnewread` trabaja en *modo texto* y recibe dos argumentos obligatorios: $\{\langle símbolo \rangle\}$ y $\{\langle verbalización NVDA \rangle\}$ los cuales se almacenan de forma interna para ser utilizados como $\{\langle argumento \rangle\}$ en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el $\{\langle símbolo \rangle\}$ al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento $\{\langle símbolo \rangle\}$ no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘`\cong`’ y cuando se utilice `\spread{\cong}` se imprimirá ‘≅’ en la salida PDF y NVDA/MathCAT verbalizará «*es congruente con*».

- El $\{\langle símbolo \rangle\}$ debe ser un único comando, pero puede ser uno definido por el usuario. Por ejemplo, con `\newcommand{\xcuad}{x^2}`, `\spnewread{\xcuad}{x-al-cuadrado}` registrará ‘`\xcuad`’ y cuando se utilice `\spread{\xcuad}` se imprimirá ‘x²’ en la salida PDF y NVDA/MathCAT verbalizará «*x al cuadrado*».
- Los espacios y los guiones bajos de $\{\langle verbalización NVDA \rangle\}$ se convierten en guiones ‘-’, por lo que {es congruente con} equivale a {es-congruente-con}. Los guiones repetidos se reducen a uno y se eliminan los del inicio y del final. Si el resultado queda vacío se devolverá un “error” y se detendrá la compilación del documento.

El comando \spread

\spread \spread{⟨*símbolo*⟩} \spread[⟨*key* = *val*⟩]{⟨*símbolo*⟩}

 \spread[⟨*read-arg* = {⟨*verbalización* NVDA⟩}⟩]{⟨*comandos* L^AT_EX⟩}

El comando `\spread` trabaja en *modo matemático* y recibe el *argumento obligatorio* de la forma $\{ \langle \textit{símbolo} \rangle \}$ previamente registrado por el comando `\spnewread` o $\{ \langle \textit{comandos L}^{\text{A}}\text{T}_{\text{E}}\text{X} \rangle \}$ para ser usado junto a la llave `read-arg`.

Si el argumento es $\{\langle \text{\textit{simbolo}} \rangle\}$, $\$ \backslash \text{spread} \{\langle \text{\textit{cong}} \rangle \$$ imprimirá ‘ $\cong$ ’ en la salida PDF y el argumento $\{\langle \text{\textit{verbalización NVDA}} \rangle\}$ establecido al registrar $\{\langle \text{\textit{simbolo}} \rangle\}$ con $\backslash \text{spnewread}$ se verbalizará en NVDA/MathCAT.

Si el argumento es `{\comandosTEX}` y no se establece la llave `read-arg` se devolverá un “error” y se detendrá la compilación del documento.

Llaves para \spread

`read-arg = {⟨verbalización NVDA⟩}` default: *no usado*

Establece la verbalización del argumento $\{\langle \text{comandos } \text{\LaTeX} \rangle\}$. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y sobrescribe la verbalización del $\{\langle \text{símbolo} \rangle\}$ si está registrado previamente. Por ejemplo: $\$ \text{\texttt{spread}}[\text{\texttt{read-arg}}=\{\text{es-paralela-a}\}]\{\text{\texttt{parallel}}\} \$$ imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela a».

`silent = {true | false}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del argumento $\{\{\textit{símbolo}\}\}$ o $\{\{\textit{comandos L\TeX}\}\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- Al utilizar `silent` no es necesario que el argumento `{\simbolo}` esté registrado o usar la llave `read-arg` con `{\comandos LATEX}`. Dependiendo de que contenga `{\argumento}` puede que `silent` no sea suficiente, en ese caso utilice `read-arg={_-}`.

El comando \spusep

`\spusep` `\spusep{\langle\{ [ ( ) ] \}\rangle}` `\spusep[key = val]{\langle\{ [ ( ) ] \}\rangle}`

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \{ [[()]] \backslash \} \rangle\}$, el cual debe ser «solo uno» de los paréntesis soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘\{’ o ‘\}’.

Si no se proporciona el *argumento opcional* `[(key = val)]`, `$\spusep{}` (`$` imprimirá ‘`'` en la salida PDF y `NVDA/MathCAT` verbalizará «*abrir paréntesis*» acorde a su propia heurística.

- 🟢 El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para \spusep

```
size = {<none | big | Big | bigg | Bigg>}                                     default: none
```

Establece el *tamaño del paréntesis* pasado como `{\argumento}` que se imprimirá en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en `none`, se imprime el `{\argumento}` en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl\bigl` antes del `{\argumento}`; si se establece en `Big`, se usa `\Bigl\Bigl` antes del `{\argumento}`; si se establece en `bigg`, se usa `\biggl\biggl` antes del `{\argumento}`; y si se establece en `Bigg`, se usa `\Biggl\Biggl` antes del `{\argumento}`.

`silent = {true | false}` default: *false*

Establece si se *silencia* en `NVDA/MathCAT` la verbalización del `{(argumento)}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, `NVDA/MathCAT` NO verbalizará la salida del comando; si se establece en `false`, `NVDA/MathCAT` sí verbalizará la salida del comando.

El comando \spdsym

`\spdsym` `\spdsym` `\spdsym[\langle key = val \rangle]`

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* [`[key = val]`] mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spdsym` imprimirá ‘:’ en la salida PDF y NV-DA/MathCAT verbalizará «dividido».

Llaves para \spdsym

<code>prefix = { <prefijo> }</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «antes» de <i>verbalizar</i> el {<valor>} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>suffix = { <sufijo> }</code>	default: <i>no usado</i>
Establece el <i>sufijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «después» de <i>verbalizar</i> el {<valor>} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>set-sym = { <old-style two-dots slash> }</code>	default: <i>two-dots</i>
Establece el <i>símbolo de división</i> que se imprimirá en la salida PDF. Si se establece en <code>old-style</code> , imprimirá '÷'; si se establece en <code>two-dots</code> , imprimirá '⋅' y si se establece <code>slash</code> , imprimirá '/'.	
<code>read-sym = { <dividido dividido-por dividido-entre> }</code>	default: <i>dividido</i>
Establece la <i>forma</i> con la cual NVDA/MathCAT verbalizará el <i>símbolo de división</i> . Si se establece en <code>dividido</code> , NVDA/MathCAT verbalizará «dividido»; si se establece en <code>dividido-por</code> , NVDA/MathCAT verbalizará «dividido por» y si se establece en <code>dividido-entre</code> , NVDA/MathCAT verbalizará «dividido entre».	

El comando \spmsym

<code>\spmsym</code>	<code>\spmsym</code>	<code>\spmsym[<key = val>]</code>
El comando <code>\spmsym</code> trabaja en <i>modo matemático</i> y recibe el <i>argumento opcional</i> [<code><key = val></code>] mediante el cual se establece la «forma» en la que se imprimirá el <i>símbolo de multiplicación</i> y como se verbalizará en NVDA/MathCAT.		
Si no se proporciona el <i>argumento opcional</i> [<code><key = val></code>], <code>\$\spmsym\$</code> imprimirá '·' en la salida PDF y NVDA/MathCAT verbalizará «por».		

Llaves para \spmsym

<code>prefix = { <prefijo> }</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «antes» de <i>verbalizar</i> el {<valor>} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>suffix = { <sufijo> }</code>	default: <i>no usado</i>
Establece el <i>sufijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «después» de <i>verbalizar</i> el {<valor>} establecido por la llave <code>read-sym</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>set-sym = { <cross dot asterisk> }</code>	default: <i>dot</i>
Establece el <i>símbolo de multiplicación</i> que se imprimirá en la salida PDF. Si se establece en <code>cross</code> , imprimirá '×'; si se establece en <code>dot</code> , imprimirá '⋅' y si se establece en <code>asterisk</code> , imprimirá '*'.	
<code>read-sym = { <por multiplicado-por veces> }</code>	default: <i>por</i>
Establece la <i>forma</i> con la cual NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i> . Si se establece en <code>por</code> , NVDA/MathCAT verbalizará «por»; si se establece en <code>multiplicado-por</code> , NVDA/MathCAT verbalizará «multiplicado por» y si se establece en <code>veces</code> , NVDA/MathCAT verbalizará «veces».	
<code>not-print = { <true false> }</code>	default: <i>false</i>
Deshabilita la <i>impresión</i> del <i>símbolo de multiplicación</i> . Cuando se establece esta llave, NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i> acorde al valor establecido por la llave <code>read-sym</code> , pero, no se imprimirá este en la salida PDF.	

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando \spdiv

<code>\spdiv</code>	<code>\spdiv{<dividendo>}{<divisor>}</code>	<code>\spdiv[<key = val>]{<dividendo>}{<divisor>}</code>
El comando <code>\spdiv</code> trabaja en <i>modo matemático</i> y recibe dos <i>argumentos obligatorios</i> {<dividendo>} y {<divisor>}.		
Si no se proporciona el <i>argumento opcional</i> [<code><key = val></code>], <code>\$\spdiv{20}{26}\$</code> imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veintiséis»		

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{factor}{factor}` `\spmul[key = val]{factor}{factor}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* { *factor* } y { *factor* }. Si no se proporciona el *argumento opcional* [*key = val*], `\spmul{20}{26}` imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte por veintiséis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` `\spnD(número)` `\spnD[key = val](número)` `\spnD(letra)` `\spnD[key = val](letra)`

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número*) que se procesa internamente por `\spnum` o (*letra*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el *argumento* (*número*), `\spnD(6)` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el *argumento* (*letra*), `\spnD(m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

🚫 El número 0 no es válido: `\spnD(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnD`

`concept` = { *true* | *false* } default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «divisores de» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix` = { *prefijo* } default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { *true* | *false* } default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del *argumento* (*número*). Si se establece en *true*, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en *false*, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num` = { *número natural* } default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(⟨número⟩)` `\spnM[⟨key = val⟩](⟨número⟩)` `\spnM(⟨letra⟩)` `\spnM[⟨key = val⟩](⟨letra⟩)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número*⟩) que se procesa internamente por `\spnum` o (⟨*letra*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el argumento (⟨*número*⟩), `\spnM(6)` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el argumento (⟨*letra*⟩), `\spnM(m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

🚫 El número **0** no es válido: `\spnM(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnM`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «múltiplos de» y el (⟨*argumento*⟩); si se establece en **false**, NVDA/MathCAT solo verbalizará el (⟨*argumento*⟩).

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «múltiplos de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del argumento (⟨*número*⟩). Si se establece en **true**, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en **false**, no se imprimirá ni verbalizará el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» y del (⟨*argumento*⟩) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmcD`

`\spmcD` `\spmcD` `\spmcD(⟨número, número, ...⟩)` `\spmcD(⟨letra, letra, ...⟩)` `\spmcD[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spmcD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (⟨*número, número, ...*⟩) que se procesa internamente por `\spnum` o (⟨*letra, letra, ...*⟩) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] ni el *argumento opcional delimitado por paréntesis*, `\spmcD` imprimirá ‘mcd’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor».

Si se usa el argumento (⟨*número, número, ...*⟩), `\spmcD(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis».

Si se usa el argumento (⟨*letra, letra, ...*⟩), `\spmcD(m, n)` imprimirá ‘mcd(*m, n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre eme y ene».

🚫 Los argumentos (⟨*número, número, ...*⟩) o (⟨*letra, letra, ...*⟩) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\langle true \rangle | \langle false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*máximo común divisor ...*» o «*m c d ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\langle terse \rangle | \langle clear \rangle}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c d*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*máximo común divisor*».

`upper-case = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCD' en la salida PDF; si se establece en *false*, se imprimirá 'mcd' en la salida PDF.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *imprime* el resultado de calcular el «*máximo común divisor*» del argumento (*número, número, ...*). Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis es igual a uno*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*máximo común divisor*».

`silent = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*máximo común divisor ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmc`

`\spmc` `\spmc` `\spmc(\langle número, número, ... \rangle)` `\spmc(\langle letra, letra, ... \rangle)` `\spmc[\langle key = val \rangle](\langle número, número, ... \rangle)`

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spmc` imprimirá 'mcm' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

Si se usa el argumento (*número, número, ...*), `\spmc(2, 3, 6)` imprimirá 'mcm(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spmc(m, n)` imprimirá 'mcm(m, n)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre eme y ene*».

Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\langle true \rangle | \langle false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*mínimo común múltiplo ...*» o «*m c m ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\langle terse \rangle | \langle clear \rangle}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`upper-case = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCM' en la salida PDF; si se establece en *false*, se imprimirá 'mcm' en la salida PDF.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\langle true \rangle | \langle false \rangle}` default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del argumento ($\langle \text{<número, número, ...>}$). Si se establece en *true*: $\$ \backslash \text{spmcm}[\text{result}=\text{true}](2, 3, 6)\$$ imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*mínimo común múltiplo*».

`silent = {<true | false>}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*mínimo común múltiplo ...*» y del ($\langle \text{argumento} \rangle$) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnZ`

`\spnZ` $\backslash \text{spnZ}\{\langle \text{número entero} \rangle\}$ $\backslash \text{spnZ}[\text{key} = \text{val}]\{\langle \text{número entero} \rangle\}$

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{\langle \text{número entero} \rangle\}$ el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {<true | false>}` default: false

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento $\{\langle \text{número entero} \rangle\}$ pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del $\{\langle \text{número entero} \rangle\}$; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {<true | false>}` default: true

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* $\{\langle \text{número entero} \rangle\}$ pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)’; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` $\backslash \text{spmQ}\{\langle \text{entero} \rangle\}\{\langle \text{numerador} \rangle\}\{\langle \text{denominador} \rangle\}$ $\backslash \text{spmQ}[\text{key} = \text{val}]\{\langle \text{entero} \rangle\}\{\langle \text{numerador} \rangle\}\{\langle \text{denominador} \rangle\}$

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: $\{\langle \text{número entero} \rangle\}$, $\{\langle \text{numerador} \rangle\}$ y $\{\langle \text{denominador} \rangle\}$, los procesa internamente y devuelve un «*número mixto*». Los tres deben ser *enteros*; el $\{\langle \text{numerador} \rangle\}$ y el $\{\langle \text{denominador} \rangle\}$ no pueden llevar signo y el $\{\langle \text{denominador} \rangle\}$ no puede ser cero. Cualquier otro valor se rechaza con un error y no se imprime nada.

Si no se proporciona el *argumento opcional* $[\text{key} = \text{val}]$, $\$ \backslash \text{spmQ}\{2\}\{5\}\{7\}\$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada $\$3\backslash \text{frac}\{5\}\{7\}\$$ con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {<entero | enteros | y>}` default: y

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, $\$ \backslash \text{spmQ}[\text{join-word}=\text{entero}]\{1\}\{5\}\{7\}\$$ imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*un entero la fracción con numerador cinco y denominador siete*»; si se establece en *enteros*, $\$ \backslash \text{spmQ}[\text{join-word}=\text{enteros}]\{2\}\{5\}\{7\}\$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros la fracción con numerador cinco y denominador siete*»; si se establece en *y*, $\$ \backslash \text{spmQ}[\text{join-word}=y]\{2\}\{5\}\{7\}\$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

`use-spnZ = {<true | false>}` default: true

Establece si el $\{\langle \text{numerador} \rangle\}$ y el $\{\langle \text{denominador} \rangle\}$ que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura *MathML* de la salida PDF. Si se establece en *true*, $\{\langle \text{numerador} \rangle\}$ y $\{\langle \text{denominador} \rangle\}$ se pasarán a `\spnZ` y, por ejemplo, $\$ \backslash \text{spmQ}\{2\}\{5\}\{7\}\$$ se verbalizará «*dos y la fracción con numerador cinco y denominador siete*»; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional y NVDA/MathCAT aplica sus propias reglas a la fracción: $\$ \backslash \text{spmQ}[\text{use-spnZ}=\text{false}]\{2\}\{5\}\{7\}\$$ se verbalizará «*dos y cinco séptimos*».

`use-dfrac = {<true | false>}` default: false

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\$ \backslash \text{spmQ}[\text{use-dfrac}=\text{true}]\{2\}\{5\}\{7\}\$$ imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en *true*, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = $\{\langle true \mid false \rangle\}$ default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

`silent` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *número mixto* sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, sí verbalizará la salida del comando. Los argumentos inválidos se rechazan siempre, también con `silent=true`.

El comando `\sprazon`

`\sprazon` $\backslash\text{sprazon}\{\langle\textit{antecedente}\rangle\}\{\langle\textit{consecuente}\rangle\}$ $\backslash\text{sprazon}[\langle\textit{key} = \textit{val}\rangle]\{\langle\textit{antecedente}\rangle\}\{\langle\textit{consecuente}\rangle\}$

El comando `\sprazon` trabaja en *modo matemático* y recibe *dos argumentos obligatorios*: $\{\langle\textit{antecedente}\rangle\}$ y $\{\langle\textit{consecuente}\rangle\}$, los tipografía como una fracción y devuelve una «razón».

Si no se proporciona el *argumento opcional* $[\langle\textit{key} = \textit{val}\rangle]$, $\text{\$}\backslash\text{sprazon}\{3\}\{4\}\text{\$}$ imprimirá ‘ $\frac{3}{4}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro».

- El $\{\langle\textit{antecedente}\rangle\}$ y el $\{\langle\textit{consecuente}\rangle\}$ se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\sprazon`

`silent` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *razón* pasada al comando sin modificar la impresión de esta. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si la fracción de la *razón* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\text{\$}\backslash\text{sprazon}[\textit{use-dfrac=true}]\{3\}\{4\}\text{\$}$ imprimirá la fracción en tamaño *display* ‘ $\frac{3}{4}$ ’, en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si la *razón* se imprime en línea con dos puntos ‘:’ en lugar de una fracción. Si se establece en *true*, $\text{\$}\backslash\text{sprazon}[\textit{inline=true}]\{3\}\{4\}\text{\$}$ imprimirá ‘3 : 4’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro»; si se establece en *false*, la razón se imprime como una fracción.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en *true*, porque en ese caso no se imprime una fracción.

El comando `\spProp`

`\spProp` $\backslash\text{spProp}\{\langle a \rangle\}\{\langle b \rangle\}\{\langle c \rangle\}\{\langle d \rangle\}$ $\backslash\text{spProp}[\langle\textit{key} = \textit{val}\rangle]\{\langle a \rangle\}\{\langle b \rangle\}\{\langle c \rangle\}\{\langle d \rangle\}$

El comando `\spProp` trabaja en *modo matemático* y recibe *cuatro argumentos obligatorios*: $\{\langle a \rangle\}$ y $\{\langle b \rangle\}$ forman la primera *razón*, $\{\langle c \rangle\}$ y $\{\langle d \rangle\}$ forman la segunda, y devuelve una «proporción» con las dos razones unidas por el signo igual.

Si no se proporciona el *argumento opcional* $[\langle\textit{key} = \textit{val}\rangle]$, $\text{\$}\backslash\text{spProp}\{3\}\{4\}\{6\}\{8\}\text{\$}$ imprimirá ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho».

- Los cuatro argumentos se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\spProp`

`silent` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *proporción* pasada al comando sin modificar la impresión de esta. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si las fracciones de la *proporción* se imprimen usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\text{\$}\backslash\text{spProp}[\textit{use-dfrac=true}]\{3\}\{4\}\{6\}\{8\}\text{\$}$ imprimirá las fracciones en tamaño *display* ‘ $\frac{3}{4} = \frac{6}{8}$ ’, en la salida PDF; si se establece en *false*, las fracciones se imprimen con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline` = $\{\langle true \mid false \rangle\}$ default: *false*

Establece si las *razones* de la *proporción* se imprimen en línea con dos puntos ‘:’ en lugar de fracciones. Si se establece en *true*, $\text{\$}\backslash\text{spProp}[\textit{inline=true}]\{3\}\{4\}\{6\}\{8\}\text{\$}$ imprimirá ‘3 : 4 = 6 : 8’ en la salida PDF

y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho»; si se establece en `false`, las razones se imprimen como fracciones.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprimen fracciones.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\{\langle \text{argumento} \rangle\}$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\{\langle \text{argumento} \rangle\}$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘`'`’ y los subíndices siempre deben escribirse usando llaves ‘`A_{...}`’ y nunca sin estas ‘`A_n`’ que funciona solo por coincidencia. .

El comando `\spcoord`

<code>\spcoord</code>	<code>\spcoord{\langle abscisa, ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa, ordenada \rangle}</code>
	<code>\spcoord{\langle abscisa; ordenada \rangle}</code>	<code>\spcoord[\langle key = val \rangle]{\langle abscisa; ordenada \rangle}</code>

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle abscisa, ordenada \rangle\}$ o $\{\langle abscisa; ordenada \rangle\}$ el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spcoord{-3, 4}` imprimirá ‘ $(-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «menos tres coma cuatro»; `\spcoord{x_{1}, y_{1}}` imprimirá ‘ (x_1, y_1) ’ y verbalizará «*x* subíndice uno coma y subíndice uno».

- El comando `\spcoord` solo acepta *un separador* ‘`,`’ o ‘`;`’ entre la abscisa y la ordenada, fuera de llaves: `\spcoord{a_{1,2}, b}` es válido porque la coma de las llaves no separa. Con ‘`;`’ las comas son decimales, por ejemplo `\spcoord{1,5; 2,5}`.

Llaves para `\spcoord`

`silent = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{\langle punto \rangle}</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa, ordenada \rangle)</code>
	<code>\spPoint{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>	<code>\spPoint[\langle key = val \rangle]{\langle punto \rangle}(\langle abscisa; ordenada \rangle)</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle punto \rangle\}$ el cual es una *única letra mayúscula* con prima ‘`'`’ o subíndice ‘`_{...}`’ y el *argumento opcional delimitado por paréntesis* $(\langle abscisa, ordenada \rangle)$ el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\spPoint{A}` imprimirá ‘ A ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A*»; `\spPoint{A_{1}}` imprimirá ‘ A_1 ’ y verbalizará «*punto A sub uno*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3,4)` imprimirá ‘ $A(-3,4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A menos tres coma cuatro*»; `\spPoint{A_{1}}(-3,4)` imprimirá ‘ $A_1(-3,4)$ ’ y verbalizará «*punto A sub uno menos tres coma cuatro*».

Llaves para `\spPoint`

`read-cmd = {\langle school | mathcat \rangle}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento $\{\langle punto \rangle\}$ sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «*punto A*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*punto*». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {\langle true | false \rangle}` default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{\langle letra \rangle}</code>	<code>\spvec{\langle punto, punto \rangle}</code>
	<code>\spvec{\langle letra \rangle}(\langle abscisa, ordenada \rangle)</code>	<code>\spvec{\langle punto, punto \rangle}(\langle abscisa, ordenada \rangle)</code>
	<code>\spvec[\langle key = val \rangle]{\langle letra \rangle}(\langle abscisa, ordenada \rangle)</code>	<code>\spvec[\langle key = val \rangle]{\langle punto, punto \rangle}(\langle abscisa, ordenada \rangle)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\{\langle letra \rangle\}$ (minúscula) o $\{\langle punto, punto \rangle\}$ junto al *argumento opcional delimitado por paréntesis* $(\langle abscisa, ordenada \rangle)$ el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma $\{\langle punto, punto \rangle\}$ con prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma $\{\langle letra \rangle\}$ (minúscula) con subíndice ‘`_{\dots}`’ opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\$ \spvec{a} \$` imprimirá ‘ $\vec{a}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector *a*»; `\$ \spvec{AB} \$` imprimirá ‘ $\overrightarrow{AB}$ ’ y verbalizará «vector *A B*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\$ \spvec{a}(-3,4) \$` imprimirá ‘ $\vec{a} = (-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector *a* igual a menos tres coma cuatro»; `\$ \spvec{AB}(-3,4) \$` imprimirá ‘ $\overrightarrow{AB} = (-3, 4)$ ’ y verbalizará «vector *A B* igual a menos tres coma cuatro».

Llaves para `\spvec`

`read-cmd = {\langle school | mathcat \rangle}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle punto, punto \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\$ \spvec{AB} \$` se verbaliza «vector *A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`autofit = {\langle true | false \rangle}` default: *false*

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento $\{\langle letra \rangle\}$. Si se establece en `true`, se aplicará `\vphantom{b}` al inicio del argumento $\{\langle letra \rangle\}$; si se establece en `false`, no se aplicará `\vphantom{b}`.

`concept = {\langle true | false \rangle}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el $\{\langle argumento \rangle\}$; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle argumento \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent = {\langle true | false \rangle}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del $\{\langle argumento \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{\langle punto, punto \rangle}</code>	<code>\sprecta{\langle letra \rangle}</code>
	<code>\sprecta[\langle key = val \rangle]{\langle punto, punto \rangle}</code>	<code>\sprecta[\langle key = val \rangle]{\langle letra \rangle}</code>

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\{\langle letra \rangle\}$ o $\{\langle punto, punto \rangle\}$ con prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales. Si el *argumento* es de la forma $\{\langle punto, punto \rangle\}$ se aplica `\overleftarrow` en la impresión; si el *argumento* es de la forma $\{\langle letra \rangle\}$ (mayúscula o minúscula), se imprimirá solo la $\{\langle letra \rangle\}$ sin aplicar `\overleftarrow`.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, `\$ \sprecta{AB} \$` imprimirá ‘ $\overleftarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta *A B*»; `\$ \sprecta{m} \$` imprimirá ‘*m*’ en la salida PDF y NVDA/MathCAT verbalizará «recta *m*».

Llaves para `\sprecta`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` } y el argumento { `<letra>` } cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\sprecta{AB}$` se verbaliza «*recta A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*recta*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

`\sprayo` { `<punto, punto>` }

`\sprayo` [`<key = val>`] { `<punto, punto>` }

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `$\sprayo{AB}$` imprimirá ' $\overrightarrow{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*rayo A B*».

Llaves para `\sprayo`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\sprayo{AB}$` se verbaliza «*rayo A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<rayo | semirrecta>` }

default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$\sprayo[read-sty=rayo]{AB}$`, NVDA/MathCAT verbalizará «*rayo A B*»; si se utiliza `$\sprayo[read-sty=semirrecta]{AB}$`, NVDA/MathCAT verbalizará «*semirrecta A B*».

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overrightarrow{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «*semirrecta*»; en México y otros países de América Latina se usa «*rayo*».

El comando `\spside`

```
\spside <\spside{<punto, punto>}
\spside[<key = val>]{<punto, punto>}
```

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}` con prima(s) ‘`’` o subíndice(s) ‘`_...`’ opcionales. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spside{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{AB}` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
read-sty = {<lado | segmento | trazo>} default: lado
```

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `\spside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `\spside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «trazo A B».

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
read-arg = {<verbalización NVDA>} default: no usado
```

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’. Por ejemplo: `\spside[read-arg={hipotenusa}]{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

```
silent = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

```
\spmside <\spmside{<punto, punto>}
\spmside[<key = val>]{<punto, punto>}
```

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}` con prima(s) ‘`’` o subíndice(s) ‘`_...`’ opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ‘ $\overline{AB}$ ’; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmside{AB}` imprimirá ‘AB’ en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
read-sty = {<lado | segmento | trazo>} default: lado
```

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*medida del*» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «*medida del*», el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `$\spmside[concept=false,read-arg={medida de la cuerda}]{AB}$` imprimirá 'AB' en la salida PDF y NVDA/MathCAT verbalizará «*medida de la cuerda A B*».

`print-m = {⟨true | false⟩}`

default: *false*

Establece *si se imprime* la notación visual 'm' antes aplicar `\overline` al {⟨*argumento*⟩}. Si se establece en *true*, `$\spmside[print-m=true]{AB}$` imprimirá 'm $\overline{AB}$ ' en la salida PDF, si se establece en *false* imprimirá 'AB' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*», del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

`\spangle {⟨punto | letra griega | número⟩}`

`\spangle {⟨punto, punto, punto⟩}`

`\spangle [key = val]{⟨punto | letra griega | número⟩}`

`\spangle [key = val]{⟨punto, punto, punto⟩}`

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma {⟨*punto* | *letra griega* | *número*⟩} o {⟨*punto, punto, punto*⟩} aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma {⟨*punto*⟩} o {⟨*punto, punto, punto*⟩} admiten prima(s) '1' o subíndice(s) '_{...}' opcionales.

Si no se proporciona el *argumento opcional* [key = val], `$\spangle{A}$` imprimirá '∠A' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo en A*»; `$\spangle{\alpha}$` imprimirá '∠α' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo alfa*»; `$\spangle{1}$` imprimirá '∠1' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo uno*» y `$\spangle{AOB}$` imprimirá '∠AOB' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo A O B*».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨*punto*⟩} o {⟨*punto, punto, punto*⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spangle{AOB}$` se verbaliza «*ángulo A O B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*ángulo*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`recto = {⟨true | false⟩}`

default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «*recto*» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `$\spangle[recto=true]{A}$` imprimirá '⊥A' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo recto en A*», `$\spangle[recto=true]{AOB}$` imprimirá '⊥AOB' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo recto A O B*»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «*recto*».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*ángulo*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto «*ángulo*» y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}` admiten prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`recto = {⟨true | false⟩}`

default: *false*

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en *false*, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* la notación visual ‘m’ antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en *true*, `\spmangle[print-m=true]{AOB}` imprimirá ‘m $\angle AOB$ ’ en la salida PDF, si se establece en *false* imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «medida del ángulo» y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

<code>\spang</code>	<code>\spang{⟨grados⟩}</code>
	<code>\spang{⟨grados : minutos⟩}</code>
	<code>\spang{⟨grados : minutos : segundos⟩}</code>

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨grados⟩}`, `{⟨grados : minutos⟩}` o `{⟨grados : minutos : segundos⟩}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{⟨punto, punto⟩}</code>	<code>\sparc{⟨punto, punto, punto⟩}</code>
	<code>\sparc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sparc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sparc{AB}` imprimirá $\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

<code>\spmarc</code>	<code>\spmarc{⟨punto, punto⟩}</code>	<code>\spmarc{⟨punto, punto, punto⟩}</code>
	<code>\spmarc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\spmarc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales colocando `'m'` antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmarc{AB}` imprimirá $m\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{AB}` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

```
\spPoly \spPoly{⟨punto, punto, punto, ...⟩}
\spPoly[⟨key = val⟩]{⟨punto, punto, punto, ...⟩}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto, punto, punto, ...} \rangle\}$, una lista de al menos tres $\{\langle \text{puntos} \rangle\}$ con prima(s) ‘ $\prime$ ’ o subíndice(s) ‘ $_{\dots}$ ’ opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «*triángulo*» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «*cuadrado*» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo y se añade a la verbalización el nombre del polígono: «*pentágono*», «*hexágono*», «*heptágono*» u «*octágono*» de cinco a ocho puntos, y «*polígono*» desde nueve puntos.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spPoly}\{ABC\} \$$ imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; $\$ \backslash \text{spPoly}\{ABCD\} \$$ imprimirá ‘ $\square ABCD$ ’ y verbalizará «*cuadrado A B C D*»; $\$ \backslash \text{spPoly}\{ABCDE\} \$$ imprimirá ‘ $ABCDE$ ’ y verbalizará «*pentágono A B C D E*».

Llaves para `\spPoly`

```
read-cmd = {\school | mathcat} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto, punto, punto, ...} \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spPoly}\{ABC\} \$$ se verbaliza «*triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {\prefijo} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el $\{\langle \text{argumento} \rangle\}$ o el concepto «*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

```
concept = {\true | false} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto («*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*») acorde al número de puntos en el $\{\langle \text{argumento} \rangle\}$ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

```
print-sym = {\true | false} default: true
```

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, $\$ \backslash \text{spPoly}[\text{print-sym}=\text{true}]\{ABC\} \$$ imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; $\$ \backslash \text{spPoly}[\text{print-sym}=\text{true}]\{ABCD\} \$$ imprimirá ‘ $\square ABCD$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C D*»; si se establece en `false`, solo se imprimirá el $\{\langle \text{argumento} \rangle\}$ en la salida PDF y la verbalización será la misma.

🔊 Esta llave no tiene efecto con cinco o más puntos, porque no hay un símbolo definido para ellos.

```
silent = {\true | false} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

```
\spAfig \spAfig(⟨punto, punto, punto, ...⟩)
\spAfig[⟨key = val⟩] \spAfig[⟨key = val⟩](⟨punto, punto, punto, ...⟩)
\spAfig(⟨1 | 2 | 3 | ... | n⟩) \spAfig((F | P | B | L | T))
\spAfig[⟨key = val⟩](⟨1 | 2 | 3 | ... | n⟩) \spAfig[⟨key = val⟩]((F | P | B | L | T))
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $(\langle \text{punto, punto, punto, ...} \rangle)$ o $((F | P | B | L | T))$ aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $(\langle \text{argumento} \rangle)$ y la llave `concept`.

Si se proporciona el *argumento* $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $\$ \backslash \text{spAfig}(1) \$$ imprimirá ‘ $\mathcal{A}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; $\$ \backslash \text{spAfig}(n) \$$ imprimirá ‘ $\mathcal{A}_n$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área ene*».

Si se proporciona el *argumento* $(\langle \text{punto, punto, punto, ...} \rangle)$, $\$ \backslash \text{spAfig}(ABC) \$$ imprimirá ‘ $\mathcal{A}_{\triangle ABC}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; $\$ \backslash \text{spAfig}(ABCD) \$$ imprimirá ‘ $\mathcal{A}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»; $\$ \backslash \text{spAfig}(ABCDE) \$$ imprimirá ‘ $\mathcal{A}_{ABCDE}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del pentágono A B C D E*». Con cinco o más puntos no

se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* ((F | P | B | L | T)), $\$ \backslash \text{spAfig}(F) \$$ imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura»; $\$ \backslash \text{spAfig}(F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura uno».

Si no se proporciona el *argumento opcional* [(key = val)] ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spAfig} \$$ imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área».

- El *argumento* ((punto, punto, punto, ...)) acepta prima(s) ' ' o subíndice(s) '_{...}'. El *argumento* ((F | P | B | L | T)) solo acepta subíndices '_{...}' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para \spAfig

`read-cmd = {<school | mathcat>}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spAfig}(ABC) \$$ se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {<prefijo>}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando. El valor de esta llave siempre debe pasarse entre llaves '{ }'.

`concept = {<true | false>}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el ((argumento)); si se establece en `false`, NVDA/MathCAT solo verbalizará el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el ((argumento)) es equivalente a usar `silent=true`.

`read-arg = {<nombre>}` default: no usado

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento ((punto, punto, punto, ...)) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave siempre debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spAfig}[\text{read-arg}=\{\text{trapezio}\}](ABCD) \$$ imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapecio A B C D».

- Esta llave no tiene efecto (se ignora y se emite un aviso) con los argumentos ((F | P | B | L | T)) y ((1 | 2 | 3 | ... | n)).

`concept* <valor prohibido>` default: no usado

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

`read-sub = {<true | false>}` default: false

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos ((1 | 2 | 3 | ... | n)) y ((F | P | B | L | T)) para NVDA/MathCAT. Si se utiliza ((1 | 2 | 3 | ... | n)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub=true}](1) \$$ imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área uno».

Si se utiliza ((F | P | B | L | T)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub=true}](F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym = {<auto | triangle | square | none>}` default: auto

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos ((punto, punto, punto, ...)) y ((1 | 2 | 3 | ... | n)) pasado al comando sin modificar la verbalización de este. Si se utiliza ((punto, punto, punto, ...)) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spAfig}[\text{print-sym=auto}](ABC) \$$ y $\$ \backslash \text{spAfig}[\text{print-sym=none}](ABC) \$$ imprimirán ' $\mathcal{A}_{\triangle ABC}$ ' y ' $\mathcal{A}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza ((1 | 2 | 3 | ... | n)) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spAfig}[\text{print-sym=triangle}](1) \$$ imprimirá ' $\mathcal{A}_{\triangle 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; $\$ \backslash \text{spAfig}[\text{print-sym=square}](1) \$$ imprimirá ' $\mathcal{A}_{\square 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

- Esta llave no se puede utilizar con el argumento ((F | P | B | L | T)).

`silent = {<true | false>}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(\langle \text{punto, punto, punto, ...} \rangle)</code>
<code>\spPfig[\langle \text{key} = \text{val} \rangle]</code>	<code>\spPfig[\langle \text{key} = \text{val} \rangle](\langle \text{punto, punto, punto, ...} \rangle)</code>	
<code>\spPfig(\langle 1 2 3 ... n \rangle)</code>	<code>\spPfig(F P B L T)</code>	
<code>\spPfig[\langle \text{key} = \text{val} \rangle](\langle 1 2 3 ... n \rangle)</code>	<code>\spPfig[\langle \text{key} = \text{val} \rangle](F P B L T)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $\langle \langle 1 | 2 | 3 | ... | n \rangle \rangle$, $\langle \langle \text{punto, punto, punto, ...} \rangle \rangle$ o $\langle \langle F | P | B | L | T \rangle \rangle$ aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $\langle \langle \text{argumento} \rangle \rangle$ y la llave `concept`.

Si se proporciona el *argumento* $\langle \langle 1 | 2 | 3 | ... | n \rangle \rangle$, `\spPfig(1)` imprimirá ' $\mathcal{P}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `\spPfig(n)` imprimirá ' $\mathcal{P}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* $\langle \langle \text{punto, punto, punto, ...} \rangle \rangle$, `\spPfig(ABC)` imprimirá ' $\mathcal{P}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `\spPfig(ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»; `\spPfig(ABCDE)` imprimirá ' $\mathcal{P}_{ABCDE}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del pentágono A B C D E». Con cinco o más puntos no se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* $\langle \langle F | P | B | L | T \rangle \rangle$, `\spPfig(F)` imprimirá ' $\mathcal{P}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `\spPfig(F_{1})` imprimirá ' $\mathcal{P}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$ ni el *argumento opcional delimitado por paréntesis*, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* $\langle \langle \text{punto, punto, punto, ...} \rangle \rangle$ acepta prima(s) $'$ o subíndice(s) $'_{\dots}$. El *argumento* $\langle \langle F | P | B | L | T \rangle \rangle$ solo acepta subíndices $'_{\dots}$ y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para `\spPfig`

`read-cmd` = $\{ \langle \text{school} | \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(ABC)` se verbaliza «perímetro del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \langle \text{argumento} \rangle \rangle$ pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves $\{ \}$.

`concept` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \langle \text{argumento} \rangle \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el $\langle \langle \text{argumento} \rangle \rangle$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\langle \langle \text{argumento} \rangle \rangle$ acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el $\langle \langle \text{argumento} \rangle \rangle$ es equivalente a usar `silent=true`.

`read-arg` = $\{ \langle \text{nombre} \rangle \}$ default: *no usado*

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento $\langle \langle \text{punto, punto, punto, ...} \rangle \rangle$ que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves $\{ \}$ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPfig[read-arg={trapecio}](ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no tiene efecto (se ignora y se emite un aviso) con los argumentos $\langle \langle F | P | B | L | T \rangle \rangle$ y $\langle \langle 1 | 2 | 3 | ... | n \rangle \rangle$.

`concept*` $\langle \text{valor prohibido} \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al $\langle \langle \text{argumento} \rangle \rangle$ pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el $\langle \langle \text{argumento} \rangle \rangle$ acorde al valor establecido por la llave `read-cmd`.

`read-sub` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ y $(\langle F \mid P \mid B \mid L \mid T \rangle)$ para NVDA/MathCAT. Si se utiliza $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ y se establece en **true**, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](1) \$$ imprimirá ‘ $\mathcal{P}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro sub uno*»; si se establece en **false**, la impresión será la misma y se verbaliza «*perímetro uno*».

Si se utiliza $(\langle F \mid P \mid B \mid L \mid T \rangle)$ y se establece en **true**, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](F_{\{1\}}) \$$ imprimirá ‘ $\mathcal{P}_{F_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la figura sub uno*»; si se establece en **false**, la impresión será la misma y se verbaliza «*perímetro de la figura uno*».

`print-sym = { \langle auto \mid triangle \mid square \mid none \rangle }`

default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ y $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ pasado al comando sin modificar la verbalización de este. Si se utiliza $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ se infiere el *símbolo* acorde al número de puntos; si establece en **auto** se imprime y si se establece en **none** se omite: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{auto}](ABC) \$$ y $\$ \backslash \text{spPfig}[\text{print-sym}=\text{none}](ABC) \$$ imprimirán ‘ $\mathcal{P}_{\triangle ABC}$ ’ y ‘ $\mathcal{P}_{ABC}$ ’ en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «*perímetro del triángulo A B C*».

Si se utiliza $(\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle)$ los valores **auto** y **none** quedan sin efecto; si se establece en **triangle** o **square** anida el argumento bajo el *símbolo*: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{triangle}](1) \$$ imprimirá ‘ $\mathcal{P}_{\triangle_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del triángulo uno*»; $\$ \backslash \text{spPfig}[\text{print-sym}=\text{square}](1) \$$ imprimirá ‘ $\mathcal{P}_{\square_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del cuadrado uno*».

Esta llave no se puede utilizar con el argumento $(\langle F \mid P \mid B \mid L \mid T \rangle)$.

`silent = { \langle true \mid false \rangle }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro del*», el concepto «*nombre de la figura*» y el $\{ \langle \text{argumento} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

`\sprad`
`\spdiam`

`\sprad[\langle key = val \rangle]`
`\spdiam[\langle key = val \rangle]`

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y reciben el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se imprimirán en la salida PDF las letras ‘*r*’ y ‘*d*’ y como se verbalizarán en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \backslash \text{sprad} \$$ imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio*»; $\$ \backslash \text{spdiam} \$$ imprimirá ‘*d*’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

Llaves para `\sprad` y `\spdiam`

`concept = { \langle true \mid false \rangle }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*radio*» o «*diámetro*» al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, $\$ \backslash \text{sprad} \$$ y $\$ \backslash \text{spdiam} \$$ se verbalizan como «*radio*» y «*diámetro*» en NVDA/MathCAT; si se establece en **false**, se verbalizarán las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd = { \langle school \mid mathcat \rangle }`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ cuando `concept=false` y `upper-case=true`. Si se establece en **school**, NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ sin el prefijo «mayúscula»: si se establece en **mathcat**, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`upper-case = { \langle true \mid false \rangle }`

default: *false*

Establece si se *imprime* usando mayúsculas. Si se establece en **true**, $\$ \backslash \text{sprad}[\text{upper-case}=\text{true}] \$$ y $\$ \backslash \text{spdiam}[\text{upper-case}=\text{true}] \$$ imprimirán ‘*R*’ y ‘*D*’ en la salida PDF; si se establece en **false** se imprimirá usando minúsculas.

La verbalización al establecer esta llave es controlada por la llave `read-cmd`.

`print-sym = { \langle true \mid false \rangle }`

default: *false*

Establece si se usará `\diameter` antes de la letra ‘*d*’ o ‘*D*’ en la impresión del comando `\spdiam`. Si se establece en **true**, $\$ \backslash \text{spdiam}[\text{print-sym}=\text{true}] \$$ imprimirá ‘ $\emptyset d$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index = { \langle subíndice \rangle }`

default: *no usado*

Establece un *subíndice* sobre las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’. Por ejemplo: $\$ \backslash \text{sprad}[\text{sub-index}=\{1\}] \$$ imprimirá ‘ r_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «*radio uno*».

`read-sub = {⟨true | false⟩}`

default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\sprad[sub-index={1}, read-sub=true]` se verbalizará «*radio sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*radio uno*».

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida exacta}]` imprimirá 'r' en la salida PDF y NVDA/MathCAT verbalizará «*radio medida exacta*»; `\sprad[concept=false, read-arg={medida exacta}]` imprimirá 'r' y NVDA/MathCAT verbalizará «*medida exacta*».

- Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de las letras 'r', 'R', 'd' o 'D' al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨centro⟩)` o `(⟨centro, radio⟩)`, donde `⟨radio⟩` es opcional y debe pasarse junto al argumento `(⟨centro⟩)` separado por una coma ',' obligatoria.

La forma en la que se procesa el `⟨radio⟩` junto a su impresión y verbalización se establece de manera interna: Si es un `⟨número⟩` entero o decimal se usa `\spnum`; si es una `⟨medida⟩` se usa `\spunit`; si es un segmento `⟨punto punto⟩` se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`; y si es una letra se imprime tal cual. Cualquier otra forma devuelve un "error" y se detiene la compilación del documento.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]` ni el *argumento opcional delimitado por paréntesis*, `\spcirc` imprimirá '⊙' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia*».

Si se proporciona el *argumento* `(⟨centro⟩)`, `\spcirc(⟨centro⟩)` imprimirá '⊙(O)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O*».

Si se proporciona el *argumento* `(⟨centro, radio⟩)`, `\spcirc(⟨centro, radio⟩)` imprimirá '⊙(O,3)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O y radio tres*».

- El argumento `(⟨centro⟩)` y el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩` aceptan prima ' ' o subíndice `_{...}`.

LLaves para `\spcirc`

El comando `\spcirc` posee las *meta-keys* `spside` pasada al comando `\spside` (v. pág. 22) y `spmside` pasada al comando `\spmside` (v. pág. 22).

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `(⟨centro⟩)`. Si se establece en *school*, NVDA/MathCAT verbaliza el argumento `(⟨centro⟩)` sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

- Esta llave tiene efecto sobre el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩`.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {⟨usual | odot⟩}`

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spcirc[print-sym=usual](O)` imprimirá 'C(O)' en la salida PDF; si se establece en *odot*, `\spcirc[print-sym=odot](O)` imprimirá '⊙(O)' en la salida PDF.

`read-sty = {⟨circun | circle⟩}`

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spcirc[read-sty=circun](O)` se verbalizará «*circunferencia de centro en O*» en NVDA/MathCAT; si se establece en *circle*, `\spcirc[read-sty=circle](O)` se verbalizará «*círculo de centro en O*» en NVDA/MathCAT.

`diameter = {⟨true | false⟩}`

default: *false*

Establece *como* se verbaliza el argumento *(radio)* al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[radio=true](O, 3)` se verbalizará «circunferencia de centro en *O* y diámetro tres» en NVDA/MathCAT; si se establece en `false`, `\spc[radio=false](O, 3)` se verbalizará «circunferencia de centro en *O* y radio tres» en NVDA/MathCAT.

`radio-sty = { (side | mside) }`

default: *mside*

Establece el *comando* que procesa internamente el argumento *(radio)* cuando tiene la forma *(punto punto)*. Si se establece en `side` se usará `\spside{(punto punto)}`, `\spc[radio-sty=side](O, AB)` imprimirá $\odot(O, \overline{AB})$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio lado *A B*»; si se establece en `mside` se usará `\spmside{(punto punto)}`, `\spc[radio-sty=mside](O, AB)` imprimirá $\odot(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio medida del lado *A B*».

- Las *(meta-keys)* `spside` y `spmside` permiten modificar la impresión y verbalización del argumento *(radio)* cuando tiene la forma *(punto punto)*.

`concept = { (true | false) }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el «concepto» y el argumento *(centro)* o *(centro, radio)*; si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento *(centro)* o *(centro, radio)* acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento *(centro)* es equivalente a usar `silent=true`.

`sub-index = { (subíndice) }`

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[sub-index={1},radio-sty=mside](O, AB)` imprimirá $\odot_1(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en *O* y radio medida del lado *A B*».

`read-sub = { (true | false) }`

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[sub-index={1},read-sub=true](O)` se verbalizará «circunferencia sub uno de centro en *O*»; si se establece en `false`, NVDA/MathCAT verbalizará «circunferencia uno de centro en *O*».

`read-arg = { (verbalización NVDA) }`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[read-arg={disco}](O)` imprimirá $\odot(O)$ en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en *O*».

`silent = { (true | false) }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento *(centro)* o *(centro, radio)* pasados al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[(key = val)]</code>
	<code>\spAcirc((centro))</code>	<code>\spAcirc[(key = val)]((centro))</code>
	<code>\spAcirc(1 2 3 ... n)</code>	<code>\spAcirc[(key = val)](1 2 3 ... n)</code>

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma *(centro)* o *(1 | 2 | 3 | ... | n)*, aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* `[(key = val)]` ni el *argumento opcional delimitado por paréntesis*, `\spAcirc` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* *(centro)*, `\spAcirc(O)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en *O*».

Si se proporciona el *argumento* *(1 | 2 | 3 | ... | n)*, `\spAcirc(1)` imprimirá ' $\mathcal{A}_{\odot_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* *(centro)* acepta prima '' o subíndice '_{...}' y no acepta un *(radio)*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spAcirc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`<centro>`). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento (`<centro>`) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty` = { `<circun | circle>` }

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spAcirc[read-sty=circun](0)` se verbalizará «área de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, `\spAcirc[read-sty=circle](0)` se verbalizará «área del círculo con centro en O» en NVDA/MathCAT.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento (`<centro>`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`<centro>`) o con el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`read-arg` = { `<verbalización NVDA>` }

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spAcirc[read-arg={esfera}](0)` se verbalizará «área de la esfera con centro en O» en NVDA/MathCAT; `\spAcirc[concept=false,read-arg={esfera}](0)` se verbalizará «esfera con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`print-sym` = { `<usual | odot>` }

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spAcirc[print-sym=usual](0)` imprimirá ' $\mathcal{A}_{C(O)}$ ' en la salida PDF; si se establece en *odot*, `\spAcirc[print-sym=odot](0)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF.

`sub-index` = { `<subíndice>` }

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`<centro>`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAcirc[sub-index={1}](0)` imprimirá ' $\mathcal{A}_{\odot_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O».

`read-sub` = { `<true | false>` }

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`<1 | 2 | 3 | ... | n>`) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spAcirc[read-sub=true](1)` se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT si verbalizará la salida del comando.

El comando `\spPcirc`

`\spPcirc` `\spPcirc`

`\spPcirc(<centro>)`

`\spPcirc(<1 | 2 | 3 | ... | n>)`

`\spPcirc[<key = val>]`

`\spPcirc[<key = val>](<centro>)`

`\spPcirc[<key = val>](<1 | 2 | 3 | ... | n>)`

El comando `\spPcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`<centro>`) o (`<1 | 2 | 3 | ... | n>`), aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el *argumento opcional delimitado por paréntesis*, `\spPcirc` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el argumento ($\langle \text{centro} \rangle$), $\$ \backslash \text{spPcirc}(0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O».

Si se proporciona el argumento ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$), $\$ \backslash \text{spPcirc}(1) \$$ imprimirá ' $\mathcal{P}_{O_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El argumento ($\langle \text{centro} \rangle$) acepta prima ' ' o subíndice '_{...}' y no acepta un $\langle \text{radio} \rangle$. Una coma ',' en el argumento siempre produce un "error".

Llaves para $\backslash \text{spPcirc}$

`read-cmd = { $\langle \text{school} \mid \text{mathcat} \rangle$ }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento ($\langle \text{centro} \rangle$). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento ($\langle \text{centro} \rangle$) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { $\langle \text{prefijo} \rangle$ }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = { $\langle \text{circun} \mid \text{circle} \rangle$ }` default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «perímetro de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, $\$ \backslash \text{spPcirc}[\text{read-sty}=\text{circun}](0) \$$ se verbalizará «perímetro de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, $\$ \backslash \text{spPcirc}[\text{read-sty}=\text{circle}](0) \$$ se verbalizará «perímetro del círculo con centro en O» en NVDA/MathCAT.

`concept = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento ($\langle \text{centro} \rangle$) o ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento ($\langle \text{centro} \rangle$) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento ($\langle \text{centro} \rangle$) o con el argumento ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$) es equivalente a usar `silent=true`.

`read-arg = { $\langle \text{verbalización NVDA} \rangle$ }` default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spPcirc}[\text{read-sty}=\text{circle}, \text{read-arg}=\{\text{disco}\}](0) \$$ se verbalizará «perímetro del disco con centro en O» en NVDA/MathCAT; $\$ \backslash \text{spPcirc}[\text{concept}=\text{false}, \text{read-arg}=\{\text{disco}\}](0) \$$ se verbalizará «disco con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$) es equivalente a usar `silent=true`.

`print-sym = { $\langle \text{usual} \mid \text{odot} \rangle$ }` default: *odot*

Establece si se *usará* $\backslash \text{bigodot}$ o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, $\$ \backslash \text{spPcirc}[\text{print-sym}=\text{usual}](0) \$$ imprimirá ' $\mathcal{P}_{C(0)}$ ' en la salida PDF; si se establece en *odot*, $\$ \backslash \text{spPcirc}[\text{print-sym}=\text{odot}](0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF.

`sub-index = { $\langle \text{subíndice} \rangle$ }` default: *no usado*

Establece un *subíndice* sobre $\backslash \text{bigodot}$ o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento ($\langle \text{centro} \rangle$). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash \text{spPcirc}[\text{sub-index}=\{1\}](0) \$$ imprimirá ' $\mathcal{P}_{O_1(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno con centro en O».

`read-sub = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash \text{spPcirc}[\text{read-sub}=\text{true}](1) \$$ se verbalizará «perímetro de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

`silent = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...», del concepto establecido por la llave `read-sty` y del argumento ($\langle \text{centro} \rangle$) o ($\langle 1 \mid 2 \mid 3 \mid \dots \mid n \rangle$) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spVol`

 $\spVol$ $\spVol$ $\spVol[\langle key = val \rangle]$

El comando `\spVol` trabaja en *modo matemático* y recibe el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \spVol \$$ imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen*».

Llaves para `\spVol`

`concept` = $\{\langle true | false \rangle\}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*volumen*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \spVol \$$ se verbaliza como «*volumen*» en NVDA/MathCAT; si se establece en *false*, se verbalizará la letra ‘ $\mathcal{V}$ ’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd` = $\{\langle school | mathcat \rangle\}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra ‘ $\mathcal{V}$ ’ cuando `concept=false`. Si se establece en *school*, NVDA/MathCAT verbalizará únicamente la letra ‘ $\mathcal{V}$ ’; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas sobre ‘ $\mathcal{V}$ ’.

`sub-index` = $\{\langle subíndice \rangle\}$ default: *no usado*

Establece un *subíndice* sobre ‘ $\mathcal{V}$ ’. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’. Por ejemplo: $\$ \spVol[\text{sub-index}=\{1\}] \$$ imprimirá ‘ $\mathcal{V}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen uno*».

`read-sub` = $\{\langle true | false \rangle\}$ default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \spVol[\text{sub-index}=\{1\}, \text{read-sub}=\text{true}] \$$ se verbalizará «*volumen sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*volumen uno*».

`read-arg` = $\{\langle verbalización NVDA \rangle\}$ default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \spVol[\text{read-arg}=\{\text{medida exacta}\}] \$$ imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen medida exacta*»; $\$ \spVol[\text{concept}=\text{false}, \text{read-arg}=\{\text{medida exacta}\}] \$$ imprimirá ‘ $\mathcal{V}$ ’ y NVDA/MathCAT verbalizará «*medida exacta*».

🔹 Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent` = $\{\langle true | false \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de ‘ $\mathcal{V}$ ’ al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

8 Utilidades para álgebra

El paquete `spintent` provee utilidades para trabajar con álgebra descritas en esta sección.

El comando `\spinterval`

 $\spinterval$ $\spinterval\{\langle intervalo \rangle\}$ $\spinterval[\langle key = val \rangle]\{\langle intervalo \rangle\}$

El comando `\spinterval` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{\langle intervalo \rangle\}$ de la forma $\{\langle (a; b) \rangle\}$, donde ‘*a*’ y ‘*b*’ son los *extremos* del intervalo separados por un punto y coma ‘*;*’ o, si no hay punto y coma, por una única coma ‘*,*’. El delimitador izquierdo puede ser ‘(’, ‘[’ o ‘]’ y el derecho ‘)’, ‘]’ o ‘[’, en cualquier combinación. Los extremos numéricos se procesan internamente bajo el comando `\spnum`; cualquier otro extremo, como una letra, un subíndice, una fracción o una raíz, se imprime tal cual y lo verbaliza NVDA/MathCAT.

Un delimitador ‘(’ o ‘]’ a la izquierda, y ‘)’ o ‘[’ a la derecha, *excluye* su extremo y el intervalo es *abierto* en él; los demás delimitadores lo *incluyen* y el intervalo es *cerrado* en él.

Si el intervalo es abierto en sus dos extremos, $\$ \spinterval\{(1,5)\} \$$ imprimirá ‘(1,5)’ en la salida PDF y NVDA/MathCAT verbalizará «*intervalo abierto de uno a cinco*»; la notación chilena $\$ \spinterval\{]1,5[\} \$$ imprimirá ‘]1,5[’ y se verbalizará igual.

Si el intervalo es cerrado en sus dos extremos, $\$ \spinterval\{[1,5]\} \$$ imprimirá ‘[1,5]’ y NVDA/MathCAT verbalizará «*intervalo cerrado de uno a cinco*».

Si el intervalo es abierto en un extremo y cerrado en el otro, $\$ \spinterval\{]1,5]\} \$$ imprimirá ‘]1,5]’ y NVDA/MathCAT verbalizará «*intervalo abierto en uno y cerrado en cinco*»; $\$ \spinterval\{[1,5[\} \$$ imprimirá ‘[1,5[’ y verbalizará «*intervalo cerrado en uno y abierto en cinco*».

Si un extremo es infinito, $\$ \backslash \text{spinterval}\{-\infty, 3\}$ imprimirá $[-\infty, 3]$ y NVDA/MathCAT verbalizará «intervalo abierto en menos infinito y cerrado en tres».

Si los extremos son números decimales con coma, el separador debe ser el punto y coma: $\$ \backslash \text{spinterval}\{3,5;4,2\}$ imprimirá $[3,5;4,2]$ y NVDA/MathCAT verbalizará «intervalo cerrado de tres coma cinco a cuatro coma dos».

El comando $\backslash \text{spinterval}$ devuelve un “error” y se detiene la compilación del documento cuando el $\{ \langle \text{intervalo} \rangle \}$ no comienza con un delimitador izquierdo y termina con uno derecho de los indicados, cuando un extremo está vacío, cuando no hay separador, cuando hay más de una coma sin punto y coma, cuando las llaves no están balanceadas o cuando el infinito está mal ubicado o se incluye en el intervalo.

- El argumento $\{ \langle \text{intervalo} \rangle \}$ siempre debe pasarse entre llaves ‘{ }’, incluso cuando comienza con un corchete: sin ellas, $\backslash \text{spinterval}[1,5]$ interpreta los corchetes como el argumento opcional $[(key = val)]$.
- El infinito ‘ ∞ ’ nunca se incluye en el intervalo: ‘ $-\infty$ ’ solo puede ir en el extremo izquierdo y ‘ ∞ ’ en el derecho, ambos junto a un delimitador que los excluya.

LLaves para $\backslash \text{spinterval}$

$\text{read-cmd} = \{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará los extremos del intervalo. Si se establece en *school*, $\$ \backslash \text{spinterval}[\text{read-cmd=school}]\{-2,+4\}$ se verbalizará «intervalo abierto de menos dos a más cuatro»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas y $\$ \backslash \text{spinterval}[\text{read-cmd=mathcat}]\{-2,+4\}$ se verbalizará «intervalo abierto de menos dos a positivo cuatro».

$\text{prefix} = \{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» intervalo. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: $\$ \backslash \text{spinterval}[\text{prefix}=\{el\}]\{(1,5)\}$ imprimirá ‘(1, 5)’ en la salida PDF y NVDA/MathCAT verbalizará «el intervalo abierto de uno a cinco».

$\text{read-arg} = \{ \langle \text{verbalización NVDA} \rangle \}$ default: *no usado*

Sobrescribe la verbalización del *intervalo* completo que será verbalizada por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: $\$ \backslash \text{spinterval}[\text{read-arg}=\{\text{rango}\}]\{[1,5]\}$ imprimirá ‘[1, 5]’ en la salida PDF y NVDA/MathCAT verbalizará «rango».

$\text{concept} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» intervalo y del prefijo establecido por la llave *prefix* al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el «concepto» y los extremos; si se establece en *false*, $\$ \backslash \text{spinterval}[\text{concept=false}]\{(1,5)\}$ se verbalizará «abierto de uno a cinco».

$\text{size} = \{ \langle \text{auto} \mid \text{none} \mid \text{big} \mid \text{Big} \mid \text{bigg} \mid \text{Bigg} \rangle \}$ default: *auto*

Establece el *tamaño* de los delimitadores del intervalo sin modificar la verbalización de este. Si se establece en *auto*, los delimitadores conservan su tamaño normal salvo que algún extremo contenga una fracción $\backslash \text{frac}$ o $\backslash \text{dfrac}$, en cualquier posición del extremo: en ese caso se usan $\backslash \text{bigl}$ y $\backslash \text{bigr}$ en estilo texto, $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$ en estilo display, y siempre $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$ cuando la fracción es $\backslash \text{dfrac}$. Si se establece en *none*, los delimitadores nunca cambian de tamaño; si se establece en *big*, *Big*, *bigg* o *Bigg*, se usan siempre $\backslash \text{bigl}$ y $\backslash \text{bigr}$, $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$, $\backslash \text{biggl}$ y $\backslash \text{biggr}$, o $\backslash \text{Biggl}$ y $\backslash \text{Biggr}$, respectivamente.

$\text{silent} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{intervalo} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando $\backslash \text{spfun}$

$\backslash \text{spfun} \quad \backslash \text{spfun}\{ \langle \text{función} \rangle \} \quad \backslash \text{spfun}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{función} \rangle \}$

El comando $\backslash \text{spfun}$ trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{ \langle \text{función} \rangle \}$ de la forma $\{ \langle f(x) \rangle \}$, donde ‘*f*’ es el *nombre* de la función y lo que va entre paréntesis son uno o más *argumentos* separados por coma ‘,’. El nombre es una letra, mayúscula o minúscula, con prima(s) ‘*’*’ o subíndice ‘*_*’...’ opcionales; el subíndice solo admite letras y dígitos. Los argumentos pueden ser letras, números o expresiones.

$\$ \backslash \text{spfun}\{f(x)\}$ imprimirá ‘*f(x)*’ en la salida PDF y NVDA/MathCAT verbalizará «*f de x*»; $\$ \backslash \text{spfun}\{f_{-1}(x)\}$ imprimirá ‘*f₁(x)*’ y verbalizará «*f sub uno de x*»; $\$ \backslash \text{spfun}\{f'(x)\}$ imprimirá ‘*f'(x)*’ y verbalizará «*f prima de x*»; $\$ \backslash \text{spfun}\{f''(x)\}$ imprimirá ‘*f''(x)*’ y verbalizará «*f doble prima de x*».

Con dos o más argumentos, $\$ \backslash \text{spfun}\{f(x,y)\}$ imprimirá ‘*f(x,y)*’ y NVDA/MathCAT verbalizará «*f de x coma y*». Un argumento puede ser una expresión: $\$ \backslash \text{spfun}\{f(x+1)\}$ imprimirá ‘*f(x + 1)*’ y verbalizará «*f de x más 1*».

Un argumento puede ser a su vez una llamada, tantas veces como se quiera: $\$ \backslash \text{spfun}\{f(g(x))\}$ imprimirá ‘*f(g(x))*’ y verbalizará «*f de g de x*»; $\$ \backslash \text{spfun}\{f(g^{-1}(y))\}$ imprimirá ‘*f(f⁻¹(y))*’ y verbalizará «*f de f inversa de y*»; $\$ \backslash \text{spfun}\{f(g(x),h(y))\}$ imprimirá ‘*f(g(x), h(y))*’. Una llamada anidada se valida igual

que la exterior. Si el argumento es otra cosa, como $g(x)+1$, se imprime y se verbaliza con normalidad pero sin anotar la llamada que contiene. Las llaves `read-cmd` y `silent` valen para todos los niveles; `concept` y `prefix` solo para la llamada exterior.

Las comas que quedan dentro de llaves, de paréntesis o de corchetes no separan argumentos. Por eso, un número decimal con coma debe escribirse entre llaves: $\$ \backslash \text{spfun}\{f(1{,}5)\}$ tiene un solo argumento.

El comando `\spfun` devuelve un “error” y se detiene la compilación del documento cuando el $\{ \langle \text{función} \rangle \}$ no comienza con un nombre válido, cuando el subíndice contiene algo distinto de letras o dígitos, cuando el nombre no va seguido de paréntesis, cuando los paréntesis, las llaves o los corchetes no están balanceados, cuando un argumento está vacío o cuando hay algo después del paréntesis que cierra. Lo mismo vale para una llamada anidada en un argumento.

Llaves para `\spfun`

`read-cmd` = $\{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el nombre de la función. Si se establece en `school`, NVDA/MathCAT verbaliza el nombre sin el prefijo «mayúscula»: $\$ \backslash \text{spfun}\{F'(x)\}$ se verbaliza «*F doble prima de x*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*función*». El valor de esta llave *siempre* debe pasarse entre llaves $\{ \}$. Solo se verbaliza cuando `concept` es `true`.

`concept` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se verbaliza el concepto «*función*» antes de la función. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto, el prefijo establecido por la llave `prefix` y la función; si se establece en `false`, solo verbalizará la función.

`silent` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{función} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. El argumento se valida igual en ambos casos.

Los comandos `\spdom`, `\sprec` y `\spcod`

`\spdom` $\backslash \text{spdom}\{ \langle \text{función} \rangle \}$ $\backslash \text{spdom}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{función} \rangle \}$

`\sprec` Los comandos `\spdom`, `\sprec` y `\spcod` trabajan en *modo matemático* y reciben el *argumento* obligatorio $\{ \langle \text{función} \rangle \}$: el nombre de una función como en `\spfun` (una letra con prima(s) o subíndice opcionales y, si se quiere, el exponente $\wedge\{-1\}$), sin argumentos. Imprimen respectivamente Dom, Rec y Cod seguidos de la función entre paréntesis.

$\$ \backslash \text{spdom}\{f\}$ imprimirá ‘Dom(*f*)’ y NVDA/MathCAT verbalizará «*dominio de f*»; $\$ \backslash \text{sprec}\{f\}$ verbalizará «*recorrido de f*» y $\$ \backslash \text{spcod}\{f\}$ «*codominio de f*». Con la inversa, $\$ \backslash \text{spdom}\{f^{\wedge\{-1\}}\}$ verbalizará «*dominio de f inversa*». El nombre de la función se verbaliza sin anunciar la mayúscula.

Los comandos devuelven un “error” y se detiene la compilación cuando el argumento no es el nombre de una función.

Llaves para `\spdom`, `\sprec` y `\spcod`

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de la palabra del concepto. El valor de esta llave *siempre* debe pasarse entre llaves $\{ \}$.

`silent` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{función} \rangle \}$ pasado al comando sin modificar la impresión de este.

El comando `\spfuncomp`

`\spfuncomp` $\backslash \text{spfuncomp}\{ \langle \text{composición} \rangle \}$ $\backslash \text{spfuncomp}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{composición} \rangle \}$

El comando `\spfuncomp` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{ \langle \text{composición} \rangle \}$: dos o más *funciones* separadas por `\circ`, como en $\{ \langle f \circ g \rangle \}$ o $\{ \langle f \circ g \circ h \rangle \}$. Cada función es un nombre como en `\spfun`: una letra, mayúscula o minúscula, con prima(s) o subíndice opcionales, y el exponente $\wedge\{-1\}$ de la función inversa. La composición puede ir entre paréntesis, $\{ \langle (f \circ g) \rangle \}$, y puede aplicarse a uno o más argumentos entre paréntesis separados por coma, $\{ \langle (f \circ g)(x) \rangle \}$. Los argumentos siguen las reglas de `\spfun`: pueden ser letras, números, expresiones o llamadas.

$\$ \backslash \text{spfuncomp}\{f \circ g\}$ imprimirá ‘*f* ◦ *g*’ en la salida PDF y NVDA/MathCAT verbalizará «*f compuesta con g*»; $\$ \backslash \text{spfuncomp}\{f \circ g \circ h\}$ imprimirá ‘*f* ◦ *g* ◦ *h*’ y verbalizará «*f compuesta con g compuesta con h*»; $\$ \backslash \text{spfuncomp}\{(f \circ g)(x)\}$ imprimirá ‘(*f* ◦ *g*)(*x*)’ y verbalizará «*f compuesta con g de x*»;

`$\spfuncomp{(f^{-1}\circ g)(x)}` imprimirá $(f^{-1}\circ g)(x)$ y verbalizará «*f inversa compuesta con g de x*». Un argumento puede ser una llamada: `$\spfuncomp{(f\circ g)(h(x))}` verbalizará «*f compuesta con g de h de x*».

El comando `\spfuncomp` devuelve un “error” y se detiene la compilación del documento cuando hay menos de dos funciones, cuando algo entre los `\circ` no es el nombre de una función, cuando los paréntesis no están balanceados, cuando lo que sigue a la composición no es una lista de argumentos entre paréntesis o cuando un argumento está vacío. La forma `f\circ g(x)`, sin paréntesis alrededor de la composición, no es válida porque es ambigua.

Llaves para `\spfuncomp`

`silent = {\true | false}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del `{\composición}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. El argumento se valida igual en ambos casos.

La composición no admite `prefix` ni `read-cmd`: no lleva palabra de concepto y su lectura es fija. Los nombres se verbalizan sin anunciar la mayúscula.

9 Utilidades para estadística

El paquete `spintent` provee utilidades para trabajar con estadística descritas en esta sección.

El comando `\spProm`

<code>\spProm</code>	<code>\spProm{\argumento}</code>	<code>\spProm[key = val]{\argumento}</code>
----------------------	----------------------------------	---

El comando `\spProm` trabaja en *modo matemático* y recibe el *argumento* obligatorio `{\argumento}`: cualquier expresión matemática, como una letra, una letra con subíndice o una expresión. Lo tipografía con una barra encima, la notación usual del «promedio».

`$\spProm{x}` imprimirá $\bar{x}$ en la salida PDF y NVDA/MathCAT verbalizará «*promedio de x*»; `$\spProm{x_{1}}` imprimirá $\bar{x}_1$ y verbalizará «*promedio de x sub uno*».

- El `{\argumento}` se tipografía directamente, sin procesamiento semántico adicional, y NVDA/MathCAT lo verbaliza con sus propias reglas.

Llaves para `\spProm`

`silent = {\true | false}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del `{\argumento}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando.

`read-arg = {\verbalización NVDA}` default: *no usado*

Sobrescribe las palabras «*promedio de*» que verbaliza NVDA/MathCAT antes del `{\argumento}`, sin modificar la impresión del comando. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`. Por ejemplo: `$\spProm[read-arg={media de}]{x}` imprimirá $\bar{x}$ en la salida PDF y NVDA/MathCAT verbalizará «*media de x*».

Los comandos `\spmedia`, `\spmediana`, `\spmoda`, `\spdesv` y `\spvarz`

<code>\spmedia</code>	<code>\spmedia</code>	<code>\spmedia[key = val]</code>
<code>\spmediana</code>	<code>\spmediana</code>	<code>\spmediana[key = val]</code>
<code>\spmoda</code>	<code>\spmoda</code>	<code>\spmoda[key = val]</code>
<code>\spdesv</code>	<code>\spdesv</code>	<code>\spdesv[key = val]</code>
<code>\spvarz</code>	<code>\spvarz</code>	<code>\spvarz[key = val]</code>

Los comandos `\spmedia`, `\spmediana`, `\spmoda`, `\spdesv` y `\spvarz` trabajan en *modo matemático* y reciben el *argumento opcional* `[key = val]` mediante el cual se establece la «*forma*» en la que se verbalizarán en NVDA/MathCAT los símbolos de la estadística descriptiva.

Si no se proporciona el *argumento opcional*, `$\spmedia$` imprimirá ‘MA’ en la salida PDF y NVDA/MathCAT verbalizará «*media aritmética*»; `$\spmediana$` imprimirá ‘MD’ y verbalizará «*mediana*»; `$\spmoda$` imprimirá ‘MO’ y verbalizará «*moda*»; `$\spdesv$` imprimirá ‘s’ y verbalizará «*desviación estándar s*»; `$\spvarz$` imprimirá ‘s²’ y verbalizará «*varianza s al cuadrado*».

Llaves para `\spmedia`, `\spmediana`, `\spmoda`, `\spdesv` y `\spvarz``sigma = {\true | false}`default: *false*

Establece si se *imprime* la letra ' σ ' en lugar de la letra '*s*'. Si se establece en `true`, `\spdesv[sigma=true]` imprimirá ' σ ' y `\spvarz[sigma=true]` imprimirá ' σ^2 ' en la salida PDF, y NVDA/MathCAT verbalizará «desviación estándar *sigma*» y «varianza *sigma al cuadrado*». Esta llave solo existe para `\spdesv` y `\spvarz`.

`sub-index = {\subíndice}`default: *no usado*

Establece un *subíndice* sobre el símbolo del comando, que NVDA/MathCAT verbaliza después del concepto con la palabra «*de*» (o «*sub*» en `\spdesv` y `\spvarz`, vea la llave `read-sub`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spmedia[sub-index={x}]` imprimirá ' MA_x ' en la salida PDF y NVDA/MathCAT verbalizará «*media aritmética de x*»; `\spdesv[sub-index={x}]` imprimirá ' s_x ' y verbalizará «*desviación estándar s sub x*»; `\spvarz[sub-index={x}]` imprimirá ' s_x^2 ' y verbalizará «*varianza s sub x al cuadrado*».

`read-sub = {\true | false}`default: *false*

Establece si se *usa* la palabra «*sub*» en lugar de «*de*» al verbalizar el subíndice establecido por la llave `sub-index`, sin modificar la impresión del comando. Si se establece en `true`, `\spmedia[sub-index={x}, read-sub=true]` se verbalizará «*media aritmética sub x*» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «*media aritmética de x*». El valor por defecto es `false`, salvo en `\spdesv` y `\spvarz`, donde es `true` porque estos comandos leen su letra y «s sub x» es más claro que «s de x» (en «*varianza s de x al cuadrado*», la «x» podría entenderse como parte del exponente). Así, `\spvarz[sub-index={x}, read-sub=false]` verbalizará «*varianza s de x al cuadrado*».

`read-arg = {\verbalización NVDA}`default: *no usado*

Sobrescribe el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spdesv[read-arg={desviación estándar poblacional}]` imprimirá '*s*' en la salida PDF y NVDA/MathCAT verbalizará «*desviación estándar poblacional*».

Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {\true | false}`default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprango`, `\spriq`, `\spmin` y `\spmax`

<code>\sprango</code>	<code>\sprango</code>	<code>\sprango[key = val]</code>
<code>\spriq</code>	<code>\spriq</code>	<code>\spriq[key = val]</code>
<code>\spmin</code>	<code>\spmin</code>	<code>\spmin[key = val]</code>
<code>\spmax</code>	<code>\spmax</code>	<code>\spmax[key = val]</code>

Los comandos `\sprango`, `\spriq`, `\spmin` y `\spmax` trabajan en *modo matemático* y reciben el *argumento opcional* `[key = val]` mediante el cual se establece la «*forma*» en la que se verbalizarán en NVDA/MathCAT estas medidas de la estadística descriptiva.

Si no se proporciona el *argumento opcional*, `\sprango` imprimirá '*R*' en la salida PDF y NVDA/MathCAT verbalizará «*rango*»; `\spriq` imprimirá '*RIQ*' y verbalizará «*rango intercuartil*»; `\spmin` imprimirá ' $x_{\min}$ ' y verbalizará «*mínimo*»; `\spmax` imprimirá ' $x_{\max}$ ' y verbalizará «*máximo*».

Llaves para `\sprango`, `\spriq`, `\spmin` y `\spmax`

Las llaves `silent`, `read-arg`, `sub-index` y `read-sub` de estos comandos funcionan como las de `\spmedia` (vea la página 37), con una excepción: `\spmin` y `\spmax` solo tienen `silent` y `read-arg`, porque su símbolo ya lleva un subíndice.

Los comandos `\spcuartil`, `\spdecil` y `\spPerc`

<code>\spcuartil</code>	<code>\spcuartil{k}</code>	<code>\spcuartil[key = val]{k}</code>
<code>\spdecil</code>	<code>\spdecil{k}</code>	<code>\spdecil[key = val]{k}</code>
<code>\spPerc</code>	<code>\spPerc{k}</code>	<code>\spPerc[key = val]{k}</code>

Los comandos `\spcuartil`, `\spdecil` y `\spPerc` trabajan en *modo matemático* y reciben el *argumento obligatorio* `{k}`: un número entero sin signo, de 1 a 3 en `\spcuartil`, de 1 a 9 en `\spdecil` y de 1 a 99 en `\spPerc`. Cualquier otro valor se rechaza con un error y no se imprime nada.

`\spcuartil{1}` imprimirá ' Q_1 ' en la salida PDF y NVDA/MathCAT verbalizará «*cuartil 1*»; `\spdecil{3}` imprimirá ' D_3 ' y verbalizará «*decil 3*»; `\spPerc{25}` imprimirá ' P_{25} ' y verbalizará «*percentil 25*».

Llaves para `\spcuartil`, `\spdecil` y `\spPerc`

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. El índice inválido se rechaza siempre, también con `silent=true`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe lo que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spcuartil[read-arg={primer cuartil}]{1}` imprimirá 'Q₁' en la salida PDF y NVDA/MathCAT verbalizará «*primer cuartil*».

El comando `\spFrec`

`\spFrec` `\spFrec{<símbolo>}` `\spFrec[<key = val>]{<símbolo>}`

El comando `\spFrec` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<símbolo>}: el símbolo de una frecuencia, escrito tal como se imprime. Los símbolos válidos son `f`, `F`, `f_{r}`, `F_{r}`, `f_{r}\%` y `F_{r}\%`; cualquier otro se rechaza con un error y no se imprime nada, salvo que se use la llave `read-arg`.

`\spFrec{f}` imprimirá '*f*' en la salida PDF y NVDA/MathCAT verbalizará «*frecuencia absoluta*»; `\spFrec{F}` imprimirá '*F*' y verbalizará «*frecuencia absoluta acumulada*»; `\spFrec{f_{r}}` imprimirá '*f_r*' y verbalizará «*frecuencia relativa*»; `\spFrec{F_{r}}` imprimirá '*F_r*' y verbalizará «*frecuencia relativa acumulada*»; `\spFrec{f_{r}\%}` imprimirá '*f_r%*' y verbalizará «*frecuencia porcentual*»; `\spFrec{F_{r}\%}` imprimirá '*F_r%*' y verbalizará «*frecuencia porcentual acumulada*».

Llaves para `\spFrec`

`sub-index` = { `<subíndice>` } default: `no usado`

Establece un *subíndice* sobre el símbolo, que NVDA/MathCAT verbaliza después del nombre de la frecuencia con la palabra «*sub*». Solo es válida con los símbolos `f` y `F`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spFrec[sub-index={i}]{f}` imprimirá '*f_i*' en la salida PDF y NVDA/MathCAT verbalizará «*frecuencia absoluta sub i*».

`read-sub` = { `<true | false>` } default: `true`

Establece si se *usa* la palabra «*sub*» en lugar de «*de*» al verbalizar el subíndice establecido por la llave `sub-index`, sin modificar la impresión del comando. Con `\spFrec[sub-index={i}, read-sub=false]{f}`, NVDA/MathCAT verbalizará «*frecuencia absoluta de i*».

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *nombre de la frecuencia* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Es la única forma de usar un símbolo que no está en la tabla anterior: `\spFrec[read-arg={frecuencia absoluta}]{n_{i}}` imprimirá '*n_i*' y verbalizará «*frecuencia absoluta*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. El símbolo inválido se rechaza siempre, también con `silent=true`.

El comando `\spcase`

`\spcase` `\spcase{<letra>}` `\spcase[<key = val>]{<letra>}`

El comando `\spcase` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<letra>}: la letra que acompaña a los casos de una probabilidad. Las letras válidas son `f`, `p` y `t`; cualquier otra se rechaza con un error y no se imprime nada.

`\spcase{f}` imprimirá '*C_f*' en la salida PDF y NVDA/MathCAT verbalizará «*casos favorables*»; `\spcase{p}` imprimirá '*C_p*' y verbalizará «*casos posibles*»; `\spcase{t}` imprimirá '*C_t*' y verbalizará «*casos totales*».

Llaves para `\spcase`

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *nombre de los casos* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spcase[read-arg={casos que cumplen}]{f}` imprimirá '*C_f*' en la salida PDF y NVDA/MathCAT verbalizará «*casos que cumplen*».

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. La letra inválida se rechaza siempre, también con `silent=true`.

El comando `\spOmega`

`\spOmega` `\spOmega` `\spOmega[⟨key = val⟩]`

El comando `\spOmega` trabaja en *modo matemático* y recibe el *argumento opcional* [`⟨key = val⟩`] mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT el espacio muestral.

Si no se proporciona el *argumento opcional*, `\spOmega` imprimirá ‘ Ω ’ en la salida PDF y NVDA/MathCAT verbalizará «*espacio muestral*».

Llaves para `\spOmega`

`card = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el símbolo ‘#’ antes de la letra, para representar la cardinalidad del espacio muestral. Si se establece en *true*, `\spOmega[card=true]` imprimirá ‘# Ω ’ en la salida PDF y NVDA/MathCAT verbalizará «*cardinalidad del espacio muestral*».

`sub-index = {⟨subíndice⟩}` default: *no usado*

Establece un *subíndice* sobre la letra, que NVDA/MathCAT verbaliza después del nombre con la palabra «*sub*». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spOmega[sub-index={1}]` imprimirá ‘ Ω_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «*espacio muestral sub 1*»; `\spOmega[card=true, sub-index={1}]` imprimirá ‘# Ω_1 ’ y verbalizará «*cardinalidad del espacio muestral sub 1*».

`read-arg = {⟨verbalización NVDA⟩}` default: *no usado*

Sobrescribe el *nombre* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spOmega[read-arg={universo}]` imprimirá ‘ Ω ’ en la salida PDF y NVDA/MathCAT verbalizará «*universo*».

🔹 La llave `sub-index` no tiene efecto al usar `read-arg`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, sí verbalizará la salida del comando.

El comando `\spPval`

`\spPval` `\spPval{⟨casos⟩}{⟨total⟩}` `\spPval[⟨key = val⟩]{⟨casos⟩}{⟨total⟩}`

El comando `\spPval` trabaja en *modo matemático* y recibe *dos argumentos obligatorios*: {⟨casos⟩} y {⟨total⟩}, los tipografía como una fracción y devuelve el «*valor de una probabilidad*».

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`], `\spPval{1}{5}` imprimirá ‘ $\frac{1}{5}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*1 de 5*».

🔹 Los argumentos {⟨casos⟩} y {⟨total⟩} se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\spPval`

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización del *valor de la probabilidad* pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {⟨true | false⟩}` default: *false*

Establece si la fracción del *valor de la probabilidad* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar `\spPval[use-dfrac=true]{1}{5}` imprimirá la fracción en tamaño *display* ‘ $\frac{1}{5}$ ’ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

El comando `\spProb`

`\spProb` `\spProb{⟨evento⟩}` `\spProb[⟨key = val⟩]{⟨evento⟩}`

El comando `\spProb` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨evento⟩}: un evento, o dos eventos unidos por *exactamente uno* de los operadores `\cap`, `\cup`, `\mid` o el carácter ‘|’, escritos tal como se imprimen. El comando imprime la probabilidad entre paréntesis precedida de la letra ‘P’. Cualquier otro argumento (vacío, con llaves o paréntesis sin cerrar, con un evento vacío o con más de un operador a la vez) se rechaza con un error y no se imprime nada. Lo que va entre llaves, paréntesis o corchetes no se examina, de modo que `\spProb{A \cap (B \cup C)}` es válido.

`\spProb{A}` imprimirá ‘P(A)’ en la salida PDF y NVDA/MathCAT verbalizará «*probabilidad de A*»; `\spProb{A \cap B}` imprimirá ‘P(A ∩ B)’ y verbalizará «*probabilidad de A y B*»; `\spProb{A \cup B}` imprimirá ‘P(A ∪ B)’ y verbalizará «*probabilidad de A o B*»; `\spProb{A \mid B}` (o `\spProb{A | B}`) imprimirá ‘P(A | B)’ y verbalizará «*probabilidad de A dado B*».

Llaves para `\spProb`

`read-arg = {\langle verbalización NVDA \rangle}`

default: *no usado*

Sobrescribe lo que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`. Por ejemplo: `\spProb[read-arg={probabilidad del suceso}]{A}` imprimirá $P(A)$ en la salida PDF y NVDA/MathCAT verbalizará «probabilidad del suceso».

`silent = {\langle true | false \rangle}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de la probabilidad al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, sí verbalizará la salida del comando. El argumento inválido se rechaza siempre, también con `silent=true`.

- Dentro del argumento de `\spProb`, los operadores `\cap`, `\cup` y `\mid` que quedan entre llaves o paréntesis (por ejemplo en `\spProb{A \cap (B \cup C)}`) también se verbalizan como «y», «o» y «dado», respectivamente.

El comando `\spcomp`

`\spcomp {\langle evento \rangle}`

`\spcomp[\langle key = val \rangle]{\langle evento \rangle}`

El comando `\spcomp` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\langle evento \rangle}`: el evento del que se quiere el complemento, escrito tal como se imprime. El comando imprime el complemento con el exponente c y se puede usar solo o dentro del argumento de `\spProb`. Si el evento contiene una operación entre conjuntos (`\cap`, `\cup`, `\setminus` o `\mid`), se imprime entre paréntesis y se verbaliza entre paréntesis para no confundir el complemento de toda la operación con el de su primer evento.

`\spcomp{A}` imprimirá A^c en la salida PDF y NVDA/MathCAT verbalizará «complemento de A»; `\spProb{A \cap \spcomp{B}}` imprimirá $P(A \cap B^c)$ y verbalizará «probabilidad de A y complemento de B»; `\spcomp{A \cup B}` imprimirá $(A \cup B)^c$ y verbalizará «complemento de se abren paréntesis A o B se cierran paréntesis».

Llaves para `\spcomp`

`bar = {\langle true | false \rangle}`

default: *false*

Establece si el complemento se imprime con una *barra horizontal* sobre el evento en lugar del exponente c . Si se establece en *true*, `\spcomp[bar=true]{A}` imprimirá $\overline{A}$ en la salida PDF; la verbalización es la misma en ambos casos.

`paren = {\langle auto | true | false \rangle}`

default: *auto*

Establece si el evento se imprime entre paréntesis cuando se usa el exponente c . Con *auto* se agregan solo si el evento contiene `\cap`, `\cup`, `\setminus` o `\mid`; con *true* siempre, y con *false* nunca. Con `bar=true` la barra cubre el evento y no se agregan paréntesis. Como NVDA/MathCAT no verbaliza la barra, cuando el evento es compuesto y se usa `bar=true` el complemento se verbaliza «complemento de la expresión» (por ejemplo, `\spcomp[bar=true]{A \cup B}` se verbaliza «complemento de la expresión A o B»), para dejar claro que abarca todo el evento.

`read-arg = {\langle verbalización NVDA \rangle}`

default: *no usado*

Sobrescribe lo que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`. Por ejemplo: `\spcomp[read-arg={no ocurre A}]{A}` imprimirá A^c en la salida PDF y NVDA/MathCAT verbalizará «no ocurre A».

`silent = {\langle true | false \rangle}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del complemento al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, sí verbalizará la salida del comando.

Los comandos `\spcomb` y `\spfact`

`\spcomb {\langle n \rangle} {\langle k \rangle}`

`\spcomb[\langle key = val \rangle]{\langle n \rangle} {\langle k \rangle}`

El comando `\spcomb` trabaja en *modo matemático* y escribe los símbolos de conteo: combinaciones, variaciones, arreglos y permutaciones, con o sin repetición, y las permutaciones circulares. Recibe el *argumento obligatorio* `{\langle n \rangle}` (el total de elementos) y un *segundo argumento entre llaves* `{\langle k \rangle}` (los elementos que se toman). Según el símbolo, el segundo argumento puede ser *la lista de repeticiones* (permutaciones con repetición) o puede no escribirse (permutaciones sin repetición y circulares, que solo llevan `{\langle n \rangle}`). Los argumentos aceptan números o símbolos; si ambos son números enteros, el comando comprueba que $k \leq n$ (salvo cuando hay repetición) y que las repeticiones no sumen más de n .

`\spcomb{5}{2}` imprimirá $\binom{5}{2}$ en la salida PDF y NVDA/MathCAT verbalizará «5 sobre 2»; `\spcomb[type=variation]{5}{2}` imprimirá V_2^5 y verbalizará «variaciones de 5 en 2»; `\spcomb[type=permutation]{n}` imprimirá P_n y verbalizará «permutaciones de n».

- El segundo argumento no va entre corchetes: el comando mira si el siguiente símbolo abre un grupo `{ }` y, si es así, lo lee como segundo argumento. Por eso, para una permutación sin segundo argumento, no se debe escribir inmediatamente una llave después del comando (`\spcomb[type=permutation]{n}\{x\}` lo leería como `{\langle k \rangle}`).

Llaves para `\spcomb`

`type = {⟨combination | variation | arrangement | permutation⟩}` default: *combination*

Establece el tipo de símbolo: combinaciones (C o $\binom{n}{k}$), variaciones (V), arreglos (A) o permutaciones (P). Las variaciones y los arreglos se escriben distinto pero se verbalizan según su nombre («*variaciones*» y «*arreglos*»). Con `type=permutation` el segundo argumento es opcional: sin él se imprime P_n («*permutaciones de n* ») y con él P_k^n («*permutaciones de n en k* »).

`rep = {⟨true | false⟩}` default: *false*

Establece si los elementos pueden repetirse. Con `rep=true` se agrega una R a la letra y a la verbalización se agrega «*con repetición*»: `\spcomb[type=variation,rep=true]{5}{2}` imprimirá VR_2^5 y verbalizará «*variaciones con repetición de 5 en 2*». Con `type=permutation` y `rep=true` el segundo argumento es la *lista de repeticiones* separada por comas, con al menos dos elementos: `\spcomb[type=permutation,rep=true]{10}{3,3,4}` imprimirá $PR_{10}^{3,3,4}$ y verbalizará «*permutaciones con repetición de 10 con repeticiones 3, 3 y 4*».

`circ = {⟨true | false⟩}` default: *false*

Establece si se trata de una *permutación circular*. Solo es válida con `type=permutation` y `rep=false`, y entonces solo recibe $\{⟨n⟩\}$: `\spcomb[type=permutation,circ=true]{n}` imprimirá PC_n y verbalizará «*permutaciones circulares de n* ».

`style = {⟨auto | binom | letter⟩}` default: *auto*

Establece cómo se imprimen las combinaciones sin repetición. Con `auto` (o `binom`) se imprime el coeficiente binomial $\binom{n}{k}$ y con `letter` se imprime la letra: `\spcomb[style=letter]{5}{2}` imprimirá C_2^5 . El coeficiente binomial se verbaliza « *n sobre k* » y la letra «*combinaciones de n en k* ». El valor `binom` solo es válido para combinaciones sin repetición.

`read-arg = {⟨verbalización NVDA⟩}` default: *no usado*

Sobrescribe lo que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves $\{ \}$. Por ejemplo: `\spcomb[read-arg={cinco sobre dos}]{5}{2}` imprimirá $\binom{5}{2}$ en la salida PDF y NVDA/MathCAT verbalizará «*cinco sobre dos*».

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del símbolo al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando.

- Un argumento inválido se rechaza siempre, también con `silent=true`, con un mensaje de error y sin imprimir nada: argumentos vacíos, una lista de repeticiones con menos de dos elementos o con un elemento vacío, $k > n$ (sin repetición), repeticiones que suman más de n , segundo argumento ausente cuando el símbolo lo necesita o presente cuando no lo admite, y combinaciones de llaves que no existen (`circ` sin `type=permutation` o con `rep`, `style=binom` fuera de las combinaciones sin repetición).

`\spfact` `\spfact{⟨n⟩}` `\spfact[⟨key = val⟩]{⟨n⟩}`

El comando `\spfact` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{⟨n⟩\}$. Imprime el factorial de $\{⟨n⟩\}$ y NVDA/MathCAT lo verbaliza como « *n factorial*». Si $\{⟨n⟩\}$ es una expresión, los paréntesis se escriben en el argumento (`\spfact{(n-1)}` imprimirá $(n-1)!$). Tiene las llaves `silent` y `read-arg`, con el mismo significado que en `\spcomb`.

`\spfact{5}` imprimirá $5!$ en la salida PDF y NVDA/MathCAT verbalizará «*5 factorial*».

10 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The L^AT_EX Project. “The L^AT_EX₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The L^AT_EX Project. “The L^AT_EX_{2_ε} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The L^AT_EX Project. “L^AT_EX News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The L^AT_EX Project. “L^AT_EX for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – L^AT_EX kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The L^AT_EX Project. “latex-lab – L^AT_EX laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

11 Change history

v0.99 (ctan), 2026-10-10 – First public release.

12 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAcirc	31
\spAfig	26
\spFrec	39
\spOmega	39
\spPcirc	32
\spPerc	38
\spPfig	28
\spPoint	19
\spPoly	26
\spProb	40, 41
\spProm	37
\spProp	18
\spPval	40
\spVol	34
\spangle	23
\spang	24
\sparc	25
\spcase	39
\spcirc	30
\spcod	36
\spcomb	41, 42
\spcomp	41
\spcoord	19
\spcuartil	38
\spdate	11
\spdecil	38
\spdesv	37, 38
\spdiam	29
\spdiv	13
\spdom	36
\spdsym	12
\spfact	42
\spfuncomp	36
\spfun	35, 36
\spinterval	34
\spmQ	17
\spmangle	24
\spmarc	25
\spmax	38
\spmcd	15
\spmcm	16
\spmediana	37
\spmedia	37, 38
\spmin	38
\spmoda	37
\spmoney	6
\spmside	22
\spmsym	13
\spmul	14
\spnD	14
\spnM	15
\spnZ	17
\spnewread	11
\spnewunit	6
\spnum	3
\sprad	29
\sprango	38
\sprayo	21
\sprazon	18
\spread	12
\sprecta	20
\sprec	36
\spriq	38
\spshort	8
\spside	22
\spsiglo	10
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
\spvarz	37, 38
\spvec	20
K	
Keys for \spAcirc provide by spintent :	
concept	32
prefix	32
print-sym	32
read-arg	32
read-cmd	32
read-sty	32
read-sub	32
silent	32
sub-index	32
Keys for \spAfig provide by spintent :	
concept*	27
concept	27
prefix	27
print-sym	27
read-arg	27
read-cmd	27
read-sub	27
silent	27
Keys for \spangle provide by spintent :	
concept	23
prefix	23
read-cmd	23
recto	23
silent	24
Keys for \sparc provide by spintent :	
concept	25
prefix	25
read-cmd	25
silent	25
Keys for \spcase provide by spintent :	
read-arg	39
silent	39
Keys for \spcirc provide by spintent :	
concept	31
diameter	30
prefix	30
print-sym	30
radio-sty	31
read-arg	31

read-cmd	30	silent	39
read-sty	30	sub-index	39
read-sub	31	Keys for \spfuncomp provide by spintent :	
silent	31	silent	37
spmside	30, 31	Keys for \spfun provide by spintent :	
spside	30, 31	concept	36
sub-index	31	prefix	36
Keys for \spcomb provide by spintent :		read-cmd	36
circ	42	silent	36
read-arg	42	Keys for \spinterval provide by spintent :	
rep	42	concept	35
silent	42	prefix	35
style	42	read-arg	35
type	42	read-cmd	35
Keys for \spcomp provide by spintent :		silent	35
bar	41	size	35
paren	41	Keys for \spmangle provide by spintent :	
read-arg	41	concept	24
silent	41	prefix	24
Keys for \spcoord provide by spintent :		print-m	24
silent	19	read-cmd	24
Keys for \spcuartil provide by spintent :		recto	24
read-arg	38	silent	24
silent	38	Keys for \spmarc provide by spintent :	
Keys for \spdecil provide by spintent :		concept	25
read-arg	38	prefix	25
silent	38	read-cmd	25
Keys for \spdesv provide by spintent :		silent	25
read-arg	38	Keys for \spmcd provide by spintent :	
read-sub	38	concept	16
sigma	37	prefix	16
silent	38	read-cmd	16
sub-index	37	result	16
Keys for \spdiam provide by spintent :		silent	16
concept	29	upper-case	16
print-sym	29	Keys for \spmcm provide by spintent :	
read-arg	30	concept	16
read-cmd	29	prefix	16
read-sub	30	read-cmd	16
silent	30	result	16
sub-index	29	silent	17
upper-case	29	upper-case	16
Keys for \spddiv provide by spintent :		Keys for \spmediana provide by spintent :	
cifras-sep	14	read-arg	38
notation-sym	14	read-sub	38
read-parens	14	silent	38
read-period-sep	14	sub-index	37
read-sign	14	Keys for \spmedia provide by spintent :	
read-sym	14	read-arg	38
set-sym	14	read-sub	38
wrap-parens	14	silent	38
Keys for \spdom provide by spintent :		sub-index	37
prefix	36	Keys for \spmoda provide by spintent :	
silent	36	read-arg	38
Keys for \spdsym provide by spintent :		read-sub	38
prefix	13	silent	38
read-sym	13	sub-index	37
set-sym	13	Keys for \spmoney provide by spintent :	
suffix	13	cifras-sep	7
Keys for \spfact provide by spintent :		currency	7
read-arg	42	money-math	7
silent	42	sub-units	7
Keys for \spFrec provide by spintent :		Keys for \spmQ provide by spintent :	
read-arg	39	join-word	17
read-sub	39	read-parens	18

silent	18	Keys for \spPerc provide by spintent :	
use-dfrac	17	read-arg	38
use-spnZ	17	silent	38
wrap-parens	18	Keys for \spPfig provide by spintent :	
Keys for \spmside provide by spintent :		concept*	28
concept	23	concept	28
prefix	23	prefix	28
print-m	23	print-sym	29
read-arg	23	read-arg	28
read-cmd	22	read-cmd	28
read-sty	22	read-sub	28
silent	23	silent	29
Keys for \spmsym provide by spintent :		Keys for \spPoint provide by spintent :	
not-print	13	concept	19
prefix	13	prefix	19
read-sym	13	read-cmd	19
set-sym	13	silent	19
suffix	13	Keys for \spPoly provide by spintent :	
Keys for \spmul provide by spintent :		concept	26
cifras-sep	14	prefix	26
notation-sym	14	print-sym	26
read-parens	14	read-cmd	26
read-period-sep	14	silent	26
read-sign	14	Keys for \spProb provide by spintent :	
read-sym	14	read-arg	41
set-sym	14	silent	41
wrap-parens	14	Keys for \spProm provide by spintent :	
Keys for \spnD provide by spintent :		read-arg	37
concept	14	silent	37
prefix	14	Keys for \spProp provide by spintent :	
result-num	14	inline	18, 19
result	14	silent	18
silent	15	use-dfrac	18, 19
Keys for \spnM provide by spintent :		Keys for \spPval provide by spintent :	
concept	15	silent	40
prefix	15	use-dfrac	40
result-num	15	Keys for \sprad provide by spintent :	
result	15	concept	29
silent	15	read-arg	30
Keys for \spnum provide by spintent :		read-cmd	29
cifras-sep	3	read-sub	30
notation-sym	4	silent	30
read-period-sep	3	sub-index	29
read-sign	3	upper-case	29
Keys for \spnZ provide by spintent :		Keys for \sprayo provide by spintent :	
cifras-sep	17	concept	21
read-parens	17	prefix	21
read-sign	17	read-cmd	21
wrap-parens	17	read-sty	21
Keys for \sp0mega provide by spintent :		silent	21
card	40	Keys for \sprazon provide by spintent :	
read-arg	40	inline	18
silent	40	silent	18
sub-index	40	use-dfrac	18
Keys for \spPcirc provide by spintent :		Keys for \spread provide by spintent :	
concept	33	read-arg	12
prefix	33	silent	12
print-sym	33	Keys for \sprecta provide by spintent :	
read-arg	33	concept	21
read-cmd	33	prefix	21
read-sty	33	read-cmd	21
read-sub	33	silent	21
silent	33	Keys for \spshort provide by spintent :	
sub-index	33	read-arg	8
		small-caps	8

Keys for \spside provide by **spintent**:

concept

prefix

read-arg

read-cmd

read-sty

silent

22

22

22

22

22

22

Keys for \spsiglo provide by **spintent**:

print-era

11

Keys for \spunit provide by **spintent**:

cifras-sep

notation-sym

print-cdot

read-cdot

read-period-sep

read-slash

4

4

4

4

4

4

Keys for \spusep provide by **spintent**:

silent

size

12

12

Keys for \spvarz provide by **spintent**:

read-arg

read-sub

sigma

silent

sub-index

38

38

37

38

37

Keys for \spvec provide by **spintent**:

autofit

concept

prefix

read-cmd

20

20

20

20

silent

20

Keys for \spVol provide by **spintent**:

concept

read-arg

read-cmd

read-sub

silent

sub-index

34

34

34

34

34

34

M

Lua module provide by **spintent**:

spintent.lua

6-11

P

Packages :

babel-spanish

babel

documentmetadata-support

enumext

expl3

fontspec

fourier-otf

hyperref

l3keys

latex-lab-mathintent

libertinus-otf

luamml

scontents

sourcecodepro

tagpdf

unicode-math

2

2, 8

1

1

3

2

2

2

3

1

2

1

1

2

1, 8, 11

1

13 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

13.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

13.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

13.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-10-10} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

13.4 Some kernel variants

Non-standard kernel variants.

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

```

```

62 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
63 \cs_generate_variant:Nn \str_if_eq:nnF { V }
64 \cs_generate_variant:Nn \str_uppercase:n { V }
65 \cs_generate_variant:Nn \str_lowercase:n { V }
66 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

13.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr` used by the function `\__spintent_luafun_right_angle_sqr:` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

13.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

```
\__spintent_luafun_spunit_lookup_alias:V
\__spintent_luafun_spmoney_lookup_data:V
\__spintent_luafun_spmQ_scale_int:Vn
\__spintent_luafun_clean_split_and_set:V
```

Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:n` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:n` (used by `\spnum`).

```
76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }
```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

13.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

```
mathcal-fallback
right-angle-sqr-fallback
```

First, we'll define two error messages in case our fallback fails.

```
80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }
```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

```
\__spintent_mathcal:n
```

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```
94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }
```

(End of definition for `\__spintent_mathcal:n`.)

```
\__spintent_right_angle_sqr:
```

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```
107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }
```

(End of definition for `\__spintent_right_angle_sqr:`.)

13.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/
intent=` (`#2`). A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use `math-AF` mode, you can remove the line generated by `AFghost=true` using the Lua script `spitool`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use `math-AF` mode, you can remove the line generated by `AFghost=true` using the Lua script `spitool`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0 "0 "2063
127 }
```

Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

13.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn __spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     __spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `__spintent_collapse_hyphens:N`.)

`__spintent_sanitize_affix:Nn`
`__spintent_sanitize_affix:cn`

The function `__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, it converts the $\{\langle argument \rangle\}$ into a string (so every character has category code 12 and it no longer matters how an underscore ‘_’ was typed), trims the bounds, converts spaces ‘ ’ and underscores ‘_’ to hyphens ‘-’, collapses multiple hyphens, and safely strips leading/trailing hyphens. Because underscores are converted *before* collapsing, any run mixing hyphens and underscores is reduced to a single hyphen, and the stored value never starts or ends with ‘-’ or ‘_’: the command that uses the value is the one that places the leading ‘_’ of the intent name. For example, if you pass a target variable and raw text:

```
__spintent_sanitize_affix:Nn \__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string foo-bar-baz inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended. In the same way, `{ __foo__bar__ }` is stored as foo-bar. The only exception is the exact value ‘_-’ (once the bounds are trimmed), which is stored as a single hyphen ‘-’. After the command prepends its own leading ‘_’, this yields the *silent* literal ‘_-’ documented for `\spread`. Any other value made only of hyphens, underscores and spaces is stored as *empty*.

```

136 \cs_new_protected:Nn __spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Nc #1 { \tl_to_str:e {#2} }
142     \tl_trim_spaces:N #1
143     \str_if_eq:VnTF #1 { _- }
144     { \tl_set:Nn #1 { - } }
145     {
146       \tl_replace_all:Nnn #1 { ~ } { - }
147       \tl_replace_all:Nee #1 { \tl_to_str:n { _ } } { - }
148       __spintent_collapse_hyphens:N #1
149       \tl_put_left:Nn #1 { \q_mark }
150       \tl_put_right:Nn #1 { \q_stop }
151       \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
152       \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
153       \tl_remove_all:Nn #1 { \q_mark }
154       \tl_remove_all:Nn #1 { \q_stop }
155     }
156   }
157 }
158 \cs_generate_variant:Nn __spintent_sanitize_affix:Nn { cn }
```

(End of definition for `__spintent_sanitize_affix:Nn`.)

13.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

13.10.1 Internal variables and error messages for `\setspintent`

`\__spintent_setspintent_input_arg_cmd_tl` The variable `\__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```

159 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\langle argument \rangle$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

160 \msg_new:nnnn { spintent } { unknown-command-setup }
161 {
162   Cannot~configure~unknown~command~'\exp_not:c{#1}'~'\msg_line_context:.
163 }
164 {
165   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
166   command~'\exp_not:c{#1}'~is~not~defined.~Please~check~for~typos.
167 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

168 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
169 {
170   \cs_if_exist:cTF { #1 }
171   {
172     \keys_set:nn { spintent / #1 } { #2 }
173   }
174   {
175     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
176   }
177 }
178 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

179 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
180 {
181   \tl_if_single:nTF { #1 }
182   {
183     \token_if_cs:NTF #1
184     {
185       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
186     }
187     {
188       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
189     }
190   }
191   {
192     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
193   }
194 }
195 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent` Now we define the user command `\setspintent`.

```

196 \NewDocumentCommand \setspintent { m m }
197 {
198   \tl_if_blank:nTF { #1 }
199   {
200     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
201   }
202   {
203     \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
204     \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
205     \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
206   }
207 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

13.11 Handling unknown keys

```
\l__spintent_command_name_str
    unknown-choice
    cmd-key-unknown
    cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “*name*” of each command within the implementation along with the messages `unknown-choice` used by *(keys)* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *(key)* `unknown` in commands that use `[⟨key = val⟩]`.

```
208 \str_new:N \l__spintent_command_name_str
209 \msg_new:nnn { spintent } { unknown-choice }
210 {
211     Invalid~value~'#3'~for~key~'#1';~use~one~of:~#2~\msg_line_context:.
212 }
213 \msg_new:nnnn { spintent } { cmd-key-unknown }
214 {
215     The~key~'#1'~is~unknown~by~command~
216     \c_backslash_str \l__spintent_command_name_str \c_space_tl
217     and~is~being~ignored~\msg_line_context:.
218 }
219 {
220     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
221     does~not~have~a~key~called~'#1'.\\
222     Check~that~you~have~spelled~the~key~name~correctly.
223 }
224 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
225 {
226     The~key~'#1=#2'~is~unknown~by~command~
227     \c_backslash_str \l__spintent_command_name_str \c_space_tl
228     and~is~being~ignored~\msg_line_context:.
229 }
230 {
231     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
232     does~not~have~a~key~called~'#1'.\\
233     Check~that~you~have~spelled~the~key~name~correctly.
234 }
```

(End of definition for `\l__spintent_command_name_str` and others.)

```
\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n
```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the *(key)* `unknown`.

```
235 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
236 {
237     \tl_if_blank:nTF {#2}
238     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
239     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
240 }
241 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
242 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
243 {
244     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
245 }
```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

13.12 Definition of core base variables

```
\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V
```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{⟨argument⟩}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “*units*”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘`_str`’ or ‘`_tl`’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```
\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str
```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘`+`’ or ‘`-`’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “*integer part*”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘`,`’ or ‘`.`’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “*finite decimal part*” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “*infinite periodic decimal part*” that follow a semicolon ‘`;`’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

246 \tl_new:N \l__spintent_luaset_sign_tl
247 \tl_new:N \l__spintent_luaset_part_int_tl
248 \tl_new:N \l__spintent_luaset_dec_sep_tl
249 \tl_new:N \l__spintent_luaset_part_dec_tl
250 \tl_new:N \l__spintent_luaset_part_period_tl
251 \str_new:N \l__spintent_luaset_has_integer_str
252 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

`\l__spintent_luaset_only_part_int_str` The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\l__spintent_luaset_only_part_int_str` for the “integer part”, in `\l__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\l__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\l__spintent_luaset_format_part_int_str` for the “integer part” and `\l__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```

253 \str_new:N \l__spintent_luaset_only_part_int_str
254 \str_new:N \l__spintent_luaset_only_part_dec_str
255 \str_new:N \l__spintent_luaset_only_part_period_str
256 \str_new:N \l__spintent_luaset_format_part_int_str
257 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

`\l__spintent_luaset_millions_str` The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

258 \str_new:N \l__spintent_luaset_millions_str
259 \str_new:N \l__spintent_luaset_decimal_period_str
260 \str_new:N \l__spintent_luaset_not_decimal_period_str
261 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
262 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
263 \str_new:N \l__spintent_luaset_has_exponent_str
264 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

`\l__spintent_luaset_has_units_str` Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§13.14) and `\spunit` (§13.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

265 \str_new:N \l__spintent_luaset_has_units_str
266 \str_new:N \l__spintent_luaset_status_str
267 \tl_new:N \l__spintent_luaset_arg_above_tl
268 \tl_new:N \l__spintent_luaset_arg_below_tl
269 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

13.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

270 \msg_new:nnnn { spintent } { need-math-mode }
271 {
272   Math~mode~required~for~#1\msg_line_context:.
273 }
274 {
275   Use~inside~$.~$.~$~or~in~math~environment.
276 }
277 \msg_new:nnnn { spintent } { need-text-mode }
278 {
279   Text~mode~required~for~#1\msg_line_context:.
280 }
281 {
282   Use~outside~of~$.~$.~$~or~math~environment.
283 }
```

13.14 The command \spnum

The user command `\spnum` is first of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

13.14.1 Definition of keys and variables for \spnum

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “positivo” or “negativo” after reading the number; if set to `terse`, it will say “más” or “menos” before reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will store the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` stores the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```

284 \bool_new:N \l__spintent_spnum_read_terse_bool
285 \str_new:N \l__spintent_spnum_read_period_sep_str
286 \tl_new:N \l__spintent_spnum_notation_sym_tl
287 \str_new:N \l__spintent_spnum_intent_read_sign_str
288 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

```

cifras-sep
read-sign
read-period-sep
notation-sym

Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.

289 \keys_define:nn { spintent/spnum }
290 {
291   cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
292   cifras-sep .initial:n = true,
293   cifras-sep .value_required:n = true,
294   read-sign .choices:nn =
295     { terse , clear }
296   {
297     \int_case:nn { \l_keys_choice_int }
298     {
299       {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
300       {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
301     }
302   },
303   read-sign .initial:n = terse,
304   read-sign .value_required:n = true,
305   read-sign / unknown .code:n =
```

```

306     \msg_error:nneee { spintent } { unknown-choice }
307     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
308 read-period-sep          .choices:nn =
309     { coma , punto }
310     {
311         \int_case:nn { \l_keys_choice_int }
312         {
313             {1} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { coma } }
314             {2} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { punto } }
315         }
316     },
317 read-period-sep          .initial:n = coma,
318 read-period-sep          .value_required:n = true,
319 read-period-sep / unknown .code:n =
320     \msg_error:nneee { spintent } { unknown-choice }
321     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
322 notation-sym             .choices:nn =
323     { times , cdot }
324     {
325         \int_case:nn { \l_keys_choice_int }
326         {
327             {1} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \times } }
328             {2} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \cdot } }
329         }
330     },
331 notation-sym             .initial:n = cdot,
332 notation-sym             .value_required:n = true,
333 notation-sym / unknown   .code:n =
334     \msg_error:nneee { spintent } { unknown-choice }
335     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
336 unknown                  .code:n =
337     {
338         \str_set:Nn \l__spintent_command_name_str { spnum }
339         \__spintent_unknown_keys_cmd:n {#1}
340     },
341 }

```

(End of definition for `cifras-sep` and others.)

13.14.2 Internal functions for `\spnum`

```

\__spintent_spnun_render_part:n
\__spintent_spnun_render_int:
\__spintent_spnun_render_dec:

```

The function `\__spintent_spnun_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnun_render_int:` and `\__spintent_spnun_render_dec:` to define the *intent function* `:number` for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

342 \cs_new_protected:Nn \__spintent_spnun_render_part:n
343 {
344     \MathMLintent { \str_use:c { \l__spintent_luaset_only_part_#1_str } :number }
345     {
346         \bool_if:NTF \l__spintent_spnun_cifras_sep_bool
347         { \str_use:c { \l__spintent_luaset_format_part_#1_str } }
348         { \tl_use:c { \l__spintent_luaset_part_#1_tl } }
349     }
350 }
351 \cs_new_protected:Nn \__spintent_spnun_render_int:
352 {
353     \__spintent_spnun_render_part:n { int }
354 }
355 \cs_new_protected:Nn \__spintent_spnun_render_dec:
356 {
357     \__spintent_spnun_render_part:n { dec }
358 }

```

(End of definition for `\__spintent_spnun_render_part:n`, `\__spintent_spnun_render_int:`, and `\__spintent_spnun_render_dec:`.)

```

\__spintent_spnun_render_only_period:
\__spintent_spnun_print_period_bar:
\__spintent_spnun_render_period_sep:

```

The function `\__spintent_spnun_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* `:number` passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_spnun_print_period_bar:`.

```

359 \cs_new_protected:Nn \__spintent_spnun_render_only_period:
360 {
361     \MathMLintent { \l__spintent_luaset_only_part_period_str :number }

```

```

362     {
363         \l__spintent_luaset_part_period_tl
364     }
365 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

366 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
367 {
368     \MathMLintent { _periódico:postfix($dp) }
369     {
370         {
371             \overline
372             {
373                 \MathMLarg { dp } { \__spintent_snum_render_only_period: }
374             }
375         }
376     }
377 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

378 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
379 {
380     \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
381     {
382         \MathMLintent { _coma:prefix($pp) }
383         {
384             \mathord{,}
385             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
386         }
387     }
388     {
389         \MathMLintent { _punto:prefix($pp) }
390         {
391             \mathord{.}
392             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
393         }
394     }
395 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

396 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
397 {
398     \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
399     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
400     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
401     \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
402     {
403         \mathord { \l__spintent_luaset_dec_sep_tl }
404         \MathMLarg { dp }
405         {
406             \__spintent_inner_sep:
407             \__spintent_snum_render_dec:
408         }
409     }
410     % period part
411     \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
412     {
413         \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
414         {
415             \__spintent_inner_sep:
416             \__spintent_snum_print_period_bar:

```

```

417     }
418     {
419         \MathMLintent { _con:prefix($pnum) }
420         {
421             \__spintent_inner_sep:
422             \MathMLarg { pnum }
423             {
424                 \__spintent_snum_print_period_bar:
425             }
426         }
427     }
428 }
429 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

430 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
431 {
432     \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
433     {
434         \__spintent_snum_render_decimal_sep:
435     }
436 }
437 \cs_new_protected:Nn \__spintent_snum_print_part_period:
438 {
439     \tl_if_empty:NF \l__spintent_luaset_part_period_tl
440     {
441         \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
442         { \__spintent_snum_render_period_sep: }
443         {
444             \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
445             {
446                 \__spintent_snum_render_period_sep:
447             }
448         }
449     }
450 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

451 \cs_new_protected:Nn \__spintent_snum_print_integer:
452 {
453     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
454     { \__spintent_snum_render_int: }
455     {
456         \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
457         {
458             \bool_if:NTF \l__spintent_snum_cifras_sep_bool
459             { \l__spintent_luaset_format_part_int_str }
460             { \l__spintent_luaset_part_int_tl }
461             \MathMLarg { next }
462         }

```

```

463         \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
464         { \__spintent_spnum_print_part_period: }
465         { \__spintent_spnum_print_part_dec: }
466     }
467 }
468 }
469 }
470 \cs_new_protected:Nn \__spintent_spnum_print_num_start:
471 {
472     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
473     {
474         \__spintent_invisible_sep: % need
475         \__spintent_spnum_print_integer:
476     }
477     {
478         \bool_if:NTF \l__spintent_spnum_read_terse_bool
479         {
480             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
481             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _más:prefix($n)} }
482             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _menos:prefix($n)} }
483         }
484         {
485             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
486             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
487             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
488         }
489         \__spintent_invisible_sep: % need
490         \MathMLintent { \l__spintent_spnum_intent_read_sign_str }
491         {
492             \l__spintent_luaset_sign_tl
493             \MathMLarg { n } { \__spintent_spnum_print_integer: }
494         }
495     }
496 }
497 \cs_new_protected:Nn \__spintent_spnum_print_number_core:
498 {
499     \__spintent_spnum_print_num_start:
500     \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }
501     {
502         \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
503         10^ { \l__spintent_luaset_exponent_tl }
504     }
505 }

```

(End of definition for `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, and `\__spintent_spnum_print_number_core:`.)

13.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

506 \msg_new:nnnn { spintent } { spnum-has-units }
507 {
508     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(' #1')~\msg_line_context
509 }
510 {
511     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
512 }
513 \msg_new:nnnn { spintent } { spnum-no-digits }
514 {
515     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value~\msg_line_con
516 }
517 {
518     You~provided~' #1',~which~contains~no~digits.
519 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

13.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[key = val]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is

not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

520 \NewDocumentCommand \spnum { O{} m }
521 {
522   \mode_if_math:TF
523   {
524     \group_begin:
525       \keys_set:nn { spintent/spnum } { #1 }
526       \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
527       \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
528       { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
529       {
530         \tl_if_empty:NTF \__spintent_luaset_part_int_tl
531         { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
532         { \__spintent_spnum_print_number_core: }
533       }
534     \group_end:
535   }
536   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
537 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

13.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the `{\argument}` to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the `{\argument}`.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

13.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

538 \str_new:N \__spintent_spunit_luaset_canonical_str
539 \str_new:N \__spintent_spunit_luaset_lookup_status_str
540 \str_new:N \__spintent_spunit_luaset_read_str
541 \str_new:N \__spintent_spunit_luaset_is_compact_str
542 \str_new:N \__spintent_spunit_luaset_compact_exp_str
543 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
544 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

545 \tl_new:N \__spintent_spunit_exponent_tl
546 \tl_new:N \__spintent_spunit_unit_name_tl
547 \str_new:N \__spintent_spunit_read_slash_str
548 \seq_new:N \__spintent_spunit_exp_seq
549 \seq_new:N \__spintent_spunit_mult_seq
550 \bool_new:N \__spintent_spunit_is_mult_bool
551 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

13.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the `{\argument}`, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

552 \msg_new:nnnn { spintent } { spunit-invalid-unit }
553 {
554   The~expression~'~#1'~contains~an~unknown~or~malformed~unit~structure~\msg_line_context:.

```

```

555 }
556 {
557   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
558 }
559 \msg_new:nnnn { spintent } { spunit-no-units }
560 {
561   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1')~\msg_line_c
562 }
563 {
564   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
565 }
566 \msg_new:nnnn { spintent } { inline-numeric-denominator }
567 {
568   Illegal~numeric~coefficient~in~inline~denominator~'#1'~\msg_line_context:.
569 }
570 {
571   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
572   does~not~allow~numbers~after~the~slash~(/).
573 }

```

(End of definition for spunit-invalid-unit, spunit-no-units, and inline-numeric-denominator.)

13.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

574 \keys_define:nn { spintent/spunit }
575 {
576   print-cdot      .bool_set:N = \l_spintent_spunit_print_cdot_bool,
577   print-cdot      .initial:n  = false,
578   print-cdot      .value_required:n = true,
579   read-cdot       .bool_set:N = \l_spintent_spunit_read_cdot_bool,
580   read-cdot       .initial:n  = false,
581   read-cdot       .value_required:n = true,
582   read-slash      .choices:nn =
583     { por , partido-por }
584     {
585       \int_case:nn { \l_keys_choice_int }
586       {
587         {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { _por } }
588         {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { _partido-por } }
589       }
590     },
591   read-slash      .initial:n  = por,
592   read-slash      .value_required:n = true,
593   read-slash / unknown .code:n =
594     \msg_error:nnee { spintent } { unknown-choice }
595     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
596   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
597   cifras-sep      .value_required:n = true,
598   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
599   read-period-sep .value_required:n = true,
600   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
601   notation-sym    .value_required:n = true,
602   unknown         .code:n =
603     {
604       \str_set:Nn \l__spintent_command_name_str { spunit }
605       \__spintent_unknown_keys_cmd:n {#1}
606     },
607 }

```

(End of definition for read-cdot and others.)

13.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:`

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

608 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
609 {
610   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
611   {
612     { ℓ } { \ell }

```

```

613     }
614     { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
615 }

```

(End of definition for `\l__spintent_spunit_format_canonical:`)

`\l__spintent_spunit_process_base_exp:n`

The function `\l__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘#1’. It first looks up the unit in the dictionary; if not found, flags it as invalid.

Otherwise, unless this is the first unit in a product (`\l__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

616 \cs_new_protected:Nn \l__spintent_spunit_process_base_exp:n
617 {
618   \tl_clear:N \l__spintent_spunit_exponent_tl
619   \l__spintent_luafun_spunit_lookup_alias:n { #1 }
620   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
621   {
622     \str_set:Nn \l__spintent_luaset_status_str { invalid }
623   }
624   {
625     \bool_if:NF \l__spintent_spunit_is_mult_bool
626     {
627       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
628       {
629         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
630         { \MathMLint { _por } { \cdot } }
631         { \MathMLint { :silent } { \cdot } }
632       }
633       {
634         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
635         { \MathMLint { _por } { \, } }
636         { \, }
637       }
638     }
639   }
640 }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spintent_spunit_luafun_spunit_lookup_status_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

639   \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
640   {
641     {
642       \MathMLint { \l__spintent_spunit_luaset_read_str }
643       {
644         \l__spintent_spunit_format_canonical:
645       }
646     }
647     ^ { \l__spintent_spunit_luaset_compact_exp_str }
648   }
649   {
650     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
651     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
652     \seq_if_empty:NF \l__spintent_spunit_exp_seq
653     {
654       \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
655     }
656     \MathMLint { \l__spintent_spunit_luaset_read_str }
657     {
658       \l__spintent_spunit_format_canonical:
659     }
660     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
661     {
662       ^ { \l__spintent_spunit_exponent_tl }
663     }
664   }
665   \bool_set_false:N \l__spintent_spunit_is_mult_bool
666 }
667 }

```

(End of definition for `\l__spintent_spunit_process_base_exp:n`.)

`\l__spintent_spunit_process_mult:n`

`\l__spintent_spunit_process_mult:V`

The function `\l__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{ \langle argument \rangle \}$ at each “star symbol” and applies `\l__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

668 \cs_new_protected:Nn \l__spintent_spunit_process_mult:n

```

```

669 {
670   \bool_set_true:N \__spintent_spunit_is_mult_bool
671   \seq_set_split:Nnn \__spintent_spunit_mult_seq { * } { #1 }
672   \seq_map_function:NN \__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
673 }
674 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

675 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
676 {
677   \tl_if_empty:NF \__spintent_luaset_arg_above_tl
678   {
679     \__spintent_spunit_process_mult:V \__spintent_luaset_arg_above_tl
680     \tl_if_empty:NF \__spintent_luaset_arg_below_tl
681     {
682       \MathMLintent { \__spintent_spunit_read_slash_str } { / }
683       \__spintent_spunit_process_mult:V \__spintent_luaset_arg_below_tl
684     }
685   }
686 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{\langle argument \rangle}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. When the number is exactly 1, with no decimal part, the reading becomes the singular literal `_grado:unit`, `_minuto:unit` or `_segundo:unit`, since neither the fixed plural names nor `MathCAT` can see the number. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

687 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
688 {
689   \str_if_eq:VnTF \__spintent_luaset_has_units_str { false }
690   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
691   {
692     \str_if_eq:VnTF \__spintent_luaset_status_str { valid }
693     {
694       \str_if_eq:VnTF
695       \__spintent_luaset_denom_has_number_str { true }
696       {
697         \msg_error:nnn { spintent }
698         { inline-numeric-denominator } { #1 }
699       }
700       {
701         \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
702         \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
703         {
704           \tl_if_empty:NF \__spintent_luaset_part_int_tl
705           {
706             \__spintent_spnum_print_number_core:
707           }
708           \bool_lazy_and:nnT
709           { \str_if_eq_p:ee { \__spintent_luaset_part_int_tl } { 1 } }
710           { \tl_if_empty_p:N \__spintent_luaset_part_dec_tl }
711           {
712             \str_set:Ne \__spintent_spunit_luaset_read_str
713             {
714               \str_case:Vn \__spintent_spunit_luaset_canonical_str
715               {
716                 { ° } { _grado:unit }
717                 { ' } { _minuto:unit }
718                 { " } { _segundo:unit }
719               }
720             }
721           }
722         }
723       }
724     }
725   }

```

```

722         \MathMLintent { \l__spinttent_spunit_luaset_read_str }
723         {
724             \mathord
725             {
726                 \str_case:Vn \l__spinttent_spunit_luaset_canonical_str
727                 {
728                     { " } { \prime \! \prime }
729                     { ' } { \prime }
730                     { ° } { ° }
731                 }
732             }
733         }
734     }
735     {
736         \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
737         {
738             \__spinttent_spnum_print_number_core: \,
739         }
740         \__spinttent_print_spunit_hierarchy:
741     }
742 }
743 }
744 { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
745 }
746 }

```

(End of definition for `\__spinttent_spunit_safe_exec:n`.)

13.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`(key = val)`] argument in ‘#1’ and a mandatory `{(unit expression)}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{(argument)}` to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

747 \NewDocumentCommand \spunit { O{} m }
748 {
749     \mode_if_math:TF
750     {
751         \group_begin:
752         \keys_set:nn { spinttent/spunit } { #1 }
753         \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
754         \__spinttent_spunit_safe_exec:n { #2 }
755         \group_end:
756     }
757     { \msg_error:nnn { spinttent } { need-math-mode } { \spunit } }
758 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

13.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spinttent.lua` via `\__spinttent_luafun_define_custom_unit:nn` and `\__spinttent_luafun_define_spunit`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to T_EX using the `\l__spinttent_spunit_luaset_status_s` variable. If a collision is detected or the `{(argument)}` is invalid, the package throws the corresponding “error” message.

13.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

759 \msg_new:nnnn { spinttent } { spnewunit-duplicate }
760 {
761     The~unit~symbol~or~alias~'#1'~is~already~defined~\msg_line_context:.
762 }
763 {
764     You~cannot~redefine~with~\token_to_str:N \spnewunit.

```

```

765 }
766 \msg_new:nnnn { spintent } { spunitalias-duplicate }
767 {
768   The~alias~'#1'~is~already~defined~or~reserved~\msg_line_context:.
769 }
770 {
771   You~cannot~redefine~an~existing~unit~or~another~alias.
772 }
773 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
774 {
775   The~expression~'#2'~for~alias~'#1'~is~invalid~\msg_line_context:.
776 }
777 {
778   It~must~contain~only~defined~units,~exponents~and~one~'/' .
779 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

\spnewunit The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

780 \NewDocumentCommand \spnewunit { m m }
781 {
782   \mode_if_math:TF
783   {
784     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
785   }
786   {
787     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
788     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { duplicate }
789     {
790       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
791     }
792   }
793 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

\spunitalias The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

794 \NewDocumentCommand \spunitalias { m m }
795 {
796   \mode_if_math:TF
797   {
798     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
799   }
800   {
801     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
802     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { duplicate }
803     {
804       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
805     }
806     \str_if_eq:VnT \__spintent_spunit_luaset_status_str { invalid-expr }
807     {
808       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
809     }
810   }
811 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

13.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{(argument)}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

```

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “*currency symbol*”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

812 \str_new:N \l__spintent_spmoney_luaset_symbol_str
813 \str_new:N \l__spintent_spmoney_luaset_position_str
814 \str_new:N \l__spintent_spmoney_luaset_sing_str
815 \str_new:N \l__spintent_spmoney_luaset_plur_str
816 \str_new:N \l__spintent_spmoney_luaset_conde_str
817 \str_new:N \l__spintent_spmoney_luaset_subsing_str
818 \str_new:N \l__spintent_spmoney_luaset_subplur_str
819 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
820 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
821 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

```

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

822 \bool_new:N \l__spintent_spmoney_sub_units_bool
823 \str_new:N \l__spintent_spmoney_intent_money_str
824 \str_new:N \l__spintent_spmoney_intent_sign_str
825 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
826 \str_new:N \l__spintent_spmoney_res_main_voice_str
827 \int_new:N \l__spintent_spmoney_subnum_int
828 \str_new:N \l__spintent_spmoney_extracted_curr_str
829 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

13.17.1 Definition of error messages

```

currency-no-subunits
invalid-currency

```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database or is not written validly: an alias or symbol with any uppercase letter, or an ISO code that is neither all lowercase nor all uppercase.

```

830 \msg_new:nnnn { spintent } { currency-no-subunits }
831 {
832   The~currency~'#1'~does~not~support~subunits.~
833   The~option~'sub-units=true'~is~ignored.
834 }
835 {
836   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
837   do~not~have~fractional~subunits~registered~in~Lua~module.~
838 }
839 \msg_new:nnnn { spintent } { invalid-currency }
840 {
841   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney \c_space_tl \msg_line_context:.
842 }
843 {
844   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
845   the~Lua~module.
846 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

13.17.2 Auxiliary functions for `\spmoney`

```

\__spintent_spmoney_set_currency:n
\__spintent_spmoney_normalize_key:n
\__spintent_spmoney_normalize_key:V

```

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\l__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

847 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
848 {
849   \__spintent_spmoney_normalize_key:n {#1}
850   \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
851   {
852     \msg_error:nnn { spintent } { invalid-currency } {#1}

```

```

853     }
854     { \str_set:Nn \l__spintenc_spmoney_luaset_currency_arg_str {#1} }
855   }
856   \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
857   {
858     \__spintenc_luafun_spmoney_normalize_key:n { \exp_not:n { #1 } }
859   }
860   \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintenc_spmoney_set_currency:n` and `\__spintenc_spmoney_normalize_key:n`.)

13.17.3 Definition of keys for `\spmoney`

Now we add the keys `currency`, `sub-units`, `money-math` and `cifras-sep`.

```

currency
sub-units
money-math
cifras-sep
861 \keys_define:nn { spintenc / spmoney }
862 {
863   currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
864   currency      .initial:n    = { pesos },
865   currency      .value_required:n = true,
866   sub-units     .bool_set:N   = \l__spintenc_spmoney_sub_units_bool,
867   sub-units     .value_required:n = true,
868   money-math    .bool_set:N   = \l__spintenc_spmoney_money_math_bool,
869   money-math    .initial:n    = { true },
870   money-math    .value_required:n = true,
871   cifras-sep    .meta:nn      = { spintenc/spnum } { cifras-sep = {#1} },
872   cifras-sep    .value_required:n = true,
873   unknown       .code:n      =
874   {
875     \str_set:Nn \l__spintenc_command_name_str { spmoney }
876     \__spintenc_unknown_keys_cmd:n {#1}
877   },
878 }

```

(End of definition for `currency` and others.)

13.17.4 Internal functions for `\spmoney`

`\__spintenc_spmoney_validate_money:` The function `\__spintenc_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\l__spintenc_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintenc_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintenc_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintenc.lua` via `\__spintenc_luafun_spmoney_lookup_data:V`.

```

879 \cs_new_protected:Nn \__spintenc_spmoney_validate_money:
880 {
881   \str_if_eq:VnT \l__spintenc_luaset_has_units_str { true }
882   {
883     \__spintenc_spmoney_normalize_key:V \l__spintenc_luaset_arg_above_tl
884     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_status_str { notfound }
885     { \msg_error:nne { spintenc } { invalid-currency } { \l__spintenc_luaset_arg_above_tl } }
886     {
887       \str_set:NV
888         \l__spintenc_spmoney_luaset_currency_arg_str
889         \l__spintenc_luaset_arg_above_tl
890     }
891   }
892   \str_set:Ne \l__spintenc_spmoney_luaset_status_str { found }
893   \__spintenc_luafun_spmoney_lookup_data:V \l__spintenc_spmoney_luaset_currency_arg_str
894 }

```

(End of definition for `\__spintenc_spmoney_validate_money:`.)

`\__spintenc_spmoney_print_symbol:` The function `\__spintenc_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintenc_spmoney_luaset_currency_arg_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”, directly in math mode when `\l__spintenc_spmoney_money_math_bool` is `true`, or wrapped inside `\text` when it is `false`. For dollar signs ‘\$’, the symbol is typeset through the control sequence `\$` in both cases.

```

895 \cs_new_protected:Nn \__spintenc_spmoney_print_symbol:
896 {
897   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_print_iso_str { true }
898   { \mathrm { \l__spintenc_spmoney_luaset_currency_arg_str } }
899   {
900     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_symbol_str { $ }
901     {
902       \bool_if:NTF \l__spintenc_spmoney_money_math_bool

```

```

903         { \ $ } { \text { \ $ } }
904     }
905     {
906         \bool_if:NTF \__spintent_spmoney_money_math_bool
907         { \__spintent_spmoney_luaset_symbol_str }
908         { \text { \__spintent_spmoney_luaset_symbol_str } }
909     }
910 }
911 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

912 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
913 {
914     \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
915     {
916         \__spintent_spmoney_print_symbol:
917         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
918     }
919     {
920         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
921         \__spintent_spmoney_print_symbol:
922     }
923 }

```

(End of definition for `\__spintent_spmoney_render_layout:`)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

924 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
925 {
926     \__spintent_spmoney_render_int:
927     \__spintent_spmoney_print_part_dec:
928 }
929 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
930 {
931     \__spintent_spmoney_render_int:
932 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘*más*’ or ‘*menos*’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

933 \cs_new_protected:Nn \__spintent_spmoney_print_int:
934 {
935     \tl_if_empty:NTF \__spintent_luaset_sign_tl
936     {
937         \bool_if:NTF \__spintent_spmoney_sub_units_bool
938         { \__spintent_spmoney_set_and_print_sub_units: }
939         { \__spintent_spmoney_intent_int: }
940     }
941     {
942         \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
943         {
944             \str_set:Nn \__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
945         }
946         {
947             \str_set:Nn \__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
948         }
949         \__spintent_invisible_sep:
950         \MathMLintent { \__spintent_spmoney_intent_sign_str }
951         {
952             \__spintent_luaset_sign_tl
953             \MathMLarg { n }

```

```

954         {
955             \bool_if:NTF \l__spintent_spmoney_sub_units_bool
956             { \__spintent_spmoney_set_and_print_sub_units: }
957             { \__spintent_spmoney_intent_int: }
958         }
959     }
960 }
961 }

```

(End of definition for `\__spintent_spmoney_print_int:`)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

962 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
963 {
964     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
965     {
966         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
967         {
968             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
969         }
970         {
971             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
972         }
973         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
974         {
975             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
976         }
977     }
978     {
979         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
980         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
981         {
982             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
983             {
984                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
985             }
986             {
987                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
988             }
989         }
990     }
991     \__spintent_invisible_sep:
992     \MathMLintent { _ \l__spintent_spmoney_intent_money_str :postfix($n) }
993     {
994         \__spintent_spmoney_render_layout:
995     }
996 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag. The flag is also disabled, without a warning, when the amount has no decimal part, since there are no “subunits” to voice.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

997 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
998 {
999     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
1000     {
1001         \msg_warning:nne { spintent } { currency-no-subunits }
1002         { \l__spintent_spmoney_luaset_currency_arg_str }
1003         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
1004     }
1005     \tl_if_empty:NT \l__spintent_luaset_dec_sep_tl
1006     { \bool_set_false:N \l__spintent_spmoney_sub_units_bool }

```

```

1007     }
1008     \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
1009     {
1010         \int_compare:nTF { \str_count:N \__spintent_luaset_only_part_dec_str == 1 }
1011         {
1012             \int_set:Nn \__spintent_spmoney_subnum_int
1013             { \__spintent_luaset_only_part_dec_str * 10 }
1014             \str_set:Ne \__spintent_luaset_only_part_dec_str
1015             { \__spintent_luaset_only_part_dec_str 0 }
1016         }
1017         {
1018             \int_set:Nn \__spintent_spmoney_subnum_int
1019             { \__spintent_luaset_only_part_dec_str }
1020         }
1021         \int_compare:nTF { \__spintent_spmoney_subnum_int == 1 }
1022         {
1023             \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
1024             { _ \__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1025         }
1026         {
1027             \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
1028             { _ \__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1029         }
1030         \str_if_eq:VnTF \__spintent_luaset_only_part_int_str { 1 }
1031         {
1032             \str_set:NV \__spintent_spmoney_res_main_voice_str
1033             \__spintent_spmoney_luaset_sing_str
1034         }
1035         {
1036             \str_set:NV \__spintent_spmoney_res_main_voice_str
1037             \__spintent_spmoney_luaset_plur_str
1038         }
1039         \str_if_eq:VnT \__spintent_luaset_millions_str { true }
1040         {
1041             \str_set:NV \__spintent_spmoney_res_main_voice_str
1042             \__spintent_spmoney_luaset_conde_str
1043         }
1044         \__spintent_spmoney_build_tree:
1045     }

```

(End of definition for __spintent_spmoney_check_sub_units: and __spintent_spmoney_set_and_print_sub_units:.)

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “subunit” component based on the layout position (pre or post) of the “currency symbol”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘_con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “subunit” intent to the decimal digits, ensuring accurate accessibility output.

```

1046 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1047 {
1048     \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
1049     {
1050         \MathMLintent { _ \__spintent_spmoney_res_main_voice_str :postfix($n) }
1051         {
1052             \__spintent_spmoney_print_symbol:
1053             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1054         }
1055         \__spintent_spmoney_print_sep_con:
1056         \__spintent_spmoney_print_dec_sub_unit:
1057     }
1058     {
1059         \MathMLintent { _ \__spintent_spmoney_res_main_voice_str :postfix($n) }
1060         {
1061             \__spintent_inner_sep:
1062             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1063         }
1064         \__spintent_spmoney_print_sep_con:
1065         \__spintent_spmoney_print_dec_sub_unit:
1066         \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1067     }
1068 }

```

```

1069 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1070 {
1071   \MathMLintent { _con }
1072   {
1073     \mathord { \l__spintent_luaset_dec_sep_tl }
1074   }
1075 }
1076 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1077 {
1078   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1079   {
1080     \__spintent_inner_sep:
1081     \MathMLarg { sub }
1082     {
1083       \__spintent_spmoney_render_dec:
1084     }
1085   }
1086 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{(argument)}` passed in ‘`#1`’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1087 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1088 {
1089   \__spintent_luafun_spmoney_prepare_input:n { \exp_not:n { #1 } }
1090   \__spintent_luafun_clean_split_and_set:n { \exp_not:V \l__spintent_spmoney_extracted_num_tl }
1091   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1092   {
1093     \str_set:Nx \l__spintent_luaset_has_units_str { true }
1094     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1095   }
1096   \__spintent_spmoney_validate_money:
1097   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1098   { \__spintent_spmoney_check_sub_units: }
1099   \__spintent_spmoney_print_int:
1100 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘`#1`’ for local key-value configurations and a mandatory argument ‘`#2`’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1101 \NewDocumentCommand \spmoney { 0{} m }
1102 {
1103   \mode_if_math:TF
1104   {
1105     \group_begin:
1106     \keys_set:nn { spintent / spmoney } { #1 }
1107     \__spintent_spmoney_parse_and_route:n { #2 }
1108     \group_end:
1109   }
1110   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1111 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

13.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on `MathML` attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that `NVDA` interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

```

\__spintent_spshort_luaset_status_str
\__spintent_spshort_luaset_layout_str
\__spintent_spshort_luaset_expand_str
\__spintent_spshort_luaset_output_str
\__spintent_spshort_luaset_base_str
\__spintent_spshort_luaset_suffix_str

```

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1112 \str_new:N \__spintent_spshort_luaset_status_str
1113 \str_new:N \__spintent_spshort_luaset_layout_str
1114 \str_new:N \__spintent_spshort_luaset_expand_str
1115 \str_new:N \__spintent_spshort_luaset_output_str
1116 \str_new:N \__spintent_spshort_luaset_base_str
1117 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

13.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1118 \msg_new:nnnn { spintent } { unknown-abbreviation }
1119 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized~\msg_line_context:. }
1120 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

13.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1121 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1122 {
1123   \tag_mc_end_push:
1124   \tag_struct_begin:n { tag=Span, E={ #1 } }
1125   \tag_mc_begin:n { tag=Span }
1126   \tag_suspend:n { \spshort }
1127   #2
1128   \tag_resume:n { \spshort }
1129   \tag_mc_end:
1130   \tag_struct_end:
1131   \tag_mc_begin_pop:n {}
1132 }
1133 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

13.18.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1134 \keys_define:nn { spintent / spshort }
1135 {
1136   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1137   small-caps .initial:n = false,
1138   small-caps .value_required:n = true,
1139   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1140   read-arg .initial:n = {},
1141   read-arg .value_required:n = true,
1142   unknown .code:n =
1143   {
1144     \str_set:Nn \__spintent_command_name_str { spshort }
1145     \__spintent_unknown_keys_cmd:n {#1}
1146   },
1147 }

```

(End of definition for `small-caps` and `read-arg`.)

13.18.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1148 \cs_new_protected:Nn \__spintent_spshort_print:nn

```

```

1149 {
1150   \str_clear:N \l__spintent_spshort_read_arg_str
1151   \keys_set:nn { spintent / spshort } {#1}
1152   \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1153   {
1154     \__spintent_spshort_process_and_render:n {#2}
1155   }
1156   {
1157     \__spintent_spshort_render_bypass:n {#2}
1158   }
1159 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1160 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1161 {
1162   \__spintent_luafun_spshort_process:n {#1}
1163   \str_case:VnF \l__spintent_spshort_luaset_status_str
1164   {
1165     { success } { \__spintent_spshort_render_success: }
1166     { fallback } { \__spintent_spshort_render_fallback: }
1167     { error } { \__spintent_spshort_render_error:n {#1} }
1168   }
1169   { \__spintent_spshort_render_error:n {#1} }
1170 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as SMALL CAPS to unrecognized or partially resolved terms.

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

`\__spintent_spshort_render_error:n`

```

1171 \cs_new_protected:Nn \__spintent_spshort_render_success:
1172 {
1173   \__spintent_spshort_tag_span:en
1174   { \l__spintent_spshort_luaset_expand_str }
1175   { \__spintent_spshort_execute_layout: }
1176 }
1177 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1178 {
1179   \__spintent_spshort_tag_span:en
1180   { \l__spintent_spshort_luaset_expand_str }
1181   {
1182     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1183     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1184     { \l__spintent_spshort_luaset_output_str }
1185   }
1186 }
1187 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1188 {
1189   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1190 }
1191 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1192 {
1193   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1194 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\LaTeX$

formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1195 \cs_new_protected:Nn \__spintenc_spshort_execute_layout:
1196 {
1197   \str_case:VnF \l__spintenc_spshort_luaset_layout_str
1198   {
1199     { linear_regular }
1200     {
1201       \bool_if:NTF \l__spintenc_spshort_smallcaps_bool
1202       { \textsc{ \l__spintenc_spshort_luaset_output_str } }
1203       { \l__spintenc_spshort_luaset_output_str }
1204     }
1205     { linear_caps }
1206     { \l__spintenc_spshort_luaset_output_str }
1207     { linear_mixed }
1208     { \l__spintenc_spshort_luaset_output_str }
1209     { superscript }
1210     {
1211       \l__spintenc_spshort_luaset_base_str
1212       \bool_if:NTF \l__spintenc_spshort_smallcaps_bool
1213       {
1214         \textsuperscript{
1215           \textsc{
1216             \str_lowercase:V \l__spintenc_spshort_luaset_suffix_str
1217           }
1218         }
1219       }
1220       {
1221         \textsuperscript{ \l__spintenc_spshort_luaset_suffix_str }
1222       }
1223     }
1224     { small_caps_pure }
1225     {
1226       \bool_if:NTF \l__spintenc_spshort_smallcaps_bool
1227       {
1228         \textsc{
1229           \str_lowercase:V \l__spintenc_spshort_luaset_output_str
1230         }
1231       }
1232       { \l__spintenc_spshort_luaset_output_str }
1233     }
1234     { acronym_long }
1235     { \l__spintenc_spshort_luaset_output_str }
1236   }
1237   { \l__spintenc_spshort_luaset_output_str }
1238 }

```

(End of definition for __spintenc_spshort_execute_layout:.)

\spshort The public document command **\spshort** manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts **\mode_leave_vertical:** and establishes a local group to call the function **__spintenc_spshort_print:nn** for processing and printing.

```

1239 \NewDocumentCommand \spshort { 0{} m }
1240 {
1241   \mode_if_math:TF
1242   {
1243     \msg_error:nnn { spintenc } { need-text-mode } { \spshort }
1244   }
1245   {
1246     \mode_leave_vertical:
1247     \group_begin:
1248       \__spintenc_spshort_print:nn {#1} {#2}
1249     \group_end:
1250   }
1251 }

```

(End of definition for \spshort. This function is documented on page 8.)

13.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible `MathML` intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```
\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str
```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```
1252 \str_new:N \l__spintent_spsiglo_luaset_error_str
1253 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1254 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1255 \str_new:N \l__spintent_spsiglo_print_era_str
```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

13.19.1 Definition of error messages for `\spsiglo`

```
invalid-century-format
```

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of **1** to **4000**, preventing incorrect speech synthesis.

```
1256 \msg_new:nnnn { spintent } { invalid-century-format }
1257 {
1258   Invalid~argument~'#2'~for~#1\msg_line_context:.
1259 }
1260 {
1261   The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1262   Ensure~the~value~is~between~1~and~4000.
1263 }
```

(End of definition for `invalid-century-format`.)

13.19.2 Definition of keys for `\spsiglo`

```
print-era
```

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```
1264 \keys_define:nn { spintent / spsiglo }
1265 {
1266   print-era .choices:nn =
1267     { none , ac , dc }
1268   {
1269     \int_case:nn { \l_keys_choice_int }
1270     {
1271       {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1272       {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1273       {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1274     }
1275   },
1276   print-era .initial:n = none,
1277   print-era .value_required:n = true,
1278   print-era / unknown .code:n =
1279     \msg_error:nnee { spintent } { unknown-choice }
1280     { print-era } { none , ~ac , ~dc } { \exp_not:n {#1} },
1281   unknown .code:n =
1282     {
1283       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1284       \__spintent_unknown_keys_cmd:n {#1}
1285     },
1286 }
```

(End of definition for `print-era`.)

13.19.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers *ac* or *dc*. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1287 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1288 {
1289   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1290 }
1291 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1292 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1293 {
1294   \str_if_eq:nnTF { #1 } { ac }
1295   {
1296     \MathMLintent { _antes-de-cristo:postfix($z) }
1297     {
1298       \__spintent_inner_sep:
1299       \MathMLarg { z }
1300       {
1301         \MathMLintent { #2:number }
1302         {
1303           \text { \textsc {#3}~a.~C. }
1304         }
1305       }
1306     }
1307   }
1308   {
1309     \MathMLintent { _después-de-cristo:postfix($z) }
1310     {
1311       \__spintent_inner_sep:
1312       \MathMLarg { z }
1313       {
1314         \MathMLintent { #2:number }
1315         {
1316           \text { \textsc {#3}~d.~C. }
1317         }
1318       }
1319     }
1320   }
1321 }
1322 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1323 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1324 {
1325   \tag_mc_end_push:
1326   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1327   \tag_mc_begin:n { tag=Span }
1328   \tag_suspend:n { \spsiglo }
1329   #2
1330   \tag_resume:n { \spsiglo }
1331   \tag_mc_end:
1332   \tag_struct_end:
1333   \tag_mc_begin_pop:n {}
1334 }
1335 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1336 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1337 {
1338   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1339   {
1340     \msg_error:nnnn { spintent } { invalid-century-format }
1341     { \spsiglo } { #1 }
1342   }
1343   {
1344     \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1345     {
1346       \__spintent_spsiglo_render_simple:VV
1347       \l__spintent_spsiglo_luaset_arabic_str
1348       \l__spintent_spsiglo_luaset_roman_min_str
1349     }
1350     {
1351       \__spintent_spsiglo_render_with_era:VVV
1352       \l__spintent_spsiglo_print_era_str
1353       \l__spintent_spsiglo_luaset_arabic_str
1354       \l__spintent_spsiglo_luaset_roman_min_str
1355     }
1356   }
1357 }
1358 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1359 {
1360   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1361   {
1362     \msg_error:nnnn { spintent } { invalid-century-format }
1363     { \spsiglo } { #1 }
1364   }
1365   {
1366     \str_case:Vn \l__spintent_spsiglo_print_era_str
1367     {
1368       { none }
1369       {
1370         \__spintent_spsiglo_tag_span:en
1371         { \l__spintent_spsiglo_luaset_arabic_str }
1372         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1373       }
1374       { ac }
1375       {
1376         \__spintent_spsiglo_tag_span:en
1377         { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1378         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1379       }
1380       { dc }
1381       {
1382         \__spintent_spsiglo_tag_span:en
1383         { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1384         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1385       }
1386     }
1387   }
1388 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1389 \NewDocumentCommand \spsiglo { 0{} m }
1390 {
1391   \group_begin:
1392   \keys_set:nn { spintent / spsiglo } { #1 }
1393   \__spintent_luafun_spsiglo_parse:n { #2 }
1394   \mode_if_math:TF
1395   { \__spintent_spsiglo_print_math:n { #2 } }
1396   {

```

```

1397         \mode_leave_vertical:
1398         \__spintent_spsiglo_print_text:n { #2 }
1399     }
1400     \group_end:
1401 }

```

(End of definition for \spsiglo. This function is documented on page 10.)

13.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\l__spintent_sptime_luaset_seconds_str
\l__spintent_sptime_luaset_has_seconds_str
\l__spintent_sptime_luaset_error_str
\l__spintent_sptime_luaset_is_fraction_str
\l__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1402 \str_new:N \l__spintent_sptime_luaset_seconds_str
1403 \str_new:N \l__spintent_sptime_luaset_has_seconds_str
1404 \str_new:N \l__spintent_sptime_luaset_error_str
1405 \str_new:N \l__spintent_sptime_luaset_is_fraction_str
1406 \str_new:N \l__spintent_sptime_luaset_base_str

```

(End of definition for \l__spintent_sptime_luaset_seconds_str and others.)

13.20.1 Definition of error messages

```
invalid-time-format
```

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1407 \msg_new:nnnn { spintent } { invalid-time-format }
1408 {
1409     Invalid~argument~'#2'~for~#1\msg_line_context:. \\\
1410     It~does~not~match~any~supported~time~format.
1411 }
1412 {
1413     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1414     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1415 }

```

(End of definition for `invalid-time-format`.)

13.20.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1416 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1417 {
1418     \MathMLintent { #1:prefix($z) }
1419     {
1420         \text { : }
1421         \MathMLarg { z }
1422         {
1423             \MathMLintent { _segundos:postfix($m) }
1424             {
1425                 \__spintent_inner_sep:
1426                 \MathMLarg { m }
1427                 {
1428                     \text { #2 }
1429                 }
1430             }
1431         }
1432     }
1433 }

```

(End of definition for `\__spintent_sptime_build_seconds:nn`.)

```
\__spintent_sptime_build_tree:
```

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```

1434 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1435 {
1436   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1437   {
1438     \MathMLintent { :time }
1439     {
1440       \text { \l__spintent_sptime_luaset_base_str }
1441     }
1442     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1443     {
1444       \__spintent_sptime_build_seconds:nn { con }
1445       { \l__spintent_sptime_luaset_seconds_str }
1446     }
1447     {
1448       \__spintent_sptime_build_seconds:nn { _minutos-con }
1449       { \l__spintent_sptime_luaset_seconds_str }
1450     }
1451   }
1452   {
1453     \MathMLintent { :time }
1454     {
1455       \text { \l__spintent_sptime_luaset_base_str }
1456     }
1457     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1458     {
1459       \MathMLintent { _minutos }
1460       {
1461         \c__spintent_invisible_sep_int
1462       }
1463     }
1464   }
1465 }

```

(End of definition for __spintent_sptime_build_tree:.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1466 \NewDocumentCommand \sptime { m }
1467 {
1468   \mode_if_math:TF
1469   {
1470     \group_begin:
1471     \__spintent_luafun_sptime_parse:n { #1 }
1472     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1473     {
1474       \msg_error:nnnn { spintent } { invalid-time-format }
1475       { \sptime } { #1 }
1476     }
1477     {
1478       \__spintent_sptime_build_tree:
1479     }
1480     \group_end:
1481   }
1482   { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1483 }

```

(End of definition for \sptime. This function is documented on page 11.)

13.21 The command \spdate

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1484 \str_new:N \l__spintent_spdate_luaset_output_str

```

```
1485 \str_new:N \__spintent_spdate_luaset_error_str
```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

13.21.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1486 \msg_new:nnnn { spintent } { invalid-date-format }
1487 {
1488   Invalid~argument~'#2'~for~#1\msg_line_context:. \\\
1489   It~does~not~match~a~supported~date~format.
1490 }
1491 {
1492   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1493   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1494 }
```

(End of definition for `invalid-date-format`.)

13.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```
1495 \cs_new_protected:Nn \__spintent_spdate_process:n
1496 {
1497   \__spintent_luafun_spdate_parse:n { #1 }
1498   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1499   {
1500     \msg_error:nnnn { spintent } { invalid-date-format }
1501     { \spdate } { #1 }
1502   }
1503   {
1504     \MathMLintent { :date }
1505     {
1506       \text { \__spintent_spdate_luaset_output_str }
1507     }
1508   }
1509 }
```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1510 \NewDocumentCommand \spdate { m }
1511 {
1512   \mode_if_math:TF
1513   {
1514     \group_begin:
1515     \__spintent_spdate_process:n { #1 }
1516     \group_end:
1517   }
1518   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1519 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

13.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

13.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *⟨keys⟩* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *⟨argument⟩*, the second this is when an input is invalid.

```

1520 \msg_new:nnnn { spintent } { spusep-empty-arg }
1521 {
1522   Empty~argument~'#1'~for~\token_to_str:N \spusep \c_space_tl \msg_line_context:.
1523 }
1524 {
1525   \token_to_str:N \spusep \c_space_tl requires~a~non-empty~argument.
1526 }
1527 \msg_new:nnn { spintent } { spusep-invalid-arg }
1528 {
1529   Invalid~argument~'#1'~for~\token_to_str:N \spusep \c_space_tl \msg_line_context:.
1530 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\l__spintent_spusep_arg_tl`
`\l__spintent_spusep_left_tl`
`\l__spintent_spusep_right_tl`

The variable `\l__spintent_spusep_arg_tl` stores the *⟨argument⟩*, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1531 \tl_new:N \l__spintent_spusep_arg_tl
1532 \tl_new:N \l__spintent_spusep_left_tl
1533 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the *⟨keys⟩* `size` and `silent`.

```

1534 \keys_define:nn { spintent / spusep }
1535 {
1536   size .choices:nn =
1537     { none , big , Big , bigg , Bigg }
1538     {
1539       \int_case:nn { \l_keys_choice_int }
1540       {
1541         { 1 }
1542         {
1543           \tl_clear:N \l__spintent_spusep_left_tl
1544           \tl_clear:N \l__spintent_spusep_right_tl
1545         }
1546         { 2 }
1547         {
1548           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1549           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1550         }
1551         { 3 }
1552         {
1553           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1554           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1555         }
1556         { 4 }
1557         {
1558           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1559           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1560         }
1561         { 5 }
1562         {
1563           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1564           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1565         }
1566       }
1567     },
1568   size .initial:n = none,
1569   size .value_required:n = true,
1570   size / unknown .code:n =
1571     \msg_error:nneee { spintent } { unknown-choice }
1572     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1573   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1574   silent .initial:n = false,

```

```

1575     silent           .value_required:n = true,
1576     unknown          .code:n =
1577     {
1578         \str_set:Nn \__spintent_command_name_str { spusep }
1579         \__spintent_unknown_keys_cmd:n {#1}
1580     },
1581 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` and then check the state of the variable `\__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1582 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1583 {
1584     \tl_put_right:cn { \__spintent_spusep_ #1 _tl } { #2 }
1585     \bool_if:NTF \__spintent_spusep_silent_bool
1586     {
1587         \MathMLintenc { :silent } { \tl_use:c { \__spintent_spusep_ #1 _tl } }
1588     }
1589     {
1590         \tl_use:c { \__spintent_spusep_ #1 _tl }
1591     }
1592 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1593 \cs_new_protected:Nn \__spintent_spusep_convert:n
1594 {
1595     \tl_set:Ne \__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1596     \tl_if_empty:NT \__spintent_spusep_arg_tl
1597     {
1598         \msg_error:nnn { spintenc } { spusep-empty-arg } {#1}
1599     }
1600     \str_case:VnF \__spintent_spusep_arg_tl
1601     {
1602         { ( }
1603         {
1604             \__spintent_spusep_set_parens:nn { left } { \lparen }
1605         }
1606         { ) }
1607         {
1608             \__spintent_spusep_set_parens:nn { right } { \rparen }
1609         }
1610         { [ }
1611         {
1612             \__spintent_spusep_set_parens:nn { left } { \lbrack }
1613         }
1614         { ] }
1615         {
1616             \__spintent_spusep_set_parens:nn { right } { \rbrack }
1617         }
1618         { \{ }
1619         {
1620             \__spintent_spusep_set_parens:nn { left } { \lbrace }
1621         }
1622         { \} }
1623         {
1624             \__spintent_spusep_set_parens:nn { right } { \rbrace }
1625         }
1626     }
1627     {
1628         \msg_error:nnn { spintenc } { spusep-invalid-arg } {#1}
1629     }
1630 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1631 \NewDocumentCommand \spusep { 0{ } m }
1632 {
1633   \mode_if_math:TF
1634   {
1635     \group_begin:
1636     \keys_set:nn { spintent / spusep } {#1}
1637     \__spintent_spusep_convert:n {#2}
1638     \group_end:
1639   }
1640   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1641 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

13.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

The “error” `spnewread-empty-args` is raised when `\spnewread` receives a blank symbol or a reading that is empty once normalized. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1642 \msg_new:nnnn { spintent } { spnewread-empty-args }
1643 {
1644   Empty~argument~in~\token_to_str:N \spnewread \c_space_tl \msg_line_context:.
1645 }
1646 {
1647   \token_to_str:N \spnewread \c_space_tl requires~two~non-empty~arguments.
1648 }
1649 \msg_new:nnnn { spintent } { spnewread-duplicate }
1650 {
1651   The~symbol~'~#1'~is~already~defined~\msg_line_context:.
1652 }
1653 {
1654   You~cannot~redefine~with~\token_to_str:N \spnewread.
1655 }
1656 \msg_new:nnnn { spintent } { spread-empty-sym }
1657 {
1658   Empty~symbol~argument~in~\token_to_str:N \spread \c_space_tl \msg_line_context:.
1659 }
1660 {
1661   \token_to_str:N \spread \c_space_tl requires~a~non-empty~symbol.
1662 }
1663 \msg_new:nnnn { spintent } { spread-unknown-sym }
1664 {
1665   The~symbol~'~#1'~is~not~registered~and~no~'read-arg'~was~given~\msg_line_context:.
1666 }
1667 {
1668   Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1669 }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

The variables `\__spintent_spread_luaset_status_str` and `\__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:nn/\__spintent_luafun_spread_lookup:n`. The variable `\__spintent_spread_read_arg_tl` stores `read-arg`.

```

1670 \str_new:N \__spintent_spread_luaset_status_str
1671 \str_new:N \__spintent_spread_luaset_reading_str
1672 \tl_new:N \__spintent_spread_read_arg_tl
1673 \tl_new:N \__spintent_spread_read_tmp_tl

```

(End of definition for `\__spintent_spread_luaset_status_str` and others.)

We define the $\langle keys \rangle$ `read-arg` and `silent`.

```

1674 \keys_define:nn { spintent / spread }
1675 {
1676   read-arg .code:n =
1677   {
1678     \__spintent_sanitize_affix:Nn \__spintent_spread_read_arg_tl {#1}

```

```

1679     },
1680     read-arg      .initial:n = ,
1681     read-arg      .value_required:n = true,
1682     silent        .bool_set:N = \l__spintent_spread_silent_bool,
1683     silent        .initial:n = false,
1684     silent        .value_required:n = true,
1685     unknown       .code:n =
1686     {
1687       \str_set:Nn \l__spintent_command_name_str { spread }
1688       \__spintent_unknown_keys_cmd:n {#1}
1689     },
1690   }

```

(End of definition for `read-arg`, `silent`, and `unknown`.)

`\__spintent_spread_exec:nn`

The function `\__spintent_spread_exec:nn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\{\langle symbol \rangle\}$ in ‘#2’. With `silent true`, the symbol is typeset inside a literal `_- \MathMLintent`, nothing else evaluated; a composite symbol (such as x^2) is not reliably silenced by `:silent` alone. In every case the symbol is preceded by `\__spintent_inner_sep:` inside the group, so that a composite symbol (a sole child, such as x^2) keeps its `intent` instead of collapsing. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintent_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1691 \cs_new_protected:Nn \__spintent_spread_exec:nn
1692 {
1693   \keys_set:nn { spintent / spread } { #1 }
1694   \tl_if_blank:nTF { #2 }
1695   { \msg_error:nn { spintent } { spread-empty-sym } }
1696   {
1697     \__spintent_invisible_sep:
1698     \bool_if:NTF \l__spintent_spread_silent_bool
1699     {
1700       \MathMLintent { _- } { \__spintent_inner_sep: #2 }
1701     }
1702     {
1703       \tl_if_empty:NTF \l__spintent_spread_read_arg_tl
1704       {
1705         \__spintent_luafun_spread_lookup:n { \exp_not:n {#2} }
1706         \str_if_eq:VnTF \l__spintent_spread_luaaset_status_str { found }
1707         {
1708           \MathMLintent { _\l__spintent_spread_luaaset_reading_str } { \__spintent_inner_sep: #2 }
1709         }
1710         {
1711           \msg_error:nne { spintent } { spread-unknown-sym } { \exp_not:n {#2} }
1712         }
1713       }
1714       {
1715         \MathMLintent { _\l__spintent_spread_read_arg_tl } { \__spintent_inner_sep: #2 }
1716       }
1717     }
1718   }
1719 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spread_exec:nn`.

```

1720 \NewDocumentCommand \spread { 0{} m }
1721 {
1722   \mode_if_math:TF
1723   {
1724     \group_begin:
1725     \__spintent_spread_exec:nn { #1 } { #2 }
1726     \group_end:
1727   }
1728   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1729 }

```

(End of definition for `\spread`. This function is documented on page 12.)

`\spnewread`

The public command `\spnewread` registers a new symbol/reading pair. It first normalizes the reading ‘#2’ with `\__spintent_sanitize_affix:Nn`, storing the result in the scratch variable `\l__spintent_spread_read_tmp_tl`.

and reports `spnewread-empty-args` when the symbol ‘`#1`’ is blank or the normalized reading is empty. Otherwise it passes the symbol and the normalized reading to Lua via `\__spintent_luafun_spread_new:nn`, reporting `spnewread-duplicate` when the symbol is already registered.

```

1730 \NewDocumentCommand \spnewread { m m }
1731 {
1732   \mode_if_math:TF
1733   {
1734     \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1735   }
1736   {
1737     \__spintent_sanitizize_affix:Nn \__spintent_spread_read_tmp_tl { #2 }
1738     \bool_lazy_or:nnTF
1739     { \tl_if_blank_p:n { #1 } } { \tl_if_empty_p:N \__spintent_spread_read_tmp_tl }
1740     { \msg_error:nn { spintent } { spnewread-empty-args } }
1741     {
1742       \__spintent_luafun_spread_new:nn { \exp_not:n {#1} } { \exp_not:V \__spintent_spread
1743       \str_if_eq:VnT \__spintent_spread_luaset_status_str { duplicate }
1744       {
1745         \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1746       }
1747     }
1748   }
1749 }

```

(End of definition for `\spnewread`. This function is documented on page 11.)

13.23.2 The command `\spdsym`

```

\__spintent_spdsym_read_sym_tl
\__spintent_spdsym_symbol_tl
\__spintent_spdsym_prefix_tl
\__spintent_spdsym_suffix_tl

```

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

1750 \tl_new:N \__spintent_spdsym_read_sym_tl
1751 \tl_new:N \__spintent_spdsym_symbol_tl
1752 \tl_new:N \__spintent_spdsym_prefix_tl
1753 \tl_new:N \__spintent_spdsym_suffix_tl

```

(End of definition for `\__spintent_spdsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1754 \keys_define:nn { spintent / spdsym }
1755 {
1756   prefix .code:n =
1757   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_prefix_tl {#1} },
1758   prefix .initial:n = ,
1759   prefix .value_required:n = true,
1760   suffix .code:n =
1761   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_suffix_tl {#1} },
1762   suffix .initial:n = ,
1763   suffix .value_required:n = true,
1764   set-sym .choices:nn =
1765   { old-style , two-dots , slash }
1766   {
1767     \int_case:nn { \l_keys_choice_int }
1768     {
1769       {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1770       {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1771       {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1772     }
1773   },
1774   set-sym .initial:n = two-dots,
1775   set-sym .value_required:n = true,
1776   set-sym / unknown .code:n =
1777   \msg_error:nneee { spintent } { unknown-choice }
1778   { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1779   read-sym .choices:nn =
1780   { dividido , dividido-por , dividido-entre }
1781   {
1782     \int_case:nn { \l_keys_choice_int }
1783     {
1784       {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1785       {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1786       {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1787     }

```

```

1788     },
1789     read-sym      .initial:n = dividido,
1790     read-sym      .value_required:n = true,
1791     read-sym / unknown .code:n =
1792       \msg_error:nneee { spintent } { unknown-choice }
1793       { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1794       { \exp_not:n {#1} },
1795     unknown      .code:n =
1796       {
1797         \str_set:Nn \l__spintent_command_name_str { spdsym }
1798         \__spintent_unknown_keys_cmd:n {#1}
1799       },
1800   }

```

(End of definition for *prefix* and others.)

\spdsym Defines the **\spdsym** operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1801 \NewDocumentCommand \spdsym { O{} }
1802 {
1803   \mode_if_math:TF
1804   {
1805     \group_begin:
1806     \keys_set:nn { spintent / spdsym } { #1 }
1807     \MathMLintent
1808     {
1809       -
1810       \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1811       { \l__spintent_spdsym_prefix_tl - }
1812       \l__spintent_spdsym_read_sym_tl
1813       \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1814       { - \l__spintent_spdsym_suffix_tl }
1815     }
1816     { \l__spintent_spdsym_symbol_tl }
1817     \group_end:
1818   }
1819   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1820 }

```

(End of definition for **\spdsym**. This function is documented on page 12.)

13.23.3 The command **\spmsym**

\l__spintent_spmsym_read_sym_tl Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for **\spmsym**.

```

\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1821 \tl_new:N \l__spintent_spmsym_read_sym_tl
1822 \tl_new:N \l__spintent_spmsym_symbol_tl
1823 \tl_new:N \l__spintent_spmsym_prefix_tl
1824 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for **\l__spintent_spmsym_read_sym_tl** and others.)

We define the *(keys)* prefix, suffix, set-sym, read-sym and not-print for **\spmsym**.

```

prefix
suffix
set-sym
read-sym
not-print
unknown
1825 \keys_define:nn { spintent / spmsym }
1826 {
1827   prefix      .code:n =
1828   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1829   prefix      .initial:n = ,
1830   prefix      .value_required:n = true,
1831   suffix      .code:n =
1832   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1833   suffix      .initial:n = ,
1834   suffix      .value_required:n = true,
1835   set-sym     .choices:nn =
1836   { cross , dot , asterisk }
1837   {
1838     \int_case:nn { \l_keys_choice_int }
1839     {
1840       {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1841       {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1842       {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1843     }
1844   },

```

```

1845   set-sym          .initial:n = dot,
1846   set-sym          .value_required:n = true,
1847   set-sym / unknown .code:n =
1848     \msg_error:nnee { spintent } { unknown-choice }
1849     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1850   read-sym         .choices:nn =
1851     { por , multiplicado-por , veces }
1852     {
1853       \int_case:nn { \l_keys_choice_int }
1854       {
1855         {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1856         {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1857         {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1858       }
1859     },
1860   read-sym         .initial:n = por,
1861   read-sym         .value_required:n = true,
1862   read-sym / unknown .code:n =
1863     \msg_error:nnee { spintent } { unknown-choice }
1864     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1865   not-print        .bool_set:N = \l__spintent_spmsym_not_print_bool,
1866   not-print        .initial:n = false,
1867   not-print        .value_required:n = true,
1868   unknown          .code:n =
1869     {
1870       \str_set:Nn \l__spintent_command_name_str { spmsym }
1871       \__spintent_unknown_keys_cmd:n {#1}
1872     },
1873 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *<keys>*, safely attaching prefixes and suffixes if present.

```

1874 \NewDocumentCommand \spmsym { 0{} }
1875 {
1876   \mode_if_math:TF
1877   {
1878     \group_begin:
1879     \keys_set:nn { spintent / spmsym } { #1 }
1880     \MathMLintent
1881     {
1882       -
1883       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1884       { \l__spintent_spmsym_prefix_tl - }
1885       \l__spintent_spmsym_read_sym_tl
1886       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1887       { - \l__spintent_spmsym_suffix_tl }
1888     }
1889     {
1890       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1891       {
1892         \invisibletimes
1893       }
1894       { \l__spintent_spmsym_symbol_tl }
1895     }
1896     \group_end:
1897   }
1898   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1899 }

```

(End of definition for `\spmsym`. This function is documented on page 13.)

13.24 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the *<dividend>* and the *<divisor>*. Both *<arguments>* are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *<dividend>* nor the *<divisor>* is *blank*.

13.24.1 Definition of keys for \spdiv

We define the *(keys)* `wrap-parens` and `read-parens` for `\spdiv` command. Number-related *(keys)* `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator *(keys)* `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1900 \keys_define:nn { spintent / spdiv }
1901 {
1902   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1903   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1904   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1905   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1906   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1907   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1908   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1909   wrap-parens     .initial:n  = false,
1910   wrap-parens     .value_required:n = true,
1911   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1912   read-parens     .initial:n  = true,
1913   read-parens     .value_required:n = true,
1914   unknown         .code:n =
1915   {
1916     \str_set:Nn \l__spintent_command_name_str { spdiv }
1917     \__spintent_unknown_keys_cmd:n {#1}
1918   },
1919 }

```

(End of definition for `cifras-sep` and others.)

13.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the *(dividend)* or the *(divisor)* is missing or *blank*.

```

1920 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1921 { The~arguments~for~\token_to_str:N \spdiv \c_space_tl cannot~be~empty~\msg_line_context:. }
1922 {
1923   You~must~provide~both~a~dividend~and~a~divisor.\\
1924 }

```

(End of definition for `spdiv-empty-argument`.)

13.24.3 Internal functions for \spdiv

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both *(arguments)* through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1925 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1926 {
1927   \spnum {#1}
1928   \MathMLintent { _ \l__spintent_spdsym_read_sym_tl }
1929   {
1930     \l__spintent_spdsym_symbol_tl
1931   }
1932   \spnum {#2}
1933 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1934 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1935 {
1936   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1937   {
1938     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1939   }
1940   {
1941     \MathMLintent { :silent } { ( }
1942     \__spintent_spdiv_render_op:nn {#1} {#2}
1943     \MathMLintent { :silent } { ) }
1944   }
1945 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn`

The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\{\langle arguments \rangle\}$, and decides on parenthesis wrapping.

```

1946 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1947 {
1948   \keys_set:nn { spintent / spdiv } { #1 }
1949   \tl_if_blank:nTF { #2 }
1950   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1951   {
1952     \tl_if_blank:nTF { #3 }
1953     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1954     {
1955       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1956       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1957       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1958     }
1959   }
1960 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1961 \NewDocumentCommand \spdiv { 0{} m m }
1962 {
1963   \mode_if_math:TF
1964   {
1965     \group_begin:
1966     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1967     \group_end:
1968   }
1969   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1970 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

13.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\{\langle first\ factor \rangle\}$ and the $\{\langle second\ factor \rangle\}$. Both $\{\langle arguments \rangle\}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{\langle first\ factor \rangle\}$ nor the $\{\langle second\ factor \rangle\}$ is *blank*.

13.25.1 Definition of keys for `\spmul`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1971 \keys_define:nn { spintent / spmul }
1972 {
1973   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1974   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1975   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1976   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1977   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1978   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1979   wrap-parens     .bool_set:N = \__spintent_spmul_wrap_parens_bool,
1980   wrap-parens     .initial:n  = false,
1981   wrap-parens     .value_required:n = true,
1982   read-parens     .bool_set:N = \__spintent_spmul_read_parens_bool,
1983   read-parens     .initial:n  = true,
1984   read-parens     .value_required:n = true,
1985   unknown         .code:n =
1986   {
1987     \str_set:Nn \__spintent_command_name_str { spmul }
1988     \__spintent_unknown_keys_cmd:n {#1}
1989   },
1990 }

```

(End of definition for `cifras-sep` and others.)

13.25.2 Definition of error messages

spmul-empty-argument

Raised when either the $\langle first\ factor \rangle$ or the $\langle second\ factor \rangle$ is missing or *blank*.

```

1991 \msg_new:nnnn { spintent } { spmul-empty-argument }
1992 { The~arguments~for~\token_to_str:N \spmul \c_space_tl cannot~be~empty~\msg_line_context:. }
1993 {
1994   You~must~provide~both~first~factor~and~second~factor.\
1995 }

```

(End of definition for spmul-empty-argument.)

13.25.3 Internal functions for \spmul

__spintent_spmul_render_op:nn

The function `\__spintent_spmul_render_op:nn` parses both $\langle arguments \rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1996 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1997 {
1998   \spnum {#1}
1999   \MathMLintent { _ \__spintent_spmsym_read_sym_tl }
2000   {
2001     \__spintent_spmsym_symbol_tl
2002   }
2003   \spnum {#2}
2004 }

```

(End of definition for __spintent_spmul_render_op:nn.)

__spintent_spmul_render_parens:nn

The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

2005 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
2006 {
2007   \bool_if:NTF \__spintent_spmul_read_parens_bool
2008   {
2009     ( \__spintent_spmul_render_op:nn {#1} {#2} )
2010   }
2011   {
2012     \MathMLintent { :silent } { ( }
2013     \__spintent_spmul_render_op:nn {#1} {#2}
2014     \MathMLintent { :silent } { ) }
2015   }
2016 }

```

(End of definition for __spintent_spmul_render_parens:nn.)

__spintent_spmul_process:nnn

The function `\__spintent_spmul_process:nnn` sets $\langle keys \rangle$, validates $\langle arguments \rangle$, and decides on parenthesis wrapping.

```

2017 \cs_new_protected:Nn \__spintent_spmul_process:nnn
2018 {
2019   \keys_set:nn { spintent / spmul } { #1 }
2020   \tl_if_blank:nTF { #2 }
2021   { \msg_error:nnn { spintent } { spmul-empty-argument } }
2022   {
2023     \tl_if_blank:nTF { #3 }
2024     { \msg_error:nnn { spintent } { spmul-empty-argument } }
2025     {
2026       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
2027       { \__spintent_spmul_render_parens:nn {#2} {#3} }
2028       { \__spintent_spmul_render_op:nn {#2} {#3} }
2029     }
2030   }
2031 }

```

(End of definition for __spintent_spmul_process:nnn.)

\spmul

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

2032 \NewDocumentCommand \spmul { 0{ } m m }
2033 {
2034   \mode_if_math:TF
2035   {
2036     \group_begin:
2037     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2038     \group_end:
2039   }
2040   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2041 }

```

(End of definition for `\spmul`. This function is documented on page 14.)

13.26 The commands `\spgcd` and `\spmc`

The user-level commands `\spgcd` and `\spmc` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

13.26.1 Variables for `\spgcd` and `\spmc`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintenc_spmcm_spmcd_luaset_error_str
\__spintenc_mc_luaset_types_str
\__spintenc_mc_luaset_all_numeric_str
\__spintenc_mc_args_seq
\__spintenc_mc_raw_args_seq
\__spintenc_spmcd_print_result_bool
\__spintenc_spmcm_print_result_bool
\__spintenc_spmcd_read_terse_bool
\__spintenc_spmcm_read_terse_bool
\__spintenc_spmcd_print_upper_bool
\__spintenc_spmcm_print_upper_bool
\__spintenc_spmcd_prefix_tl
\__spintenc_spmcm_prefix_tl
\__spintenc_spmcd_suffix_tl
\__spintenc_spmcm_suffix_tl
```

```
2042 \str_new:N \__spintenc_spmcm_spmcd_luaset_error_str
2043 \clist_new:N \__spintenc_mc_luaset_types_clist
2044 \str_new:N \__spintenc_mc_luaset_all_numeric_str
2045 \seq_new:N \__spintenc_mc_args_seq
2046 \seq_new:N \__spintenc_mc_raw_args_seq
2047 \seq_new:N \__spintenc_mc_types_seq
2048 \int_new:N \__spintenc_mc_idx_int
2049 \clist_map_inline:nn { mcd , mcm }
2050 {
2051   \tl_new:c { \__spintenc_sp #1 _luaset_ #1 _value_tl }
2052   \bool_new:c { \__spintenc_sp #1 _read_terse_bool }
2053   \tl_new:c { \__spintenc_sp #1 _prefix_tl }
2054 }
```

(End of definition for `\__spintenc_spmcm_spmcd_luaset_error_str` and others.)

13.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
2055 \msg_new:nnnn { spintenc } { mc-invalid-arguments }
2056 {
2057   Invalid~arguments~for~#1\msg_line_context:.
2058 }
2059 {
2060   The~arguments~must~be~a~comma-separated~list. \\
2061 }
```

(End of definition for `mc-invalid-arguments`.)

13.26.3 Keys for `\spgcd` and `\spmc`

result
prefix
read-cmd
upper-case
concept
silent
unknown

We define the shared *⟨keys⟩* `result`, `prefix`, `read-cmd`, `upper-case`, `concept` and `silent`.

```
2062 \clist_map_inline:nn { mcd , mcm }
2063 {
2064   \keys_define:nn { spintenc / sp #1 }
2065   {
2066     result      .bool_set:c = { \__spintenc_sp #1 _print_result_bool },
2067     result      .initial:n = false,
2068     result      .value_required:n = true,
2069     prefix      .code:n =
2070       { \__spintenc_sanitiz_affe:cn { \__spintenc_sp #1 _prefix_tl } {##1} },
2071     prefix      .initial:n = ,
2072     prefix      .value_required:n = true,
2073     read-cmd    .choices:nn =
2074       { terse , clear }
2075       {
2076         \int_case:nn { \keys_choice_int }
2077         {
2078           {1} { \bool_set_true:c { \__spintenc_sp #1 _read_terse_bool } }
2079           {2} { \bool_set_false:c { \__spintenc_sp #1 _read_terse_bool } }
2080         }
2081       },
2082     read-cmd    .initial:n = clear,
2083     read-cmd    .value_required:n = true,
2084     read-cmd / unknown .code:n =
2085       \msg_error:nneee { spintenc } { unknown-choice }
2086       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2087     upper-case  .bool_set:c = { \__spintenc_sp #1 _print_upper_bool },
2088     upper-case  .initial:n = false,
2089     upper-case  .value_required:n = true,
2090     concept     .bool_set:c = { \__spintenc_sp #1 _concept_bool },
2091     concept     .initial:n = true,
```

```

2092         concept                .value_required:n = true,
2093         silent                  .bool_set:c = { l__spintent_sp #1 _silent_bool },
2094         silent                  .initial:n = false,
2095         silent                  .value_required:n = true,
2096         unknown                 .code:n =
2097         {
2098             \str_set:Nn \l__spintent_command_name_str { sp #1 }
2099             \__spintent_unknown_keys_cmd:n {##1}
2100         },
2101     }
2102 }

```

(End of definition for *result* and others.)

13.26.4 Internal functions for \spmcd and \spmcm

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintent_mc_raw_args_seq` and `\l__spintent_mc_luaset_types_clist` into `\l__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is *number*, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2103 \cs_new_protected:Nn \__spintent_mc_process_args:nn
2104 {
2105     \__spintent_luafun_mc_classify_args:n {#2}
2106     \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#2}
2107     \seq_set_from_clist:NN \l__spintent_mc_types_seq \l__spintent_mc_luaset_types_clist
2108     \seq_clear:N \l__spintent_mc_args_seq
2109     \int_zero:N \l__spintent_mc_idx_int
2110     \seq_map_inline:Nn \l__spintent_mc_raw_args_seq
2111     {
2112         \int_incr:N \l__spintent_mc_idx_int
2113         \str_if_eq:eeTF
2114         { \seq_item:Nn \l__spintent_mc_types_seq { \l__spintent_mc_idx_int } } { number }
2115         {
2116             \seq_put_right:Nn \l__spintent_mc_args_seq { \spnum {##1} }
2117         }
2118         { \seq_put_right:Nn \l__spintent_mc_args_seq {##1} }
2119     }

```

Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2120     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2121     {
2122         \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
2123         {
2124             \MathMLintent { _- } { \__spintent_inner_sep: ##1 }
2125         }
2126         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2127         \MathMLintent { :silent } { ( }
2128         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "intervalo"
2129         \seq_use:Nnnn \l__spintent_mc_args_seq
2130         { \MathMLintent { :silent } { , } }
2131         { \MathMLintent { :silent } { , } }
2132         { \MathMLintent { :silent } { , } }
2133     }
2134     {
2135         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2136         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2137         {
2138             \MathMLintent { _entre } { ( }
2139         }
2140         {
2141             \MathMLintent { :silent } { ( }
2142         }
2143         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "intervalo"
2144         \seq_use:Nnnn \l__spintent_mc_args_seq
2145         {
2146             \MathMLintent { :silent } { , }
2147             \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" medio
2148         }
2149         {
2150             \MathMLintent { :silent } { , }

```

```

2151     }
2152     {
2153         \MathMLintent { :silent } { , }
2154         \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2155     }
2156 }
2157 \MathMLintent { :silent } { ) }
2158 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmc`. Takes the short identifier in `#1`, its full reading string in `#2`, the local `(keys)` in `#3`, and the optional arguments in `#4`.

The operator name is silenced with `silent=true` — `\__spintent_mc_process_args:nn` silences the arguments too, in that case, each wrapped in a literal `_-` intent, and the printed result is silenced as well — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal intent instead — `#1` or `#2` per `read-cmd`, prefixed with `prefix` when given.

With `#4` given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintent_mc_process_args:n` prints the argument list itself either way.

- We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintent_invisible_sep_int` is applied.

```

2159 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2160 {
2161     \keys_set:nn { spintent / sp #1 } { #3 }
2162     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
2163     {
2164         \MathMLintent { :silent }
2165         {
2166             \mathrm
2167             {
2168                 \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2169                 { \text_uppercase:n {#1} } { #1 }
2170             }
2171         }
2172     }
2173     {
2174         \bool_if:cTF { \__spintent_sp #1 _concept_bool }
2175         {
2176             \MathMLintent
2177             {
2178                 -
2179                 \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2180                 {
2181                     \tl_use:c { \__spintent_sp #1 _prefix_tl } -
2182                 }
2183                 \bool_if:cTF { \__spintent_sp #1 _read_terse_bool }
2184                 { #1 } { #2 }
2185             }
2186             {
2187                 \mathrm
2188                 {
2189                     \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2190                     { \text_uppercase:n {#1} } { #1 }
2191                 }
2192             }
2193         }
2194     }
2195     \MathMLintent { :silent }
2196     {
2197         \mathrm
2198         {
2199             \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2200             { \text_uppercase:n {#1} } { #1 }
2201         }
2202     }
2203 }
2204 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2205 \tl_if_novalue:nF { #4 }
2206 {
2207   \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#4}
2208   \int_compare:nNnTF { \seq_count:N \l__spintent_mc_raw_args_seq } < { 2 }
2209   {
2210     \msg_error:nne { spintent } { mc-too-few-arguments } { \exp_not:c { sp #1 } }
2211   }
2212   {
2213     \__spintent_luafun_mc_classify_args:n {#4}
2214     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2215     {
2216       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2217       \str_if_eq:VnT \l__spintent_spmcm_spmcd_luaset_error_str { true }
2218       {
2219         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2220       }
2221     }
2222     \__spintent_mc_process_args:nn {#1} {#4}
2223     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2224     {
2225       \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2226       {
2227         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2228         {
2229           \MathMLint { :silent } { = }
2230           \MathMLint { _- }
2231           {
2232             \__spintent_inner_sep:
2233             \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2234           }
2235         }
2236         {
2237           \MathMLint { _es-igual-a } { = }
2238           \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2239         }
2240       }
2241     }
2242   }
2243 }
2244 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2245 \NewDocumentCommand \spmc { 0{} d() }
2246 {
2247   \mode_if_math:TF
2248   {
2249     \group_begin:
2250     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2251     \group_end:
2252   }
2253   { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2254 }
2255 \NewDocumentCommand \spmcm { 0{} d() }
2256 {
2257   \mode_if_math:TF
2258   {
2259     \group_begin:
2260     \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2261     \group_end:
2262   }
2263   { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }

```

```
2264 }
```

(End of definition for `\spmc` and `\spmcm`. These functions are documented on page 15.)

13.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

13.27.1 Definition of variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```
\__spintent_md_out_seq
\__spintent_spm_md_luaset_is_number_str
\__spintent_spm_spnD_luaset_error_str
\__spintent_spm_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spm_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spm_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spm_print_result_num_tl
\__spintent_spnD_print_result_num_tl

2265 \seq_new:N \__spintent_md_out_seq
2266 \str_new:N \__spintent_spm_spnD_luaset_error_str
2267 \str_new:N \__spintent_spm_md_luaset_is_number_str
2268 \clist_map_inline:nn { nM , nD }
2269 {
2270   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2271   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2272   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2273 }
```

(End of definition for `\__spintent_md_out_seq` and others.)

13.27.2 Keys for `\spnM` and `\spnD`

We define the shared *keys* `result`, `silent`, `result-num`, `prefix` and `concept`.

```
result
silent
result-num
prefix
concept
unknown

2274 \clist_map_inline:nn { nM , nD }
2275 {
2276   \keys_define:nn { spintent / sp #1 }
2277   {
2278     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2279     result      .initial:n = false,
2280     result      .value_required:n = true,
2281     silent      .bool_set:c = { \__spintent_sp #1 _silent_bool },
2282     silent      .initial:n = false,
2283     silent      .value_required:n = true,
2284     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2285     result-num .initial:n = ,
2286     prefix      .code:n =
2287       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2288     prefix      .initial:n = ,
2289     prefix      .value_required:n = true,
2290     concept     .bool_set:c = { \__spintent_sp #1 _concept_bool },
2291     concept     .initial:n = true,
2292     concept     .value_required:n = true,
2293     unknown     .code:n =
2294       {
2295         \str_set:Nn \__spintent_command_name_str { sp #1 }
2296         \__spintent_unknown_keys_cmd:n {##1}
2297       },
2298   }
2299 }
```

(End of definition for `result` and others.)

13.27.3 Definition of error messages for `\spnM` and `\spnD`

Exception triggered when the argument for multiples or divisors is invalid.

```
md-invalid-argument
mc-too-few-arguments

2300 \msg_new:nnnn { spintent } { md-invalid-argument }
2301 {
2302   Invalid~argument~for~#1\msg_line_context:.
2303 }
2304 {
2305   The~argument~must~be~a~single~positive~integer. \l
2306 }
```

Exception triggered when fewer than two arguments are provided.

```
2307 \msg_new:nnnn { spintent } { mc-too-few-arguments }
2308 {
2309   Too~few~arguments~for~#1\msg_line_context:.
2310 }
2311 {
2312   At~least~two~arguments~are~required. \l
2313 }
```

(End of definition for `md-invalid-argument` and `mc-too-few-arguments`.)

13.27.4 Internal functions for `\spnM` and `\spnD`

`\__spintent_spnM_spnD_process:nnnn`

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier `#1`, its printed mathematical symbol `#2`, its base reading string `#3`, the local key-value options `#4`, and the optional argument `#5`. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spnM_and_D_exec:nnnn`.

```
2314 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnn
2315 {
2316   \keys_set:nn { spintent / sp #1 } { #4 }
2317   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2318 }
```

(End of definition for `\__spintent_spnM_spnD_process:nnnn`.)

`\__spintent_spnM_spnD_intent:nn`

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier `#1` (such as “nM” or “nD”) and the base reading string `#2`. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends `-de`.

```
2319 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2320 {
2321   -
2322   \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2323   { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
2324   \str_case:nnF { #2 } { { múltiplos } { múltiplos } }
2325   { #2 }
2326   -de
2327 }
```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol `#1` (such as “D” or “M”) and the variable or explicit argument `#2`. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules. The argument is wrapped in a literal `_-` intent, not in `:silent`, so that intents of its own (such as those of `\spnum`) are silenced too.

```
2328 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2329 {
2330   \MathMLintent { :silent } { \operatorname { #1 } }
2331   \MathMLintent { :silent } { ( }
2332   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2333   \MathMLintent { _- } { \__spintent_inner_sep: #2 }
2334   \MathMLintent { :silent } { ) }
2335 }
```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:nn`

The internal function `\__spintent_spnM_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier `#1` and the classified numeric argument `#2`. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```
2336 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2337 {
2338   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2339   {
2340     \tl_use:c { \__spintent_sp #1 _print_result_num_tl }
2341   }
2342   \str_if_eq:VnTF \__spintent_spnM_spnD_luaset_error_str { true }
2343   {
2344     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2345   }
2346   {
2347     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2348     \MathMLintent { _son } { = \{ }
2349     \seq_set_from_clist:Ne
2350     \l__spintent_md_out_seq { \tl_use:c { \__spintent_sp #1 _luaset_result_tl } }
2351     \seq_use:Nnnn \l__spintent_md_out_seq
```

```

2352     {
2353       \MathMLintent { :silent } { , }
2354       \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" (solo dos)
2355     }
2356     { \MathMLintent { :silent } { , } }
2357     {
2358       \MathMLintent { :silent } { , }
2359       \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { false }
2360       {
2361         \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2362       }
2363     }
2364     \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2365     {
2366       \MathMLintent { :silent } { , } ~ \dots
2367     }
2368     \MathMLintent { :silent } { \} }
2369   }
2370 }

```

(End of definition for `\__spintent_spnM_spnD_result:nn`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spintent_luafun_spnmd_classify:n` decides whether it is wrapped through `\spnum`—and, when it is, whether `\__spintent_spnM_spnD_result:nn` also prints the computed set, per `result`—or left verbatim, with no result attempted. A zero is flagged as an error by `\__spintent_luafun_spnmd_classify:n` and rejected right away with `md-invalid-argument`. With `silent=true` the printed result is wrapped in a literal `_-` intent, so it stays on the page but is not verbalized.

Either way, `silent` silences the whole thing through `\__spintent_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal intent from `\__spintent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2371 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2372 {
2373   \tl_if_novalue:nTF { #4 }
2374   {
2375     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2376     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2377     {
2378       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2379       {
2380         \MathMLintent
2381         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2382         { \operatorname { #2 } }
2383       }
2384       { \MathMLintent { :silent } { \operatorname { #2 } } }
2385       \MathMLintent { :silent } { ( }
2386       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2387       n
2388       \MathMLintent { :silent } { ) }
2389     }
2390   }
2391   {
2392     \__spintent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2393     \str_if_eq:VnT \__spintent_spnM_spnD_luaset_error_str { true }
2394     { \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } } }
2395     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2396     {
2397       \str_if_eq:VnTF \__spintent_spnmd_luaset_is_number_str { true }
2398       { \__spintent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2399       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2400     }
2401     {
2402       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2403       {
2404         \MathMLintent
2405         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2406         { \operatorname { #2 } }
2407       }

```

```

2408         { \MathMLintent { :silent } { \operatorname { #2 } } }
2409     \MathMLintent { :silent } { ( }
2410     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2411     \str_if_eq:VnTF \l__spintent_spnmd_luaset_is_number_str { true }
2412         { \spnum { #4 } }
2413         { #4 }
2414     \MathMLintent { :silent } { ) }
2415 }
2416 \str_if_eq:VnT \l__spintent_spnmd_luaset_is_number_str { true }
2417 {
2418     \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2419     {
2420         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2421         {
2422             \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2423             \MathMLintent { _- }
2424             {
2425                 \__spintent_inner_sep:
2426                 \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2427             }
2428             {
2429                 \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2430             }
2431         }
2432     }
2433 }
2434 }
2435 }

```

(End of definition for `\__spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spnM_spnD_process:nnnnn`, passing along the optional $\langle\textit{keys}\rangle$ ‘#1’ and the numeric $\{\langle\textit{argument}\rangle\}$ ‘#2’.

```

2436 \NewDocumentCommand \spnM { O{} d() }
2437 {
2438     \mode_if_math:TF
2439     {
2440         \group_begin:
2441         \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2442         \group_end:
2443     }
2444     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2445 }
2446 \NewDocumentCommand \spnD { O{} d() }
2447 {
2448     \mode_if_math:TF
2449     {
2450         \group_begin:
2451         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2452         \group_end:
2453     }
2454     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2455 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 15.)

13.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

13.28.1 Definition of keys for `\sprazon`

silent We define the $\langle\textit{keys}\rangle$ `silent`, `use-dfrac` and `inline`.

```

2456 \keys_define:nn { spintent / sprazon }
2457 {
2458     silent .bool_set:N = \l__spintent_sprazon_silent_bool,
2459     silent .initial:n = false,
2460     silent .value_required:n = true,
2461     use-dfrac .bool_set:N = \l__spintent_sprazon_use_dfrac_bool,
2462     use-dfrac .initial:n = false,

```

```

2463     use-dfrac    .value_required:n = true,
2464     inline      .bool_set:N = \l__spintent_sprazon_inline_bool,
2465     inline      .initial:n = false,
2466     inline      .value_required:n = true,
2467     unknown     .code:n =
2468     {
2469         \str_set:Nn \l__spintent_command_name_str { sprazon }
2470         \__spintent_unknown_keys_cmd:n {#1}
2471     },
2472 }

```

(End of definition for *silent* and others.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under *silent*, the whole thing is silenced as one unit — `\frac/\dfrac` (per `use-dfrac`) normally, or `#2 : #3` under *inline*.

Otherwise, with `inline=false` (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With `inline=true`, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2473 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2474 {
2475     \keys_set:nn { spintent / sprazon } { #1 }
2476     \__spintent_invisible_sep:
2477     \bool_if:NTF \l__spintent_sprazon_silent_bool
2478     {
2479         \MathMLintent { _- }
2480         {
2481             \__spintent_inner_sep:
2482             \bool_if:NTF \l__spintent_sprazon_inline_bool
2483             { #2 : #3 }
2484             {
2485                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2486                 { \dfrac { #2 } { #3 } }
2487                 { \frac { #2 } { #3 } }
2488             }
2489         }
2490     }
2491     {
2492         \bool_if:NTF \l__spintent_sprazon_inline_bool
2493         {
2494             \MathMLintent { _es-a:infix($num,$denom) }
2495             {
2496                 \MathMLarg { num } { #2 }
2497                 :
2498                 \MathMLarg { denom } { #3 }
2499             }
2500         }
2501         {
2502             \MathMLintent { _es-a:infix($num,$denom) }
2503             {
2504                 \__spintent_inner_sep:
2505                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2506                 {
2507                     \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2508                 }
2509                 { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2510             }
2511         }
2512     }
2513 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2514 \NewDocumentCommand \sprazon { 0{} m m }
2515 {
2516     \mode_if_math:TF
2517     {
2518         \group_begin:
2519         \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }

```

```

2520     \group_end:
2521   }
2522   { \msg_error:nnn { spinttent } { need-math-mode } { \sprazon } }
2523 }

```

(End of definition for \sprazon. This function is documented on page 18.)

13.29 The command \spProp

The user command \spProp typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing _spinttent_sprazon_exec:nnn directly (not the public \sprazon command) for each ratio.

\spProp The public command \spProp enforces *math mode*, and forwards the same $\langle keys \rangle$ in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native \MathMLintent reading _como over the $=$ sign.

```

2524 \NewDocumentCommand \spProp { O{} m m m m }
2525 {
2526   \mode_if_math:TF
2527   {
2528     \group_begin:
2529     \_spinttent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2530     \bool_if:NTF \_spinttent_sprazon_silent_bool
2531     {
2532       \MathMLintent { :silent } { = }
2533     }
2534     { \MathMLintent { _como } { = } }
2535     \_spinttent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2536   \group_end:
2537   }
2538   { \msg_error:nnn { spinttent } { need-math-mode } { \spProp } }
2539 }

```

(End of definition for \spProp. This function is documented on page 18.)

13.30 The command \spang

The command \spang is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function _spinttent_luafun_spang_parse:n. This separates the degrees, minutes, and seconds to construct a MathML intent tree with the correct postfixes for NVDA/MathCAT.

```

\_spinttent_spang_luaset_grado_str
\_spinttent_spang_luaset_minuto_str
\_spinttent_spang_luaset_segundo_str
\_spinttent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module spinttent.lua. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2540 \str_new:N \_spinttent_spang_luaset_grado_str
2541 \str_new:N \_spinttent_spang_luaset_minuto_str
2542 \str_new:N \_spinttent_spang_luaset_segundo_str
2543 \str_new:N \_spinttent_spang_luaset_error_str

```

(End of definition for _spinttent_spang_luaset_grado_str and others.)

13.30.1 Definition of error messages for \spang

invalid-angle-format The error message invalid-angle-format defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2544 \msg_new:nnnn { spinttent } { invalid-angle-format }
2545 { Invalid~argument~'~#2'~for~#1\msg_line_context:. }
2546 {
2547   The~argument~'~#2'~could~not~be~parsed. \
2548 }

```

(End of definition for invalid-angle-format.)

13.30.2 Internal functions for \spang

_spinttent_spang_process:n The internal function _spinttent_spang_process:n parses the angle input using _spinttent_luafun_spang_parse:n after resetting the error flag in _spinttent_spang_luaset_error_str. If an invalid format is detected, it issues the invalid-angle-format error. Otherwise, it checks the extracted string variables _spinttent_spang_luaset_grado_str, _spinttent_spang_luaset_minuto_str, and _spinttent_spang_luaset_segundo_str. For each non-empty variable, it constructs the corresponding MathML intent structure with the correct measurement symbols and postfixes for NVDA/MathCAT. Each value is expanded with \exp_args:N before it reaches \spunit, which protects its $\langle argument \rangle$ from expansion.

```

2549 \cs_new_protected:Nn \_spinttent_spang_process:n
2550 {

```

```

2551 \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2552 \__spintent_luafun_spang_parse:n { #1 }
2553 \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2554 {
2555   \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2556 }
2557 {
2558   \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2559   {
2560     \exp_args:Ne \spunit { \l__spintent_spang_luaset_grado_str ° }
2561   }
2562   \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2563   {
2564     \MathMLintend { _minutos:postfix($m) }
2565     {
2566       \MathMLarg { m }
2567       {
2568         \exp_args:Ne \spunit { \l__spintent_spang_luaset_minuto_str ' }
2569       }
2570     }
2571   }
2572   \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2573   {
2574     \MathMLintend { _segundos:postfix($s) }
2575     {
2576       \MathMLarg { s }
2577       {
2578         \exp_args:Ne \spunit { \l__spintent_spang_luaset_segundo_str " }
2579       }
2580     }
2581   }
2582 }
2583 }

```

(End of definition for `\__spintent_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2584 \NewDocumentCommand \spang { m }
2585 {
2586   \mode_if_math:TF
2587   {
2588     \group_begin:
2589     \__spintent_spang_process:n { #1 }
2590     \group_end:
2591   }
2592   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2593 }

```

(End of definition for `\spang`. This function is documented on page 24.)

13.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

13.31.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2594 \keys_define:nn { spintent / spnZ }
2595 {
2596   cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2597   read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2598   wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2599   wrap-parens .initial:n = false,
2600   wrap-parens .value_required:n = true,

```

```

2601     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2602     read-parens .initial:n = true,
2603     read-parens .value_required:n = true,
2604     unknown .code:n =
2605     {
2606         \str_set:Nn \l__spintent_command_name_str { spnZ }
2607         \__spintent_unknown_keys_cmd:n {#1}
2608     },
2609 }

```

(End of definition for *cifras-sep* and others.)

13.31.2 Definition of error messages

spnz-not-integer

Raised when the $\langle \textit{argument} \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2610 \msg_new:nnnn { spintent } { spnz-not-integer }
2611 { The~value~!#1!~is~not~a~valid~integer~\msg_line_context:. }
2612 {
2613     Command~\token_to_str:N \spnZ \c_space_tl only~accepts~integers.\
2614 }

```

(End of definition for *spnz-not-integer*.)

13.31.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintent_spnz_read_parens_bool` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent` function *intent* to the parentheses while printing the number.

```

2615 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2616 {
2617     \bool_if:NTF \l__spintent_spnz_read_parens_bool
2618     {
2619         ( \__spintent_spnum_print_num_start: )
2620     }
2621     {
2622         \MathMLintent { :silent } { ( }
2623         \__spintent_spnum_print_num_start:
2624         \MathMLintent { :silent } { ) }
2625     }
2626 }

```

(End of definition for `\__spintent_spnz_render_parens:`.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the $\langle \textit{keys} \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```

2627 \cs_new_protected:Nn \__spintent_spnz_process:nn
2628 {
2629     \keys_set:nn { spintent / spnZ } {#1}
2630     \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2631     \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2632     {
2633         \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
2634         { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2635         {
2636             \bool_if:NTF \l__spintent_spnz_wrap_parens_bool
2637             { \__spintent_spnz_render_parens: }
2638             { \__spintent_spnum_print_num_start: }
2639         }
2640     }
2641     { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2642 }

```

(End of definition for `\__spintent_spnz_process:nn`.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2643 \NewDocumentCommand \spnZ { O{} m }
2644 {
2645   \mode_if_math:TF
2646   {
2647     \group_begin:
2648     \__spintent_spnz_process:nn { #1 } { #2 }
2649     \group_end:
2650   }
2651   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2652 }

```

(End of definition for `\spnZ`. This function is documented on page 17.)

13.32 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

13.32.1 Definition of variables for `\spmQ`

The variable `\l__spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘-’ are present in the integer part, used by the function `\__spintent_spmQ_print_scale_int:n`.

The variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The boolean variable `\l__spintent_spmQ_error_bool` is set to `true` by `\__spintent_spmQ_check_part:nn` when any of the three arguments is rejected, so that nothing is typeset. The boolean variable `\l__spintent_spmQ_silent_bool` stores the value of the key `silent`.

The integer variable `\l__spintent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2653 \str_new:N \l__spintent_spmQ_join_word_str
2654 \str_new:N \l__spintent_spmQ_intent_read_sign_str
2655 \tl_new:N \l__spintent_spmQ_wrap_parens_l_tl
2656 \tl_new:N \l__spintent_spmQ_wrap_parens_r_tl
2657 \bool_new:N \l__spintent_spmQ_error_bool
2658 \bool_new:N \l__spintent_spmQ_silent_bool
2659 \int_new:N \l__spintent_saved_luamml_flag_int

```

(End of definition for `\l__spintent_spmQ_join_word_str` and others.)

13.32.2 Definition of error messages for `\spmQ`

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when any of the three arguments is not actually an integer. The third error `spmQ-signed-part` is returned when the numerator or the denominator carries a sign: only the integer part may be signed.

```

2660 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2661 {
2662   Zero~denominator~in~\token_to_str:N \spmQ \c_space_tl \msg_line_context:.
2663 }
2664 {
2665   The~denominator~of~a~fraction~cannot~be~zero.\\
2666 }
2667 \msg_new:nnnn { spintent } { spmQ-not-integer }
2668 {
2669   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2670 }
2671 {
2672   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2673 }
2674 \msg_new:nnnn { spintent } { spmQ-signed-part }
2675 {
2676   The~value~'#1'~cannot~have~a~sign~\msg_line_context:.
2677 }

```

```

2678 {
2679     The~numerator~and~the~denominator~of~\token_to_str:N \spmQ \c_space_tl
2680     must~be~integers~without~sign;~only~the~integer~part~may~have~one.\
2681 }

```

(End of definition for `spmQ-zero-denominator`, `spmQ-not-integer`, and `spmQ-signed-part`.)

13.32.3 Definition of keys for `\spmQ`

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens`, `use-dfrac` and `silent`.

```

join-word 2682 \keys_define:nn { spinttent / spmQ }
use-spnZ   2683 {
wrap-parens 2684     join-word           .choices:nn =
read-parens 2685     { entero , enteros , y }
use-dfrac   2686     {
silent       2687         \int_case:nn { \l_keys_choice_int }
unknown      2688         {
2689             {1} { \str_set:Nn \l__spinttent_spmQ_join_word_str { entero } }
2690             {2} { \str_set:Nn \l__spinttent_spmQ_join_word_str { enteros } }
2691             {3} { \str_set:Nn \l__spinttent_spmQ_join_word_str { y } }
2692         }
2693     },
2694     join-word           .initial:n = y,
2695     join-word           .value_required:n = true,
2696     join-word/unknown .code:n =
2697         \msg_error:nneee { spinttent } { unknown-choice }
2698         { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2699     use-spnZ           .bool_set:N = \l__spinttent_spmQ_use_spnZ_bool,
2700     use-spnZ           .initial:n = true,
2701     use-spnZ           .value_required:n = true,
2702     use-dfrac          .bool_set:N = \l__spinttent_spmQ_use_dfrac_bool,
2703     use-dfrac          .initial:n = false,
2704     use-dfrac          .value_required:n = true,
2705     wrap-parens        .bool_set:N = \l__spinttent_spmQ_wrap_parens_bool,
2706     wrap-parens        .initial:n = false,
2707     wrap-parens        .value_required:n = true,
2708     read-parens        .bool_set:N = \l__spinttent_spmQ_read_parens_bool,
2709     read-parens        .initial:n = true,
2710     read-parens        .value_required:n = true,
2711     silent             .bool_set:N = \l__spinttent_spmQ_silent_bool,
2712     silent             .initial:n = false,
2713     silent             .value_required:n = true,
2714     unknown            .code:n =
2715         {
2716             \str_set:Nn \l__spinttent_command_name_str { spmQ }
2717             \__spinttent_unknown_keys_cmd:n {#1}
2718         },
2719 }

```

(End of definition for `join-word` and others.)

13.32.4 Internal functions for `\spmQ`

`\__spinttent_spmQ_set_wrap_parens_size:`

The function `\__spinttent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spinttent_spmQ_wrap_parens_l_tl` and `\l__spinttent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigr\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `use-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2720 \cs_new_protected:Nn \__spinttent_spmQ_set_wrap_parens_size:
2721 {
2722     \int_case:nnF { \mathstyle }
2723     {
2724         { 0 }
2725         {
2726             \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2727             \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2728         }
2729         { 1 }
2730         {
2731             \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2732             \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \Bigr \rparen }

```

```

2733     }
2734     { 2 }
2735     {
2736         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2737         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2738     }
2739     { 3 }
2740     {
2741         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2742         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2743     }
2744 }
2745 {
2746     \tl_set:Nn \l__spintrent_spmQ_wrap_parens_l_tl { \lparen }
2747     \tl_set:Nn \l__spintrent_spmQ_wrap_parens_r_tl { \rparen }
2748 }

```

If the key `use-dfrac` is active, we must reset the variables `\l__spintrent_spmQ_wrap_parens_l_tl` and `\l__spintrent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2749     \bool_if:NT \l__spintrent_spmQ_use_dfrac_bool
2750     {
2751         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2752         \tl_set:Nn \l__spintrent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2753     }
2754 }

```

(End of definition for `\__spintrent_spmQ_set_wrap_parens_size:`)

```

\__spintrent_spmQ_start_wrap_parens:
\__spintrent_spmQ_stop_wrap_parens:

```

The functions `\__spintrent_spmQ_start_wrap_parens:` and `\__spintrent_spmQ_stop_wrap_parens:` open and close the parentheses around the “*mixed number*” when the key `wrap-parens` is active. They first call `\__spintrent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintrent_spmQ_wrap_parens_l_tl` or `\l__spintrent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintrent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintenc { :silent }` so that NVDA/MathCAT does not verbalize them.

```

2755 \cs_new_protected:Nn \__spintrent_spmQ_start_wrap_parens:
2756 {
2757     \bool_if:NT \l__spintrent_spmQ_wrap_parens_bool
2758     {
2759         \__spintrent_spmQ_set_wrap_parens_size:
2760         \bool_if:NTF \l__spintrent_spmQ_read_parens_bool
2761             { \l__spintrent_spmQ_wrap_parens_l_tl }
2762             { \MathMLintenc { :silent } { \l__spintrent_spmQ_wrap_parens_l_tl } }
2763     }
2764 }
2765 \cs_new_protected:Nn \__spintrent_spmQ_stop_wrap_parens:
2766 {
2767     \bool_if:NT \l__spintrent_spmQ_wrap_parens_bool
2768     {
2769         \__spintrent_spmQ_set_wrap_parens_size:
2770         \bool_if:NTF \l__spintrent_spmQ_read_parens_bool
2771             { \l__spintrent_spmQ_wrap_parens_r_tl }
2772             { \MathMLintenc { :silent } { \l__spintrent_spmQ_wrap_parens_r_tl } }
2773     }
2774 }

```

(End of definition for `\__spintrent_spmQ_start_wrap_parens:` and `\__spintrent_spmQ_stop_wrap_parens:`)

```
\__spintrent_spmQ_set_join_word:
```

The function `\__spintrent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintrent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintrent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintenc`, which NVDA/MathCAT will verbalize as «*entero*», «*enteros*» or «*y*» respectively.

```

2775 \cs_new_protected:Nn \__spintrent_spmQ_set_join_word:
2776 {
2777     \str_case:Nn \l__spintrent_spmQ_join_word_str
2778     {
2779         { entero } { \MathMLintenc { _entero } { \c__spintrent_invisible_sep_int } }
2780         { enteros } { \MathMLintenc { _enteros } { \c__spintrent_invisible_sep_int } }
2781         { y } { \MathMLintenc { _y- } { \c__spintrent_invisible_sep_int } }

```

```

2782     }
2783 }

```

(End of definition for `\__spintrent_spmQ_set_join_word:`.)

`\__spintrent_spmQ_print_scale_int:`

The function `\__spintrent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintrent_luafun_spmQ_sca` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintrent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintrent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintrent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `use-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2784 \cs_new_protected:Nn \__spintrent_spmQ_print_scale_int:
2785 {
2786   \int_set_eq:Nc \l__spintrent_saved_luamml_flag_int { l__luamml_flag_int }
2787   \int_set:Nn \l__luamml_flag_int { 0 }
2788   \bool_if:NTF \l__spintrent_spmQ_use_dfrac_bool
2789     { \__spintrent_luafun_spmQ_scale_int:Vn \l__spintrent_luaset_part_int_tl { dfrac } }
2790     { \__spintrent_luafun_spmQ_scale_int:Vn \l__spintrent_luaset_part_int_tl { frac } }
2791   \int_set_eq:cN { l__luamml_flag_int } \l__spintrent_saved_luamml_flag_int
2792 }

```

(End of definition for `\__spintrent_spmQ_print_scale_int:`.)

`\__spintrent_spmQ_print_scale_int:n`

The function `\__spintrent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘`#1`’. It first calls `\__spintrent_luafun_clean_split_and_set:n` on ‘`#1`’ to parse and populate the shared Lua variables, then checks the status of `\l__spintrent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintrent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintrent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintrent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\__spintrent_invisible_sep:` call between the sign and the `\MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2793 \cs_new_protected:Nn \__spintrent_spmQ_print_scale_int:n
2794 {
2795   \__spintrent_luafun_clean_split_and_set:n { \exp_not:n { #1 } }
2796   \str_if_eq:VnTF \l__spintrent_luaset_has_integer_str { true }
2797   {
2798     \tl_if_empty:NTF \l__spintrent_luaset_sign_tl
2799     {
2800       \__spintrent_spmQ_print_scale_int:
2801     }
2802     {
2803       \str_if_eq:VnTF \l__spintrent_luaset_sign_tl { + }
2804       {
2805         \str_set:Nn
2806           \l__spintrent_spmQ_intent_read_sign_str { _más:prefix($e) }
2807       }
2808       {
2809         \str_set:Nn
2810           \l__spintrent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2811       }
2812       \MathMLintent { \l__spintrent_spmQ_intent_read_sign_str }
2813       {
2814         \l__spintrent_luaset_sign_tl \c__spintrent_invisible_sep_int % \__spintrent_invisible_s
2815         \MathMLarg { e } { \__spintrent_spmQ_print_scale_int: }
2816       }
2817     }
2818   }
2819   { \msg_error:nnn { spintrent } { spmQ-not-integer } { #1 } }
2820 }

```

(End of definition for `\__spintrent_spmQ_print_scale_int:n`.)

`\__spintent_spmQ_print_spnZ_dfrac:nn`

The function `\__spintent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’, both already validated by `\__spintent_spmQ_check_part:nn`. The function dispatches on the two boolean variables `\l__spintent_spmQ_use_spnZ_bool` and `\l__spintent_spmQ_use_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `use-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `use-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```
2821 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2822 {
2823   \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
2824   {
2825     \use:c { \bool_if:NT \l__spintent_spmQ_use_dfrac_bool { d } frac }
2826     { \spnZ { #1 } } { \spnZ { #2 } }
2827   }
2828   {
2829     \use:c { \bool_if:NT \l__spintent_spmQ_use_dfrac_bool { d } frac }
2830     { #1 } { #2 }
2831   }
2832 }
```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `use-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```
2833 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2834 {
2835   \__spintent_spmQ_start_wrap_parens:
2836   \__spintent_spmQ_print_scale_int:n { #1 }
2837   \__spintent_spmQ_set_join_word:
2838   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2839   \__spintent_spmQ_stop_wrap_parens:
2840 }
```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_check_part:nn`

The function `\__spintent_spmQ_check_part:nn` validates one argument of `\spmQ`. It receives the role of the argument in ‘#1’ (int, num or den) and the argument itself in ‘#2’, and calls `\__spintent_luafun_clean_split_a` on it. If `\l__spintent_luafun_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised. If the argument is an integer but carries a sign and its role is not `int`, the `spmQ-signed-part` “error” is raised. If the role is `den` and the value is zero, the `spmQ-zero-denominator` “error” is raised. Every rejected argument sets `\l__spintent_spmQ_error_bool` to `true`; the arguments are rejected whatever the value of the key `silent`.

```
2841 \cs_new_protected:Nn \__spintent_spmQ_check_part:nn
2842 {
2843   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2844   \str_if_eq:VnTF \l__spintent_luafun_has_integer_str { true }
2845   {
2846     \bool_lazy_or:nnTF
2847     { \tl_if_empty_p:N \l__spintent_luafun_sign_tl }
2848     { \str_if_eq_p:nn {#1} { int } }
2849     {
2850       \str_if_eq:nnT {#1} { den }
2851       {
2852         \int_compare:nNnT { \l__spintent_luafun_part_int_tl } = { 0 }
2853         {
2854           \msg_error:nn { spintent } { spmQ-zero-denominator }
2855           \bool_set_true:N \l__spintent_spmQ_error_bool
2856         }
2857       }
2858     }
2859     {
2860       \msg_error:nnn { spintent } { spmQ-signed-part } { #2 }
2861       \bool_set_true:N \l__spintent_spmQ_error_bool
2862     }
2863   }
```

```

2864     {
2865         \msg_error:nnn { spintent } { spmQ-not-integer } { #2 }
2866         \bool_set_true:N \l__spintent_spmQ_error_bool
2867     }
2868 }

```

(End of definition for `\l__spintent_spmQ_check_part:nn`.)

`\l__spintent_spmQ_exec:nnnn`

The internal function `\l__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then validates the three arguments with `\l__spintent_spmQ_check_part:nn`. If any of them is rejected, nothing is typeset.

Otherwise, it calls `\l__spintent_spmQ_print:nnn` to format the “mixed number”. With the key `silent` set to `true`, the whole “mixed number” is wrapped in a single `\MathMLintent{_-}`, preceded by the outer invisible separator and with the inner one first inside it, so MathCAT verbalizes nothing.

```

2869 \cs_new_protected:Nn \l__spintent_spmQ_exec:nnnn
2870 {
2871     \keys_set:nn { spintent / spmQ } { #1 }
2872     \bool_set_false:N \l__spintent_spmQ_error_bool
2873     \l__spintent_spmQ_check_part:nn { int } { #2 }
2874     \l__spintent_spmQ_check_part:nn { num } { #3 }
2875     \l__spintent_spmQ_check_part:nn { den } { #4 }
2876     \bool_if:NF \l__spintent_spmQ_error_bool
2877     {
2878         \bool_if:NTF \l__spintent_spmQ_silent_bool
2879         {
2880             \l__spintent_invisible_sep:
2881             \MathMLintent { _- }
2882             {
2883                 \l__spintent_inner_sep:
2884                 \l__spintent_spmQ_print:nnn { #2 } { #3 } { #4 }
2885             }
2886         }
2887         { \l__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2888     }
2889 }

```

(End of definition for `\l__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\l__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```

2890 \NewDocumentCommand \spmQ { O{} m m m }
2891 {
2892     \mode_if_math:TF
2893     {
2894         \group_begin:
2895         \l__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2896         \group_end:
2897     }
2898     { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2899 }

```

(End of definition for `\spmQ`. This function is documented on page 17.)

13.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaset_error_str`, `\l__spintent_geo_luaset_intent_str`, `\l__spintent_geo_luaset_print_tl`, `\l__spintent_geo_luaset_sub_str` and `\l__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\l__spintent_mathmlarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

13.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str

```

```

2904 \str_new:N \__spintent_geo_luaset_is_name_str
2905 \str_new:N \__spintent_geo_luaset_poly_sym_str
2906 \str_new:N \__spintent_geo_luaset_angulo_case_str
2907 \str_new:N \__spintent_geo_luaset_concept_str
2908 \str_new:N \__spintent_geo_luaset_letra_word_str
2909 \str_new:N \__spintent_geo_luaset_sub_spoken_str
2910 \str_new:N \__spintent_geo_luaset_nombre_es_num_str
2911 \str_new:N \__spintent_geo_luaset_letra_str
2912 \str_new:N \__spintent_geo_luaset_sub_str
2913 \str_new:N \__spintent_geo_luaset_tipo_str

```

(End of definition for `\__spintent_geo_luaset_error_str` and others.)

13.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x, y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as más/menos — never positivo/negativo, which is not how MathCAT reads a bare sign in this position.

13.34.1 Definition of variables for `\spcoord`

Variables set by the function `\__spintent_luafun_spcoord_parse_and_set:n`. The string variable `\__spintent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\__spintent_spcoord_luaset_ #1 _sign_str` is `+`, `-`, or empty; the variables `\__spintent_spcoord_luaset_ #1 _type_str` is `number` or `raw`; the variables `\__spintent_spcoord_luaset_ #1 _value_str` is the value with the sign already removed; and the variables `\__spintent_spcoord_luaset_ #1 _has_frac_str` is `none`, `frac`, or `dfrac`; and the variables `\__spintent_spcoord_luaset_ #1 _composite_str` is `true` when the value is a sole subscripted, superscripted or radical node, `false` otherwise.

```

2914 \str_new:N \__spintent_spcoord_luaset_error_str
2915 \str_new:N \__spintent_spcoord_luaset_sep_str
2916 \clist_map_inline:nn { x , y }
2917 {
2918   \str_new:c { \__spintent_spcoord_luaset_ #1 _sign_str }
2919   \str_new:c { \__spintent_spcoord_luaset_ #1 _type_str }
2920   \str_new:c { \__spintent_spcoord_luaset_ #1 _value_str }
2921   \str_new:c { \__spintent_spcoord_luaset_ #1 _has_frac_str }
2922   \str_new:c { \__spintent_spcoord_luaset_ #1 _composite_str }
2923 }

```

(End of definition for `\__spintent_spcoord_luaset_error_str` and others.)

The variable `\__spintent_spcoord_raw_arg_tl` holds the `{(argument)}` exactly as given. The variable `\__spintent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\__spintent_spcoord_wrap_parens_l_tl`/`\__spintent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\__spintent_spcoord_set_wrap_parens_size:.`

```

2924 \tl_new:N \__spintent_spcoord_raw_arg_tl
2925 \str_new:N \__spintent_spcoord_intent_str
2926 \tl_new:N \__spintent_spcoord_wrap_parens_l_tl
2927 \tl_new:N \__spintent_spcoord_wrap_parens_r_tl

```

(End of definition for `\__spintent_spcoord_raw_arg_tl` and others.)

13.34.2 Definition of error messages for `\spcoord`

spcoord-invalid-arg

```

2928 \msg_new:nnn { spintent } { spcoord-invalid-arg }
2929 {
2930   Invalid~argument~'#1'~for~\token_to_str:N \spcoord \c_space_tl \msg_line_context:.
2931 }

```

(End of definition for `spcoord-invalid-arg`.)

13.34.3 Definition of keys for `\spcoord`

We define the key `silent`.

```

2932 \keys_define:nn { spintent / spcoord }
2933 {
2934   silent .bool_set:N = \__spintent_spcoord_silent_bool,
2935   silent .initial:n = false,
2936   silent .value_required:n = true,
2937   unknown .code:n =
2938     {
2939       \str_set:Nn \__spintent_command_name_str { spcoord }

```

```

2940     \__spintrent_unknown_keys_cmd:n {#1}
2941   },
2942 }

```

(End of definition for *silent*.)

13.34.4 Internal functions for \spcoord

__spintrent_spcoord_set_wrap_parens_size:

With neither side carrying a fraction, the parentheses stay unsized. A fraction counts wherever it appears in the value (`\frac`, `\dfrac` or a `\spmQ`, which prints one), except inside a subscript or superscript, where it does not make the line taller. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `\dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2943 \cs_new_protected:Nn \__spintrent_spcoord_set_wrap_parens_size:
2944 {
2945   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \lparen }
2946   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \rparen }
2947   \bool_if:nT
2948   {
2949     \bool_lazy_or_p:nn
2950     { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { none } } }
2951     { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { none } } }
2952   }
2953   {
2954     \int_case:nnF { \mathstyle }
2955     {
2956       { 0 }
2957       {
2958         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2959         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2960       }
2961       { 1 }
2962       {
2963         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2964         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2965       }
2966       { 2 }
2967       {
2968         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2969         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2970       }
2971       { 3 }
2972       {
2973         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2974         \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2975       }
2976     }
2977   }
2978   \bool_if:nT
2979   {
2980     \bool_lazy_or_p:nn
2981     { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { dfraction } }
2982     { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { dfraction } }
2983   }
2984   {
2985     \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2986     \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2987   }
2988 }
2989 }

```

(End of definition for `\__spintrent_spcoord_set_wrap_parens_size:`.)

__spintrent_spcoord_value:n

With `\l__spintrent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\l__spintrent_spcoord_luaset_ #1 _value_str`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘`#1`’.

```

2990 \cs_new_protected:Nn \__spintrent_spcoord_value:n
2991 {
2992   \str_if_eq:eeTF
2993   { \str_use:c { \l__spintrent_spcoord_luaset_ #1 _type_str } } { number }
2994   {
2995     \exp_args:Ne \spnum { \str_use:c { \l__spintrent_spcoord_luaset_ #1 _value_str } }
2996   }

```

```

2997     { \str_use:c { l__spintent_spcoord_luaset_ #1 _value_str } }
2998   }

```

(End of definition for `\__spintent_spcoord_value:n`.)

`\__spintent_spcoord_print:n`

With `\l__spintent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintent_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read as más/menos`. With no sign, delegates directly to `\__spintent_spcoord_value:n`. Receives `x` or `y` in `'#1'`.

```

2999 \cs_new_protected:Nn \__spintent_spcoord_print:n
3000 {
3001   \str_case:enF { \str_use:c { l__spintent_spcoord_luaset_ #1 _sign_str } }
3002   {
3003     { + }
3004     {
3005       \MathMLint { _más:prefix ($vp) }
3006       {
3007         \str_use:c { l__spintent_spcoord_luaset_ #1 _sign_str }
3008         \MathMLarg{ vp } { \__spintent_spcoord_value:n { #1 } }
3009       }
3010     }
3011     { - }
3012     {
3013       \MathMLint { _menos :prefix($vn) }
3014       {
3015         \str_use:c { l__spintent_spcoord_luaset_ #1 _sign_str }
3016         \MathMLarg{ vn } { \__spintent_spcoord_value:n { #1 } }
3017       }
3018     }
3019   }
3020   { \__spintent_spcoord_value:n { #1 } }
3021 }

```

(End of definition for `\__spintent_spcoord_print:n`.)

`\__spintent_spcoord_inner_sep:n`

Receives `x` or `y` in `'#1'`. With `\l__spintent_spcoord_luaset_ #1 _composite_str` equal to `true`, places `\__spintent_inner_sep:` first, so that the group `\MathMLarg` wraps holds two children and keeps its `arg`: `luamml` drops it from a sole node that carries a subscript, a superscript or a radical.

```

3022 \cs_new_protected:Nn \__spintent_spcoord_inner_sep:n
3023 {
3024   \str_if_eq:eeT
3025   { \str_use:c { l__spintent_spcoord_luaset_ #1 _composite_str } } { true }
3026   { \__spintent_inner_sep: }
3027 }

```

(End of definition for `\__spintent_spcoord_inner_sep:n`.)

`\__spintent_spcoord_intent_infix:`

Sets `\l__spintent_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\l__spintent_spcoord_intent_str`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintent_spcoord_print:n`'s own output.

```

3028 \cs_new_protected:Nn \__spintent_spcoord_intent_infix:
3029 {
3030   \str_if_eq:VnTF \l__spintent_spcoord_luaset_sep_str { ; }
3031   { \str_set:Nn \l__spintent_spcoord_intent_str { _punto-y-coma:infix } }
3032   { \str_set:Nn \l__spintent_spcoord_intent_str { _coma:infix } }
3033   \MathMLint{ \l__spintent_spcoord_intent_str ($x, $y)}
3034   {
3035     \MathMLarg{x}{ \__spintent_spcoord_inner_sep:n { x } \__spintent_spcoord_print:n { x } }
3036     \MathMLint { :silent } { \l__spintent_spcoord_luaset_sep_str }
3037     \MathMLarg{y}{ \__spintent_spcoord_inner_sep:n { y } \__spintent_spcoord_print:n { y } }
3038   }
3039 }

```

(End of definition for `\__spintent_spcoord_intent_infix:.`)

`\__spintent_spcoord_mathml:`

With `silent=true`, everything — parentheses and argument alike — is wrapped in a single `_`, reading nothing at all: `\l__spintent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:.` Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```

3040 \cs_new_protected:Nn \__spintent_spcoord_mathml:
3041 {
3042   \bool_if:NTF \l__spintent_spcoord_silent_bool

```

```

3043     {
3044       \MathMLintent { _- }
3045       {
3046         \l__spinttent_spcoord_wrap_parens_l_tl
3047         \l__spinttent_spcoord_raw_arg_tl
3048         \l__spinttent_spcoord_wrap_parens_r_tl
3049       }
3050     }
3051     {
3052       \MathMLintent { :silent } { \l__spinttent_spcoord_wrap_parens_l_tl }
3053       \l__spinttent_spcoord_intent_infix:
3054       \MathMLintent { :silent } { \l__spinttent_spcoord_wrap_parens_r_tl }
3055     }
3056   }

```

(End of definition for `\l__spinttent_spcoord_mathml:`)

`\l__spinttent_spcoord_exec:nn` Sets the `(keys)` in `'#1'` and `\l__spinttent_spcoord_raw_arg_tl` to `'#2'`. Calls `\l__spinttent_luafun_spcoord_parse_and_set:n` and reports `spcoord-invalid-arg` on error, whatever the value of `silent`: an invalid `{(argument)}` is always rejected. Then sizes the parentheses, and delegates to `\l__spinttent_spcoord_mathml:` unless there was an error, preceded by an invisible separator — placed *before* the opening parenthesis, as `\l__spinttent_svec_exec:nnn` and `\l__spinttent_spPoint_exec:nnn` do — and the whole pair is wrapped in `\mathord`. The wrapper is what keeps MathCAT from reading `(1,2)` as an interval when an equals sign or `\in` follows it; a silent separator after the closing parenthesis does not help, as it makes the relation disappear.

```

3057 \cs_new_protected:Nn \l__spinttent_spcoord_exec:nn
3058   {
3059     \keys_set:nn { spinttent / spcoord } {#1}
3060     \tl_set:Nn \l__spinttent_spcoord_raw_arg_tl {#2}
3061     \l__spinttent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
3062     \str_if_eq:VnT \l__spinttent_spcoord_luaset_error_str { true }
3063       { \msg_error:nnn { spinttent } { spcoord-invalid-arg } { \exp_not:n {#2} } }
3064     \l__spinttent_spcoord_set_wrap_parens_size:
3065     \str_if_eq:VnF \l__spinttent_spcoord_luaset_error_str { true }
3066       {
3067         \mathord
3068         {
3069           \MathMLintent { :silent } { \c__spinttent_invisible_sep_int } % previene lectura de inter
3070           \l__spinttent_spcoord_mathml:
3071         }
3072       }
3073   }

```

(End of definition for `\l__spinttent_spcoord_exec:nn`)

`\spcoord` The public command enforces *math mode*, opens a local group, and delegates to `\l__spinttent_spcoord_exec:nn`.

```

3074 \NewDocumentCommand \spcoord { 0{} m }
3075   {
3076     \mode_if_math:TF
3077     {
3078       \group_begin:
3079       \attribute \l__spinttent_scope_attr_int = 1
3080       \l__spinttent_spcoord_exec:nn {#1} {#2}
3081       \group_end:
3082     }
3083     { \msg_error:nnn { spinttent } { need-math-mode } { \spcoord } }
3084   }

```

(End of definition for `\spcoord`. This function is documented on page 19.)

13.35 The command `\spinterval`

The user command `\spinterval` receives the `{(argument)}` written as a left delimiter `(, [` or `]`, two endpoints separated by a semicolon or by a single comma, and a right delimiter `), ]` or `[`. A delimiter `(` or `]` on the left, and `)` or `[` on the right, excludes its endpoint; the others include it. The type of interval, the endpoints and the separator are automatically detected by the Lua module.

13.35.1 Definition of variables for \spinterval

Variables set by the function `\__spintrent_luafun_spinterval_parse_and_set:n`.

```

3085 \str_new:N \__spintrent_spinterval_luaset_error_str
3086 \str_new:N \__spintrent_spinterval_luaset_error_code_str
3087 \str_new:N \__spintrent_spinterval_luaset_left_sym_str
3088 \str_new:N \__spintrent_spinterval_luaset_right_sym_str
3089 \str_new:N \__spintrent_spinterval_luaset_left_open_str
3090 \str_new:N \__spintrent_spinterval_luaset_right_open_str
3091 \str_new:N \__spintrent_spinterval_luaset_type_str
3092 \str_new:N \__spintrent_spinterval_luaset_sep_str
3093 \str_new:N \__spintrent_spinterval_luaset_tall_str

```

(End of definition for `\__spintrent_spinterval_luaset_error_str` and others.)

The same variables for each endpoint, a and b.

```

3094 \clist_map_inline:nn { a , b }
3095 {
3096   \str_new:c { \__spintrent_spinterval_luaset_ #1 _sign_str }
3097   \str_new:c { \__spintrent_spinterval_luaset_ #1 _type_str }
3098   \str_new:c { \__spintrent_spinterval_luaset_ #1 _value_str }
3099   \str_new:c { \__spintrent_spinterval_luaset_ #1 _inf_str }
3100   \str_new:c { \__spintrent_spinterval_luaset_ #1 _tall_str }
3101   \str_new:c { \__spintrent_spinterval_luaset_ #1 _composite_str }
3102 }

```

(End of definition for `\__spintrent_spinterval_luaset_ #1 _sign_str` and others.)

The variable `\__spintrent_spinterval_raw_arg_tl` stores the $\{\langle argument \rangle\}$ as it was received. The variables `\__spintrent_spinterval_mode_str`, `\__spintrent_spinterval_prefix_tl`, `\__spintrent_spinterval_read_arg_tl` and `\__spintrent_spinterval_size_str` store the values set by the keys `read-cmd`, `prefix`, `read-arg` and `size`, and `\__spintrent_spinterval_reason_str` the text of the error reason. The variables `\__spintrent_spinterval_word_w_tl` and `\__spintrent_spinterval_word_g_tl` store the word that names the interval and the word that joins its endpoints, and `\__spintrent_spinterval_intent_arg_tl` and `\__spintrent_spinterval_intent_group_tl` the intents of the whole interval and of the group of endpoints. The variables `\__spintrent_spinterval_sym_l_tl`, `\__spintrent_spinterval_sym_r_tl`, `\__spintrent_spinterval_size_l_tl`, `\__spintrent_spinterval_size_r_tl`, `\__spintrent_spinterval_delim_l_tl` and `\__spintrent_spinterval_delim_r_tl` store the delimiter symbols, their size commands and the delimiters ready to be typeset; `\__spintrent_spinterval_tmp_tl` is a scratch variable.

```

3103 \tl_new:N \__spintrent_spinterval_raw_arg_tl
3104 \tl_new:N \__spintrent_spinterval_prefix_tl
3105 \tl_new:N \__spintrent_spinterval_read_arg_tl
3106 \str_new:N \__spintrent_spinterval_mode_str
3107 \str_new:N \__spintrent_spinterval_size_str
3108 \str_new:N \__spintrent_spinterval_reason_str
3109 \tl_new:N \__spintrent_spinterval_word_w_tl
3110 \tl_new:N \__spintrent_spinterval_word_g_tl
3111 \tl_new:N \__spintrent_spinterval_intent_arg_tl
3112 \tl_new:N \__spintrent_spinterval_intent_group_tl
3113 \tl_new:N \__spintrent_spinterval_sym_l_tl
3114 \tl_new:N \__spintrent_spinterval_sym_r_tl
3115 \tl_new:N \__spintrent_spinterval_size_l_tl
3116 \tl_new:N \__spintrent_spinterval_size_r_tl
3117 \tl_new:N \__spintrent_spinterval_delim_l_tl
3118 \tl_new:N \__spintrent_spinterval_delim_r_tl
3119 \tl_new:N \__spintrent_spinterval_tmp_tl

```

(End of definition for `\__spintrent_spinterval_raw_arg_tl` and others.)

13.35.2 Definition of error messages for \spinterval

The “error” `spinterval-invalid-arg` is raised when the Lua module rejects the $\{\langle argument \rangle\}$; it reports the argument and the reason set by `\__spintrent_spinterval_set_reason:.`

```

3120 \msg_new:nnnn { spintrent } { spinterval-invalid-arg }
3121 {
3122   Invalid~argument~'!~for~\token_to_str:N \spinterval \c_space_tl (#2)~\msg_line_context:.
3123 }
3124 {
3125   See~the~documentation~of~\token_to_str:N \spinterval \c_space_tl for~the~accepted~argument~forms.
3126 }

```

(End of definition for `spinterval-invalid-arg`.)

13.35.3 Definition of keys for \spinterval

We define the *(keys)* read-cmd, prefix, read-arg, size, silent and concept.

```

3127 \keys_define:nn { spintent / spinterval }
3128 {
3129   read-cmd      .choices:nn = { school , mathcat }
3130   {
3131     \int_case:nn { \l_keys_choice_int }
3132     {
3133       {1} { \str_set:Nn \l__spintent_spinterval_mode_str { school } }
3134       {2} { \str_set:Nn \l__spintent_spinterval_mode_str { mathcat } }
3135     }
3136   },
3137   read-cmd      .initial:n = school,
3138   read-cmd      .value_required:n = true,
3139   read-cmd / unknown .code:n =
3140     \msg_error:nneee { spintent } { unknown-choice }
3141     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3142   prefix        .code:n =
3143     { \__spintent_sanitizize_affix:Nn \l__spintent_spinterval_prefix_tl {#1} },
3144   prefix        .initial:n = ,
3145   prefix        .value_required:n = true,
3146   read-arg      .code:n =
3147     { \__spintent_sanitizize_affix:Nn \l__spintent_spinterval_read_arg_tl {#1} },
3148   read-arg      .initial:n = ,
3149   read-arg      .value_required:n = true,
3150   size          .choices:nn = { auto , none , big , Big , bigg , Bigg }
3151   {
3152     \int_case:nn { \l_keys_choice_int }
3153     {
3154       {1} { \str_set:Nn \l__spintent_spinterval_size_str { auto } }
3155       {2} { \str_set:Nn \l__spintent_spinterval_size_str { none } }
3156       {3} { \str_set:Nn \l__spintent_spinterval_size_str { big } }
3157       {4} { \str_set:Nn \l__spintent_spinterval_size_str { Big } }
3158       {5} { \str_set:Nn \l__spintent_spinterval_size_str { bigg } }
3159       {6} { \str_set:Nn \l__spintent_spinterval_size_str { Bigg } }
3160     }
3161   },
3162   size          .initial:n = auto,
3163   size          .value_required:n = true,
3164   size / unknown .code:n =
3165     \msg_error:nneee { spintent } { unknown-choice }
3166     { size } { auto,~none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
3167   silent        .bool_set:N = \l__spintent_spinterval_silent_bool,
3168   silent        .initial:n = false,
3169   silent        .value_required:n = true,
3170   concept       .bool_set:N = \l__spintent_spinterval_concept_bool,
3171   concept       .initial:n = true,
3172   concept       .value_required:n = true,
3173   unknown       .code:n =
3174     {
3175       \str_set:Nn \l__spintent_command_name_str { spinterval }
3176       \__spintent_unknown_keys_cmd:n {#1}
3177     },
3178 }

```

(End of definition for read-cmd and others.)

13.35.4 Internal functions for \spinterval

`\__spintent_spinterval_set_reason:` The internal function `\__spintent_spinterval_set_reason:` stores in `\l__spintent_spinterval_reason_str` the text that describes the error code returned by the Lua module, or invalid argument when the code is not known.

```

3179 \cs_new_protected:Nn \__spintent_spinterval_set_reason:
3180 {
3181   \str_case:VnF \l__spintent_spinterval_luaset_error_code_str
3182   {
3183     { delim }
3184     {
3185       \str_set:Nn \l__spintent_spinterval_reason_str
3186       { it~must~start~with~'(' ,~'['~or~'{'~and~end~with~')' ,~']'~or~'}' }
3187     }

```

```

3188 { vacio }
3189 { \str_set:Nn \l__spintent_spinterval_reason_str { an~endpoint~is~empty } }
3190 { sep }
3191 {
3192   \str_set:Nn \l__spintent_spinterval_reason_str
3193   { there~is~no~separator:~use~a~comma~or~a~semicolon }
3194 }
3195 { ambiguo }
3196 {
3197   \str_set:Nn \l__spintent_spinterval_reason_str
3198   { more~than~one~separator:~use~a~single~semicolon~and~the~commas~are~decimal }
3199 }
3200 { llaves }
3201 { \str_set:Nn \l__spintent_spinterval_reason_str { unbalanced~braces } }
3202 { inf-orden }
3203 {
3204   \str_set:Nn \l__spintent_spinterval_reason_str
3205   { minus~infinity~goes~on~the~left~and~infinity~on~the~right }
3206 }
3207 { inf-cerrado }
3208 {
3209   \str_set:Nn \l__spintent_spinterval_reason_str
3210   { an~infinite~endpoint~cannot~be~included:~use~'('~or~')'~on~the~left,~')'~or~'['~or~']' }
3211 }
3212 }
3213 { \str_set:Nn \l__spintent_spinterval_reason_str { invalid~argument } }
3214 }

```

(End of definition for `\l__spintent_spinterval_set_reason:`)

`\l__spintent_spinterval_set_delims:`

The internal function `\l__spintent_spinterval_set_delims:` sets the symbols of the left and right delimiters from the characters returned by the Lua module, and their size commands from the key `size`. With `big`, `Big`, `bigg` and `Bigg` the size is fixed, and with `none` there is no size command. With `auto` there is none unless an endpoint contains a fraction, in which case it is `\Bigl/\Bigr` in display style and `\bigl/\bigr` in text style, and always `\Bigl/\Bigr` when the fraction is a `\dfrac`. Without a size command each delimiter is wrapped in `\mathopen` or `\mathclose`; with one, the delimiter follows the size command.

```

3215 \cs_new_protected:Nn \l__spintent_spinterval_set_delims:
3216 {
3217   \str_case:Nn \l__spintent_spinterval_luasets_left_sym_str
3218   {
3219     { ( ) } { \tl_set:Nn \l__spintent_spinterval_sym_l_tl { \lparen } }
3220     { [ ] } { \tl_set:Nn \l__spintent_spinterval_sym_l_tl { \lbrack } }
3221     { } } { \tl_set:Nn \l__spintent_spinterval_sym_l_tl { \rbrack } }
3222   }
3223   \str_case:Nn \l__spintent_spinterval_luasets_right_sym_str
3224   {
3225     { ) } { \tl_set:Nn \l__spintent_spinterval_sym_r_tl { \rparen } }
3226     { [ ] } { \tl_set:Nn \l__spintent_spinterval_sym_r_tl { \lbrack } }
3227     { } } { \tl_set:Nn \l__spintent_spinterval_sym_r_tl { \rbrack } }
3228   }
3229   \tl_clear:N \l__spintent_spinterval_size_l_tl
3230   \tl_clear:N \l__spintent_spinterval_size_r_tl
3231   \str_case:Nn \l__spintent_spinterval_size_str
3232   {
3233     { big }
3234     {
3235       \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3236       \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3237     }
3238     { Big }
3239     {
3240       \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3241       \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3242     }
3243     { bigg }
3244     {
3245       \tl_set:Nn \l__spintent_spinterval_size_l_tl { \biggl }
3246       \tl_set:Nn \l__spintent_spinterval_size_r_tl { \biggr }
3247     }
3248     { Bigg }
3249     {

```

```

3250     \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Biggl }
3251     \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Biggr }
3252   }
3253   { auto }
3254   {
3255     \str_if_eq:VnF \l__spintent_spinterval_luaset_tall_str { none }
3256     {
3257       \int_case:nnF { \mathstyle }
3258       {
3259         { 0 }
3260         {
3261           \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3262           \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3263         }
3264         { 1 }
3265         {
3266           \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3267           \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3268         }
3269         { 2 }
3270         {
3271           \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3272           \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3273         }
3274         { 3 }
3275         {
3276           \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3277           \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3278         }
3279       }
3280     }
3281     \str_if_eq:VnT \l__spintent_spinterval_luaset_tall_str { dfraction }
3282     {
3283       \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3284       \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3285     }
3286   }
3287 }
3288 }
3289 \tl_if_empty:NTF \l__spintent_spinterval_size_l_tl
3290 {
3291   \tl_set:Nn \l__spintent_spinterval_delim_l_tl
3292   { \exp_not:N \mathopen { \exp_not:V \l__spintent_spinterval_sym_l_tl } }
3293   \tl_set:Nn \l__spintent_spinterval_delim_r_tl
3294   { \exp_not:N \mathclose { \exp_not:V \l__spintent_spinterval_sym_r_tl } }
3295 }
3296 {
3297   \tl_set:Nn \l__spintent_spinterval_delim_l_tl
3298   { \exp_not:V \l__spintent_spinterval_size_l_tl \exp_not:V \l__spintent_spinterval_sym_l_tl }
3299   \tl_set:Nn \l__spintent_spinterval_delim_r_tl
3300   { \exp_not:V \l__spintent_spinterval_size_r_tl \exp_not:V \l__spintent_spinterval_sym_r_tl }
3301 }
3302 }

```

(End of definition for `\__spintent_spinterval_set_delims:`)

`\__spintent_spinterval_set_intents:`

The internal function `\__spintent_spinterval_set_intents:` sets the two words used in the intents from the type returned by the Lua module: `abierto-de` and `a` for an open interval, `cerrado-de` and `a` for a closed one, `abierto-en` and `y-cerrado-en` for an open-closed one, and `cerrado-en` and `y-abierto-en` for a closed-open one. The intent of the group of endpoints is `<word>:infix($a,$b)` with the second word. The intent of the whole interval is `_` when `silent` is `true`, `_` followed by the value of `read-arg` when it is not empty, and otherwise `<word>:prefix($ab)` with the first word, preceded by the `prefix`, if any, and `intervalo-` when `concept` is `true`.

```

3303 \cs_new_protected:Nn \__spintent_spinterval_set_intents:
3304 {
3305   \str_case:Vn \l__spintent_spinterval_luaset_type_str
3306   {
3307     { open }
3308     {
3309       \tl_set:Nn \l__spintent_spinterval_word_w_tl { abierto-de }
3310       \tl_set:Nn \l__spintent_spinterval_word_g_tl { a }

```

```

3311     }
3312     { closed }
3313     {
3314         \tl_set:Nn \l__spintent_spinterval_word_w_tl { cerrado-de }
3315         \tl_set:Nn \l__spintent_spinterval_word_g_tl { a }
3316     }
3317     { open-closed }
3318     {
3319         \tl_set:Nn \l__spintent_spinterval_word_w_tl { abierto-en }
3320         \tl_set:Nn \l__spintent_spinterval_word_g_tl { y-cerrado-en }
3321     }
3322     { closed-open }
3323     {
3324         \tl_set:Nn \l__spintent_spinterval_word_w_tl { cerrado-en }
3325         \tl_set:Nn \l__spintent_spinterval_word_g_tl { y-abierto-en }
3326     }
3327 }
3328 \tl_set:Nn \l__spintent_spinterval_intent_group_tl
3329 { _ \l__spintent_spinterval_word_g_tl :infix($a,$b) }
3330 \bool_if:NTF \l__spintent_spinterval_silent_bool
3331 { \tl_set:Nn \l__spintent_spinterval_intent_arg_tl { _- } }
3332 {
3333     \tl_if_empty:NTF \l__spintent_spinterval_read_arg_tl
3334     {
3335         \tl_set:Nn \l__spintent_spinterval_intent_arg_tl
3336         {
3337             -
3338             \bool_if:NT \l__spintent_spinterval_concept_bool
3339             {
3340                 \tl_if_empty:NF \l__spintent_spinterval_prefix_tl
3341                 { \tl_use:N \l__spintent_spinterval_prefix_tl - }
3342                 intervalo-
3343             }
3344             \l__spintent_spinterval_word_w_tl :prefix($ab)
3345         }
3346     }
3347     {
3348         \tl_set:Nn \l__spintent_spinterval_intent_arg_tl
3349         { _ \l__spintent_spinterval_read_arg_tl }
3350     }
3351 }
3352 }

```

(End of definition for `\__spintent_spinterval_set_intents:`.)

`\__spintent_spinterval_inner_sep:n`

The internal function `\__spintent_spinterval_inner_sep:n` calls `\__spintent_inner_sep:` when the endpoint a or b passed in ‘#1’ is composite.

```

3353 \cs_new_protected:Nn \__spintent_spinterval_inner_sep:n
3354 {
3355     \str_if_eq:eeT
3356     { \str_use:c { \l__spintent_spinterval_luaset_ #1 _composite_str } } { true }
3357     { \__spintent_inner_sep: }
3358 }

```

(End of definition for `\__spintent_spinterval_inner_sep:n`.)

`\__spintent_spinterval_plain_number:n`

The internal function `\__spintent_spinterval_plain_number:n` typesets the value of the endpoint passed in ‘#1’ as a number, preceded by `\__spintent_inner_sep:` and with each comma protected by braces.

```

3359 \cs_new_protected:Nn \__spintent_spinterval_plain_number:n
3360 {
3361     \__spintent_inner_sep:
3362     \tl_set:Nn \l__spintent_spinterval_tmp_tl
3363     { \str_use:c { \l__spintent_spinterval_luaset_ #1 _value_str } }
3364     \tl_replace_all:Nnn \l__spintent_spinterval_tmp_tl { , } { {,} }
3365     \tl_use:N \l__spintent_spinterval_tmp_tl
3366 }

```

(End of definition for `\__spintent_spinterval_plain_number:n`.)

`\__spintent_spinterval_value:n` The internal function `\__spintent_spinterval_value:n` typesets the value of the endpoint passed in ‘#1’ according to its type: `\infty` for infinity; for a number, `\__spintent_inner_sep:` followed by `\spnum` when `read-cmd` is `school`, or `\__spintent_spinterval_plain_number:n` otherwise; and the value as written for any other type.

```

3367 \cs_new_protected:Nn \__spintent_spinterval_value:n
3368 {
3369   \str_case:enF { \str_use:c { l__spintent_spinterval_luaset_ #1 _type_str } }
3370   {
3371     { infinity } { \infty }
3372     { number }
3373     {
3374       \str_if_eq:VnTF \__spintent_spinterval_mode_str { school }
3375       {
3376         \__spintent_inner_sep:
3377         \exp_args:Ne \spnum { \str_use:c { l__spintent_spinterval_luaset_ #1 _value_str } }
3378       }
3379       { \__spintent_spinterval_plain_number:n { #1 } }
3380     }
3381   }
3382   { \str_use:c { l__spintent_spinterval_luaset_ #1 _value_str } }
3383 }

```

(End of definition for `\__spintent_spinterval_value:n`.)

`\__spintent_spinterval_print:n` The internal function `\__spintent_spinterval_print:n` typesets the endpoint passed in ‘#1’ with its sign. With `read-cmd=school` a sign + or - is wrapped, with its value, in a native `\MathMLintent` with the intent `_más:prefix($vp)` or `_menos:prefix($vn)`, the value tagged with `\MathMLarg` and preceded by `\__spintent_spinterval_inner_sep:n`; an endpoint without sign is typeset through `\__spintent_spinterval_value:n`. With `read-cmd=mathcat` the sign is typeset as is, followed by the value.

```

3384 \cs_new_protected:Nn \__spintent_spinterval_print:n
3385 {
3386   \str_if_eq:VnTF \__spintent_spinterval_mode_str { school }
3387   {
3388     \str_case:enF { \str_use:c { l__spintent_spinterval_luaset_ #1 _sign_str } }
3389     {
3390       { + }
3391       {
3392         \MathMLintent { _más:prefix($vp) }
3393         {
3394           +
3395           \MathMLarg { vp }
3396           { \__spintent_spinterval_inner_sep:n { #1 } \__spintent_spinterval_value:n }
3397         }
3398       }
3399       { - }
3400       {
3401         \MathMLintent { _menos:prefix($vn) }
3402         {
3403           -
3404           \MathMLarg { vn }
3405           { \__spintent_spinterval_inner_sep:n { #1 } \__spintent_spinterval_value:n }
3406         }
3407       }
3408     }
3409     { \__spintent_spinterval_value:n { #1 } }
3410   }
3411   {
3412     \str_use:c { l__spintent_spinterval_luaset_ #1 _sign_str }
3413     \__spintent_spinterval_value:n { #1 }
3414   }
3415 }

```

(End of definition for `\__spintent_spinterval_print:n`.)

`\__spintent_spinterval_mathml:` The internal function `\__spintent_spinterval_mathml:` builds the annotated interval with a native `\MathMLintent` that carries the intent of the whole interval. It contains the left delimiter silenced, a `\MathMLarg` named `ab` that wraps `\__spintent_inner_sep:` and a `\MathMLintent` with the intent of the group of endpoints, and the right delimiter silenced. The group holds the endpoints `a` and `b`, each tagged with `\MathMLarg` and typeset by `\__spintent_spinterval_print:n`, separated by the silenced separator.

```

3416 \cs_new_protected:Nn \__spintent_spinterval_mathml:

```

```

3417 {
3418   \MathMLintent { \l__spinttent_spinterval_intent_arg_tl }
3419   {
3420     \MathMLintent { :silent } { \l__spinttent_spinterval_delim_l_tl }
3421     \MathMLarg { ab }
3422     {
3423       \__spinttent_inner_sep:
3424       \MathMLintent { \l__spinttent_spinterval_intent_group_tl }
3425       {
3426         \MathMLarg { a }
3427         { \__spinttent_spinterval_inner_sep:n { a } \__spinttent_spinterval_print:n { a } }
3428         \MathMLintent { :silent } { \l__spinttent_spinterval_luaset_sep_str }
3429         \MathMLarg { b }
3430         { \__spinttent_spinterval_inner_sep:n { b } \__spinttent_spinterval_print:n { b } }
3431       }
3432     }
3433     \MathMLintent { :silent } { \l__spinttent_spinterval_delim_r_tl }
3434   }
3435 }

```

(End of definition for __spinttent_spinterval_mathml:.)

__spinttent_spinterval_exec:nn

The internal function `\__spinttent_spinterval_exec:nn` receives the optional `(keys)` passed in ‘#1’ and the `{(argument)}` passed in ‘#2’. It calls `\__spinttent_luafun_spinterval_parse_and_set:n`; if the Lua module reports an error, it reports `spinterval-invalid-arg` whatever the value of `silent`: an invalid `{(argument)}` is always rejected. When there is no error, it calls `\__spinttent_spinterval_set_delims:` and `\__spinttent_spinterval_set_intents:`, places `\__spinttent_invisible_sep:` and delegates the rendering to `\__spinttent_spinterval_mathml:`.

```

3436 \cs_new_protected:Nn \__spinttent_spinterval_exec:nn
3437 {
3438   \keys_set:nn { spinttent / spinterval } {#1}
3439   \tl_set:Nn \l__spinttent_spinterval_raw_arg_tl {#2}
3440   \__spinttent_luafun_spinterval_parse_and_set:n { \exp_not:n {#2} }
3441   \str_if_eq:VnTF \l__spinttent_spinterval_luaset_error_str { true }
3442   {
3443     \__spinttent_spinterval_set_reason:
3444     \msg_error:nnee { spinttent } { spinterval-invalid-arg }
3445     { \exp_not:n {#2} } { \l__spinttent_spinterval_reason_str }
3446   }
3447   {
3448     \__spinttent_spinterval_set_delims:
3449     \__spinttent_spinterval_set_intents:
3450     \__spinttent_invisible_sep:
3451     \__spinttent_spinterval_mathml:
3452   }
3453 }

```

(End of definition for __spinttent_spinterval_exec:nn.)

\spinterval

The public command enforces *math mode*, opens a local group, and delegates to `\__spinttent_spinterval_exec:nn`.

```

3454 \NewDocumentCommand \spinterval { 0{ } m }
3455 {
3456   \mode_if_math:TF
3457   {
3458     \group_begin:
3459     \attribute \l__spinttent_scope_attr_int = 1
3460     \__spinttent_spinterval_exec:nn {#1} {#2}
3461     \group_end:
3462   }
3463   { \msg_error:nnn { spinttent } { need-math-mode } { \spinterval } }
3464 }

```

(End of definition for \spinterval. This function is documented on page 34.)

13.36 The command \spfun

The user command `\spfun` typesets a function call such as $f(x)$, $f_{\{1\}}(x)$, $f'(x,y)$. The Lua function `\__spinttent_luafun_spfun_parse_and_set:n` splits the `{(argument)}` into the function name (a letter with an optional subscript or primes, left in the shared `\l_@@_geo_luaset_*` variables) and the list of arguments, cut at the top-level commas, each one stored in `\l_@@_spfun_luaset_arg_<i>_tl` together with a flag `\l_@@_spfun_luaset_simple_<i>_str` telling whether it is a single node. The whole call is tagged with a native `\MathMLintent` whose `intent` is `$f($a1,$a2,...)`; the name is tagged `arg="f"` and

each argument `arg="a<i>"`, while the parentheses and the commas are silenced. The name may carry the inverse exponent `^{-1}` (flag `\l_@@_spfun_luaset_inv_str`); in that case the literal `intent` of the name ends in `-inversa`, the exponent is typeset silenced and the parentheses may be omitted (no arguments, `$f` as the whole `intent`). An argument that is itself a call, `f(g(x))` or `f(g^{ -1 }(y))`, is analysed recursively by Lua (kind `call`) and typeset by `\__spintent_spfun_nested:n`, which runs the same code in a group, one level deeper; the `arg` names carry the depth.

13.36.1 Definition of variables for `\spfun`

The variable `\__spintent_spfun_mode_str` stores the value set by the key `read-cmd` and `\__spintent_spfun_p` the one set by `prefix`. The two `luaset` variables are set from Lua.

```
\__spintent_spfun_mode_str
\__spintent_spfun_prefix_tl
\__spintent_spfun_intent_arg_tl
\__spintent_spfun_kv_tl
\__spintent_spfun_refs_tl
\__spintent_spfun_tmp_str
\__spintent_spfun_nargs_int
\__spintent_spfun_luaset_error_str
\__spintent_spfun_luaset_nargs_str

3465 \str_new:N \__spintent_spfun_mode_str
3466 \tl_new:N \__spintent_spfun_prefix_tl
3467 \tl_new:N \__spintent_spfun_intent_arg_tl
3468 \tl_new:N \__spintent_spfun_kv_tl
3469 \tl_new:N \__spintent_spfun_refs_tl
3470 \str_new:N \__spintent_spfun_tmp_str
3471 \int_new:N \__spintent_spfun_nargs_int
3472 \bool_new:N \__spintent_spfun_silent_bool
3473 \bool_new:N \__spintent_spfun_concept_bool
3474 \str_new:N \__spintent_spfun_luaset_error_str
3475 \str_new:N \__spintent_spfun_luaset_nargs_str
3476 \str_new:N \__spintent_spfun_luaset_inv_str
3477 \tl_new:N \__spintent_spfun_call_tl
3478 \int_new:N \__spintent_spfun_depth_int
```

(End of definition for `\__spintent_spfun_mode_str` and others.)

13.36.2 Definition of error messages for `\spfun`

`spfun-invalid-arg` The “error” `spfun-invalid-arg` is raised when the argument fails to parse: no valid name, no parentheses (except for the inverse function), a malformed or misplaced inverse exponent, unbalanced delimiters, an empty argument or text after the closing parenthesis. It is raised whatever the value of `silent`.

```
3479 \msg_new:nnn { spintent } { spfun-invalid-arg }
3480 {
3481   Invalid~argument~'#1'~for~\token_to_str:N \spfun \c_space_tl \msg_line_context:.
3482 }
```

(End of definition for `spfun-invalid-arg`.)

13.36.3 Definition of keys for `\spfun`

`read-cmd` We define the `(keys)` `read-cmd`, `prefix`, `concept` and `silent`. Unlike the geometry commands, `concept` is `false` by default: *f de x* is already how the function is read.

```
3483 \keys_define:nn { spintent / spfun }
3484 {
3485   read-cmd .choices:nn = { school , mathcat }
3486   {
3487     \int_case:nn { \l_keys_choice_int }
3488     {
3489       {1} { \str_set:Nn \__spintent_spfun_mode_str { school } }
3490       {2} { \str_set:Nn \__spintent_spfun_mode_str { mathcat } }
3491     }
3492   },
3493   read-cmd .initial:n = school,
3494   read-cmd .value_required:n = true,
3495   read-cmd / unknown .code:n =
3496     \msg_error:nneee { spintent } { unknown-choice }
3497     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3498   prefix .code:n =
3499     { \__spintent_sanitize_affix:Nn \__spintent_spfun_prefix_tl {#1} },
3500   prefix .initial:n = ,
3501   prefix .value_required:n = true,
3502   silent .bool_set:N = \__spintent_spfun_silent_bool,
3503   silent .initial:n = false,
3504   silent .value_required:n = true,
3505   concept .bool_set:N = \__spintent_spfun_concept_bool,
3506   concept .initial:n = false,
3507   concept .value_required:n = true,
3508   unknown .code:n =
3509     {
3510       \str_set:Nn \__spintent_command_name_str { spfun }
3511       \__spintent_unknown_keys_cmd:n {#1}
```

```

3512     },
3513 }

```

(End of definition for `read-cmd` and others.)

13.36.4 Internal functions for `\spfun`

`\__spinttent_spfun_fname:` Names of the `arg` attributes at the current nesting depth `\l__spinttent_spfun_depth_int`. At depth 1 they are `f` and `a<i>`; at a deeper level `f<d>` and `a<d>x<i>`, so that the references of a call nested in an argument never clash with those of the call around it.

```

3514 \cs_new:Nn \__spinttent_spfun_fname:
3515 {
3516   f
3517   \int_compare:nNnF { \l__spinttent_spfun_depth_int } = { 1 }
3518     { \int_use:N \l__spinttent_spfun_depth_int }
3519 }
3520 \cs_new:Nn \__spinttent_spfun_aname:n
3521 {
3522   a
3523   \int_compare:nNnF { \l__spinttent_spfun_depth_int } = { 1 }
3524     { \int_use:N \l__spinttent_spfun_depth_int x }
3525   #1
3526 }

```

(End of definition for `\__spinttent_spfun_fname:` and `\__spinttent_spfun_aname:n`.)

`\__spinttent_spfun_name_plain:` Typesets the name without any annotation: the letter, followed by the subscript or by the primes when there are any.

```

3527 \cs_new_protected:Nn \__spinttent_spfun_name_plain:
3528 {
3529   \l__spinttent_geo_luaset_letra_str
3530   \str_case:VnF \l__spinttent_geo_luaset_tipo_str
3531     {
3532       { sub } { \c_math_subscript_token { \l__spinttent_geo_luaset_sub_str } }
3533       { prima } { \l__spinttent_geo_luaset_sub_str }
3534     }
3535     { }
3536   \str_if_eq:VnT \l__spinttent_spfun_luaset_inv_str { true }
3537     { \c_math_superscript_token { -1 } }
3538 }

```

(End of definition for `\__spinttent_spfun_name_plain:.`)

`\__spinttent_spfun_name_school:` For `read-cmd=school` the name is tagged `arg="f"` with a literal `intent(_f-sub-uno-,_F-doble-prima-,...)` through the shared queue, so `MathCAT` neither announces a capital letter nor reads the Unicode double prime as seconds. A subscript is typeset silenced; a prime is not, since it is not spoken on the base. The inverse exponent `^{-1}`, when present, is typeset silenced too: the literal `intent(_f-inversa-)` already says everything.

```

3539 \cs_new_protected:Nn \__spinttent_spfun_name_school:
3540 {
3541   \tl_set:Ne \l__spinttent_spfun_kv_tl
3542     { arg = " \__spinttent_spfun_fname: ", intent = " _ \l__spinttent_geo_luaset_intent_str -" }
3543   \__spinttent_luafun_mrow_intent_arg:nn { } { \l__spinttent_spfun_kv_tl }
3544   \str_case:VnF \l__spinttent_geo_luaset_tipo_str
3545     {
3546       { sub }
3547       {
3548         \l__spinttent_geo_luaset_letra_str
3549         \c_math_subscript_token
3550         { \MathMLintent { :silent } { \l__spinttent_geo_luaset_sub_str } }
3551       }
3552       { prima }
3553       { \l__spinttent_geo_luaset_letra_str \l__spinttent_geo_luaset_sub_str }
3554     }
3555     { \l__spinttent_geo_luaset_letra_str }
3556   \str_if_eq:VnT \l__spinttent_spfun_luaset_inv_str { true }
3557     {
3558       \c_math_superscript_token
3559       { \MathMLintent { :silent } { -1 } }
3560     }
3561 }

```

(End of definition for `\__spinttent_spfun_name_school:.`)

`\__spintent_sfun_name_mathcat:` For `read-cmd=mathcat` the name is tagged `arg="f"` through a native `\MathMLarg`, never mixed with the shared queue for the same name. With a subscript or primes the inner separator keeps the `arg` on a wrapping `<mrow>`.

```

3562 \cs_new_protected:Nn \__spintent_sfun_name_mathcat:
3563 {
3564   \bool_lazy_and:nnTF
3565     { \str_if_eq_p:Vn \__spintent_geo_luaset_tipo_str { } }
3566     { \str_if_eq_p:Vn \__spintent_sfun_luaset_inv_str { false } }
3567     { \exp_args:Ne \MathMLarg { \__spintent_sfun_fname: } { \__spintent_sfun_name_plain: } }
3568     { \exp_args:Ne \MathMLarg { \__spintent_sfun_fname: } { \__spintent_inner_sep: \__spintent
3569   }

```

(End of definition for `\__spintent_sfun_name_mathcat:.`)

`\__spintent_sfun_arg:n` Tags argument number `#1` with `arg="a<i>"` (see `\__spintent_sfun_aname:n`). A single node (letter, number, a command such as `\alpha`) carries its `arg` directly; any other argument starts with the inner separator, which forces an `<mrow>` to hold the `arg`. An argument that is itself a call (`g(x)`, `g^{ -1 }(y)`) is typeset by `\__spintent_sfun_nested:n`.

```

3570 \cs_new_protected:Nn \__spintent_sfun_arg:n
3571 {
3572   \str_set_eq:Nc \__spintent_sfun_tmp_str { \__spintent_sfun_luaset_simple_ #1 _str }
3573   \str_case:VnF \__spintent_sfun_tmp_str
3574     {
3575       { true }
3576       {
3577         \exp_args:Ne \MathMLarg { \__spintent_sfun_aname:n {#1} }
3578         { \tl_use:c { \__spintent_sfun_luaset_arg_ #1 _tl } }
3579       }
3580       { call }
3581       {
3582         \exp_args:Ne \MathMLarg { \__spintent_sfun_aname:n {#1} }
3583         { \__spintent_inner_sep: \__spintent_sfun_nested:n {#1} }
3584       }
3585     }
3586     {
3587       \exp_args:Ne \MathMLarg { \__spintent_sfun_aname:n {#1} }
3588       { \__spintent_inner_sep: \tl_use:c { \__spintent_sfun_luaset_arg_ #1 _tl } }
3589     }
3590 }

```

(End of definition for `\__spintent_sfun_arg:n`.)

`\__spintent_sfun_args:` Walks the arguments, silencing the comma between them.

```

3591 \cs_new_protected:Nn \__spintent_sfun_args:
3592 {
3593   \int_step_inline:nn { \__spintent_sfun_nargs_int }
3594   {
3595     \int_compare:nNnF {##1} = {1}
3596     { \MathMLintent { :silent } { , } }
3597     \__spintent_sfun_arg:n {##1}
3598   }
3599 }

```

(End of definition for `\__spintent_sfun_args:.`)

`\__spintent_sfun_args_plain:` Typesets the arguments with no annotation (`silent`).

```

3600 \cs_new_protected:Nn \__spintent_sfun_args_plain:
3601 {
3602   \int_step_inline:nn { \__spintent_sfun_nargs_int }
3603   {
3604     \int_compare:nNnF {##1} = {1} { , }
3605     \tl_use:c { \__spintent_sfun_luaset_arg_ ##1 _tl }
3606   }
3607 }

```

(End of definition for `\__spintent_sfun_args_plain:.`)

`\__spintent_sfun_set_intent:` Builds the concept `intent` in `\__spintent_sfun_intent_arg_tl: $f($a1,$a2,...)` when `concept` is `false`, and `_prefix-función:prefix($f($a1,$a2,...))` otherwise. Without arguments (the inverse function written bare) the call is just `$f`.

```

3608 \cs_new_protected:Nn \__spintent_sfun_set_intent:

```

```

3609 {
3610   \tl_clear:N \l__spintent_sfun_refs_tl
3611   \int_step_inline:nn { \l__spintent_sfun_nargs_int }
3612   {
3613     \tl_put_right:Ne \l__spintent_sfun_refs_tl
3614     { \int_compare:nNnF {##1} = {1} { , } $ \l__spintent_sfun_aname:n {##1} }
3615   }
3616   \int_compare:nNnTF { \l__spintent_sfun_nargs_int } = { 0 }
3617   { \tl_set:Ne \l__spintent_sfun_call_tl { $ \l__spintent_sfun_fname: } }
3618   { \tl_set:Ne \l__spintent_sfun_call_tl { $ \l__spintent_sfun_fname: ( \l__spintent_sfun_r
3619   \bool_if:NTF \l__spintent_sfun_concept_bool
3620   {
3621     \tl_set:Ne \l__spintent_sfun_intent_arg_tl
3622     {
3623       -
3624       \tl_if_empty:NF \l__spintent_sfun_prefix_tl
3625       { \tl_use:N \l__spintent_sfun_prefix_tl - }
3626       función:prefix( \l__spintent_sfun_call_tl )
3627     }
3628   }
3629   {
3630     \tl_set_eq:NN \l__spintent_sfun_intent_arg_tl \l__spintent_sfun_call_tl
3631   }
3632 }

```

(End of definition for `\l__spintent_sfun_set_intent:`)

`\l__spintent_sfun_mathml:` Calls `\l__spintent_invisible_sep:` first, then wraps name, parentheses and arguments in a native `\MathMLintent`. With `silent` set to `true` the intent is `-` and nothing inside is annotated.

```

3633 \cs_new_protected:Nn \l__spintent_sfun_mathml:
3634 {
3635   \l__spintent_invisible_sep:
3636   \l__spintent_sfun_body:
3637 }

```

(End of definition for `\l__spintent_sfun_mathml:`)

`\l__spintent_sfun_body:` The call itself, without the outer separator: used by the top-level call and by every call nested in an argument.

```

3638 \cs_new_protected:Nn \l__spintent_sfun_body:
3639 {
3640   \bool_if:NTF \l__spintent_sfun_silent_bool
3641   {
3642     \MathMLintent { - }
3643     {
3644       \l__spintent_inner_sep:
3645       \l__spintent_sfun_name_plain:
3646       \int_compare:nNnF { \l__spintent_sfun_nargs_int } = { 0 }
3647       {
3648         (
3649         \l__spintent_sfun_args_plain:
3650         )
3651       }
3652     }
3653   }
3654   {
3655     \l__spintent_sfun_set_intent:
3656     \MathMLintent { \l__spintent_sfun_intent_arg_tl }
3657     {
3658       \int_compare:nNnT { \l__spintent_sfun_nargs_int } = { 0 }
3659       { \l__spintent_inner_sep: }
3660       \str_if_eq:VnTF \l__spintent_sfun_mode_str { school }
3661       { \l__spintent_sfun_name_school: }
3662       { \l__spintent_sfun_name_mathcat: }
3663       \int_compare:nNnF { \l__spintent_sfun_nargs_int } = { 0 }
3664       {
3665         \MathMLintent { :silent } { ( }
3666         \l__spintent_sfun_args:
3667         \MathMLintent { :silent } { ) }
3668       }
3669     }
3670   }
3671 }

```

(End of definition for `\__spintent_spfun_body:n`.)

`\__spintent_spfun_nested:n` Typesets argument number **#1** as a call of its own. Opens a group, so the values that Lua leaves in the variables for the nested call (name, arguments, number of arguments) vanish when it ends and the outer call keeps its own. The nested call raises the depth, never applies the `concept` and goes through `\__spintent_spfun_body:` with no outer separator. It cannot fail to parse: the first analysis already validated every level.

```

3672 \cs_new_protected:Nn \__spintent_spfun_nested:n
3673 {
3674   \group_begin:
3675   \exp_args:Ne \__spintent_luafun_spfun_parse_and_set:n
3676   { \exp_not:v { \__spintent_spfun_luaaset_arg_ #1 _tl } }
3677   \int_incr:N \__spintent_spfun_depth_int
3678   \bool_set_false:N \__spintent_spfun_concept_bool
3679   \int_set:Nn \__spintent_spfun_nargs_int { \__spintent_spfun_luaaset_nargs_str }
3680   \__spintent_spfun_body:
3681   \group_end:
3682 }

```

(End of definition for `\__spintent_spfun_nested:n`.)

`\__spintent_spfun_exec:nn` Receives the optional `<keys>` in ‘**#1**’ and the `{<argument>}` in ‘**#2**’. Parses first, reporting `spfun-invalid-arg` on error whatever `silent` is.

```

3683 \cs_new_protected:Nn \__spintent_spfun_exec:nn
3684 {
3685   \keys_set:nn { spintent / spfun } {#1}
3686   \__spintent_luafun_spfun_parse_and_set:n { \exp_not:n {#2} }
3687   \str_if_eq:VnTF \__spintent_spfun_luaaset_error_str { true }
3688   {
3689     \msg_error:nne { spintent } { spfun-invalid-arg } { \exp_not:n {#2} }
3690   }
3691   {
3692     \int_set:Nn \__spintent_spfun_nargs_int { \__spintent_spfun_luaaset_nargs_str }
3693     \int_set:Nn \__spintent_spfun_depth_int { 1 }
3694     \__spintent_spfun_mathml:
3695   }
3696 }

```

(End of definition for `\__spintent_spfun_exec:nn`.)

`\spfun` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spfun_exec:nn`.

```

3697 \NewDocumentCommand \spfun { O{} m }
3698 {
3699   \mode_if_math:TF
3700   {
3701     \group_begin:
3702     \attribute \__spintent_scope_attr_int = 1
3703     \__spintent_spfun_exec:nn {#1} {#2}
3704     \group_end:
3705   }
3706   { \msg_error:nnn { spintent } { need-math-mode } { \spfun } }
3707 }

```

(End of definition for `\spfun`. This function is documented on page 35.)

13.37 The command `\spfuncomp`

The user command `\spfuncomp` typesets the linear composition of functions: $f\circ g$, $f\circ g\circ h$, $(f\circ g)$ and the composition applied to arguments, $(f\circ g)(x)$. The Lua function `\__spintent_luafun_sp` splits the `{<argument>}` at the top-level `\circ` (or the Unicode character U+2218); every function is a name as in `\spfun` and gets its literal `intent` (`_f-`, `_f-inversa-`, ...), so MathCAT neither announces a capital letter nor reads the letter `h` (U+210E) as Planck’s constant. The arguments of the application, if any, are left in the `\spfun` variables and typeset with its own functions, so an argument may be a call. The whole is tagged with a native `\MathMLintent` whose `intent` is `_compuesta-con:infix($k1,$k2,...)`; the application adds `$c($a1,...)` around it, with the composition tagged `arg="c"`. With two or more arguments they are grouped under their own `intent`, `_coma:infix($a1,$a2,...)`, tagged `arg="l"`, because MathCAT drops the commas of a call whose head is not a single letter. The `\circ` and the parentheses are silenced.

13.37.1 Definition of variables for \spfuncomp

Number of functions, the references and the `intent` being built, and the values that Lua leaves: error flag, number of functions, whether the composition is in parentheses and whether it is applied to arguments.

```

3708 \int_new:N \__spintent_spfuncomp_n_int
3709 \tl_new:N \__spintent_spfuncomp_refs_tl
3710 \tl_new:N \__spintent_spfuncomp_intent_tl
3711 \bool_new:N \__spintent_spfuncomp_silent_bool
3712 \tl_new:N \__spintent_spfuncomp_kv_tl
3713 \str_new:N \__spintent_spfuncomp_tipo_str
3714 \str_new:N \__spintent_spfuncomp_luaset_error_str
3715 \str_new:N \__spintent_spfuncomp_luaset_n_str
3716 \str_new:N \__spintent_spfuncomp_luaset_paren_str
3717 \str_new:N \__spintent_spfuncomp_luaset_apply_str

```

(End of definition for `\__spintent_spfuncomp_n_int` and others.)

13.37.2 Definition of error messages for \spfuncomp

`spfuncomp-invalid-arg`

Raised when the argument is not a composition of at least two function names, optionally in parentheses and optionally applied to arguments written in parentheses, whatever the value of `silent`.

```

3718 \msg_new:nnn { spintent } { spfuncomp-invalid-arg }
3719 {
3720   Invalid~argument~'#1'~for~\token_to_str:N \spfuncomp \c_space_tl \msg_line_context:.
3721 }

```

(End of definition for `spfuncomp-invalid-arg`.)

13.37.3 Definition of keys for \spfuncomp

`silent`
`unknown`

Only `silent`: the composition has no concept word and its reading is fixed, so `prefix` and `read-cmd` would have nothing to act on.

```

3722 \keys_define:nn { spintent / spfuncomp }
3723 {
3724   silent .bool_set:N = \__spintent_spfuncomp_silent_bool,
3725   silent .initial:n = false,
3726   silent .value_required:n = true,
3727   unknown .code:n =
3728   {
3729     \str_set:Nn \__spintent_command_name_str { spfuncomp }
3730     \__spintent_unknown_keys_cmd:n {#1}
3731   },
3732 }

```

(End of definition for `silent` and `unknown`.)

13.37.4 Internal functions for \spfuncomp

`\__spintent_spfuncomp_fun:n`

Function number `#1` of the chain. Its first letter is tagged `arg="k<i>"` with the literal `intent` of the name (`_f-`, `_f-inversa-`, ...) through the shared queue, exactly as `\spfun` does with its name: a native `\MathMLintent` around the name would leave its letter free for the queue to claim by mistake when the arguments contain calls. A subscript is typeset silenced; a prime is not; the inverse exponent `^{\-1}` is silenced too.

```

3733 \cs_new_protected:Nn \__spintent_spfuncomp_fun:n
3734 {
3735   \tl_set:Ne \__spintent_spfuncomp_kv_tl
3736   {
3737     arg = " k #1 ",
3738     intent = "_ \use:c { \__spintent_spfuncomp_luaset_intent_ #1 _str } -"
3739   }
3740   \__spintent_luafun_mrow_intent_arg:nn { } { \__spintent_spfuncomp_kv_tl }
3741   \str_set_eq:Nc \__spintent_spfuncomp_tipo_str
3742   { \__spintent_spfuncomp_luaset_tipo_ #1 _str }
3743   \str_case:VnF \__spintent_spfuncomp_tipo_str
3744   {
3745     { sub }
3746     {
3747       \use:c { \__spintent_spfuncomp_luaset_letra_ #1 _str }
3748       \c_math_subscript_token
3749       {
3750         \MathMLintent { :silent }
3751         { \use:c { \__spintent_spfuncomp_luaset_sub_ #1 _str } }
3752       }
3753     }
3754     { prima }

```

```

3755         {
3756             \use:c { l__spintent_sfuncomp_luaset_letra_ #1 _str }
3757             \use:c { l__spintent_sfuncomp_luaset_sub_ #1 _str }
3758         }
3759     }
3760     { \use:c { l__spintent_sfuncomp_luaset_letra_ #1 _str } }
3761     \str_set_eq:Nc \l__spintent_sfuncomp_tipo_str
3762     { l__spintent_sfuncomp_luaset_inv_ #1 _str }
3763     \str_if_eq:VnT \l__spintent_sfuncomp_tipo_str { true }
3764     {
3765         \c_math_superscript_token
3766         { \MathMLintent { :silent } { -1 } }
3767     }
3768 }

```

(End of definition for `\__spintent_sfuncomp_fun:n`.)

`\__spintent_sfuncomp_group:` The composition, with its own `intent`, in parentheses when the argument was written in parentheses.

```

3769 \cs_new_protected:Nn \__spintent_sfuncomp_group:
3770 {
3771     \tl_clear:N \l__spintent_sfuncomp_refs_tl
3772     \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3773     {
3774         \tl_put_right:Ne \l__spintent_sfuncomp_refs_tl
3775         { \int_compare:nNnF {##1} = {1} { , } $k ##1 }
3776     }
3777     \tl_set:Ne \l__spintent_sfuncomp_intent_tl
3778     { _compuesta-con:infix( \l__spintent_sfuncomp_refs_tl ) }
3779     \exp_args:NV \MathMLintent \l__spintent_sfuncomp_intent_tl
3780     {
3781         \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true }
3782         { \MathMLintent { :silent } { ( } }
3783         \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3784         {
3785             \int_compare:nNnF {##1} = {1}
3786             { \MathMLintent { :silent } { \circ } }
3787             \__spintent_sfuncomp_fun:n {##1}
3788         }
3789         \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true }
3790         { \MathMLintent { :silent } { ) } }
3791     }
3792 }

```

(End of definition for `\__spintent_sfuncomp_group:.`)

`\__spintent_sfuncomp_plain:` Typesets the whole with no annotation (`silent`).

```

3793 \cs_new_protected:Nn \__spintent_sfuncomp_plain:
3794 {
3795     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true } { ( }
3796     \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3797     {
3798         \int_compare:nNnF {##1} = {1} { \circ }
3799         \tl_use:c { l__spintent_sfuncomp_luaset_print_ ##1 _tl }
3800     }
3801     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true } { ) }
3802     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_apply_str { true }
3803     {
3804         ( \__spintent_sfun_args_plain: )
3805     }
3806 }

```

(End of definition for `\__spintent_sfuncomp_plain:.`)

`\__spintent_sfuncomp_mathml:` Calls `\__spintent_invisible_sep:` first. With `silent` the `intent` is `_-`. Otherwise an application wraps the composition (`arg="c"`) and the arguments in `$c($a1,...)`; the arguments reuse the `\spfun` machinery, including calls nested in them.

```

3807 \cs_new_protected:Nn \__spintent_sfuncomp_mathml:
3808 {
3809     \__spintent_invisible_sep:
3810     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_apply_str { true }
3811     {
3812         \int_set:Nn \l__spintent_sfun_nargs_int { \l__spintent_sfun_luaset_nargs_str }

```

```

3813     }
3814     \int_set:Nn \l__spintent_sfun_depth_int { 1 }
3815     \str_set:Nn \l__spintent_sfun_mode_str { school }
3816     \bool_set_false:N \l__spintent_sfun_silent_bool
3817     \bool_set_false:N \l__spintent_sfun_concept_bool
3818     \bool_if:NTF \l__spintent_sfuncomp_silent_bool
3819     {
3820       \MathMLintent { _- }
3821       { \__spintent_inner_sep: \__spintent_sfuncomp_plain: }
3822     }
3823     {
3824       \str_if_eq:VnTF \l__spintent_sfuncomp_luaset_apply_str { true }
3825       {
3826         \tl_clear:N \l__spintent_sfun_refs_tl
3827         \int_step_inline:nn { \l__spintent_sfun_nargs_int }
3828         {
3829           \tl_put_right:Ne \l__spintent_sfun_refs_tl
3830           { \int_compare:nNnF {##1} = {1} { , } $ \__spintent_sfun_aname:n {##1} }
3831         }
3832         \int_compare:nNnTF { \l__spintent_sfun_nargs_int } > { 1 }
3833         {
3834           \tl_set:Ne \l__spintent_sfun_call_tl { $c ( $l ) }
3835           \exp_args:NV \MathMLintent \l__spintent_sfun_call_tl
3836           {
3837             \MathMLarg { c } { \__spintent_inner_sep: \__spintent_sfuncomp_group: }
3838             \MathMLintent { :silent } { ( }
3839             \MathMLarg { l }
3840             {
3841               \__spintent_inner_sep:
3842               \exp_args:Ne \MathMLintent
3843               { _coma:infix ( \l__spintent_sfun_refs_tl ) }
3844               { \__spintent_sfun_args: }
3845             }
3846             \MathMLintent { :silent } { ) }
3847           }
3848         }
3849       }
3850       \tl_set:Ne \l__spintent_sfun_call_tl { $c ( \l__spintent_sfun_refs_tl ) }
3851       \exp_args:NV \MathMLintent \l__spintent_sfun_call_tl
3852       {
3853         \MathMLarg { c } { \__spintent_inner_sep: \__spintent_sfuncomp_group: }
3854         \MathMLintent { :silent } { ( }
3855         \__spintent_sfun_args:
3856         \MathMLintent { :silent } { ) }
3857       }
3858     }
3859   }
3860   { \__spintent_sfuncomp_group: }
3861 }
3862 }

```

(End of definition for `\__spintent_sfuncomp_mathml:`.)

`\__spintent_sfuncomp_exec:nn` Receives the optional $\langle keys \rangle$ in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. Parses first, reporting `spfuncomp-invalid-arg` on error whatever `silent` is.

```

3863 \cs_new_protected:Nn \__spintent_sfuncomp_exec:nn
3864 {
3865   \keys_set:nn { spintent / spfuncomp } {#1}
3866   \__spintent_luafun_sfuncomp_parse_and_set:n { \exp_not:n {#2} }
3867   \str_if_eq:VnTF \l__spintent_sfuncomp_luaset_error_str { true }
3868   {
3869     \msg_error:nne { spintent } { spfuncomp-invalid-arg }
3870     { \exp_not:n {#2} }
3871   }
3872   {
3873     \int_set:Nn \l__spintent_sfuncomp_n_int { \l__spintent_sfuncomp_luaset_n_str }
3874     \__spintent_sfuncomp_mathml:
3875   }
3876 }

```

(End of definition for `\__spintent_sfuncomp_exec:nn`.)

`\spfuncomp` The public command enforces *math mode*, opens a local group (with the scope attribute, for the calls that an argument may contain) and delegates to `\__spintent_spfuncomp_exec:nn`.

```

3877 \NewDocumentCommand \spfuncomp { O{} m }
3878 {
3879   \mode_if_math:TF
3880   {
3881     \group_begin:
3882       \attribute \__spintent_scope_attr_int = 1
3883       \__spintent_spfuncomp_exec:nn {#1} {#2}
3884     \group_end:
3885   }
3886   { \msg_error:nnn { spintent } { need-math-mode } { \spfuncomp } }
3887 }

```

(End of definition for `\spfuncomp`. This function is documented on page 36.)

13.38 The commands `\spdom`, `\sprec` and `\spcod`

The user commands `\spdom`, `\sprec` and `\spcod` typeset the domain `\operatorname{Dom}(f)`, the range `\operatorname{Rec}(f)` and the codomain `\operatorname{Cod}(f)` of a function. They share one implementation, `\__spintent_spdom_exec:nnnnn`, which receives the command name, the operator, the spoken word, the *(keys)* and the *{(argument)}*. The Lua function `\__spintent_luafun_spdom_parse_and_set:n` accepts exactly a function name, as `\spfun` does (a letter with an optional subscript or primes, and the optional exponent `^{\{-1\}}`), and leaves it in the shared `\l_@_geo_luaset_*` variables. The whole is tagged with a native `\MathMLintent` whose `intent` is `_ [prefix-]dominio-de:prefix($f)` (recorrido, codominio); the operator, the parentheses are silenced and the name carries `arg="f"` wrapped in its own literal `intent` (`_f-`, `_F-doble-prima-`, `_f-inversa-`, ...) so `MathCAT` neither announces a capital letter nor reads the inverse as a power. No `read-cmd` nor `concept` keys exist: the native reading of `Dom(f)` is not useful, and the concept is already the spoken word.

13.38.1 Definition of variables for `\spdom`

Command name (for the messages), operator and word (set by each public command), the value of the key `prefix`, the `intent` being built and the error flag set from Lua.

```

3888 \str_new:N \__spintent_spdom_cmd_str
3889 \tl_new:N \__spintent_spdom_op_tl
3890 \tl_new:N \__spintent_spdom_word_tl
3891 \tl_new:N \__spintent_spdom_prefix_tl
3892 \tl_new:N \__spintent_spdom_intent_arg_tl
3893 \bool_new:N \__spintent_spdom_silent_bool
3894 \str_new:N \__spintent_spdom_luaset_error_str

```

(End of definition for `\__spintent_spdom_cmd_str` and others.)

13.38.2 Definition of error messages for `\spdom`

`spdom-invalid-arg` The “error” `spdom-invalid-arg` is raised when the argument is not a function name. It receives the command name and the argument, and is raised whatever the value of `silent`.

```

3895 \msg_new:nnn { spintent } { spdom-invalid-arg }
3896 {
3897   Invalid~argument~'#2'~for~\c_backslash_str #1~\msg_line_context:.
3898 }

```

(End of definition for `spdom-invalid-arg`.)

13.38.3 Definition of keys for `\spdom`

`prefix` We define the *(keys)* `prefix` and `silent`.
`silent`
`unknown`

```

3899 \keys_define:nn { spintent / spdom }
3900 {
3901   prefix .code:n =
3902     { \__spintent_sanitizé_affix:Nn \__spintent_spdom_prefix_tl {#1} },
3903   prefix .initial:n = ,
3904   prefix .value_required:n = true,
3905   silent .bool_set:N = \__spintent_spdom_silent_bool,
3906   silent .initial:n = false,
3907   silent .value_required:n = true,
3908   unknown .code:n =
3909     {
3910       \str_set_eq:NN \__spintent_command_name_str \__spintent_spdom_cmd_str
3911       \__spintent_unknown_keys_cmd:n {#1}
3912     },
3913 }

```

(End of definition for `prefix`, `silent`, and `unknown`.)

13.38.4 Internal functions for \spdom

`\__spintent_spdom_name:` Tags the name with `arg="f"` through a native `\MathMLarg`; inside it, a native `\MathMLintent` gives the name its literal `intent`. Both groups start with the inner separator, so each one gets its own `<mrow>` even when the name is a single node.

```

3914 \cs_new_protected:Nn \__spintent_spdom_name:
3915 {
3916   \MathMLarg { f }
3917   {
3918     \__spintent_inner_sep:
3919     \MathMLintent { _ \l__spintent_geo_luaset_intent_str - }
3920     { \__spintent_inner_sep: \__spintent_spfun_name_plain: }
3921   }
3922 }
```

(End of definition for `\__spintent_spdom_name:`.)

`\__spintent_spdom_mathml:` Calls `\__spintent_invisible_sep:` first, then wraps operator, parentheses and name in a native `\MathMLintent`. With `silent` set to `true` the `intent` is `_` and nothing inside is annotated.

```

3923 \cs_new_protected:Nn \__spintent_spdom_mathml:
3924 {
3925   \__spintent_invisible_sep:
3926   \bool_if:NTF \l__spintent_spdom_silent_bool
3927   {
3928     \MathMLintent { _- }
3929     {
3930       \__spintent_inner_sep:
3931       \operatorname { \l__spintent_spdom_op_tl }
3932       ( \__spintent_spfun_name_plain: )
3933     }
3934   }
3935   {
3936     \tl_set:Nx \l__spintent_spdom_intent_arg_tl
3937     {
3938       -
3939       \tl_if_empty:NF \l__spintent_spdom_prefix_tl
3940       { \tl_use:N \l__spintent_spdom_prefix_tl - }
3941       \l__spintent_spdom_word_tl -de:prefix( $f )
3942     }
3943     \MathMLintent { \l__spintent_spdom_intent_arg_tl }
3944     {
3945       \MathMLintent { :silent } { \operatorname { \l__spintent_spdom_op_tl } }
3946       \MathMLintent { :silent } { ( ) }
3947       \__spintent_spdom_name:
3948       \MathMLintent { :silent } { ) }
3949     }
3950   }
3951 }
```

(End of definition for `\__spintent_spdom_mathml:`.)

`\__spintent_spdom_exec:nnnnn` Receives the command name in ‘#1’, the operator in ‘#2’, the spoken word in ‘#3’, the optional `<keys>` in ‘#4’ and the `{<argument>}` in ‘#5’. Parses first, reporting `spdom-invalid-arg` on error whatever `silent` is.

```

3952 \cs_new_protected:Nn \__spintent_spdom_exec:nnnnn
3953 {
3954   \str_set:Nn \l__spintent_spdom_cmd_str {#1}
3955   \tl_set:Nn \l__spintent_spdom_op_tl {#2}
3956   \tl_set:Nn \l__spintent_spdom_word_tl {#3}
3957   \keys_set:nn { spintent / spdom } {#4}
3958   \__spintent_luafun_spdom_parse_and_set:n { \exp_not:n {#5} }
3959   \str_if_eq:VnTF \l__spintent_spdom_luaset_error_str { true }
3960   {
3961     \msg_error:nnee { spintent } { spdom-invalid-arg }
3962     {#1} { \exp_not:n {#5} }
3963   }
3964   { \__spintent_spdom_mathml: }
3965 }
```

(End of definition for `\__spintent_spdom_exec:nnnnn`.)

```

\spdom The public commands enforce math mode, open a local group, and delegate to \__spintent_spdom_exec:nnnnn.
\sprec
\spcod
3966 \NewDocumentCommand \spdom { O{} m }
3967 {
3968   \mode_if_math:TF
3969   {
3970     \group_begin:
3971     \__spintent_spdom_exec:nnnnn { spdom } { Dom } { dominio } {#1} {#2}
3972     \group_end:
3973   }
3974   { \msg_error:nnn { spintent } { need-math-mode } { \spdom } }
3975 }
3976 \NewDocumentCommand \sprec { O{} m }
3977 {
3978   \mode_if_math:TF
3979   {
3980     \group_begin:
3981     \__spintent_spdom_exec:nnnnn { sprec } { Rec } { recorrido } {#1} {#2}
3982     \group_end:
3983   }
3984   { \msg_error:nnn { spintent } { need-math-mode } { \sprec } }
3985 }
3986 \NewDocumentCommand \spcod { O{} m }
3987 {
3988   \mode_if_math:TF
3989   {
3990     \group_begin:
3991     \__spintent_spdom_exec:nnnnn { spcod } { Cod } { codominio } {#1} {#2}
3992     \group_end:
3993   }
3994   { \msg_error:nnn { spintent } { need-math-mode } { \spcod } }
3995 }

```

(End of definition for `\spdom`, `\sprec`, and `\spcod`. These functions are documented on page 36.)

13.39 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent` state, looked up by name (#2).

```

\__spintent_mathmlarg_point_ref_tl
\__spintent_mathmlarg_point_mode_str
\__spintent_mathmlarg_point_kv_tl
\__spintent_mathmlarg_point_silent_bool
3996 \tl_new:N \__spintent_mathmlarg_point_ref_tl
3997 \str_new:N \__spintent_mathmlarg_point_mode_str
3998 \tl_new:N \__spintent_mathmlarg_point_kv_tl
3999 \bool_new:N \__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

```

\__spintent_mathmlarg_point:nnn
\__spintent_mathmlarg_point:nnV

```

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. A subscript of more than one digit is not annotated natively — the queue would hand its digits to the next point's claim — but typeset plain, with one extra silent claim per digit. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

4000 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
4001 {
4002   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
4003   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
4004   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_bool }
4005   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
4006   \__spintent_invisible_sep:
4007   \str_case:VnF \__spintent_geo_luaset_tipo_str
4008   {
4009     { sub }
4010     {
4011       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
4012       {
4013         \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
4014         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
4015         \__spintent_luafun_mrow_intent_arg:nn
4016         { } { \__spintent_mathmlarg_point_kv_tl }
4017         \int_compare:nNnTF { \str_count:N \__spintent_geo_luaset_sub_str } > { 1 }

```

```

4018         {
4019             \int_step_inline:nnnn { 1 } { 1 } { \str_count:N \l__spintent_geo_luaset_sub_
4020             { \l__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } }
4021             {
4022                 \l__spintent_geo_luaset_letra_str
4023                 \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4024             }
4025         }
4026         {
4027             {
4028                 \l__spintent_geo_luaset_letra_str
4029                 \c_math_subscript_token
4030                 { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4031             }
4032         }
4033     }
4034     {
4035         {
4036             \l__spintent_inner_sep:
4037             \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
4038             {
4039                 {
4040                     \l__spintent_geo_luaset_letra_str
4041                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4042                 }
4043             }
4044         }
4045     }
4046 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark `unsilenced` (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

4047     { prima }
4048     {
4049         \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
4050         {
4051             \tl_set:Ne \l__spintent_mathmlarg_point_kv_tl
4052             { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_luas
4053             \l__spintent_luafun_mrow_intent_arg:nn
4054             { } { \l__spintent_mathmlarg_point_kv_tl }
4055             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4056         }
4057         {
4058             {
4059                 \l__spintent_inner_sep:
4060                 \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
4061                 {
4062                     \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4063             }
4064         }
4065     }
4066 }
4067 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

4068     {
4069         \bool_if:NTF \l__spintent_mathmlarg_point_silent_bool
4070         {
4071             \tl_set:Ne \l__spintent_mathmlarg_point_kv_tl
4072             { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "-_" }
4073             \l__spintent_luafun_mrow_intent_arg:nn
4074             { } % debe ir vacío
4075             { \l__spintent_mathmlarg_point_kv_tl }
4076             { \l__spintent_geo_luaset_print_tl }
4077         }
4078         {
4079             \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
4080             {

```

```

4081         \tl_set:Nx \l__spintent_mathmlarg_point_kv_tl
4082         { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_lua
4083         \__spintent_luafun_mrow_intent_arg:nn
4084         { } % debe ir vacío
4085         { \l__spintent_mathmlarg_point_kv_tl }
4086     { \l__spintent_geo_luaaset_print_tl }
4087     }
4088     {
4089         \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
4090         { \l__spintent_geo_luaaset_print_tl }
4091     }
4092 }
4093 }
4094 }
4095 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\__spintent_mathmlarg_point:nnn`.)

13.40 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

13.40.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
4096 \str_new:N \l__spintent_spPoint_mode_str
4097 \tl_new:N \l__spintent_spPoint_prefix_tl
4098 \tl_new:N \l__spintent_spPoint_intent_arg_tl

```

(End of definition for `\l__spintent_spPoint_mode_str`, `\l__spintent_spPoint_prefix_tl`, and `\l__spintent_spPoint_intent_arg_tl`.)

13.40.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

4099 \msg_new:nnnn { spintent } { spPoint-invalid-point }
4100 {
4101     Invalid~argument~'~#1'~for~\token_to_str:N \spPoint \c_space_tl \msg_line_context:.
4102 }
4103 {
4104     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
4105     subscript~(\_{...})~or~prime~(').\
4106 }

```

(End of definition for `spPoint-invalid-point`.)

13.40.3 Definition of keys for `\spPoint`

`read-cmd`

We define the `(keys)` `read-cmd`, `prefix`, `silent` and `concept`.

`prefix`

`silent`

`unknown`

```

4107 \keys_define:nn { spintent / spPoint }
4108 {
4109     read-cmd
4110         {
4111             \int_case:nn { \l_keys_choice_int }
4112             {
4113                 {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
4114                 {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
4115             }
4116         },
4117     read-cmd
4118         .initial:n = school,
4119     read-cmd
4120         .value_required:n = true,
4121     read-cmd / unknown
4122         .code:n =
4123         \msg_error:nneee { spintent } { unknown-choice }
4124         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4125     prefix
4126         .code:n =
4127         { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
4128     prefix
4129         .initial:n = ,
4130     prefix
4131         .value_required:n = true,
4132     silent
4133         .bool_set:N = \l__spintent_spPoint_silent_bool,
4134     silent
4135         .initial:n = false,
4136     silent
4137         .value_required:n = true,
4138     concept
4139         .bool_set:N = \l__spintent_spPoint_concept_bool,
4140     concept
4141         .initial:n = true,

```

```

4131     concept                .value_required:n = true,
4132     unknown                .code:n =
4133     {
4134         \str_set:Nn \l__spintent_command_name_str { spPoint }
4135         \__spintent_unknown_keys_cmd:n {#1}
4136     },
4137 }

```

(End of definition for read-cmd and others.)

13.40.4 Internal functions for \spPoint

`\__spintent_spPoint_print:n` The internal function `\__spintent_spPoint_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\__spintent_luafun_geo_point_parse_and_set:n`, reporting spPoint-invalid-point on error. If `silent` is `true` or `concept` is `false`, sets `\l__spintent_spPoint_intent_arg_tl` to `_{\$pt}` — forced further to `_-` when `silent` is `true`, whatever `read-cmd` is: a composite point (one with a subscript) is not reliably silenced by the per-point `:silent` alone. Otherwise sets it to `_punto:prefix(\$pt)`, with `prefix` placed after the leading `_` when given. Finally calls `\__spintent_spPoint_mathml:n`.

```

4138 \cs_new_protected:Nn \__spintent_spPoint_print:n
4139 {
4140     \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
4141     \str_if_eq:NnT \l__spintent_geo_luaset_error_str { true }
4142     {
4143         \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
4144     }
4145     \bool_if:nTF
4146     {
4147         \bool_lazy_or_p:nn
4148         { \l__spintent_spPoint_silent_bool }
4149         { ! \l__spintent_spPoint_concept_bool }
4150     }
4151     {
4152         \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _{ \$pt } }
4153         \bool_if:NT \l__spintent_spPoint_silent_bool
4154         {
4155             \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _- }
4156         }
4157     }
4158     {
4159         \tl_set:Ne \l__spintent_spPoint_intent_arg_tl
4160         {
4161             -
4162             \tl_if_empty:NF \l__spintent_spPoint_prefix_tl
4163             {
4164                 \tl_use:N \l__spintent_spPoint_prefix_tl -
4165             }
4166             punto:prefix(\$pt)
4167         }
4168     }
4169     \__spintent_spPoint_mathml:n { #1 }
4170 }

```

(End of definition for `\__spintent_spPoint_print:n`.)

`\__spintent_spPoint_mathml:n` The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\l__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_inner_sep:` first inside that group.

```

4171 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
4172 {
4173     \__spintent_invisible_sep:
4174     \MathMLintent { \l__spintent_spPoint_intent_arg_tl }
4175     {
4176         \__spintent_inner_sep:
4177         \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
4178     }
4179 }

```

(End of definition for `\__spintent_spPoint_mathml:n`.)

`\__spintent_spPoint_exec:nnn` The function `\__spintent_spPoint_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the point $\langle argument \rangle$ in ‘#2’, delegating to `\__spintent_spPoint_print:n`. With ‘#3’ given (the optional coordinate),

silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\__spintent_spcoord_silent_bool` to `\__spintent_spPoint_silent_bool` and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error (whatever `silent` says), then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:`

```

4180 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
4181 {
4182   \keys_set:nn { spintent / spPoint } { #1 }
4183   \__spintent_spPoint_print:n { #2 }
4184   \tl_if_novalue:nF {#3}
4185   {
4186     \MathMLint { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
4187     \bool_set_eq:NN \__spintent_spcoord_silent_bool \__spintent_spPoint_silent_bool
4188     \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#3}
4189     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
4190     \str_if_eq:VnT \__spintent_spcoord_luaset_error_str { true }
4191     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
4192     \__spintent_spcoord_set_wrap_parens_size:
4193     \str_if_eq:VnF \__spintent_spcoord_luaset_error_str { true }
4194     { \__spintent_spcoord_mathml: }
4195   }
4196 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

4197 \NewDocumentCommand \spPoint { 0{} m d() }
4198 {
4199   \mode_if_math:TF
4200   {
4201     \group_begin:
4202     \attribute \__spintent_scope_attr_int = 1
4203     \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
4204     \group_end:
4205   }
4206   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
4207 }

```

(End of definition for `\spPoint`. This function is documented on page 19.)

13.41 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle argument \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle argument \rangle$.

13.41.1 Definition of variables for `\spvec`

The variable `\__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

4208 \str_new:N \__spintent_spvec_mode_str
4209 \tl_new:N \__spintent_spvec_prefix_tl
4210 \tl_new:N \__spintent_spvec_intent_arg_tl
4211 \tl_new:N \__spintent_spvec_kv_tl
4212 \seq_new:N \__spintent_spvec_pt_seq
4213 \tl_new:N \__spintent_spvec_refs_tl
4214 \int_new:N \__spintent_spvec_pt_idx_int

```

(End of definition for `\__spintent_spvec_mode_str` and others.)

13.41.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

4215 \msg_new:nnn { spintent } { spvec-invalid-arg }
4216 {
4217   Invalid~argument~'!#1'~for~\token_to_str:N \spvec \c_space_tl \msg_line_context:.
4218 }

```

(End of definition for `spvec-invalid-arg`.)

13.41.3 Definition of keys for \spvec

We define the *(keys)* `silent`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

silent
concept 4219 \keys_define:nn { spintent / spvec }
autofit 4220 {
read-cmd 4221   silent      .bool_set:N = \__spintent_spvec_silent_bool,
prefix   4222   silent      .initial:n = false,
unknown  4223   silent      .value_required:n = true,
          4224   concept     .bool_set:N = \__spintent_spvec_concept_bool,
          4225   concept     .initial:n = true,
          4226   concept     .value_required:n = true,
          4227   autofit     .bool_set:N = \__spintent_spvec_autofit_bool,
          4228   autofit     .initial:n = false,
          4229   autofit     .value_required:n = true,
          4230   read-cmd     .choices:nn = { school , mathcat }
          4231   {
          4232     \int_case:nn { \l_keys_choice_int }
          4233     {
          4234       {1} { \str_set:Nn \__spintent_spvec_mode_str { school } }
          4235       {2} { \str_set:Nn \__spintent_spvec_mode_str { mathcat } }
          4236     }
          4237   },
          4238   read-cmd     .initial:n = school,
          4239   read-cmd     .value_required:n = true,
          4240   read-cmd / unknown .code:n =
          4241     \msg_error:nneee { spintent } { unknown-choice }
          4242     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          4243   prefix       .code:n =
          4244     { \__spintent_sanitizize_affix:Nn \__spintent_spvec_prefix_tl {#1} },
          4245   prefix       .initial:n = { },
          4246   prefix       .value_required:n = true,
          4247   unknown      .code:n =
          4248     {
          4249       \str_set:Nn \__spintent_command_name_str { spvec }
          4250       \__spintent_unknown_keys_cmd:n {#1}
          4251     },
          4252   }

```

(End of definition for `silent` and others.)

13.41.4 Internal functions for \spvec

`\__spintent_spvec_print:n` The internal function `\__spintent_spvec_print:n` processes the *(argument)* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\__spintent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_spvec_mathml:n`.

```

4253 \cs_new_protected:Nn \__spintent_spvec_print:n
4254 {
4255   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
4256   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { true }
4257   {
4258     \str_if_eq:VnT \__spintent_geo_luaset_tipo_str { prima }
4259     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
4260   }
4261   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { false }
4262   {
4263     \int_compare:nT { \clist_count:N \__spintent_geo_luaset_points_clist != 2 }
4264     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
4265   }
4266   \str_if_eq:VnTF \__spintent_geo_luaset_error_str { true }
4267   { \msg_error:nne { spintent } { spvec-invalid-arg } { \exp_not:n {#1} } }
4268   { \__spintent_spvec_mathml:n { #1 } }
4269 }

```

(End of definition for `\__spintent_spvec_print:n`.)

`\__spintent_spvec_mathml:n` The function `\__spintent_spvec_mathml:n` branches on `\__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_( $\$$ pt)` when `silent` is `true` or `concept` is `false` — forced further to `_` when `silent` is `true`, whatever `read-cmd` is; otherwise `_vector:prefix( $\$$ pt)`, with `prefix` placed after the leading `_` when given. With `\__spintent_geo_luaset_tipo` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native

`\MathMLarg` instead, the subscript silenced only under `silent`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same kv logic wraps `\vec{print\_tl}` as a whole.

```

4270 \cs_new_protected:Nn \__spintent_sptvec_mathml:n
4271 {
4272   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
4273   {
4274     \bool_if:nTF
4275     {
4276       \bool_lazy_or_p:nn
4277       { \l__spintent_sptvec_silent_bool }
4278       { ! \l__spintent_sptvec_concept_bool }
4279     }
4280     {
4281       \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _($pt) }
4282       \bool_if:NT \l__spintent_sptvec_silent_bool
4283       { \tl_set:Nn \l__spintent_sptvec_intent_arg_tl { _- } }
4284     }
4285     {
4286       \tl_set:Ne \l__spintent_sptvec_intent_arg_tl
4287       {
4288         -
4289         \tl_if_empty:NF \l__spintent_sptvec_prefix_tl
4290         { \tl_use:N \l__spintent_sptvec_prefix_tl - }
4291         vector:prefix($pt)
4292       }
4293     }
4294   \str_case:VnF \l__spintent_geo_luaset_tipo_str
4295   {
4296     { sub }
4297     {
4298       \str_if_eq:VnTF \l__spintent_sptvec_mode_str { school }
4299       {
4300         \bool_if:NTF \l__spintent_sptvec_silent_bool
4301         { \tl_set:Nn \l__spintent_sptvec_kv_tl { arg = "pt", intent = "_-" } }
4302         {
4303           \tl_set:Ne \l__spintent_sptvec_kv_tl
4304           { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4305         }
4306         \__spintent_invisible_sep:
4307         \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
4308         {
4309           \__spintent_inner_sep:
4310           \__spintent_luafun_mrow_intent_arg:nn
4311           { } { \l__spintent_sptvec_kv_tl }
4312         }
4313         \vec
4314         {
4315           \bool_if:NT \l__spintent_sptvec_autofit_bool
4316           { \vphantom{ b } }
4317           \l__spintent_geo_luaset_letra_str
4318         }
4319         \c_math_subscript_token
4320         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4321       }
4322     }
4323   }
4324   {
4325     \__spintent_invisible_sep:
4326     \MathMLintent { \l__spintent_sptvec_intent_arg_tl }
4327     {
4328       \__spintent_inner_sep:
4329       \vec
4330       {
4331         \bool_if:NT \l__spintent_sptvec_autofit_bool
4332         { \vphantom{ b } }
4333         \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
4334       }
4335       \bool_if:NT \l__spintent_sptvec_silent_bool
4336       { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
4337     }

```

```

4338         \bool_if:NF \l__spintent_sspec_silent_bool
4339         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
4340     }
4341 }
4342 }
4343 {
4344     \bool_if:NTF \l__spintent_sspec_silent_bool
4345     { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "-" } }
4346     {
4347         \tl_set:Nn \l__spintent_sspec_kv_tl
4348         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4349     }
4350     \__spintent_invisible_sep:
4351     \MathMLintent { \l__spintent_sspec_intent_arg_tl }
4352     {
4353         \__spintent_inner_sep:
4354         \__spintent_luafun_mrow_intent_arg:nn
4355         { } { \l__spintent_sspec_kv_tl }
4356     }
4357     \vec
4358     {
4359         \bool_if:NT \l__spintent_sspec_autofit_bool
4360         { \vphantom{ b } }
4361         \l__spintent_geo_luaset_print_tl
4362     }
4363 }
4364 }
4365 }
4366 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sspec_pt_seq` and walked to build `\l__spintent_sspec_refs_tl`, the $\$pts1, \$pts2$ reference list. The concept `intent` is `_` when `silent` is `true`, `_` (refs) when `concept` is `false` or `_vector:prefix`(refs) otherwise, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4367     {
4368         \seq_set_from_clist:NN \l__spintent_sspec_pt_seq \l__spintent_geo_luaset_points_clist
4369         \tl_clear:N \l__spintent_sspec_refs_tl
4370         \int_zero:N \l__spintent_sspec_pt_idx_int
4371         \seq_map_inline:Nn \l__spintent_sspec_pt_seq
4372         {
4373             \int_incr:N \l__spintent_sspec_pt_idx_int
4374             \tl_if_empty:NF \l__spintent_sspec_refs_tl
4375             { \tl_put_right:Nn \l__spintent_sspec_refs_tl { , } }
4376             \tl_put_right:Nn \l__spintent_sspec_refs_tl { $ pts \int_use:N \l__spintent_sspec_pt_idx_int }
4377         }
4378         \bool_if:NTF
4379         {
4380             \bool_lazy_or_p:nn
4381             { \l__spintent_sspec_silent_bool }
4382             { ! \l__spintent_sspec_concept_bool }
4383         }
4384         {
4385             \tl_set:Nn \l__spintent_sspec_intent_arg_tl { _ ( \l__spintent_sspec_refs_tl ) }
4386             \bool_if:NTF \l__spintent_sspec_silent_bool
4387             { \tl_set:Nn \l__spintent_sspec_intent_arg_tl { _- } }
4388         }
4389         {
4390             \tl_set:Nn \l__spintent_sspec_intent_arg_tl
4391             {
4392                 -
4393                 \tl_if_empty:NF \l__spintent_sspec_prefix_tl
4394                 { \tl_use:N \l__spintent_sspec_prefix_tl - }
4395                 vector:prefix( \l__spintent_sspec_refs_tl )
4396             }
4397         }
4398         \__spintent_invisible_sep:
4399         \MathMLintent { \l__spintent_sspec_intent_arg_tl }
4400         {
4401             \__spintent_inner_sep:
4402             \overrightarrow

```

```

4403         {
4404             \int_zero:N \__spintent_svec_pt_idx_int
4405             \seq_map_inline:Nn \__spintent_svec_pt_seq
4406             {
4407                 \int_incr:N \__spintent_svec_pt_idx_int
4408                 \__spintent_mathmlarg_point:nnn
4409                 { pts \int_use:N \__spintent_svec_pt_idx_int }
4410                 { svec }
4411                 { ##1 }
4412             }
4413         }
4414     }
4415 }
4416 }

```

(End of definition for `\__spintent_svec_mathml:n`.)

`\__spintent_svec_exec:nnn` The function `\__spintent_svec_exec:nnn` receives the optional *keys* in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_spcord_silent_bool` to `\__spintent_svec_silent_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcord_parse_and_set:n` directly, reporting `spcord-invalid-arg` on error (whatever `silent` says), then `\__spintent_spcord_set_wrap_parens_size:` and `\__spintent_spcord_mathml:` — the same rendering functions `\spcord` itself uses.

```

4417 \cs_new_protected:Nn \__spintent_svec_exec:nnn
4418 {
4419     \keys_set:nn { spintent / svec } { #1 }
4420     \__spintent_svec_print:n { #2 }
4421     \tl_if_novalue:nF {#3}
4422     {
4423         \bool_if:NTF \__spintent_svec_silent_bool
4424         { \MathMLint { :silent } { \equal } }
4425         { \equal } % notación de vector =( , )
4426         \MathMLint { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
4427         \bool_set_eq:NN \__spintent_spcord_silent_bool \__spintent_svec_silent_bool
4428         \tl_set:Nn \__spintent_spcord_raw_arg_tl {#3}
4429         \__spintent_luafun_spcord_parse_and_set:n { \exp_not:n {#3} }
4430         \str_if_eq:VnT \__spintent_spcord_luaseta_error_str { true }
4431         { \msg_error:nnn { spintent } { spcord-invalid-arg } { \exp_not:n {#3} } }
4432         \__spintent_spcord_set_wrap_parens_size:
4433         \str_if_eq:VnF \__spintent_spcord_luaseta_error_str { true }
4434         { \__spintent_spcord_mathml: }
4435     }
4436 }

```

(End of definition for `\__spintent_svec_exec:nnn`.)

`\spvec` The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_svec_exec:nnn`.

```

4437 \NewDocumentCommand \spvec { O{} m d() }
4438 {
4439     \mode_if_math:TF
4440     {
4441         \group_begin:
4442         \attribute \__spintent_scope_attr_int = 1
4443         \__spintent_svec_exec:nnn { #1 } { #2 } { #3 }
4444         \group_end:
4445     }
4446     { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
4447 }

```

(End of definition for `\spvec`. This function is documented on page 20.)

13.42 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\{\langle argument \rangle\}$: when the argument contains two points (separated by an optional comma), `\overleftarrow{\rightarrow}` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

13.42.1 Definition of variables for \sprecta

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

4448 \str_new:N \__spintent_sprecta_mode_str
4449 \tl_new:N \__spintent_sprecta_prefix_tl
4450 \seq_new:N \__spintent_sprecta_pt_seq
4451 \tl_new:N \__spintent_sprecta_refs_tl
4452 \tl_new:N \__spintent_sprecta_intent_arg_tl
4453 \tl_new:N \__spintent_sprecta_kv_tl
4454 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

13.42.2 Definition of error messages for \sprecta

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

4455 \msg_new:nnnn { spintent } { sprecta-invalid-points }
4456 {
4457   Invalid~argument~'#1'~for~\token_to_str:N \sprecta \c_space_tl \msg_line_context:.
4458 }
4459 {
4460   See~the~documentation~of~\token_to_str:N \sprecta \c_space_tl for~the~accepted~argument~forms
4461 }

```

(End of definition for `sprecta-invalid-points`.)

13.42.3 Definition of keys for \sprecta

We define the *(keys)* `read-cmd`, `prefix` and `silent`.

```

read-cmd 4462 \keys_define:nn { spintent / sprecta }
prefix   4463 {
silent   4464   read-cmd           .choices:nn = { school , mathcat }
unknown  4465     {
          4466       \int_case:nn { \_keys_choice_int }
          4467       {
          4468         {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
          4469         {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
          4470       }
          4471     },
          4472   read-cmd           .initial:n = school,
          4473   read-cmd           .value_required:n = true,
          4474   read-cmd / unknown .code:n =
          4475     \msg_error:nneee { spintent } { unknown-choice }
          4476     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
          4477   prefix           .code:n =
          4478     { \__spintent_sanitize_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
          4479   prefix           .initial:n = ,
          4480   prefix           .value_required:n = true,
          4481   silent           .bool_set:N = \__spintent_sprecta_silent_bool,
          4482   silent           .initial:n = false,
          4483   silent           .value_required:n = true,
          4484   concept          .bool_set:N = \__spintent_sprecta_concept_bool,
          4485   concept          .initial:n = true,
          4486   concept          .value_required:n = true,
          4487   unknown          .code:n =
          4488     {
          4489       \str_set:Nn \__spintent_command_name_str { sprecta }
          4490       \__spintent_unknown_keys_cmd:n {#1}
          4491     },
          4492   }

```

(End of definition for `read-cmd` and others.)

13.42.4 Internal functions for \sprecta

The internal function `\__spintent_sprecta_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `sprecta-invalid-points` on error, then `\__spintent_`

```

4493 \cs_new_protected:Nn \__spintent_sprecta_print:n
4494 {
4495   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
4496   \str_if_eq:VnT \__spintent_geo_luaerror_str { true }
4497   {
4498     \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }

```

```

4499     }
4500     \__spintent_spectra_mathml:n { #1 }
4501 }

```

(End of definition for __spintent_spectra_print:n.)

__spintent_spectra_mathml:n

The function `\__spintent_spectra_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept `intent` is `_( $\$pt$ )` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true` and `read-cmd` is not `school`; otherwise `_recta:prefix( $\$pt$ )`, with `prefix` placed after the leading `_` when given. With `read-cmd=school`, the sub and prima cases tag the letter with a literal kv that is `_-` when `silent` is `true`.

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

4502 \cs_new_protected:Nn \__spintent_spectra_mathml:n
4503 {
4504   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
4505   {
4506     \__spintent_invisible_sep:
4507     \bool_if:nTF
4508     {
4509       \bool_lazy_or_p:nn
4510       { \l__spintent_spectra_silent_bool }
4511       { ! \l__spintent_spectra_concept_bool }
4512     }
4513     {
4514       \tl_set:Nn \l__spintent_spectra_intent_arg_tl { _( $\$pt$ ) }
4515     \bool_if:NT \l__spintent_spectra_silent_bool
4516     {
4517       \str_if_eq:VnF \l__spintent_spectra_mode_str { school }
4518       { \tl_set:Nn \l__spintent_spectra_intent_arg_tl { _- } }
4519     }
4520   }
4521   {
4522     \tl_set:Ne \l__spintent_spectra_intent_arg_tl
4523     {
4524       -
4525       \tl_if_empty:NF \l__spintent_spectra_prefix_tl
4526       { \tl_use:N \l__spintent_spectra_prefix_tl - }
4527       recta:prefix( $\$pt$ )
4528     }
4529   }
4530   \str_case:VnF \l__spintent_geo_luaset_tipo_str
4531   {
4532     { sub }
4533     {
4534       \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
4535       {
4536         \bool_if:NTF \l__spintent_spectra_silent_bool
4537         { \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "_-" } }
4538         {
4539           \tl_set:Ne \l__spintent_spectra_kv_tl
4540           { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4541         }
4542         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4543         {
4544           \__spintent_inner_sep:
4545           \__spintent_luafun_mrow_intent_arg:nn
4546           { } { \l__spintent_spectra_kv_tl }
4547           {
4548             \l__spintent_geo_luaset_letra_str
4549             \c_math_subscript_token
4550             { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4551           }
4552         }
4553       }
4554     }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

4554   {

```

```

4555         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4556     {
4557     {
4558         \__spintent_inner_sep:
4559         \MathMLarg { pt }
4560     {
4561     {
4562         \l__spintent_geo_luaset_letra_str
4563         \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4564     }
4565     }
4566     }
4567     }
4568     }
4569     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

4570     { prima }
4571     {
4572         \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
4573     {
4574         \bool_if:NTF \l__spintent_spectra_silent_bool
4575         { \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "--" } }
4576         {
4577             \tl_set:Ne \l__spintent_spectra_kv_tl
4578             { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str -" }
4579         }
4580         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4581         {
4582             \__spintent_inner_sep:
4583             \__spintent_luafun_mrow_intent_arg:nn
4584             { } { \l__spintent_spectra_kv_tl }
4585             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4586         }
4587     }
4588     {
4589         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4590         {
4591         {
4592             \__spintent_inner_sep:
4593             \MathMLarg { pt }
4594         {
4595             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4596             }
4597         }
4598         }
4599     }
4600     }
4601     }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

4602     {
4603         \bool_if:NTF \l__spintent_spectra_silent_bool
4604         {
4605             \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "--" }
4606         }
4607         {
4608             \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
4609             {
4610                 \tl_set:Ne \l__spintent_spectra_kv_tl
4611                 { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str -" }
4612             }
4613             {
4614                 \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt" }
4615             }
4616         }
4617         \__spintent_luafun_mrow_intent_arg:nn
4618         { \l__spintent_spectra_intent_arg_tl }
4619         { \l__spintent_spectra_kv_tl }

```

```

4620         { \l__spintent_geo_luaset_print_tl }
4621     }
4622 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_` - when `silent` is `true`, `_`(refs) when `concept` is `false` or `_recta:prefix`(refs) otherwise, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4623 {
4624   \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
4625   \tl_clear:N \l__spintent_sprecta_refs_tl
4626   \int_zero:N \l__spintent_sprecta_pt_idx_int
4627   \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
4628   {
4629     \int_incr:N \l__spintent_sprecta_pt_idx_int
4630     \tl_if_empty:NF \l__spintent_sprecta_refs_tl
4631     { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
4632     \tl_put_right:Ne \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
4633   }
4634   \bool_if:nTF
4635   {
4636     \bool_lazy_or_p:nn
4637     { \l__spintent_sprecta_silent_bool }
4638     { ! \l__spintent_sprecta_concept_bool }
4639   }
4640   {
4641     \tl_set:Ne \l__spintent_sprecta_intent_arg_tl { _ ( \l__spintent_sprecta_refs_tl ) }
4642     \bool_if:NT \l__spintent_sprecta_silent_bool
4643     { \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { _- } }
4644   }
4645   {
4646     \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
4647     {
4648       -
4649       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
4650       { \tl_use:N \l__spintent_sprecta_prefix_tl - }
4651       recta:prefix( \l__spintent_sprecta_refs_tl )
4652     }
4653   }
4654   \__spintent_invisible_sep:
4655   \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
4656   {
4657     \__spintent_inner_sep:
4658     \overleftrightharrow
4659     {
4660       \int_zero:N \l__spintent_sprecta_pt_idx_int
4661       \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
4662       {
4663         \int_incr:N \l__spintent_sprecta_pt_idx_int
4664         \__spintent_mathmlarg_point:nnn
4665         { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
4666         { sprecta }
4667         { ##1 }
4668       }
4669     }
4670   }
4671 }
4672 }

```

(End of definition for `\__spintent_sprecta_mathml:n`.)

`\__spintent_sprecta_exec:nn`

The function `\__spintent_sprecta_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *(argument)* passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

4673 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
4674 {
4675   \keys_set:nn { spintent / sprecta } { #1 }
4676   \__spintent_sprecta_print:n { #2 }
4677 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

4678 \NewDocumentCommand \sprecta { 0{} m }
4679 {
4680   \mode_if_math:TF
4681   {
4682     \group_begin:
4683     \attribute \__spintent_scope_attr_int = 1
4684     \__spintent_sprecta_exec:nn { #1 } { #2 }
4685     \group_end:
4686   }
4687   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
4688 }

```

(End of definition for `\sprecta`. This function is documented on page 20.)

13.43 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

13.43.1 Definition of variables for `\sprayo`

`\__spintent_sprayo_mode_str` The variable `\__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

4689 \str_new:N \__spintent_sprayo_mode_str
4690 \tl_new:N \__spintent_sprayo_prefix_tl
4691 \str_new:N \__spintent_sprayo_read_sty_str

```

(End of definition for `\__spintent_sprayo_mode_str`, `\__spintent_sprayo_prefix_tl`, and `\__spintent_sprayo_read_sty_str`.)

13.43.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points`

Raised when the argument is not a comma-separated pair of uppercase letters.

```

4692 \msg_new:nnnn { spintent } { sprayo-invalid-points }
4693 {
4694   Invalid~argument~'#1'~for~\token_to_str:N \sprayo \c_space_tl \msg_line_context:.
4695 }
4696 {
4697   See~the~documentation~of~\token_to_str:N \sprayo \c_space_tl for~the~accepted~point~formats.
4698 }

```

(End of definition for `sprayo-invalid-points`.)

13.43.3 Definition of keys for `\sprayo`

We define the `(keys)` `read-cmd`, `prefix`, `silent`, `concept` and `read-sty`.

```

4699 \keys_define:nn { spintent / sprayo }
4700 {
4701   read-cmd .choices:nn = { school , mathcat }
4702   {
4703     \int_case:nn { \l_keys_choice_int }
4704     {
4705       {1} { \str_set:Nn \__spintent_sprayo_mode_str { school } }
4706       {2} { \str_set:Nn \__spintent_sprayo_mode_str { mathcat } }
4707     }
4708   },
4709   read-cmd .initial:n = school,
4710   read-cmd .value_required:n = true,
4711   read-cmd / unknown .code:n =
4712     \msg_error:nneee { spintent } { unknown-choice }
4713     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4714   prefix .code:n =
4715     { \__spintent_sanitize_affix:Nn \__spintent_sprayo_prefix_tl {#1} },
4716   prefix .initial:n = ,
4717   prefix .value_required:n = true,
4718   silent .bool_set:N = \__spintent_sprayo_silent_bool,
4719   silent .initial:n = false,
4720   silent .value_required:n = true,
4721   concept .bool_set:N = \__spintent_sprayo_concept_bool,
4722   concept .initial:n = true,
4723   concept .value_required:n = true,

```

```

4724     read-sty           .choices:nn = { rayo , semirrecta }
4725     {
4726         \int_case:nn { \l_keys_choice_int }
4727         {
4728             {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
4729             {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
4730         }
4731     },
4732     read-sty           .initial:n = rayo,
4733     read-sty           .value_required:n = true,
4734     read-sty / unknown .code:n =
4735         \msg_error:nnee { spintent } { unknown-choice }
4736         { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
4737     unknown           .code:n =
4738         {
4739             \str_set:Nn \l__spintent_command_name_str { sprayo }
4740             \__spintent_unknown_keys_cmd:n {#1}
4741         },
4742 }

```

(End of definition for read-cmd and others.)

13.43.4 Internal functions for \sprayo

Auxiliary variables used by __spintent_sprayo_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1,\$pts2,... reference list.

```

\l__spintent_sprayo_pt_seq
\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
4743 \seq_new:N \l__spintent_sprayo_pt_seq
4744 \tl_new:N \l__spintent_sprayo_refs_tl
4745 \tl_new:N \l__spintent_sprayo_intent_arg_tl
4746 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for \l__spintent_sprayo_pt_seq and others.)

__spintent_sprayo_print:n The internal function __spintent_sprayo_print:n process the {<argument>} passed in ‘#1’. It calls __spintent_luafun_geo_parse_and_set:n to process the {<argument>}, returning an “error” if the input is invalid, then calls __spintent_sprayo_mathml:. Unlike \sprecta, \sprayo has no proper-name argument: it always takes a list of points, so no intent is built here.

```

4747 \cs_new_protected:Nn \__spintent_sprayo_print:n
4748 {
4749     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
4750     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4751     {
4752         \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
4753     }
4754     \__spintent_sprayo_mathml:
4755 }

```

(End of definition for __spintent_sprayo_print:n.)

__spintent_sprayo_mathml: The function __spintent_sprayo_mathml: splits \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element) into \l__spintent_sprayo_pt_seq and walks it once to build \l__spintent_sprayo_refs_tl, the comma-separated \$pts1,\$pts2,... reference list, naming each point by position.

The concept intent is _- when silent is true, _(refs) when concept is false; otherwise it is _\l__spintent_sprayo_read_sty_str:prefix(refs) — the concept word itself (rayo or semirrecta) comes from the read-sty key, not a literal, since \sprayo verbalizes under two names — prefixed with prefix when given. It is applied with a native \MathMLintent wrapping \overrightarrow, preceded by __spintent_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>). The sequence is walked a second time, calling __spintent_mathmlarg_point:nnn once per point with its ptsn reference name and silent/read-cmd looked up under sprayo: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

4756 \cs_new_protected:Nn \__spintent_sprayo_mathml:
4757 {
4758     \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
4759     \tl_clear:N \l__spintent_sprayo_refs_tl
4760     \int_zero:N \l__spintent_sprayo_pt_idx_int
4761     \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
4762     {
4763         \int_incr:N \l__spintent_sprayo_pt_idx_int
4764         \tl_if_empty:NF \l__spintent_sprayo_refs_tl
4765         { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }

```

```

4766         \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx
4767     }
4768     \bool_if:nTF
4769     {
4770         \bool_lazy_or_p:nn
4771         { \l__spintent_sprayo_silent_bool }
4772         { ! \l__spintent_sprayo_concept_bool }
4773     }
4774     {
4775         \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _( \l__spintent_sprayo_refs_tl ) }
4776         \bool_if:NT \l__spintent_sprayo_silent_bool
4777         { \tl_set:Nn \l__spintent_sprayo_intent_arg_tl { _- } }
4778     }
4779     {
4780         \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
4781         {
4782             -
4783             \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
4784             { \tl_use:N \l__spintent_sprayo_prefix_tl - }
4785             \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
4786         }
4787     }
4788     \__spintent_invisible_sep:
4789     \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
4790     {
4791         \__spintent_inner_sep:
4792         \overrightarrow
4793         {
4794             \int_zero:N \l__spintent_sprayo_pt_idx_int
4795             \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
4796             {
4797                 \int_incr:N \l__spintent_sprayo_pt_idx_int
4798                 \__spintent_mathmlarg_point:nnn
4799                 { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
4800                 { sprayo }
4801                 { ##1 }
4802             }
4803         }
4804     }
4805 }

```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn`

The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argument \rangle \}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

4806 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
4807 {
4808     \keys_set:nn { spintent / sprayo } { #1 }
4809     \__spintent_sprayo_print:n { #2 }
4810 }

```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo`

The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

4811 \NewDocumentCommand \sprayo { 0{} m }
4812 {
4813     \mode_if_math:TF
4814     {
4815         \group_begin:
4816         \attribute \l__spintent_scope_attr_int = 1
4817         \__spintent_sprayo_exec:nn { #1 } { #2 }
4818         \group_end:
4819     }
4820     { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
4821 }

```

(End of definition for `\sprayo`. This function is documented on page 21.)

13.44 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

13.44.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spinttent_spside_mode_str` and `\__spinttent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spinttent_spside_prefix_tl` and `\__spinttent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\__spinttent_spside_read_sty_str` and `\__spinttent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\__spinttent_spside_read_arg_tl` and `\__spinttent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```
4822 \clist_map_inline:nn { side , mside }
4823 {
4824   \str_new:c { \__spinttent_sp #1 _mode_str }
4825   \tl_new:c { \__spinttent_sp #1 _prefix_tl }
4826   \str_new:c { \__spinttent_sp #1 _read_sty_str }
4827   \tl_new:c { \__spinttent_sp #1 _read_arg_tl }
4828 }
```

(End of definition for `\__spinttent_spside_mode_str` and others.)

13.44.2 Definition of error messages for `\spside` and `\spmside`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘`#1`’ to the command.

```
4829 \clist_map_inline:nn { side , mside }
4830 {
4831   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
4832   {
4833     Invalid~argument~'##2'~for~##1~\msg_line_context:.
4834   }
4835   {
4836     See~the~documentation~of~##1~for~the~accepted~point~formats.
4837   }
4838 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

13.44.3 Definition of keys for `\spside` and `\spmside`

We define the shared `\keys` `read-cmd`, `prefix`, `silent`, `concept`, `read-sty`, `read-arg` and `print-m`.

```
read-cmd
prefix
silent
concept
read-sty
read-arg
print-m
unknown
4839 \clist_map_inline:nn { side , mside }
4840 {
4841   \keys_define:nn { spinttent / sp #1 }
4842   {
4843     read-cmd .choices:nn = { school , mathcat }
4844     {
4845       \int_case:nn { \_keys_choice_int }
4846       {
4847         {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
4848         {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
4849       }
4850     },
4851     read-cmd .initial:n = school,
4852     read-cmd .value_required:n = true,
4853     read-cmd / unknown .code:n =
4854       \msg_error:nneee { spinttent } { unknown-choice }
4855       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4856     prefix .code:n =
4857       { \__spinttent_sanitize_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
4858     prefix .initial:n = ,
4859     prefix .value_required:n = true,
4860     silent .bool_set:c = { \__spinttent_sp #1 _silent_bool },
4861     silent .initial:n = false,
```

```

4862     silent                .value_required:n = true,
4863     concept               .bool_set:c = { l__spintent_sp #1 _concept_bool },
4864     concept               .initial:n = true,
4865     concept               .value_required:n = true,
4866     read-sty              .choices:nn = { lado , segmento , trazo }
4867     {
4868         \int_case:nn { \l_keys_choice_int }
4869         {
4870             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
4871             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
4872             {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
4873         }
4874     },
4875     read-sty              .initial:n = lado,
4876     read-sty              .value_required:n = true,
4877     read-sty / unknown    .code:n =
4878         \msg_error:nnee { spintent } { unknown-choice }
4879         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
4880     read-arg              .code:n =
4881     {
4882         \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
4883     },
4884     read-arg              .initial:n = ,
4885     read-arg              .value_required:n = true,
4886     print-m               .bool_set:c = { l__spintent_sp #1 _print_m_bool },
4887     print-m               .initial:n = false,
4888     print-m               .value_required:n = true,
4889     unknown               .code:n =
4890     {
4891         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4892         \__spintent_unknown_keys_cmd:n {##1}
4893     },
4894 }
4895 }

```

🌱 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

```

\l__spintent_spside_intent_tl
\l__spintent_spside_pt_seq
\l__spintent_spside_refs_tl
\l__spintent_spside_intent_arg_tl
\l__spintent_spside_pt_idx_int

```

The variable `\l__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\l__spintent_geo_luasets_point` and build the `$pts1, $pts2, ...` reference list.

```

4896 \tl_new:N \l__spintent_spside_intent_tl
4897 \seq_new:N \l__spintent_spside_pt_seq
4898 \tl_new:N \l__spintent_spside_refs_tl
4899 \tl_new:N \l__spintent_spside_intent_arg_tl
4900 \int_new:N \l__spintent_spside_pt_idx_int

```

(End of definition for `\l__spintent_spside_intent_tl` and others.)

```
\__spintent_spside_process:nn
```

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘`#1`’ and the `{\argument}` in ‘`#2`’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘`#2`’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘`medida-del-`’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘`medida-del-`’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

4901 \cs_new_protected:Nn \__spintent_spside_process:nn
4902 {
4903     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4904     \str_if_eq:VnT \l__spintent_geo_luasets_error_str { true }
4905     {
4906         \msg_error:nnee { spintent } { sp #1 -invalid-points }
4907         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4908     }
4909     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4910     {
4911         \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str }
4912         \str_if_eq:eeT {#1} { mside }
4913         {
4914             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }

```

```

4915     }
4916   }
4917   {
4918     \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
4919     \bool_if:cT { l__spintent_sp #1 _concept_bool }
4920     {
4921       \str_if_eq:eeT {#1} { mside }
4922       {
4923         \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
4924       }
4925     }
4926   }
4927   \l__spintent_spside_mathml:n { #1 }
4928 }

```

(End of definition for \l__spintent_spside_process:nn.)

\l__spintent_spside_mathml:n

The function `\l__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

```

4929 \cs_new_protected:Nn \l__spintent_spside_mathml:n
4930 {
4931   \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
4932   \tl_clear:N \l__spintent_spside_refs_tl
4933   \int_zero:N \l__spintent_spside_pt_idx_int
4934   \seq_map_inline:Nn \l__spintent_spside_pt_seq
4935   {
4936     \int_incr:N \l__spintent_spside_pt_idx_int
4937     \tl_if_empty:NF \l__spintent_spside_refs_tl
4938     { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
4939     \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
4940   }

```

The concept `intent` is `_` when `silent` is `true`; otherwise, if `read_arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read_arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read_arg` (see `\l__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

4941   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
4942   {
4943     \tl_set:Nn \l__spintent_spside_intent_arg_tl { _- }
4944   }
4945   {
4946     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4947     {
4948       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4949       {
4950         \tl_set:Ne \l__spintent_spside_intent_arg_tl
4951         {
4952           -
4953           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4954           { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4955           \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4956         }
4957       }
4958       {
4959         \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
4960       }
4961     }
4962     {
4963       \tl_set:Ne \l__spintent_spside_intent_arg_tl
4964       {
4965         -
4966         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4967         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4968         \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4969       }
4970     }
4971   }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

4972   \__spintent_invisible_sep: % (luamml issues #26)
4973   \MathMLintent { \__spintent_spside_intent_arg_tl }
4974   {
4975     \__spintent_inner_sep:
4976     \str_if_eq:eeTF {#1} { side }
4977     {
4978       \overline { \__spintent_spside_points_loop:n {#1} }
4979     }
4980     {
4981       \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
4982       {
4983         \__spintent_luafun_mrow_intent_arg:nn
4984         { } { intent = ":silent" }
4985         { \operatorname { m } }
4986         \overline { \__spintent_spside_points_loop:n {#1} }
4987       }
4988       {
4989         \__spintent_spside_points_loop:n {#1}
4990       }
4991     }
4992   }
4993 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in `'#1'` again since a separate macro does not inherit its caller's own `#1`.

```

4994 \cs_new_protected:Nn \__spintent_spside_points_loop:n
4995 {
4996   \int_zero:N \__spintent_spside_pt_idx_int
4997   \seq_map_inline:Nn \__spintent_spside_pt_seq
4998   {
4999     \int_incr:N \__spintent_spside_pt_idx_int
5000     \__spintent_mathmlarg_point:nnn
5001     { pts \int_use:N \__spintent_spside_pt_idx_int }
5002     { sp #1 }
5003     { ##1 }
5004   }
5005 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn`

The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in `'#1'`, the optional `<keys>` passed in `'#2'` and the `<argument>` passed in `'#3'`, delegating the rendering to `\__spintent_spside_process:nn`.

```

5006 \cs_new_protected:Nn \__spintent_spside_exec:nnn
5007 {
5008   \keys_set:nn { spintent / sp #1 } { #2 }
5009   \__spintent_spside_process:nn { #1 } { #3 }
5010 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

5011 \NewDocumentCommand \spside { O{} m }
5012 {
5013   \mode_if_math:TF
5014   {
5015     \group_begin:

```

```

5016         \attribute \l__spintent_scope_attr_int = 1
5017         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
5018     \group_end:
5019 }
5020 { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
5021 }
5022 \NewDocumentCommand \spmside { 0{} m }
5023 {
5024     \mode_if_math:TF
5025     {
5026         \group_begin:
5027         \attribute \l__spintent_scope_attr_int = 1
5028         \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
5029         \group_end:
5030     }
5031     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
5032 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 22.)

13.45 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

13.45.1 Definition of variables for `\spangle` and `\spmangle`

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

5033 \clist_map_inline:nn { angle , mangle }
5034 {
5035     \str_new:c { l__spintent_sp #1 _mode_str }
5036     \tl_new:c { l__spintent_sp #1 _prefix_tl }
5037 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

13.45.2 Definition of error messages for `\spangle` and `\spmangle`

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{\argument}` passed in ‘#1’ to the command.

```

5038 \clist_map_inline:nn { angle , mangle }
5039 {
5040     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5041     {
5042         Invalid~argument~'##2'~for~##1~\msg_line_context:.
5043     }
5044     {
5045         See~the~documentation~of~##1~for~the~accepted~argument~forms.
5046     }
5047 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

13.45.3 Definition of keys for `\spangle` and `\spmangle`

We define the shared `\keys` `read-cmd`, `prefix`, `recto`, `print-m`, `silent` and `concept`,

```

read-cmd
prefix
recto
print-m
silent
concept
unknown
5048 \clist_map_inline:nn { angle , mangle }
5049 {
5050     \keys_define:nn { spintent / sp #1 }
5051     {
5052         read-cmd .choices:nn = { school , mathcat }
5053         {
5054             \int_case:nn { \l_keys_choice_int }
5055             {
5056                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5057                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5058             }

```

```

5059     },
5060     read-cmd          .initial:n = school,
5061     read-cmd          .value_required:n = true,
5062     read-cmd / unknown .code:n =
5063       \msg_error:nneee { spintent } { unknown-choice }
5064       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5065     prefix            .code:n =
5066       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5067     prefix            .initial:n = ,
5068     prefix            .value_required:n = true,
5069     recto              .bool_set:c = { l__spintent_sp #1 _recto_bool },
5070     recto              .initial:n = false,
5071     recto              .value_required:n = true,
5072     print-m            .bool_set:c = { l__spintent_sp #1 _print_m_bool },
5073     print-m            .initial:n = false,
5074     print-m            .value_required:n = true,
5075     silent             .bool_set:c = { l__spintent_sp #1 _silent_bool },
5076     silent             .initial:n = false,
5077     silent             .value_required:n = true,
5078     concept            .bool_set:c = { l__spintent_sp #1 _concept_bool },
5079     concept            .initial:n = true,
5080     concept            .value_required:n = true,
5081     unknown            .code:n =
5082       {
5083         \str_set:Nn \__spintent_command_name_str { sp #1 }
5084         \__spintent_unknown_keys_cmd:n {##1}
5085       },
5086   }
5087 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

13.45.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n`

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

5088 \cs_new_protected:Nn \__spintent_spangle_sym:n
5089 {
5090   \str_if_eq:eeTF {#1} { mangle }
5091   {
5092     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
5093     {
5094       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5095       { \__spintent_right_angle_sqr: } { \angle }
5096     }
5097     {
5098       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5099       { \measuredrightangle } { \measuredangle }
5100     }
5101   }
5102   {
5103     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5104     { \__spintent_right_angle_sqr: } { \angle }
5105   }
5106 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl`
`\l__spintent_spangle_pt_seq`
`\l__spintent_spangle_refs_tl`
`\l__spintent_spangle_intent_arg_tl`
`\l__spintent_spangle_pt_idx_int`
`\l__spintent_spangle_print_m_now_bool`

The variable `\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_luaset_points_clist` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

```

5107 \tl_new:N \l__spintent_spangle_intent_tl
5108 \seq_new:N \l__spintent_spangle_pt_seq
5109 \tl_new:N \l__spintent_spangle_refs_tl
5110 \tl_new:N \l__spintent_spangle_intent_arg_tl
5111 \int_new:N \l__spintent_spangle_pt_idx_int
5112 \str_new:N \l__spintent_spangle_kv_tl
5113 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn`

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘#1’ and the $\langle\langle argument \rangle\rangle$ in ‘#2’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is `true`, and calls `\__spintent_spangle_mathml:n`.

```

5114 \cs_new_protected:Nn \__spintent_spangle_process:nn
5115 {
5116   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
5117   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5118   {
5119     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5120     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5121   }
5122   \str_if_eq:VnT \__spintent_geo_luaset_angulo_case_str { nombre }
5123   {
5124     \str_if_eq:VnF \__spintent_geo_luaset_nombre_es_num_str { true }
5125     {
5126       \int_compare:nT { \str_count:N \__spintent_geo_luaset_letra_str = 1 }
5127       {
5128         \str_if_eq:eeF { \__spintent_geo_luaset_letra_str } { \str_uppercase:n { \__spintent_geo_luaset_letra_str } }
5129         {
5130           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5131           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5132         }
5133       }
5134     }
5135   }
5136   \str_if_eq:eeTF {#1} { mangle }
5137   {
5138     \tl_set:Nn \__spintent_spangle_intent_tl { medida-del-ángulo }
5139   }
5140   {
5141     \tl_set:Nn \__spintent_spangle_intent_tl { ángulo }
5142   }
5143   \bool_if:cT { l__spintent_sp #1 _recto_bool }
5144   {
5145     \tl_put_right:Nn \__spintent_spangle_intent_tl { -recto }
5146   }
5147   \__spintent_spangle_mathml:n {#1}
5148 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_arg` on its own, with `\__spintent_spangle_intent_tl` alone (bare, no $\$$ -reference: `nombre` never refers to more than one thing) — `-` when `silent` is `true` or `concept` is `false`. With `silent true`, the name that follows is silenced as a whole.

```

5149 \cs_new_protected:Nn \__spintent_spangle_mathml:n
5150 {
5151   \__spintent_invisible_sep: % (luamml issues #26)
5152   \str_if_eq:VnTF \__spintent_geo_luaset_angulo_case_str { nombre }
5153   {
5154     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5155     {
5156       \tl_set:Nn \__spintent_spangle_intent_arg_tl { - }
5157     }
5158     {
5159       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5160       {
5161         \tl_set:Nn \__spintent_spangle_intent_arg_tl
5162         {
5163           -
5164           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5165           { \tl_use:c { l__spintent_sp #1 _prefix_tl } } - }
5166         \__spintent_spangle_intent_tl

```

```

5167         }
5168     }
5169     {
5170         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
5171     }
5172 }
5173 \tl_set:Ne \l__spintent_spangle_kv_tl
5174 { intent = "\l__spintent_spangle_intent_arg_tl" }
5175 \__spintent_luafun_mrow_intent_arg:nn
5176 { } { \l__spintent_spangle_kv_tl }
5177 { \__spintent_spangle_sym:n {#1} }
5178 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5179 {
5180     \__spintent_luafun_mrow_intent_arg:nn { } { intent = "-_" }
5181     { \l__spintent_geo_luaset_print_tl }
5182 }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

5183     {
5184         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
5185         {
5186             \tl_set:Ne \l__spintent_spangle_kv_tl
5187             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
5188             \__spintent_luafun_mrow_intent_arg:nn
5189             { } { \l__spintent_spangle_kv_tl }
5190             { \l__spintent_geo_luaset_print_tl }
5191         }
5192     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

5192     {
5193         \str_case:VnF \l__spintent_geo_luaset_tipo_str
5194         {
5195             { sub }
5196             {
5197                 \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
5198                 {
5199                     \tl_set:Ne \l__spintent_spangle_kv_tl
5200                     { intent = "_ \l__spintent_geo_luaset_intent_str -" }
5201                     \__spintent_luafun_mrow_intent_arg:nn
5202                     { } { \l__spintent_spangle_kv_tl }
5203                     {
5204                         \l__spintent_geo_luaset_letra_str
5205                         \c_math_subscript_token
5206                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
5207                     }
5208                 }
5209                 {
5210                     \l__spintent_geo_luaset_letra_str
5211                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
5212                 }
5213             }
5214         }
5215     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

5214     { prima }
5215     {
5216         \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
5217         {
5218             \tl_set:Ne \l__spintent_spangle_kv_tl
5219             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
5220             \__spintent_luafun_mrow_intent_arg:nn
5221             { } { \l__spintent_spangle_kv_tl }
5222             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
5223         }
5224         {
5225             \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
5226         }
5227     }

```

```

5227         }
5228     }
5229     { \l__spintent_geo_luaset_print_tl }
5230 }
5231 }
5232 }

```

For tres-puntos/un-punto, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(ref - _ -` when `silent` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\l__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\l__spintent_mathmlarg_point:nnn` once per point.

```

5233 {
5234   \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
5235   \tl_clear:N \l__spintent_spangle_refs_tl
5236   \int_zero:N \l__spintent_spangle_pt_idx_int
5237   \seq_map_inline:Nn \l__spintent_spangle_pt_seq
5238   {
5239     \int_incr:N \l__spintent_spangle_pt_idx_int
5240     \tl_if_empty:NF \l__spintent_spangle_refs_tl
5241     { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
5242     \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_
5243   }
5244   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5245   {
5246     \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _ - }
5247   }
5248   {
5249     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5250     {
5251       \tl_set:Ne \l__spintent_spangle_intent_arg_tl
5252       {
5253         -
5254         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5255         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5256         \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
5257       }
5258     }
5259     {
5260       \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _( \l__spintent_spangle_refs_tl )
5261     }
5262   }
5263   \bool_set_false:N \l__spintent_spangle_print_m_now_bool
5264   \str_if_eq:eeT {#1} { mangle }
5265   {
5266     \bool_if:cT { l__spintent_sp #1 _print_m_bool }
5267     { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
5268   }
5269   \MathMLintent { \l__spintent_spangle_intent_arg_tl }
5270   {
5271     \l__spintent_inner_sep:
5272     \bool_if:NT \l__spintent_spangle_print_m_now_bool
5273     {
5274       \l__spintent_luafun_mrow_intent_arg:nn
5275       { } { intent = ":silent" }
5276       { \operatorname { m } }
5277     }
5278     \l__spintent_luafun_mrow_intent_arg:nn
5279     { } { intent = ":silent" }
5280     { \l__spintent_spangle_sym:n {#1} }
5281     \int_zero:N \l__spintent_spangle_pt_idx_int
5282     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
5283     {
5284       \int_incr:N \l__spintent_spangle_pt_idx_int
5285       \l__spintent_mathmlarg_point:nnn
5286       { pt \int_use:N \l__spintent_spangle_pt_idx_int }
5287       { sp#1 }
5288       { ##1 }
5289     }
5290   }

```

```

5291     }
5292 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn` The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

5293 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
5294 {
5295   \keys_set:nn { spintent / sp #1 } { #2 }
5296   \__spintent_spangle_process:nn { #1 } { #3 }
5297 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle` Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

5298 \NewDocumentCommand \spangle { 0{} m }
5299 {
5300   \mode_if_math:TF
5301   {
5302     \group_begin:
5303     \attribute \__spintent_scope_attr_int = 1
5304     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
5305     \group_end:
5306   }
5307   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
5308 }
5309 \NewDocumentCommand \spmangle { 0{} m }
5310 {
5311   \mode_if_math:TF
5312   {
5313     \group_begin:
5314     \attribute \__spintent_scope_attr_int = 1
5315     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
5316     \group_end:
5317   }
5318   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
5319 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 23.)

13.46 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

13.46.1 Definition of variables for `\sparc` and `\spmarc`

`\__spintent_sparc_mode_str` Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_smparc_prefix_tl` store the values set by the key `prefix`.

```

5320 \clist_map_inline:nn { arc , marc }
5321 {
5322   \str_new:c { \__spintent_sp #1 _mode_str }
5323   \tl_new:c { \__spintent_sp #1 _prefix_tl }
5324 }

```

(End of definition for `\__spintent_sparc_mode_str` and others.)

13.46.2 Definition of error messages for `\sparc` and `\spmarc`

`sparc-invalid-points` We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid *{argument}* passed in ‘#1’ to the command.

```

5325 \clist_map_inline:nn { arc , marc }
5326 {
5327   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
5328   {
5329     Invalid~argument~'##2'~for~##1~\msg_line_context:.
5330   }
5331 }

```

```

5332         See~the~documentation~of~##1~for~the~accepted~point~formats.
5333     }
5334 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

13.46.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared *(keys)* `read-cmd`, `prefix`, `silent` and `concept`.

```

read-cmd 5335 \clist_map_inline:nn { arc , marc }
prefix   5336 {
silent    5337   \keys_define:nn { spintent / sp #1 }
concept   5338   {
unknown   5339     read-cmd           .choices:nn = { school , mathcat }
           5340     {
           5341       \int_case:nn { \l_keys_choice_int }
           5342       {
           5343         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
           5344         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
           5345       }
           5346     },
           5347     read-cmd           .initial:n = school,
           5348     read-cmd           .value_required:n = true,
           5349     read-cmd / unknown .code:n =
           5350       \msg_error:nneee { spintent } { unknown-choice }
           5351       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
           5352     prefix           .code:n =
           5353       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
           5354     prefix           .initial:n = ,
           5355     prefix           .value_required:n = true,
           5356     silent            .bool_set:c = { l__spintent_sp #1 _silent_bool },
           5357     silent            .initial:n = false,
           5358     silent            .value_required:n = true,
           5359     concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
           5360     concept           .initial:n = true,
           5361     concept           .value_required:n = true,
           5362     unknown           .code:n =
           5363       {
           5364         \str_set:Nn \l__spintent_command_name_str { sp #1 }
           5365         \__spintent_unknown_keys_cmd:n {##1}
           5366       },
           5367   }
           5368 }

```

(End of definition for `read-cmd` and others.)

13.46.4 Internal functions for `\sparc` and `\spmarc`

`\__spintent_sparc_wide_arc:n`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

5369 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
5370 {
5371   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
5372 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

`\__spintent_sparc_print_arc:n`

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

5373 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
5374 {
5375   \cs_if_exist:NTF \widearc
5376   { \widearc {#1} }
5377   { \__spintent_sparc_wide_arc:n {#1} }
5378 }

```

(End of definition for `\__spintent_sparc_print_arc:n`.)

`\l__spintent_sparc_intent_tl`

The variable `\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list used in the concept intent.

```

5379 \tl_new:N \l__spintent_sparc_intent_tl

```

```

5380 \seq_new:N \l__spintent_sparc_pt_seq
5381 \tl_new:N \l__spintent_sparc_refs_tl
5382 \tl_new:N \l__spintent_sparc_intent_arg_tl
5383 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_math` builds after splitting the points.

```

5384 \cs_new_protected:Nn \__spintent_sparc_process:nn
5385 {
5386   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
5387   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5388   {
5389     \msg_error:nnee { spintent } { sp #1 -invalid-points }
5390     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5391   }
5392   \str_if_eq:eeTF {#1} { marc }
5393   { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
5394   { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
5395   \__spintent_sparc_mathml:n {#1}
5396 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated $\$pt_1, \$pt_2, \dots$ reference list, naming each point by position.

```

5397 \cs_new_protected:Nn \__spintent_sparc_mathml:n
5398 {
5399   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
5400
5401   \tl_clear:N \l__spintent_sparc_refs_tl
5402   \int_zero:N \l__spintent_sparc_pt_idx_int
5403   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
5404   {
5405     \int_incr:N \l__spintent_sparc_pt_idx_int
5406     \tl_if_empty:NF \l__spintent_sparc_refs_tl
5407     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
5408     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int
5409   }

```

The concept `intent` is `_` when `silent` is `true`, `_(refs)` when `concept` is `false`; otherwise `_\l__spintent_sparc` with `prefix` placed after the leading `_` when given.

```

5410   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5411   {
5412     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
5413   }
5414   {
5415     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5416     {
5417       \tl_set:Ne \l__spintent_sparc_intent_arg_tl
5418       {
5419         -
5420         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5421         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5422         \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
5423       }
5424     }
5425     {
5426       \tl_set:Ne \l__spintent_sparc_intent_arg_tl { _ ( \l__spintent_sparc_refs_tl ) }
5427     }
5428   }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}`

precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

5429   \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
5430   \MathMLintent { \__spintent_sparc_intent_arg_tl }
5431   {
5432     \__spintent_inner_sep:
5433     \str_if_eq:eeT {#1} { marc }
5434     {
5435       \__spintent_luafun_mrow_intent_arg:nn
5436       { } { intent = ":silent" }
5437       { \operatorname { m } } }
5438     }
5439     \__spintent_sparc_print_arc:n
5440     {
5441       \int_zero:N \__spintent_sparc_pt_idx_int
5442       \seq_map_inline:Nn \__spintent_sparc_pt_seq
5443       {
5444         \int_incr:N \__spintent_sparc_pt_idx_int
5445         \__spintent_mathmlarg_point:nnn
5446         { pt \int_use:N \__spintent_sparc_pt_idx_int }
5447         { sp #1 }
5448         { ##1 }
5449       }
5450     }
5451   }
5452 }

```

(End of definition for `\__spintent_sparc_mathml:n`.)

`\__spintent_sparc_exec:nnn` The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

5453 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
5454 {
5455   \keys_set:nn { spintent / sp #1 } { #2 }
5456   \__spintent_sparc_process:nn { #1 } { #3 }
5457 }

```

(End of definition for `\__spintent_sparc_exec:nnn`.)

`\sparc` Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

5458 \NewDocumentCommand \sparc { O{} m }
5459 {
5460   \mode_if_math:TF
5461   {
5462     \group_begin:
5463     \attribute \__spintent_scope_attr_int = 1
5464     \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
5465     \group_end:
5466   }
5467   { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
5468 }
5469 \NewDocumentCommand \spmarc { O{} m }
5470 {
5471   \mode_if_math:TF
5472   {
5473     \group_begin:
5474     \attribute \__spintent_scope_attr_int = 1
5475     \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
5476     \group_end:
5477   }
5478   { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
5479 }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 25.)

13.47 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaaset_intent_str` (the full *intent*, with the figure name).

13.47.1 Definition of variables for \spPoly

The variable `\l__spinttent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spinttent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```
5480 \str_new:N \l__spinttent_spPoly_mode_str
5481 \tl_new:N \l__spinttent_spPoly_prefix_tl
```

(End of definition for `\l__spinttent_spPoly_mode_str` and `\l__spinttent_spPoly_prefix_tl`.)

13.47.2 Definition of error messages for \spPoly

`spPoly-invalid-points` Raised when the argument is not a comma-separated list of at least three uppercase letters.

```
5482 \msg_new:nnnn { spinttent } { spPoly-invalid-points }
5483 {
5484   Invalid~argument~'#1'~for~\token_to_str:N \spPoly \c_space_tl \msg_line_context:.
5485 }
5486 {
5487   See~the~documentation~of~\token_to_str:N \spPoly \c_space_tl for~the~accepted~point~formats.
5488 }
```

(End of definition for `spPoly-invalid-points`.)

13.47.3 Definition of keys for \spPoly

We define the *(keys)* `read-cmd`, `prefix`, `print-sym`, `silent` and `concept`.

```
read-cmd 5489 \keys_define:nn { spinttent / spPoly }
prefix
print-sym 5490 {
silent 5491   read-cmd .choices:nn = { school , mathcat }
concept 5492   {
unknown 5493     \int_case:nn { \l_keys_choice_int }
5494       {
5495         {1} { \str_set:Nn \l__spinttent_spPoly_mode_str { school } }
5496         {2} { \str_set:Nn \l__spinttent_spPoly_mode_str { mathcat } }
5497       }
5498     },
5499     read-cmd .initial:n = school,
5500     read-cmd .value_required:n = true,
5501     read-cmd / unknown .code:n =
5502       \msg_error:nneee { spinttent } { unknown-choice }
5503       { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
5504     prefix .code:n =
5505       { \__spinttent_sanitize_affix:Nn \l__spinttent_spPoly_prefix_tl {#1} },
5506     prefix .initial:n = ,
5507     prefix .value_required:n = true,
5508     print-sym .bool_set:N = \l__spinttent_spPoly_print_sym_bool,
5509     print-sym .initial:n = true,
5510     print-sym .value_required:n = true,
5511     silent .bool_set:N = \l__spinttent_spPoly_silent_bool,
5512     silent .initial:n = false,
5513     silent .value_required:n = true,
5514     concept .bool_set:N = \l__spinttent_spPoly_concept_bool,
5515     concept .initial:n = true,
5516     concept .value_required:n = true,
5517     unknown .code:n =
5518       {
5519         \str_set:Nn \l__spinttent_command_name_str { spPoly }
5520         \__spinttent_unknown_keys_cmd:n {#1}
5521       },
5522   }
```

(End of definition for `read-cmd` and others.)

13.47.4 Internal functions for \spPoly

Scratch variables to split `\l__spinttent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list, and to hold the final concept `intent` once built.

```
5523 \seq_new:N \l__spinttent_spPoly_pt_seq
5524 \tl_new:N \l__spinttent_spPoly_refs_tl
5525 \tl_new:N \l__spinttent_spPoly_intent_arg_tl
5526 \int_new:N \l__spinttent_spPoly_pt_idx_int
```

(End of definition for `\l__spinttent_spPoly_pt_seq` and others.)

`\__spintent_spPoly_sym:` Unchanged: reads `\__spintent_geo_luaset_poly_sym_str` and typesets the corresponding symbol.

```

5527 \cs_new_protected:Nn \__spintent_spPoly_sym:
5528 {
5529   \str_case:Vn \__spintent_geo_luaset_poly_sym_str
5530   {
5531     { triangle } { \triangle }
5532     { square } { \mdlgwhtsquare }
5533   }
5534 }

```

(End of definition for `\__spintent_spPoly_sym:`.)

`\__spintent_spPoly_build_refs:` Splits `\__spintent_geo_luaset_points_clist` into `\__spintent_spPoly_pt_seq` and walks it once to build `\__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

5535 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
5536 {
5537   \seq_set_from_clist:NN \__spintent_spPoly_pt_seq \__spintent_geo_luaset_points_clist
5538   \tl_clear:N \__spintent_spPoly_refs_tl
5539   \int_zero:N \__spintent_spPoly_pt_idx_int
5540   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
5541   {
5542     \int_incr:N \__spintent_spPoly_pt_idx_int
5543     \tl_if_empty:NF \__spintent_spPoly_refs_tl
5544     { \tl_put_right:Nn \__spintent_spPoly_refs_tl { , } }
5545     \tl_put_right:Ne \__spintent_spPoly_refs_tl { $ pt \int_use:N \__spintent_spPoly_pt_idx_int }
5546   }
5547 }

```

(End of definition for `\__spintent_spPoly_build_refs:`.)

`\__spintent_spPoly_build_concept:` Builds `\__spintent_spPoly_intent_arg_tl`, independently of `print-sym:` `_-` when `silent` is `true`, `_(refs)` when `concept` is `false`; otherwise `\__spintent_geo_luaset_concept_str:prefix(refs)`, with `prefix` placed after the leading `_` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

5548 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
5549 {
5550   \bool_if:NTF \__spintent_spPoly_silent_bool
5551   {
5552     \tl_set:Nn \__spintent_spPoly_intent_arg_tl { _- }
5553   }
5554   {
5555     \bool_if:NTF \__spintent_spPoly_concept_bool
5556     {
5557       \tl_set:Ne \__spintent_spPoly_intent_arg_tl
5558       {
5559         -
5560         \tl_if_empty:NF \__spintent_spPoly_prefix_tl
5561         { \tl_use:N \__spintent_spPoly_prefix_tl - }
5562         \__spintent_geo_luaset_concept_str :prefix( \__spintent_spPoly_refs_tl )
5563       }
5564     }
5565     {
5566       \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _( \__spintent_spPoly_refs_tl ) }
5567     }
5568   }
5569 }

```

(End of definition for `\__spintent_spPoly_build_concept:`.)

`\__spintent_spPoly_points_loop:` Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

5570 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
5571 {
5572   \int_zero:N \__spintent_spPoly_pt_idx_int
5573   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
5574   {
5575     \int_incr:N \__spintent_spPoly_pt_idx_int
5576     \__spintent_mathmlarg_point:nnn
5577     { pt \int_use:N \__spintent_spPoly_pt_idx_int }

```

```

5578         { spPoly }
5579         { ##1 }
5580     }
5581 }

```

(End of definition for `\__spintent_spPoly_points_loop:`)

`\__spintent_spPoly_process:n` Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly`-invalid-points on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

5582 \cs_new_protected:Nn \__spintent_spPoly_process:n
5583 {
5584     \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
5585     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5586     {
5587         \msg_error:nne { spintent } { spPoly-invalid-points }
5588         { \exp_not:n {#1} }
5589     }
5590     \__spintent_spPoly_mathml:
5591 }

```

(End of definition for `\__spintent_spPoly_process:n`.)

`\__spintent_spPoly_mathml:` Builds the reference list and the concept `intent` (`\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_concept:`) then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).

The keys `print-sym` and `concept/silent` are independent: the former only decides whether the glyph (`\__spintent_spPoly_sym:`, silenced with `\__spintent_luafun_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

5592 \cs_new_protected:Nn \__spintent_spPoly_mathml:
5593 {
5594     \__spintent_invisible_sep: % (luamml issues #26)
5595     \__spintent_spPoly_build_refs:
5596     \__spintent_spPoly_build_concept:
5597     \MathMLintent { \__spintent_spPoly_intent_arg_tl }
5598     {
5599         \__spintent_inner_sep:
5600         \bool_if:NT \__spintent_spPoly_print_sym_bool
5601         {
5602             \str_if_empty:NF \__spintent_geo_luaset_poly_sym_str
5603             {
5604                 \__spintent_luafun_mrow_intent_arg:nn
5605                 { } { intent = ":silent" }
5606                 { \__spintent_spPoly_sym: }
5607             }
5608         }
5609         \__spintent_spPoly_points_loop:
5610     }
5611 }

```

(End of definition for `\__spintent_spPoly_mathml:`.)

`\__spintent_spPoly_exec:nn` The function `\__spintent_spPoly_exec:nn` receives the optional keys in `#1` and the argument in `#2`, delegating the rendering to `\__spintent_spPoly_process:n`.

```

5612 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
5613 {
5614     \keys_set:nn { spintent / spPoly } { #1 }
5615     \__spintent_spPoly_process:n { #2 }
5616 }

```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly` The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

5617 \NewDocumentCommand \spPoly { 0{ } m }
5618 {
5619     \mode_if_math:TF
5620     {
5621         \group_begin:
5622         \attribute \__spintent_scope_attr_int = 1

```

```

5623         \__spintent_spPoly_exec:nn { #1 } { #2 }
5624     \group_end:
5625 }
5626 { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
5627 }

```

(End of definition for `\spPoly`. This function is documented on page 26.)

13.48 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{<:silent>}`.

13.48.1 Definition of variables for `\spAfig` and `\spPfig`

```
\__spintent_geo_final_print_tl
```

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. The variable `\__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```
5628 \tl_new:N \__spintent_geo_final_print_tl
```

(End of definition for `\__spintent_geo_final_print_tl`.)

```
\__spintent_spAfig_mode_str
```

```
\__spintent_spPfig_mode_str
```

```
\__spintent_spAfig_print_sym_str
```

```
\__spintent_spPfig_print_sym_str
```

```
\__spintent_spAfig_prefix_tl
```

```
\__spintent_spPfig_prefix_tl
```

```
\__spintent_spAfig_read_arg_tl
```

```
\__spintent_spPfig_read_arg_tl
```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

5629 \clist_map_inline:nn { Afig , Pfig }
5630 {
5631     \str_new:c { \__spintent_sp #1 _mode_str      }
5632     \str_new:c { \__spintent_sp #1 _print_sym_str }
5633     \tl_new:c  { \__spintent_sp #1 _prefix_tl     }
5634     \tl_new:c  { \__spintent_sp #1 _read_arg_tl   }
5635 }

```

(End of definition for `\__spintent_spAfig_mode_str` and others.)

13.48.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```

5636 \clist_map_inline:nn { Afig , Pfig }
5637 {
5638     \msg_new:nnnn { spintent } { sp #1 -invalid-points }
5639     {
5640         Invalid~argument~'##2'~for~##1~\msg_line_context:.
5641     }
5642     {
5643         See~the~documentation~of~##1~for~the~accepted~point~formats.
5644     }
5645     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5646     {
5647         Invalid~argument~'##2'~for~##1~\msg_line_context:.
5648     }
5649     {
5650         See~the~documentation~of~##1~for~the~accepted~argument~forms.
5651     }
5652     \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
5653     {
5654         print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
5655     }
5656     {
5657         See~the~documentation~of~##1~for~details~on~print-sym.
5658     }
5659     \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
5660     {
5661         print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
5662     }
5663     {
5664         See~the~documentation~of~##1~for~details~on~print-sym.
5665     }

```

```

5666 \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
5667 {
5668   read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
5669 }
5670 {
5671   See~the~documentation~of~##1~for~details~on~read-arg.
5672 }
5673 }

```

(End of definition for `spAfig`-invalid-points and others.)

13.48.3 Definition of keys for `\spAfig` and `\spPfig`

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle`/square force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle`/square nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

5674 \clist_map_inline:nn { Afig , Pfig }
5675 {
5676   \keys_define:nn { spintent / sp #1 }
5677   {
5678     read-cmd .choices:nn = { school , mathcat }
5679     {
5680       \int_case:nn { \l_keys_choice_int }
5681       {
5682         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5683         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5684       }
5685     },
5686     read-cmd .initial:n = school,
5687     read-cmd .value_required:n = true,
5688     read-cmd / unknown .code:n =
5689       \msg_error:nneee { spintent } { unknown-choice }
5690       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5691     prefix .code:n =
5692       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5693     prefix .initial:n = ,
5694     prefix .value_required:n = true,
5695     print-sym .choices:nn = { auto , triangle , square , none }
5696     {
5697       \int_case:nn { \l_keys_choice_int }
5698       {
5699         {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
5700         {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
5701         {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
5702         {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
5703       }
5704     },
5705     print-sym .initial:n = auto,
5706     print-sym .value_required:n = true,
5707     print-sym / unknown .code:n =
5708       \msg_error:nneee { spintent } { unknown-choice }
5709       { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
5710     read-arg .code:n =
5711       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
5712     read-arg .initial:n = ,
5713     read-arg .value_required:n = true,
5714     read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5715     read-sub .initial:n = false,
5716     read-sub .value_required:n = true,
5717     silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
5718     silent .initial:n = false,
5719     silent .value_required:n = true,
5720     concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
5721     concept .initial:n = true,
5722     concept .value_required:n = true,
5723     concept* .meta:n =
5724     {
5725       concept = false,

```

```

5726         read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
5727     },
5728     concept*           .value_forbidden:n = true,
5729     unknown            .code:n =
5730     {
5731         \str_set:Nn \l__spintent_command_name_str { sp #1 }
5732         \__spintent_unknown_keys_cmd:n {##1}
5733     },
5734 }
5735 }

```

(End of definition for read-cmd and others.)

13.48.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

5736 \cs_new_protected:Nn \__spintent_spfig_sym:
5737 {
5738     \str_case:Nn \l__spintent_geo_print_sym_str
5739     {
5740         { triangle } { \triangle }
5741         { square } { \mdlgwhtsquare }
5742     }
5743 }

```

(End of definition for `\__spintent_spfig_sym:`.)

`\l__spintent_spfig_op_tl` The variable `\l__spintent_spfig_op_tl` holds `área` or `perímetro`. The variable `\l__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list.

```

\l__spintent_spfig_op_tl
\l__spintent_spfig_intent_tl
\l__spintent_spfig_symbol_tl
\l__spintent_spfig_pt_seq
\l__spintent_spfig_refs_tl
\l__spintent_spfig_intent_arg_tl
\l__spintent_spfig_pt_idx_int
\l__spintent_geo_print_sym_str
\l__spintent_geo_temp_tl
5744 \tl_new:N \l__spintent_spfig_op_tl
5745 \tl_new:N \l__spintent_spfig_intent_tl
5746 \tl_new:N \l__spintent_spfig_desc_tl
5747 \tl_new:N \l__spintent_spfig_symbol_tl
5748 \seq_new:N \l__spintent_spfig_pt_seq
5749 \tl_new:N \l__spintent_spfig_refs_tl
5750 \tl_new:N \l__spintent_spfig_intent_arg_tl
5751 \int_new:N \l__spintent_spfig_pt_idx_int
5752 \str_new:N \l__spintent_geo_print_sym_str
5753 \tl_new:N \l__spintent_geo_temp_tl

```

(End of definition for `\l__spintent_spfig_op_tl` and others.)

`\__spintent_spfig_process:nn` Receives the short command name (Afig or Pfig) in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. Calls `\__spintent_luafun_geo` or `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and warns with `sp#1-read-arg-key` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect and is ignored. Sets `\l__spintent_spfig_op_tl` to `área`/`perímetro`, then delegates to `\__spintent_spfig_mathml:n`.

```

5754 \cs_new_protected:Nn \__spintent_spfig_process:nn
5755 {
5756     \str_if_eq:eeTF {#1} { Afig }
5757     { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5758     { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5759     \str_if_eq:NnT \l__spintent_geo_luaset_error_str { true }
5760     {
5761         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5762         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5763     }
5764     \clist_if_empty:NT \l__spintent_geo_luaset_points_clist
5765     {
5766         \tl_if_empty:cF { \l__spintent_sp #1 _read_arg_tl }
5767         {
5768             \msg_warning:nne { spintent } { sp #1 -read-arg-key }
5769             { \c_backslash_str sp #1 }
5770         }
5771     }
5772     \str_if_eq:eeTF {#1} { Afig }
5773     { \tl_set:Nn \l__spintent_spfig_op_tl { área } }
5774     { \tl_set:Nn \l__spintent_spfig_op_tl { perímetro } }
5775     \str_set:Ne \l__spintent_geo_print_sym_str { \str_use:c { \l__spintent_sp #1 _print_sym_str } }
5776     \__spintent_spfig_mathml:n {#1}
5777 }

```

(End of definition for `\__spintent_sfig_process:nn`.)

`\__spintent_sfig_build_simple:n`

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\l__spintent_geo_final_print_tl` (the visual placed under `\mathcal`'s subscript) and `\l__spintent_sfig_desc_tl` (the descriptive part, without *área/perímetro*). Receives the short command name in `#1`.

With an empty argument, both are cleared for `print-sym auto/ none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

5778 \cs_new_protected:Nn \__spintent_sfig_build_simple:n
5779 {
5780   \tl_if_empty:NTF { \l__spintent_geo_luaset_print_tl }
5781   {
5782     \str_case:VnF \l__spintent_geo_print_sym_str
5783     {
5784       { auto }
5785       {
5786         \tl_clear:N \l__spintent_geo_final_print_tl
5787         \tl_clear:N \l__spintent_sfig_desc_tl
5788       }
5789       { none }
5790       {
5791         \tl_clear:N \l__spintent_geo_final_print_tl
5792         \tl_clear:N \l__spintent_sfig_desc_tl
5793       }
5794     }
5795   }
5796   \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_sfig_sym: }
5797   \tl_set:Ne \l__spintent_sfig_desc_tl
5798   {
5799     \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
5800     { triángulo } { cuadrado }
5801   }
5802 }
5803

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

5804 {
5805   \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
5806   {
5807     \str_case:VnF \l__spintent_geo_print_sym_str
5808     {
5809       { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_letra_word_str } }
5810       { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_letra_word_str } }
5811     }
5812   }
5813   \tl_set:Nn \l__spintent_geo_final_print_tl
5814   {
5815     \__spintent_sfig_sym:
5816     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
5817   }
5818 }
5819 \str_case:VnF \l__spintent_geo_print_sym_str
5820 {
5821   { auto }
5822   {
5823     \tl_set:Ne \l__spintent_sfig_desc_tl
5824     {
5825       \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5826       \l__spintent_geo_luaset_sub_spoken_str
5827     }
5828   }
5829   { none }
5830   {
5831     \tl_set:Ne \l__spintent_sfig_desc_tl
5832     {
5833       \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5834       \l__spintent_geo_luaset_sub_spoken_str
5835     }
5836   }

```

```

5837     }
5838     {
5839         \tl_set:Ne \l__spintent_spfig_desc_tl
5840         {
5841             del-
5842             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
5843             { triángulo } { cuadrado }
5844             -
5845             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5846             \l__spintent_geo_luaset_sub_spoken_str
5847         }
5848     }
5849 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value `as-is`; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

5850     {
5851         \str_case:VnF \l__spintent_geo_print_sym_str
5852         {
5853             { auto } { }
5854             { none } { }
5855         }
5856         {
5857             \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
5858             { \c_backslash_str sp #1 }
5859         }
5860         \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
5861         \tl_set:Ne \l__spintent_spfig_desc_tl
5862         {
5863             \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
5864             { \l__spintent_geo_luaset_letra_conector_str - }
5865             \l__spintent_geo_luaset_letra_word_str
5866             \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
5867             {
5868                 -
5869                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5870                 \l__spintent_geo_luaset_sub_spoken_str
5871             }
5872         }
5873     }
5874 }

```

The variable `\l__spintent_spfig_intent_tl` is always the word- shape, without the leading `_`: `\l__spintent_spfig_intent_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent=false` and `concept false`, it becomes `\l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro* nor, for a table letter, its connector *de-la/de*l); with `concept true` it is `_`, then `prefix` when given, and then `\l__spintent_spfig_intent_tl`.

```

5875     \tl_set:Ne \l__spintent_spfig_intent_tl
5876     {
5877         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
5878         { \l__spintent_spfig_op_tl - }
5879         { \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
5880     }
5881     \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5882     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
5883     {
5884         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5885         {
5886             \tl_set:Ne \l__spintent_spfig_intent_arg_tl
5887             {
5888                 -
5889                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
5890                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
5891                 \l__spintent_spfig_intent_tl
5892             }
5893         }
5894         {
5895             \tl_set_eq:NN \l__spintent_geo_temp_tl \l__spintent_spfig_desc_tl

```

```

5896         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
5897         {
5898             \tl_replace_once:Nee \l__spintent_geo_temp_tl
5899             { \l__spintent_geo_luaset_letra_conector_str - } { }
5900         }
5901         \tl_set:N \l__spintent_spfig_intent_arg_tl { _ \l__spintent_geo_temp_tl - }
5902     }
5903 }
5904 }

```

(End of definition for \l__spintent_spfig_build_simple:n.)

\l__spintent_spfig_build_refs: Splits \l__spintent_geo_luaset_points_clist into \l__spintent_spfig_pt_seq and walks it once to build \l__spintent_spfig_refs_tl, the comma-separated \$pt1,\$pt2,... reference list, naming each vertex by position.

```

5905 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
5906 {
5907     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
5908     \tl_clear:N \l__spintent_spfig_refs_tl
5909     \int_zero:N \l__spintent_spfig_pt_idx_int
5910     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
5911     {
5912         \int_incr:N \l__spintent_spfig_pt_idx_int
5913         \tl_if_empty:NF \l__spintent_spfig_refs_tl
5914         { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
5915         \tl_put_right:Ne \l__spintent_spfig_refs_tl { $ pt \int_use:N \l__spintent_spfig_pt_idx_int
5916     }
5917 }

```

(End of definition for \l__spintent_spfig_build_refs:.)

\l__spintent_spfig_build_concept_vertices:n Handles the vertices case. Receives the short command name in ‘#1’. Must be called after \l__spintent_spfig_build_refs:n. The key read-arg, when given, overrides the auto-detected figure name in \l__spintent_spfig_intent_tl: with concept true it is still prefixed with área-del-/perímetro-del-, with concept false it is used verbatim. With read-arg empty, \l__spintent_spfig_intent_tl is simply \l__spintent_geo_luaset_concept_

```

5918 \cs_new_protected:Nn \l__spintent_spfig_build_concept_vertices:n
5919 {
5920     \tl_if_empty:cTF { \l__spintent_sp #1_read_arg_tl }
5921     { \tl_set:N \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
5922     {
5923         \bool_if:cTF { \l__spintent_sp #1_concept_bool }
5924         {
5925             \tl_set:N \l__spintent_spfig_intent_tl
5926             {
5927                 \l__spintent_spfig_op_tl -del-
5928                 \tl_use:c { \l__spintent_sp #1_read_arg_tl }
5929             }
5930         }
5931         {
5932             \tl_set:N \l__spintent_spfig_intent_tl { \tl_use:c { \l__spintent_sp #1_read_arg_tl } }
5933         }
5934     }

```

Determines \l__spintent_spfig_symbol_tl (empty when none applies): for print-sym=auto, the symbol matching \l__spintent_geo_luaset_poly_sym_str (\triangle/ \mdlgwhtsquare), or none when that is empty; print-sym=none always clears it; a forced print-sym requires it to match the actual vertex count, reporting sp#1-sym-cardinality-conflict otherwise.

```

5935     \str_case:VnF \l__spintent_geo_print_sym_str
5936     {
5937         { auto }
5938         {
5939             \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
5940             {
5941                 { triangle } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \triangle } }
5942                 { square } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \mdlgwhtsquare } }
5943             }
5944             \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
5945             { \tl_clear:N \l__spintent_spfig_symbol_tl }
5946         }
5947         { none }
5948         { \tl_clear:N \l__spintent_spfig_symbol_tl }

```

```

5949     }
5950     {
5951         \str_if_eq:VVTF \__spintent_geo_print_sym_str \__spintent_geo_luaset_poly_sym_str
5952         { \tl_set:Nn \__spintent_spfig_symbol_tl { \__spintent_spfig_sym: } }
5953         {
5954             \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
5955             { \c_backslash_str sp #1 }
5956         }
5957     }

```

The key `silent` sets `\__spintent_spfig_intent_arg_tl` to `_`. Otherwise, with `read-arg` empty, `concept` chooses between `\__spintent_spfig_intent_tl:prefix(refs)` (with `prefix` placed after the leading `_` when given) and `_ (refs)` alone; with `read-arg` given, it is always `\__spintent_spfig_intent_tl:— read-arg` is never silenced by `concept` alone, only by `silent`.

```

5958     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
5959     {
5960         \tl_set:Nn \__spintent_spfig_intent_arg_tl { _- }
5961     }
5962     {
5963         \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5964         {
5965             \bool_if:cTF { \__spintent_sp #1 _concept_bool }
5966             {
5967                 \tl_set:Ne \__spintent_spfig_intent_arg_tl
5968                 {
5969                     -
5970                     \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
5971                     { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
5972                     \__spintent_spfig_intent_tl :prefix( \__spintent_spfig_refs_tl )
5973                 }
5974             }
5975             {
5976                 \tl_set:Ne \__spintent_spfig_intent_arg_tl { _ ( \__spintent_spfig_refs_tl ) }
5977             }
5978         }
5979         {
5980             \tl_set:Ne \__spintent_spfig_intent_arg_tl
5981             {
5982                 -
5983                 \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
5984                 { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
5985                 \__spintent_spfig_intent_tl :prefix( \__spintent_spfig_refs_tl )
5986             }
5987         }
5988     }
5989 }

```

(End of definition for `\__spintent_spfig_build_concept_vertices:n`)

`\__spintent_spfig_points_loop:n`

Walks `\__spintent_spfig_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in `'#1'`.

```

5990 \cs_new_protected:Nn \__spintent_spfig_points_loop:n
5991 {
5992     \int_zero:N \__spintent_spfig_pt_idx_int
5993     \seq_map_inline:Nn \__spintent_spfig_pt_seq
5994     {
5995         \int_incr:N \__spintent_spfig_pt_idx_int
5996         \__spintent_mathmlarg_point:nnn
5997         { pt \int_use:N \__spintent_spfig_pt_idx_int }
5998         { sp #1 }
5999         { ##1 }
6000     }
6001 }

```

(End of definition for `\__spintent_spfig_points_loop:n`)

`\__spintent_spfig_mathml:n`

Receives the short command name in `'#1'`. Branches on whether `\__spintent_geo_luaset_points_clist` is empty.

Without vertices, `\__spintent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all. With vertices, `\__spintent_spfig_build_refs:` then `\__spintent_spfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside,

`\__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`'s own symbol); `\__spintent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\__spintent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback's recursive lookup through a noad's sub field, not just its nucleus.

The function `\__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

6002 \cs_new_protected:Nn \__spintent_spfig_mathml:n
6003 {
6004   \__spintent_invisible_sep: % (luamml issues #26)
6005   \clist_if_empty:NTF \__spintent_geo_luaset_points_clist
6006   {
6007     \__spintent_spfig_build_simple:n {#1}
6008     \MathMLintent { \__spintent_spfig_intent_arg_tl }
6009     {
6010       \__spintent_inner_sep:
6011       \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
6012       \tl_if_empty:NF \__spintent_geo_final_print_tl
6013       {
6014         \c_math_subscript_token { \__spintent_geo_final_print_tl }
6015       }
6016     }
6017   }
6018   {
6019     \__spintent_spfig_build_refs:
6020     \__spintent_spfig_build_concept_vertices:n {#1}
6021     \MathMLintent { \__spintent_spfig_intent_arg_tl }
6022     {
6023       \__spintent_inner_sep:
6024       \__spintent_luafun_mrow_intent_arg:nn
6025       { } { intent = ":silent" }
6026       {
6027         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:
6028       }
6029       \c_math_subscript_token
6030       {
6031         \tl_if_empty:NF \__spintent_spfig_symbol_tl
6032         {
6033           \__spintent_luafun_mrow_intent_arg:nn
6034           { } { intent = ":silent" }
6035           { \tl_use:N \__spintent_spfig_symbol_tl }
6036         }
6037         \__spintent_spfig_points_loop:n {#1}
6038       }
6039     }
6040   }
6041 }

```

(End of definition for `\__spintent_spfig_mathml:n`.)

`\__spintent_spfig_exec:nnn` The function `\__spintent_spfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spfig_process:nn`.

```

6042 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
6043 {
6044   \keys_set:nn { spintent / sp #1 } { #2 }
6045   \__spintent_spfig_process:nn { #1 } { #3 }
6046 }

```

(End of definition for `\__spintent_spfig_exec:nnn`.)

`\spAfig` Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

```

6047 \NewDocumentCommand \spAfig { 0{} } d() {
6048   {
6049     \mode_if_math:TF
6050     {
6051       \group_begin:
6052       \attribute \__spintent_scope_attr_int = 1
6053       \tl_if_novalue:NTF {#2}
6054       { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }

```

```

6055         { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
6056     \group_end:
6057 }
6058 { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
6059 }
6060 \NewDocumentCommand \spPfig { 0{} d() }
6061 {
6062     \mode_if_math:TF
6063     {
6064         \group_begin:
6065         \attribute \__spintent_scope_attr_int = 1
6066         \tl_if_novalue:nTF {#2}
6067         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
6068         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
6069         \group_end:
6070     }
6071     { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
6072 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 26.)

13.49 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle.

13.49.1 Definition of variables for `\sprad` and `\spdiam`

The variables `\__spintent_sp_rad_diam_letra_tl` and `\__spintent_sp_rad_diam_word_tl` are scratch variables used by `\__spintent_sp_rad_diam_print:n`. `\__spintent_sp_rad_diam_read_arg_full_tl` is used by `\__spintent_sp_rad_diam_read_arg_print:n` to hold the text to voice when `read-arg` is given.

```

6073 \tl_new:N \__spintent_sp_rad_diam_letra_tl
6074 \tl_new:N \__spintent_sp_rad_diam_word_tl
6075 \tl_new:N \__spintent_sp_rad_diam_read_arg_full_tl

```

(End of definition for `\__spintent_sp_rad_diam_letra_tl`, `\__spintent_sp_rad_diam_word_tl`, and `\__spintent_sp_rad_diam_read_arg_full_tl`.)

13.49.2 Definition of keys for `\sprad` and `\spdiam`

We define the shared *(keys)* `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent` and `print-sym`.

```

6076 \clist_map_inline:nn { rad , diam }
6077 {
6078     \keys_define:nn { spintent / sp #1 }
6079     {
6080         read-cmd .choices:nn = { school , mathcat }
6081         {
6082             \int_case:nn { \l_keys_choice_int }
6083             {
6084                 {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
6085                 {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
6086             }
6087         },
6088         read-cmd .initial:n = school,
6089         read-cmd .value_required:n = true,
6090         read-cmd / unknown .code:n =
6091         { \msg_error:nneee { spintent } { unknown-choice }
6092           { read-cmd } { school,~mathcat } { \exp_not:n {##1} } },
6093         upper-case .bool_set:c = { \__spintent_sp #1 _upper_case_bool },
6094         upper-case .initial:n = false,
6095         upper-case .value_required:n = true,
6096         sub-index .code:n =
6097         { \str_set:cn { \__spintent_sp #1 _sub_index_str } {##1} },
6098         sub-index .initial:n = ,
6099         sub-index .value_required:n = true,
6100         read-sub .bool_set:c = { \__spintent_sp #1 _read_sub_bool },
6101         read-sub .initial:n = false,
6102         read-sub .value_required:n = true,
6103         read-arg .code:n =
6104         { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
6105         read-arg .initial:n = ,
6106         read-arg .value_required:n = true,

```

```

6107     concept      .bool_set:c = { l__spintent_sp #1 _concept_bool },
6108     concept      .initial:n = true,
6109     concept      .value_required:n = true,
6110     silent       .bool_set:c = { l__spintent_sp #1 _silent_bool },
6111     silent       .initial:n = false,
6112     silent       .value_required:n = true,
6113     print-sym    .bool_set:c = { l__spintent_sp #1 _print_sym_bool },
6114     print-sym    .initial:n = false,
6115     print-sym    .value_required:n = true,
6116     unknown      .code:n =
6117     {
6118         \str_set:Nn \l__spintent_command_name_str { sp #1 }
6119         \__spintent_unknown_keys_cmd:n {##1}
6120     },
6121 }
6122 }

```

• The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

13.49.3 Internal functions for `\sprad` and `\spdiam`

`\__spintent_sp_rad_diam_print:n`

The function `\__spintent_sp_rad_diam_print:n` sets `\l__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per upper-case, and, for `\spdiam` with `print-sym` true, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (rad/diam).

```

6123 \cs_new_protected:Nn \__spintent_sp_rad_diam_print:n
6124 {
6125     \bool_if:cTF { l__spintent_sp #1 _upper_case_bool }
6126     {
6127         \str_if_eq:eeTF {#1} { rad }
6128         { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { R } }
6129         { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { D } }
6130     }
6131     {
6132         \str_if_eq:eeTF {#1} { rad }
6133         { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { r } }
6134         { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { d } }
6135     }
6136     \__spintent_invisible_sep:
6137     \str_if_eq:eeT {#1} { diam }
6138     {
6139         \bool_if:NT \l__spintent_spdiam_print_sym_bool
6140         { \MathMLintent { :silent } { \diameter } }
6141     }

```

With `silent` true, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

6142     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
6143     {
6144         \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
6145         {
6146             \__spintent_luafun_mrow_intent_arg:nn
6147             { } { intent = "_-" }
6148             { \l__spintent_sp_rad_diam_letra_tl }
6149         }
6150         {
6151             \MathMLintent { _- }
6152             {
6153                 \__spintent_inner_sep:
6154                 \l__spintent_sp_rad_diam_letra_tl
6155                 \c_math_subscript_token
6156                 {
6157                     \MathMLintent { :silent }
6158                     { \str_use:c { l__spintent_sp #1 _sub_index_str } }
6159                 }
6160             }
6161         }
6162     }

```

Otherwise, with `concept` true (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing sub-*⟨value⟩* or *⟨value⟩* alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school`

(subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

6163     {
6164       \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
6165       {
6166         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
6167         {
6168           \str_if_eq:eeTF {#1} { rad }
6169           { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { radio } }
6170           { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { diámetro } }
6171         \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
6172         {
6173           \__spintent_luafun_mrow_intent_arg:nn
6174           { } { intent = "_ \l__spintent_sp_rad_diam_word_tl :nofix" }
6175           { \l__spintent_sp_rad_diam_letra_tl }
6176         }
6177         {
6178           \str_if_eq:eeTF
6179           { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
6180           {
6181             \MathMLintent
6182             {
6183               _ \l__spintent_sp_rad_diam_word_tl -
6184               \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
6185               \str_use:c { l__spintent_sp #1 _sub_index_str }
6186             }
6187             {
6188               \__spintent_inner_sep:
6189               \l__spintent_sp_rad_diam_letra_tl
6190               \c_math_subscript_token
6191               {
6192                 \MathMLintent { :silent }
6193                 { \str_use:c { l__spintent_sp #1 _sub_index_str } }
6194               }
6195             }
6196           }
6197           {
6198             \__spintent_luafun_mrow_intent_arg:nn
6199             { } { intent = "_ \l__spintent_sp_rad_diam_word_tl :nofix" }
6200             { \l__spintent_sp_rad_diam_letra_tl }
6201             \c_math_subscript_token
6202             { \str_use:c { l__spintent_sp #1 _sub_index_str } }
6203           }
6204         }
6205       }

```

With `concept=false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by us.

```

6206     {
6207       \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
6208       {
6209         \str_if_eq:eeTF
6210         { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
6211         {
6212           \__spintent_luafun_mrow_intent_arg:nn
6213           { } { intent = "_ \l__spintent_sp_rad_diam_letra_tl -" }
6214           { \l__spintent_sp_rad_diam_letra_tl }
6215         }
6216         { \l__spintent_sp_rad_diam_letra_tl }
6217       }
6218       {
6219         \str_if_eq:eeTF
6220         { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
6221         {
6222           \MathMLintent
6223           {
6224             _ \l__spintent_sp_rad_diam_letra_tl -
6225             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
6226             \str_use:c { l__spintent_sp #1 _sub_index_str }

```

```

6227         }
6228         {
6229             \__spintent_inner_sep:
6230             \__spintent_sp_rad_diam_letra_tl
6231             \c_math_subscript_token
6232             {
6233                 \MathMLintent { :silent }
6234                 { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6235             }
6236         }
6237     }
6238     {
6239         \__spintent_sp_rad_diam_letra_tl
6240         \c_math_subscript_token
6241         { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6242     }
6243 }
6244 }
6245 }
6246 { \__spintent_sp_rad_diam_read_arg_print:n {#1} }
6247 }
6248 }

```

(End of definition for __spintent_sp_rad_diam_print:n)

__spintent_sp_rad_diam_read_arg_print:n

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true` (the default), `\__spintent_sp_rad_diam_read_arg_full_tl` becomes `radio/diámetro` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively, regardless of `read-cmd`: with no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent`.

```

6249 \cs_new_protected:Nn \__spintent_sp_rad_diam_read_arg_print:n
6250 {
6251     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
6252     {
6253         \str_if_eq:eeTF {#1} { rad }
6254         {
6255             \tl_set:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6256             { radio- \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
6257         }
6258         {
6259             \tl_set:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6260             { diámetro- \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
6261         }
6262     }
6263     {
6264         \tl_set_eq:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6265         { \__spintent_sp #1 _read_arg_tl }
6266     }
6267     \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
6268     {
6269         \__spintent_luafun_mrow_intent_arg:nn
6270         { } { intent = "_ \__spintent_sp_rad_diam_read_arg_full_tl :nofix" }
6271         { \__spintent_sp_rad_diam_letra_tl }
6272     }
6273     {
6274         \MathMLintent
6275         { _ \__spintent_sp_rad_diam_read_arg_full_tl :nofix }
6276         {
6277             \__spintent_inner_sep:
6278             \__spintent_sp_rad_diam_letra_tl
6279             \c_math_subscript_token
6280             {
6281                 \MathMLintent { :silent }
6282                 { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6283             }
6284         }
6285     }
6286 }

```

(End of definition for __spintent_sp_rad_diam_read_arg_print:n)

```

\__spintent_sp_rad_diam_exec:nn
6287 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
6288 {
6289   \keys_set:nn { spintent / sp #1 } {#2}
6290   \__spintent_sp_rad_diam_print:n {#1}
6291 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`)

`\sprad` Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

\spdiam
6292 \NewDocumentCommand \sprad { 0{} }
6293 {
6294   \mode_if_math:TF
6295   {
6296     \group_begin:
6297     \attribute \__spintent_scope_attr_int = 1
6298     \__spintent_sp_rad_diam_exec:nn { rad } {#1}
6299     \group_end:
6300   }
6301   { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
6302 }
6303 \NewDocumentCommand \spdiam { 0{} }
6304 {
6305   \mode_if_math:TF
6306   {
6307     \group_begin:
6308     \attribute \__spintent_scope_attr_int = 1
6309     \__spintent_sp_rad_diam_exec:nn { diam } {#1}
6310     \group_end:
6311   }
6312   { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
6313 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page 29.)

13.50 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

13.50.1 Definition of variables for `\spcirc`

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. The variable `\__spintent_geo_luaset_circ_radio_type_str` holds one of `number`, `measure`, `label`, `segment`, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

6314 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
6315 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (`usual` or `odot`). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

6316 \str_new:N \__spintent_spcirc_mode_str
6317 \tl_new:N \__spintent_spcirc_prefix_tl
6318 \str_new:N \__spintent_spcirc_print_sym_str
6319 \str_new:N \__spintent_spcirc_read_sty_str
6320 \str_new:N \__spintent_spcirc_radio_sty_str
6321 \str_new:N \__spintent_spcirc_sub_index_str
6322 \str_new:N \__spintent_spcirc_articulo_str
6323 \tl_new:N \__spintent_spcirc_read_arg_tl

```

(End of definition for `\l__spintent_spcirc_mode_str` and others.)

`\l__spintent_spcirc_word_tl` Scratch variables used by `\l__spintent_spcirc_process:n` to assemble the chosen word (circunferencia/círculo or the `read-arg` override) and the final literal `intent` string.

```
6324 \tl_new:N \l__spintent_spcirc_word_tl
```

(End of definition for `\l__spintent_spcirc_word_tl`.)

13.50.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg` Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```
6325 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
6326 {
6327   Invalid~argument~'#1'~for~\token_to_str:N \spcirc \c_space_tl \msg_line_context:.
6328 }
6329 {
6330   See~the~documentation~of~\token_to_str:N \spcirc \c_space_tl for~the~accepted~argument~forms.
6331 }
```

(End of definition for `spcirc-invalid-arg`.)

13.50.3 Definition of keys for `\spcirc`

`read-cmd` The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `prefix` `read-sty` chooses the verbalized word and its article: `circun` (default) for «la circunferencia», `circle` for «el círculo». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`mside`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```
6332 \keys_define:nn { spintent / spcirc }
6333 {
6334   read-cmd      .choices:nn = { school , mathcat }
6335   {
6336     \int_case:nn { \l_keys_choice_int }
6337     {
6338       {1}
6339       {
6340         \str_set:Nn \l__spintent_spcirc_mode_str { school }
6341         \keys_set:nn { spintent / spside } { read-cmd = school }
6342         \keys_set:nn { spintent / spmside } { read-cmd = school }
6343       }
6344       {2}
6345       {
6346         \str_set:Nn \l__spintent_spcirc_mode_str { mathcat }
6347         \keys_set:nn { spintent / spside } { read-cmd = mathcat }
6348         \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
6349       }
6350     }
6351   },
6352   read-cmd      .initial:n = school,
6353   read-cmd      .value_required:n = true,
6354   prefix        .code:n =
6355   { \l__spintent_sanitize_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
6356   prefix        .initial:n = ,
6357   prefix        .value_required:n = true,
6358   print-sym     .choices:nn = { usual , odot }
6359   {
6360     \int_case:nn { \l_keys_choice_int }
6361     {
6362       {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
6363       {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
6364     }
6365   },
6366   print-sym     .initial:n = odot,
6367   print-sym     .value_required:n = true,
6368   read-sty      .choices:nn = { circun , circle }
6369   {
6370     \int_case:nn { \l_keys_choice_int }
6371     {
6372       {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
6373       {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
6374     }
6375   }
6376 }
```

```

6375     },
6376     read-sty      .initial:n = circun,
6377     read-sty      .value_required:n = true,
6378     diameter      .bool_set:N = \l__spintent_spcirc_diameter_bool,
6379     diameter      .initial:n = false,
6380     diameter      .value_required:n = true,
6381     radio-sty     .choices:nn = { side , mside }
6382     {
6383       \int_case:nn { \l_keys_choice_int }
6384       {
6385         {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
6386         {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
6387       }
6388     },
6389     radio-sty     .initial:n = mside,
6390     radio-sty     .value_required:n = true,
6391     sub-index     .code:n =
6392     { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
6393     sub-index     .initial:n = ,
6394     sub-index     .value_required:n = true,
6395     read-sub      .bool_set:N = \l__spintent_spcirc_read_sub_bool,
6396     read-sub      .initial:n = false,
6397     read-sub      .value_required:n = true,
6398     read-arg      .code:n =
6399     { \__spintent_sanitizize_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
6400     read-arg      .initial:n = ,
6401     read-arg      .value_required:n = true,
6402     silent        .bool_set:N = \l__spintent_spcirc_silent_bool,
6403     silent        .initial:n = false,
6404     silent        .value_required:n = true,
6405     concept       .bool_set:N = \l__spintent_spcirc_concept_bool,
6406     concept       .initial:n = true,
6407     concept       .value_required:n = true,
6408     spside        .meta:nn = { spintent / spside } { #1 },
6409     spside        .value_required:n = true,
6410     spmside       .meta:nn = { spintent / spmside } { #1 },
6411     spmside       .value_required:n = true,
6412     unknown       .code:n =
6413     {
6414       \str_set:Nn \l__spintent_command_name_str { spcirc }
6415       \__spintent_unknown_keys_cmd:n {#1}
6416     },
6417   }

```

(End of definition for read-cmd and others.)

13.50.4 Internal functions for \spcirc

```

\l__spintent_spcirc_has_arg_bool
\l__spintent_spcirc_intent_tl
\l__spintent_spcirc_intent_arg_tl
\l__spintent_spcirc_connector_tl
\l__spintent_spcirc_connector_arg_tl
\l__spintent_spcirc_radio_arg_tl

```

The variable `\l__spintent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintent_spcirc_intent_tl` holds the word (`circunferencia` or `círculo`, with an optional `sub-index` suffix) paired with the symbol. The variable `\l__spintent_spcirc_connector_tl` holds `de-centro-en`, paired with the `(center)` group; the center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintent_spcirc_radio_arg_tl`, is spoken directly on the `,` delimiter after the center, replacing its own reading rather than pairing with a group.

```

6418 \bool_new:N \l__spintent_spcirc_has_arg_bool
6419 \tl_new:N \l__spintent_spcirc_intent_tl
6420 \tl_new:N \l__spintent_spcirc_intent_arg_tl
6421 \tl_new:N \l__spintent_spcirc_connector_tl
6422 \tl_new:N \l__spintent_spcirc_connector_arg_tl
6423 \tl_new:N \l__spintent_spcirc_radio_arg_tl

```

(End of definition for `\l__spintent_spcirc_has_arg_bool` and others.)

`\__spintent_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

6424 \cs_new_protected:Nn \__spintent_spcirc_sym:
6425 {
6426   \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
6427   {
6428     \group_begin:
6429     \textstyle\bigodot

```

```

6430         \group_end:
6431     }
6432     { C }
6433 }

```

(End of definition for `\__spintent_spcirc_sym:`.)

`\__spintent_spcirc_word_and_article:`

Sets `\__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```

6434 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
6435 {
6436     \tl_if_empty:NTF \__spintent_spcirc_read_arg_tl
6437     {
6438         \str_if_eq:VnTF \__spintent_spcirc_read_sty_str { circun }
6439         {
6440             \tl_set:Nn \__spintent_spcirc_word_tl { circunferencia }
6441             \str_set:Nn \__spintent_spcirc_articulo_str { la }
6442         }
6443         {
6444             \tl_set:Nn \__spintent_spcirc_word_tl { círculo }
6445             \str_set:Nn \__spintent_spcirc_articulo_str { el }
6446         }
6447     }
6448     {
6449         \tl_set:Nn \__spintent_spcirc_word_tl { \__spintent_spcirc_read_arg_tl }
6450         \str_if_eq:VnTF \__spintent_spcirc_read_sty_str { circun }
6451         { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
6452         { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
6453     }
6454 }

```

(End of definition for `\__spintent_spcirc_word_and_article:`.)

`\__spintent_spcirc_radio_print:`

Dispatches on `\__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```

6455 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
6456 {
6457     \str_case:Vn \__spintent_geo_luaset_circ_radio_type_str
6458     {
6459         { number } { \exp_args:Ne \spnum { \__spintent_geo_luaset_circ_radio_raw_str } }
6460         { measure } { \exp_args:Ne \spunit { \__spintent_geo_luaset_circ_radio_raw_str } }
6461         { label } { \__spintent_geo_luaset_circ_radio_raw_str }
6462         { segment }
6463         {
6464             \str_if_eq:VnTF \__spintent_spcirc_radio_sty_str { side }
6465             { \exp_args:Ne \spside { \__spintent_geo_luaset_circ_radio_raw_str } }
6466             { \exp_args:Ne \spmside { \__spintent_geo_luaset_circ_radio_raw_str } }
6467         }
6468     }
6469 }

```

(End of definition for `\__spintent_spcirc_radio_print:`.)

`\__spintent_spcirc_build_concept:`

Builds two words: `\__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\__spintent_spcirc_connector_tl` (`de-centro-en`), paired with the (center) group — built even when `\__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```

6470 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
6471 {
6472     \__spintent_spcirc_word_and_article:
6473     \tl_set:Ne \__spintent_spcirc_intent_tl { \__spintent_spcirc_word_tl }
6474     \str_if_empty:NF \__spintent_spcirc_sub_index_str
6475     {
6476         \tl_put_right:Ne \__spintent_spcirc_intent_tl
6477         {
6478             -
6479             \bool_if:NT \__spintent_spcirc_read_sub_bool { sub - }
6480             \__spintent_spcirc_sub_index_str

```

```

6481     }
6482   }
6483   \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }

```

The key `silent`, when `true`, overrides `\l__spintent_spcirc_intent_arg_tl`, `\l__spintent_spcirc_connector_arg_tl` and `\l__spintent_spcirc_radio_arg_tl` with `_-`, regardless of `read-arg`: a composite center (one with a subscript or a prime) is not reliably silenced by the per-point `:silent` alone.

```

6484   \bool_if:NTF \l__spintent_spcirc_silent_bool
6485   {
6486     \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
6487     \tl_set:Nn \l__spintent_spcirc_connector_arg_tl { _- }
6488     \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
6489   }

```

With `concept true`, `\l__spintent_spcirc_intent_arg_tl` is `_ \l__spintent_spcirc_intent_tl` alone (bare `_` — the group it wraps has nothing to reference, only a silenced symbol), with `prefix` placed after the leading `_` when given; `\l__spintent_spcirc_connector_arg_tl` is `_ \l__spintent_spcirc_connector_tl:prefix` and `\l__spintent_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\l__spintent_spcirc_radio_print:`’s independent reading follows as the next sibling.

```

6490   {
6491     \bool_if:NTF \l__spintent_spcirc_concept_bool
6492     {
6493       \tl_set:Nn \l__spintent_spcirc_intent_arg_tl
6494       {
6495         _
6496         \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
6497         { \tl_use:N \l__spintent_spcirc_prefix_tl - }
6498         \l__spintent_spcirc_intent_tl
6499       }
6500       \tl_set:Nn \l__spintent_spcirc_connector_arg_tl
6501       { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
6502       \bool_if:NTF \l__spintent_spcirc_diameter_bool
6503       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
6504       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
6505     }

```

With `concept false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\l__spintent_spside_process:nn`).

```

6506   {
6507     \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
6508     {
6509       \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
6510       \tl_set:Nn \l__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
6511       \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
6512     }
6513     {
6514       \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _ \l__spintent_spcirc_intent_tl }
6515       \tl_set:Nn \l__spintent_spcirc_connector_arg_tl
6516       { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
6517       \bool_if:NTF \l__spintent_spcirc_diameter_bool
6518       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
6519       { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
6520     }
6521   }
6522 }
6523 }

```

(End of definition for `\l__spintent_spcirc_build_concept:`)

`\l__spintent_spcirc_process:n`

Receives the `d()` argument in `#1`, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\l__spintent_spcirc_exec:nn`) — `\l__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\l__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\l__spintent_spcirc_mathml:`, which calls `\l__spintent_spcirc_build_concept:` itself.

```

6524 \cs_new_protected:Nn \l__spintent_spcirc_process:n
6525 {
6526   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl

```

```

6527     {
6528       \bool_set_false:N \__spintent_spcirc_has_arg_bool
6529     }
6530     {
6531       \bool_set_true:N \__spintent_spcirc_has_arg_bool
6532       \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
6533       \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
6534       {
6535         \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
6536       }
6537     }
6538     \__spintent_spcirc_mathml:
6539   }

```

(End of definition for `\__spintent_spcirc_process:n`.)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — except with `silent=true`, when it is wrapped in a literal `_\MathMLintent`, which silences whatever it produces. Otherwise, wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_intent_arg_tl` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`), needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

6540 \cs_new_protected:Nn \__spintent_spcirc_mathml:
6541   {
6542     \__spintent_invisible_sep: % (luamml issues #26)
6543     \__spintent_spcirc_build_concept:
6544     \MathMLintent { \__spintent_spcirc_intent_arg_tl }
6545     {
6546       \__spintent_inner_sep:
6547       \__spintent_luafun_mrow_intent_arg:nn
6548       { } { intent = ":silent" }
6549       { \__spintent_spcirc_sym: }
6550       \str_if_empty:NF \__spintent_spcirc_sub_index_str
6551       {
6552         \c_math_subscript_token
6553         {
6554           \__spintent_luafun_mrow_intent_arg:nn
6555           { } { intent = ":silent" }
6556           { \__spintent_spcirc_sub_index_str }
6557         }
6558       }
6559     }
6560     \bool_if:NT \__spintent_spcirc_has_arg_bool
6561     {
6562       \str_if_empty:NTF \__spintent_geo_luaset_circ_radio_type_str
6563       {
6564         \MathMLintent { \__spintent_spcirc_connector_arg_tl }
6565         {
6566           \__spintent_inner_sep:
6567           \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
6568           \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_

```

```

6569         \__spintenc_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
6570     }
6571 }
6572 {
6573     \MathMLintent { \__spintenc_spcirc_connector_arg_tl }
6574     {
6575         \__spintenc_inner_sep:
6576         \__spintenc_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
6577         \__spintenc_mathmlarg_point:nnV { pt } { spcirc } \__spintenc_geo_luasets_points_
6578     )
6579     \MathMLintent { \__spintenc_spcirc_radio_arg_tl } { , }
6580     \bool_if:NTF \__spintenc_spcirc_silent_bool
6581     {
6582         \MathMLintent { _- }
6583         {
6584             \__spintenc_inner_sep:
6585             \__spintenc_spcirc_radio_print:
6586         }
6587     }
6588     { \__spintenc_spcirc_radio_print: }
6589     \MathMLintent { :silent } { \rparen }
6590 }
6591 }
6592 }

```

(End of definition for `\__spintenc_spcirc_mathml:`)

`\__spintenc_spcirc_exec:nn`

The function `\__spintenc_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintenc_spcirc_print:n`.

```

6593 \cs_new_protected:Nn \__spintenc_spcirc_exec:nn
6594 {
6595     \keys_set:nn { spintenc / spcirc } { #1 }
6596     \__spintenc_spcirc_process:n { #2 }
6597 }

```

(End of definition for `\__spintenc_spcirc_exec:nn`)

`\spcirc`

The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintenc_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcdd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

6598 \NewDocumentCommand \spcirc { 0{} d() }
6599 {
6600     \mode_if_math:TF
6601     {
6602         \group_begin:
6603         \attribute \__spintenc_scope_attr_int = 1
6604         \tl_if_novalue:nTF {#2}
6605         { \__spintenc_spcirc_exec:nn { #1 } { } }
6606         { \__spintenc_spcirc_exec:nn { #1 } { #2 } }
6607     }
6608     \group_end:
6609     { \msg_error:nnn { spintenc } { need-math-mode } { \spcirc } }
6610 }

```

(End of definition for `\spcirc`. This function is documented on page 30.)

13.51 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

13.51.1 Definition of variables for `\spAcirc` and `\spPcirc`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```

6611 \clist_map_inline:nn { Acirc , Pcirc }
6612 {
6613     \str_new:c { \__spintenc_sp #1 _mode_str }
6614     \str_new:c { \__spintenc_sp #1 _print_sym_str }
6615     \str_new:c { \__spintenc_sp #1 _read_sty_str }

```

```

6616     \str_new:c { l__spintent_sp #1 _radio_sty_str }
6617     \str_new:c { l__spintent_sp #1 _sub_index_str }
6618     \tl_new:c { l__spintent_sp #1 _prefix_tl }
6619     \tl_new:c { l__spintent_sp #1 _read_arg_tl }
6620 }

```

(End of definition for `\l__spintent_spAcirc_mode_str` and others.)

`\l__spintent_geo_circfig_mode_str`

The variable `\l__spintent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (bare).

```

6621 \str_new:N \l__spintent_geo_circfig_mode_str

```

(End of definition for `\l__spintent_geo_circfig_mode_str`.)

13.51.2 Definition of error messages for `\spAcirc` and `\spPcirc`

`spAcirc-invalid-arg`
`spPcirc-invalid-arg`

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```

6622 \clist_map_inline:nn { Acirc , Pcirc }
6623 {
6624     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
6625     {
6626         Invalid~argument~'##2'~for~##1~\msg_line_context:.
6627     }
6628     {
6629         See~the~documentation~of~##1~for~the~accepted~argument.
6630     }

```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

13.51.3 Definition of keys for `\spAcirc` and `\spPcirc`

`read-cmd`
`prefix`
`print-sym`
`read-sty`
`concept`
`sub-index`
`read-sub`
`read-arg`
`silent`
`unknown`

We define the shared (*keys*) `read-cmd`, `prefix`, `print-sym`, `read-sty`, `concept`, `sub-index`, `read-sub`, `read-arg` and `silent`.

```

6631 \keys_define:nn { spintent / sp #1 }
6632 {
6633     read-cmd .choices:nn = { school , mathcat }
6634     {
6635         \int_case:nn { \l_keys_choice_int }
6636         {
6637             {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
6638             {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
6639         }
6640     },
6641     read-cmd .initial:n = school,
6642     read-cmd .value_required:n = true,
6643     read-cmd / unknown .code:n =
6644         \msg_error:nneee { spintent } { unknown-choice }
6645         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
6646     prefix .code:n =
6647         { \__spintent_sanitizize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
6648     prefix .initial:n = ,
6649     prefix .value_required:n = true,
6650     print-sym .choices:nn = { usual , odot }
6651     {
6652         \int_case:nn { \l_keys_choice_int }
6653         {
6654             {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
6655             {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
6656         }
6657     },
6658     print-sym .initial:n = odot,
6659     print-sym .value_required:n = true,
6660     print-sym / unknown .code:n =
6661         \msg_error:nneee { spintent } { unknown-choice }
6662         { print-sym } { usual,~odot } { \exp_not:n {##1} },
6663     read-sty .choices:nn = { circun , circle }
6664     {
6665         \int_case:nn { \l_keys_choice_int }
6666         {
6667             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
6668             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
6669         }

```

```

6670         },
6671         read-sty          .initial:n = circun,
6672         read-sty          .value_required:n = true,
6673         concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
6674         concept           .initial:n = true,
6675         concept           .value_required:n = true,
6676         sub-index         .code:n =
6677         { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
6678         sub-index         .initial:n = ,
6679         sub-index         .value_required:n = true,
6680         read-sub          .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
6681         read-sub          .initial:n = false,
6682         read-sub          .value_required:n = true,
6683         read-arg          .code:n =
6684         { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
6685         read-arg          .initial:n = ,
6686         read-arg          .value_required:n = true,
6687         silent            .bool_set:c = { l__spintent_sp #1 _silent_bool },
6688         silent            .initial:n = false,
6689         silent            .value_required:n = true,
6690         unknown           .code:n =
6691         {
6692             \str_set:Nn \l__spintent_command_name_str { sp #1 }
6693             \__spintent_unknown_keys_cmd:n {##1}
6694         },
6695     }
6696 }

```

(End of definition for `read-cmd` and others.)

```

\l__spintent_spcircfig_short_bool
\l__spintent_spcircfig_op_tl
\l__spintent_spcircfig_intent_tl
\l__spintent_spcircfig_intent_arg_tl

```

The variable `\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_p` the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds *área* or *perímetro*. The variable `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

6697 \bool_new:N \l__spintent_spcircfig_short_bool
6698 \tl_new:N \l__spintent_spcircfig_op_tl
6699 \tl_new:N \l__spintent_spcircfig_del_word_tl
6700 \tl_new:N \l__spintent_spcircfig_intent_tl
6701 \tl_new:N \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\l__spintent_spcircfig_short_bool` and others.)

```
\__spintent_spcircfig_sym:n
```

Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

6702 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
6703 {
6704     \str_if_eq:vnTF { l__spintent_sp #1 _print_sym_str } { odot }
6705     { \bigodot }
6706     { C }
6707 }

```

(End of definition for `\__spintent_spcircfig_sym:n`.)

```
\__spintent_spcircfig_word_and_article:n
```

Sets `\l__spintent_spcirc_word_tl` to `read-arg` or *circunferencia/círculo* from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

6708 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
6709 {
6710     \str_if_eq:vnTF { l__spintent_sp #1 _read_sty_str } { circun }
6711     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
6712     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
6713     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
6714     {
6715         \str_if_eq:vnTF { l__spintent_sp #1 _read_sty_str } { circun }
6716         { \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia } }
6717         { \tl_set:Nn \l__spintent_spcirc_word_tl { círculo } }
6718     }

```

```

6719     {
6720         \tl_set:Nc \__spintent_spcirc_word_tl
6721         {
6722             \tl_use:c { \__spintent_sp #1 _read_arg_tl }
6723         }
6724     }
6725 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n`

Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`'s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘#1’.

```

6726 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
6727 {
6728     \__spintent_spcircfig_word_and_article:n {#1}
6729     \str_if_eq:VnTF \__spintent_spcirc_articulo_str { el }
6730     {
6731         \tl_set:Nc \__spintent_spcircfig_del_word_tl { del- \__spintent_spcirc_word_tl }
6732     }
6733     {
6734         \tl_set:Nc \__spintent_spcircfig_del_word_tl { de-la- \__spintent_spcirc_word_tl }
6735     }
6736 }

```

(End of definition for `\__spintent_spcircfig_del_word:n`.)

`\__spintent_spcircfig_build_short:n`

Handles the short number/letter case: builds `\__spintent_spcircfig_intent_arg_tl` as `\__spintent_spcircfig_intent_arg_tl` (*ber word*), with `prefix` placed after the leading `_` when given, with no `:prefix` (no reference — there is nothing to point to), or `_-` when `silent` is `true` or `concept` is `false`. Receives the short command name in ‘#1’.

```

6737 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
6738 {
6739     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
6740     {
6741         \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
6742     }
6743     {
6744         \bool_if:cTF { \__spintent_sp #1 _concept_bool }
6745         {
6746             \__spintent_spcircfig_del_word:n {#1}
6747             \tl_set:Nc \__spintent_spcircfig_intent_arg_tl
6748             {
6749                 -
6750                 \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
6751                 { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
6752                 \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl -
6753                 \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
6754                 \__spintent_geo_luaset_sub_spoken_str
6755             }
6756         }
6757         {
6758             \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
6759         }
6760     }
6761 }

```

(End of definition for `\__spintent_spcircfig_build_short:n`.)

`\__spintent_spcircfig_finish_intent:nn`

Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in ‘#1’ and the argument in ‘#2’.

```

6762 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
6763 {
6764     \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
6765     {
6766         \tl_put_right:Nc \__spintent_spcircfig_intent_tl
6767         {
6768             -
6769             \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
6770             \str_use:c { \__spintent_sp #1 _sub_index_str }

```

```

6771     }
6772   }
6773   \tl_if_empty:nF {#2}
6774   {
6775     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
6776   }
6777 }

```

(End of definition for `\__spintent_spcircfig_finish_intent:nn`.)

`\__spintent_spcircfig_build_concept:nn`

Builds `\l__spintent_spcircfig_intent_arg_tl`. `_` when `silent` is `true`, whatever `#2` is, or when `false` and `concept` is `false` with `read-arg` empty (`_( $\$$ pt)` instead when `#2` is non-empty — the center is still read). Otherwise `\l__spintent_spcircfig_intent_tl` — with `:prefix( $\$$ pt)` appended only when `#2` is non-empty — with `prefix` placed after the leading `_` when given, where `\l__spintent_spcircfig_intent_tl` is: with `concept` `true`, `\l__spintent_spcircfig_op_tl`–`\l__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del`–/de-la–, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\l__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user’s to write, same convention as `\__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

6778 \cs_new_protected:Nn \__spintent_spcircfig_build_concept:nn
6779 {
6780   \bool_if:cTF { l__spintent_sp #1 _silent_bool }
6781   {
6782     \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
6783   }
6784   {
6785     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
6786     {
6787       \__spintent_spcircfig_del_word:n {#1}
6788       \tl_set:Ne \l__spintent_spcircfig_intent_tl
6789       {
6790         \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl
6791       }
6792       \__spintent_spcircfig_finish_intent:nn {#1} {#2}
6793       \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
6794       {
6795         -
6796         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
6797         {
6798           \tl_use:c { l__spintent_sp #1 _prefix_tl } -
6799         }
6800         \l__spintent_spcircfig_intent_tl
6801         \tl_if_empty:nF {#2} { :prefix (  $\$$  pt ) }
6802       }
6803     }
6804     {
6805       \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
6806       {
6807         \tl_if_empty:nTF {#2}
6808         { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
6809         { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ (  $\$$  pt ) } }
6810       }
6811       {
6812         \tl_set:Ne \l__spintent_spcircfig_intent_tl
6813         { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
6814         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
6815         \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
6816         {
6817           -
6818           \l__spintent_spcircfig_intent_tl
6819           \tl_if_empty:nF {#2} { :prefix (  $\$$  pt ) }
6820         }
6821       }
6822     }
6823   }
6824 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

__spintent_spcircfig_process:nn

Receives the short command name in ‘#1’ and the argument in ‘#2’. Sets \l__spintent_spcircfig_op_tl to área/perímetro. With #2 empty, \l__spintent_spcircfig_short_bool is set false and nothing else is parsed. With a comma in #2, reports sp#1-invalid-arg immediately — \@@_luafun_geo_circ_parse_and_set: (shared with \spcirc) still accepts a second, comma-separated part as a generic label-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls __spintent_luafun_geo_circ_parse_and_set:n; on error, retries through __spintent_luafun_geo_afig_parse_and_set:n/__spintent_luafun_geo_pfig_parse_and_set:n (the same entry point \spAfig/\spPfig use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports sp#1-invalid-arg on failure either way, then delegates to __spintent_spcircfig_build_short:n/__spintent_spcircfig_build_concept:nn and __spintent_spcircfig_mathml:nn.

- Unlike \spcirc, there is no radius/diameter here at all: a formula’s left-hand side (A_0) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

6825 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
6826 {
6827   \str_if_eq:eeTF {#1} { Acirc }
6828   { \tl_set:Nn \l__spintent_spcircfig_op_tl { área } }
6829   { \tl_set:Nn \l__spintent_spcircfig_op_tl { perímetro } }
6830   \tl_if_empty:nTF {#2}
6831   {
6832     \bool_set_false:N \l__spintent_spcircfig_short_bool
6833   }
6834   {
6835     \bool_set_false:N \l__spintent_spcircfig_short_bool
6836     \tl_if_in:nnTF {#2} { , }
6837     {
6838       \msg_error:nnee { spintent } { sp #1 -invalid-arg }
6839       { \c_backslash_str sp #1 } { \exp_not:n {#2} }
6840     }
6841     {
6842       \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
6843       \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
6844       {
6845         \str_if_eq:eeTF {#1} { Acirc }
6846         { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
6847         { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
6848         \str_if_eq:VnF \l__spintent_geo_luaset_error_str { true }
6849         {
6850           \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
6851           { \bool_set_true:N \l__spintent_spcircfig_short_bool }
6852           { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
6853         }
6854       }
6855       \bool_if:NF \l__spintent_spcircfig_short_bool
6856       {
6857         \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
6858         {
6859           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
6860           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
6861         }
6862       }
6863     }
6864   }
6865   \bool_if:NTF \l__spintent_spcircfig_short_bool
6866   { \__spintent_spcircfig_build_short:n {#1} }
6867   { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
6868   \__spintent_spcircfig_mathml:nn { #1 } { #2 }
6869 }

```

(End of definition for __spintent_spcircfig_process:nn.)

__spintent_spcircfig_mathml:nn

Renders \l__spintent_spcircfig_intent_arg_tl, already built by __spintent_spcircfig_process:nn. Three cases: the short case prints the symbol with the short value as its own subscript; an empty #2 prints the symbol alone; a real center prints the symbol (plus optional sub-index subscript) and the center between parentheses. In every case a single native \MathMLintent wraps __spintent_mathcal:n{⟨A | P⟩} and the whole subscript together — splitting the symbol and the center into separate \MathMLintent siblings inside the same subscript was tried and reads as <msub>’s own native structural narration instead. The center is read through __spintent_mathmlarg_point:nnn, referenced as \$pt (same convention as \spcirc). Receives the short command name in ‘#1’ and the argument in ‘#2’.

```

6870 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn

```

```

6871 {
6872   \__spintent_invisible_sep: % (luamml issues #26)
6873   \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
6874   {
6875     \__spintent_inner_sep:
6876     \__spintent_luafun_mrow_intent_arg:nn
6877     { } { intent = ":silent" }
6878     {
6879       \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
6880     }
6881     \bool_if:NTF \__spintent_spcircfig_short_bool
6882     {
6883       \c_math_subscript_token
6884       {
6885         \__spintent_spcircfig_sym:n {#1}
6886         \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
6887       }
6888     }
6889     {
6890       \c_math_subscript_token
6891       {
6892         \MathMLintent { :silent } { \__spintent_spcircfig_sym:n {#1} }
6893         \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
6894         {
6895           \c_math_subscript_token
6896           {
6897             \MathMLintent { :silent }
6898             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6899           }
6900         }
6901         \tl_if_empty:nF {#2}
6902         {
6903           \MathMLintent { :silent } { \lparen }
6904           \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_poin
6905           \MathMLintent { :silent } { \rparen }
6906         }
6907       }
6908     }
6909   }
6910 }

```

(End of definition for __spintent_spcircfig_mathml:nn.)

__spintent_spcircfig_exec:nnn

The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

6911 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
6912 {
6913   \keys_set:nn { spintent / sp #1 } { #2 }
6914   \__spintent_spcircfig_process:nn { #1 } { #3 }
6915 }

```

(End of definition for __spintent_spcircfig_exec:nnn.)

\spAcirc
\spPcirc

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

```

6916 \NewDocumentCommand \spAcirc { 0{} d() }
6917 {
6918   \mode_if_math:TF
6919   {
6920     \group_begin:
6921     \attribute \__spintent_scope_attr_int = 1
6922     \tl_if_novalue:nTF {#2}
6923     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
6924     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
6925     \group_end:
6926   }
6927   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
6928 }
6929 \NewDocumentCommand \spPcirc { 0{} d() }
6930 {
6931   \mode_if_math:TF
6932   {

```

```

6933     \group_begin:
6934     \attribute \l__spintenc_scope_attr_int = 1
6935     \tl_if_novalue:nTF {#2}
6936     { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { } }
6937     { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
6938     \group_end:
6939   }
6940   { \msg_error:nnn { spintenc } { need-math-mode } { \spPcirc } }
6941 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page 31.)

13.52 The command `\spVol`

The user-level command `\spVol` provides a semantic interface for the volume of a solid, always typesetting `\mathcal{V}`.

13.52.1 Definition of variables for `\spVol`

The variable `\l__spintenc_spVol_mode_str` holds the `read-cmd` choice (`school` or `mathcat`); `\l__spintenc_spVol_sub_index_str` the raw `sub-index` text; `\l__spintenc_spVol_read_arg_tl` the sanitized `read-arg` text. `\l__spintenc_spVol_read_arg_tl` and `\l__spintenc_spVol_suffix_tl` are scratch variables holding the word to voice (volumen) and the text to attach after it; `\l__spintenc_spVol_native_bool` records whether that text should be combined natively with the subscript (silenced) or left for MathCAT to read on its own. The variable `\l__spintenc_spVol_read_arg_full_tl` holds the text to voice when `read-arg` is given.

```

6942 \str_new:N \l__spintenc_spVol_mode_str
6943 \str_new:N \l__spintenc_spVol_sub_index_str
6944 \tl_new:N \l__spintenc_spVol_read_arg_tl
6945 \tl_new:N \l__spintenc_spVol_word_tl
6946 \tl_new:N \l__spintenc_spVol_suffix_tl
6947 \bool_new:N \l__spintenc_spVol_native_bool
6948 \tl_new:N \l__spintenc_spVol_read_arg_full_tl

```

(End of definition for `\l__spintenc_spVol_mode_str` and others.)

13.52.2 Definition of keys for `\spVol`

We define the *(keys)* `read-cmd`, `sub-index`, `read-sub`, `read-arg`, `concept` and `silent`.

```

read-cmd  \keys_define:nn { spintenc / spVol }
sub-index {
read-sub  {
read-arg  read-cmd .choices:nn = { school , mathcat }
concept   {
silent    \int_case:nn { \l_keys_choice_int }
unknown  {
        {1} { \str_set:Nn \l__spintenc_spVol_mode_str { school } }
        {2} { \str_set:Nn \l__spintenc_spVol_mode_str { mathcat } }
      }
    },
read-cmd .initial:n = school,
read-cmd .value_required:n = true,
read-cmd / unknown .code:n =
  \msg_error:nneee { spintenc } { unknown-choice }
  { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
sub-index .code:n =
  { \str_set:Nn \l__spintenc_spVol_sub_index_str {#1} },
sub-index .initial:n = ,
sub-index .value_required:n = true,
read-sub .bool_set:N = \l__spintenc_spVol_read_sub_bool,
read-sub .initial:n = false,
read-sub .value_required:n = true,
read-arg .code:n =
  { \__spintenc_sanitizize_affix:Nn \l__spintenc_spVol_read_arg_tl {#1} },
read-arg .initial:n = ,
read-arg .value_required:n = true,
concept .bool_set:N = \l__spintenc_spVol_concept_bool,
concept .initial:n = true,
concept .value_required:n = true,
silent .bool_set:N = \l__spintenc_spVol_silent_bool,
silent .initial:n = false,
silent .value_required:n = true,
unknown .code:n =
  {
    \str_set:Nn \l__spintenc_command_name_str { spVol }
    \__spintenc_unknown_keys_cmd:n {#1}
  }

```

```

6985     },
6986 }

```

(End of definition for `read-cmd` and others.)

13.52.3 Internal functions for `\spVol`

`\__spinttent_spVol_print:` The function `\__spinttent_spVol_print:` dispatches to `\__spinttent_spVol_silent:` when `silent` is `true`, or to `\__spinttent_spVol_verbal:` otherwise.

```

6987 \cs_new_protected:Nn \__spinttent_spVol_print:
6988 {
6989   \__spinttent_invisible_sep:
6990   \bool_if:NTF \l__spinttent_spVol_silent_bool
6991     { \__spinttent_spVol_silent: }
6992     { \__spinttent_spVol_verbal: }
6993 }

```

(End of definition for `\__spinttent_spVol_print:`.)

`\__spinttent_spVol_silent:` The function `\__spinttent_spVol_silent:` typesets `\mathcal{V}` with the intent `_` — the symbol alone through the shared queue when there is no `sub-index`, or symbol and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

6994 \cs_new_protected:Nn \__spinttent_spVol_silent:
6995 {
6996   \str_if_empty:NTF \l__spinttent_spVol_sub_index_str
6997     {
6998       \__spinttent_luafun_mrow_intent_arg:nn
6999       { } { intent = "_" }
7000       { \mathcal{V} }
7001     }
7002     {
7003       \MathMLintent { _- }
7004       {
7005         \__spinttent_inner_sep:
7006         \mathcal{V}
7007         \c_math_subscript_token
7008         {
7009           \MathMLintent { :silent }
7010           { \l__spinttent_spVol_sub_index_str }
7011         }
7012       }
7013     }
7014 }

```

(End of definition for `\__spinttent_spVol_silent:`.)

`\__spinttent_spVol_verbal:` The function `\__spinttent_spVol_verbal:` delegates to `\__spinttent_spVol_read_arg_print:` when `read-arg` is given, and otherwise, with `concept false`, to `\__spinttent_spVol_symbol_print:`. With `concept true` it sets `\l__spinttent_spVol_word_tl` to `volumen` and `\l__spinttent_spVol_suffix_tl` to `sub-⟨value⟩/⟨value⟩` (per `read-sub`), combined natively only under `read-cmd=school` — under `read-cmd=mathcat` the suffix is left for MathCAT to read on its own. Delegates the actual typesetting to `\__spinttent_spVol_attach_subscript:`

```

7015 \cs_new_protected:Nn \__spinttent_spVol_verbal:
7016 {
7017   \tl_if_empty:NTF \l__spinttent_spVol_read_arg_tl
7018     {
7019       \bool_if:NTF \l__spinttent_spVol_concept_bool
7020       {
7021         \tl_set:Nn \l__spinttent_spVol_word_tl { volumen }
7022         \str_if_eq:VnTF \l__spinttent_spVol_mode_str { school }
7023         { \bool_set_true:N \l__spinttent_spVol_native_bool }
7024         { \bool_set_false:N \l__spinttent_spVol_native_bool }
7025         \tl_set:Ne \l__spinttent_spVol_suffix_tl
7026         {
7027           \bool_if:NT \l__spinttent_spVol_read_sub_bool { sub- }
7028           \l__spinttent_spVol_sub_index_str
7029         }
7030         \__spinttent_spVol_attach_subscript:
7031       }
7032       { \__spinttent_spVol_symbol_print: }
7033     }
7034     { \__spinttent_spVol_read_arg_print: }
7035 }

```

(End of definition for `\__spintnt_spVol_verbal:`.)

`\__spintnt_spVol_symbol_print:`

With `concept false`, the symbol itself is read instead — but only when `read-cmd` is `school`: with no `sub-index`, the shared queue tags the symbol with the intent `_V-`; with one, symbol and subscript are wrapped together in one native `\MathMLintent` (subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the symbol and its subscript, unmarked by us.

```

7036 \cs_new_protected:Nn \__spintnt_spVol_symbol_print:
7037 {
7038   \str_if_empty:NTF \__spintnt_spVol_sub_index_str
7039   {
7040     \str_if_eq:VnTF \__spintnt_spVol_mode_str { school }
7041     {
7042       \__spintnt_luafun_mrow_intent_arg:nn
7043       { } { intent = "_V-" }
7044       { \mathcal{V} }
7045     }
7046     { \mathcal{V} }
7047   }
7048   {
7049     \str_if_eq:VnTF \__spintnt_spVol_mode_str { school }
7050     {
7051       \MathMLintent
7052       {
7053         _V-
7054         \bool_if:NT \__spintnt_spVol_read_sub_bool { sub- }
7055         \__spintnt_spVol_sub_index_str
7056       }
7057       {
7058         \__spintnt_inner_sep:
7059         \mathcal{V}
7060         \c_math_subscript_token
7061         {
7062           \MathMLintent { :silent }
7063           { \__spintnt_spVol_sub_index_str }
7064         }
7065       }
7066     }
7067     {
7068       \mathcal{V}
7069       \c_math_subscript_token
7070       { \__spintnt_spVol_sub_index_str }
7071     }
7072   }
7073 }

```

(End of definition for `\__spintnt_spVol_symbol_print:`.)

`\__spintnt_spVol_attach_subscript:`

The function `\__spintnt_spVol_attach_subscript:` typesets `\mathcal{V}`. An empty `sub-index` tags the symbol alone through the shared queue (`\__spintnt_spVol_suffix_tl` is discarded, since there's no subscript to attach it to). A given one combines symbol, subscript, and `\__spintnt_spVol_suffix_tl` in one native `\MathMLintent` (subscript silenced) when `\__spintnt_spVol_native_bool` is `true`, or tags the symbol alone through the shared queue with the subscript left bare when `false`.

```

7074 \cs_new_protected:Nn \__spintnt_spVol_attach_subscript:
7075 {
7076   \str_if_empty:NTF \__spintnt_spVol_sub_index_str
7077   {
7078     \__spintnt_luafun_mrow_intent_arg:nn
7079     { } { intent = "_ \__spintnt_spVol_word_tl :nofix" }
7080     { \mathcal{V} }
7081   }
7082   {
7083     \bool_if:NTF \__spintnt_spVol_native_bool
7084     {
7085       \MathMLintent
7086       { _ \__spintnt_spVol_word_tl - \__spintnt_spVol_suffix_tl :nofix }
7087       {
7088         \__spintnt_inner_sep:
7089         \mathcal{V}
7090         \c_math_subscript_token
7091         {
7092           \MathMLintent { :silent }

```

```

7093             { \l__spintent_spVol_sub_index_str }
7094         }
7095     }
7096 }
7097 {
7098     \l__spintent_luafun_mrow_intent_arg:nn
7099     { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
7100     { \mathcal{V} }
7101     \c_math_subscript_token
7102     { \l__spintent_spVol_sub_index_str }
7103 }
7104 }
7105 }

```

(End of definition for `\l__spintent_spVol_attach_subscript:`.)

`\l__spintent_spVol_read_arg_print:`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true`, `\l__spintent_spVol_read_arg_full_tl` becomes `volumen` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively regardless of `read-cmd:` with no `sub-index`, the shared queue tags the symbol alone; with one, symbol and subscript are wrapped together in one native `\MathMLintent`.

```

7106 \cs_new_protected:Nn \l__spintent_spVol_read_arg_print:
7107 {
7108     \bool_if:NTF \l__spintent_spVol_concept_bool
7109     {
7110         \tl_set:Nx \l__spintent_spVol_read_arg_full_tl
7111         { volumen- \tl_use:N \l__spintent_spVol_read_arg_tl }
7112     }
7113     { \tl_set_eq:NN \l__spintent_spVol_read_arg_full_tl \l__spintent_spVol_read_arg_tl }
7114     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
7115     {
7116         \l__spintent_luafun_mrow_intent_arg:nn
7117         { } { intent = "_ \l__spintent_spVol_read_arg_full_tl :nofix" }
7118         { \mathcal{V} }
7119     }
7120     {
7121         \MathMLintent
7122         { _ \l__spintent_spVol_read_arg_full_tl :nofix }
7123         {
7124             \l__spintent_inner_sep:
7125             \mathcal{V}
7126             \c_math_subscript_token
7127             {
7128                 \MathMLintent { :silent }
7129                 { \l__spintent_spVol_sub_index_str }
7130             }
7131         }
7132     }
7133 }

```

(End of definition for `\l__spintent_spVol_read_arg_print:`.)

`\spVol` The public command `\spVol` enforces *math mode*, opens a local group, and delegates execution to `\l__spintent_spVol_print:`.

```

7134 \NewDocumentCommand \spVol { 0{} }
7135 {
7136     \mode_if_math:TF
7137     {
7138         \group_begin:
7139         \attribute \l__spintent_scope_attr_int = 1
7140         \keys_set:nn { spintent / spVol } {#1}
7141         \l__spintent_spVol_print:
7142         \group_end:
7143     }
7144     { \msg_error:nnn { spintent } { need-math-mode } { \spVol } }
7145 }

```

(End of definition for `\spVol`. This function is documented on page 34.)

13.53 The command `\spProm`

The user command `\spProm` typesets its $\langle argument \rangle$ with a bar on top (`\bar`), the usual notation for a mean, and reads it «*promedio de*» followed by the argument. The argument is not parsed: it is typeset as given and read by MathCAT with its own rules.

13.53.1 Definition of variables for `\spProm`

```
\l__spintent_spProm_silent_bool
\l__spintent_spProm_read_arg_tl
\l__spintent_spProm_intent_tl
```

The variable `\l__spintent_spProm_silent_bool` holds the state of the `silent` key. The variable `\l__spintent_spProm_read_arg_tl` holds the normalized text of the `read-arg` key, and `\l__spintent_spProm_intent_tl` holds the name of the intent used by `\l__spintent_spProm_exec:nn`.

```
7146 \bool_new:N \l__spintent_spProm_silent_bool
7147 \tl_new:N \l__spintent_spProm_read_arg_tl
7148 \tl_new:N \l__spintent_spProm_intent_tl
```

(End of definition for `\l__spintent_spProm_silent_bool`, `\l__spintent_spProm_read_arg_tl`, and `\l__spintent_spProm_intent_tl`.)

13.53.2 Definition of keys for `\spProm`

```
silent
read-arg
unknown
```

We define the $\langle keys \rangle$ `silent` and `read-arg`. The value of `read-arg` goes through `\l__spintent_sanitize_affix:Nn` so spaces and underscores become hyphens.

```
7149 \keys_define:nn { spintent / spProm }
7150 {
7151   silent      .bool_set:N = \l__spintent_spProm_silent_bool,
7152   silent      .initial:n = false,
7153   silent      .value_required:n = true,
7154   read-arg    .code:n =
7155     { \l__spintent_sanitize_affix:Nn \l__spintent_spProm_read_arg_tl {#1} },
7156   read-arg    .initial:n = ,
7157   read-arg    .value_required:n = true,
7158   unknown     .code:n =
7159     {
7160       \str_set:Nn \l__spintent_command_name_str { spProm }
7161       \l__spintent_unknown_keys_cmd:n {#1}
7162     },
7163 }
```

(End of definition for `silent`, `read-arg`, and `unknown`.)

13.53.3 Internal functions for `\spProm`

```
\l__spintent_spProm_exec:nn
```

The function `\l__spintent_spProm_exec:nn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. Under `silent` the whole expression is silenced as one unit with the intent `_-`.

Otherwise the $\langle argument \rangle$ is tagged through a native `\MathMLarg` (a) inside `\bar`, and the outer `\MathMLintent` reads `_promedio-de:prefix($a)`. When `read-arg` is not empty, its text replaces `promedio-de`. The inner ghost `\l__spintent_inner_sep:` keeps the content of the intent from collapsing into a single child; a second one inside the `\MathMLarg` does the same for the argument, whose `arg` attribute would otherwise be lost when the argument is a single compound element, such as a letter with a subscript.

```
7164 \cs_new_protected:Nn \l__spintent_spProm_exec:nn
7165 {
7166   \keys_set:nn { spintent / spProm } {#1}
7167   \l__spintent_invisible_sep:
7168   \bool_if:NTF \l__spintent_spProm_silent_bool
7169     {
7170       \MathMLintent { _- }
7171       {
7172         \l__spintent_inner_sep:
7173         \bar { #2 }
7174       }
7175     }
7176     {
7177       \tl_if_empty:NTF \l__spintent_spProm_read_arg_tl
7178       { \tl_set:Nn \l__spintent_spProm_intent_tl { promedio-de } }
7179       { \tl_set_eq:NN \l__spintent_spProm_intent_tl \l__spintent_spProm_read_arg_tl }
7180       \MathMLintent { _ \l__spintent_spProm_intent_tl :prefix($a) }
7181       {
7182         \l__spintent_inner_sep:
7183         \bar { \MathMLarg { a } { \l__spintent_inner_sep: #2 } }
7184       }
7185     }
7186 }
```

(End of definition for `\l__spintent_spProm_exec:nn`.)

`\spProm` The public command `\spProm` enforces *math mode*, opens a local group, and delegates execution to `__spintent_spProm_exec:nn`.

```

7187 \NewDocumentCommand \spProm { 0{ } m }
7188 {
7189   \mode_if_math:TF
7190   {
7191     \group_begin:
7192     \attribute \__spintent_scope_attr_int = 1
7193     \__spintent_spProm_exec:nn { #1 } { #2 }
7194     \group_end:
7195   }
7196   { \msg_error:nnn { spintent } { need-math-mode } { \spProm } }
7197 }

```

(End of definition for `\spProm`. This function is documented on page 37.)

13.54 The commands of the stat family

The user commands `\spmedia`, `\spmediana`, `\spmoda`, `\spdesv`, `\spvarz`, `\sprango`, `\spriq`, `\spmin` and `\spmax` typeset the usual symbols of descriptive statistics: MA (arithmetic mean), MD (median), MO (mode), s (standard deviation), s^2 (variance), R (range), RIQ (interquartile range), $x_{\min}$ (minimum) and $x_{\max}$ (maximum). Each one is read with the name of the concept; `\spdesv` and `\spvarz` also read their letter.

13.54.1 Definition of variables for the stat family

```

\__spintent_spmedia_silent_bool
\__spintent_spmedia_read_sub_bool
\__spintent_spmedia_sub_index_str
\__spintent_spmedia_read_arg_tl
\__spintent_spdesv_sigma_bool

```

Each command of the family has its own copy of the variables `\__spintent_⟨name⟩_silent_bool`, `\__spintent_⟨name⟩_read_sub_bool`, `\__spintent_⟨name⟩_sub_index_str` and `\__spintent_⟨name⟩_read_arg_tl`, so each command can be configured separately with `\setspintent`. The variables are shown here for `\spmedia`. `\spdesv` and `\spvarz` also have `\__spintent_⟨name⟩_sigma_bool`. `\spmin` and `\spmax` have no `sub-index` nor `read-sub` keys, since their symbol already carries a subscript; their two variables `_sub_index_str` and `_read_sub_bool` exist only because `__spintent_stat_print:nnnnnn` reads them, and always keep their initial values.

```

7198 \clist_map_inline:nn
7199 {
7200   spmedia , spmediana , spmoda , spdesv , spvarz , sprango , spriq ,
7201   spmin , spmax , spFrec , spcase , spOmega
7202 }
7203 {
7204   \bool_new:c { \__spintent_ #1 _silent_bool }
7205   \bool_new:c { \__spintent_ #1 _read_sub_bool }
7206   \str_new:c { \__spintent_ #1 _sub_index_str }
7207   \tl_new:c { \__spintent_ #1 _read_arg_tl }
7208 }
7209 \clist_map_inline:nn { spdesv , spvarz }
7210 { \bool_new:c { \__spintent_ #1 _sigma_bool } }
7211 \bool_set_true:N \__spintent_spOmega_read_sub_bool

```

(End of definition for `\__spintent_spmedia_silent_bool` and others.)

13.54.2 Definition of keys for the stat family

```

silent
sub-index
read-sub
read-arg
sigma
unknown

```

We define the *⟨keys⟩* `silent`, `sub-index`, `read-sub` and `read-arg` for the five commands, and the same `silent`, `sub-index`, `read-sub` and `read-arg` for `\sprango` and `\spriq`. `\spmin` and `\spmax` only get `silent` and `read-arg`, since their symbol already carries a subscript. The commands `\spFrec` and `\spOmega` (see their own sections) share the variables and the keys of the first block, and `\spcase` those of `\spmin` and `\spmax`. The key `sigma` exists only for `\spdesv` and `\spvarz`. For these two, which read a letter, `read-sub` starts in `true` (`s sub x` sounds more natural than `s de x`, and it avoids hearing the *⟨x⟩* as part of the exponent in *varianza s de x al cuadrado*); for the rest it starts in `false`. As elsewhere, `sub-index` is kept as a string and `read-arg` goes through `\__spintent_sanitize_affix:cn`.

```

7212 \clist_map_inline:nn
7213 {
7214   spmedia , spmediana , spmoda , spdesv , spvarz , sprango , spriq ,
7215   spFrec
7216 }
7217 {
7218   \keys_define:nn { spintent / #1 }
7219   {
7220     silent      .bool_set:c = { \__spintent_ #1 _silent_bool },
7221     silent      .initial:n = false,
7222     silent      .value_required:n = true,
7223     sub-index   .code:n =

```

```

7224         { \str_set:cn { l__spintent_ #1 _sub_index_str } {##1} },
7225         sub-index .initial:n = ,
7226         sub-index .value_required:n = true,
7227         read-sub .bool_set:c = { l__spintent_ #1 _read_sub_bool },
7228         read-sub .initial:n = false,
7229         read-sub .value_required:n = true,
7230         read-arg .code:n =
7231         { \__spintent_sanitizize_affix:cn { l__spintent_ #1 _read_arg_tl } {##1} },
7232         read-arg .initial:n = ,
7233         read-arg .value_required:n = true,
7234         unknown .code:n =
7235         {
7236             \str_set:Nn \__spintent_command_name_str { #1 }
7237             \__spintent_unknown_keys_cmd:n {##1}
7238         },
7239     }
7240 }
7241 \clist_map_inline:nn { spdesv , spvarz }
7242 {
7243     \keys_define:nn { spintent / #1 }
7244     {
7245         sigma .bool_set:c = { l__spintent_ #1 _sigma_bool },
7246         sigma .initial:n = false,
7247         sigma .value_required:n = true,
7248         read-sub .initial:n = true,
7249     }
7250 }
7251 \keys_define:nn { spintent / spFrec }
7252 { read-sub .initial:n = true }
7253 \clist_map_inline:nn { spmin , spmax , spcase }
7254 {
7255     \keys_define:nn { spintent / #1 }
7256     {
7257         silent .bool_set:c = { l__spintent_ #1 _silent_bool },
7258         silent .initial:n = false,
7259         silent .value_required:n = true,
7260         read-arg .code:n =
7261         { \__spintent_sanitizize_affix:cn { l__spintent_ #1 _read_arg_tl } {##1} },
7262         read-arg .initial:n = ,
7263         read-arg .value_required:n = true,
7264         unknown .code:n =
7265         {
7266             \str_set:Nn \__spintent_command_name_str { #1 }
7267             \__spintent_unknown_keys_cmd:n {##1}
7268         },
7269     }
7270 }

```

(End of definition for *silent* and others.)

13.54.3 Internal functions for the stat family

__spintent_stat_typeset:nnn

The function `\__spintent_stat_typeset:nnn` typesets the symbol. It receives the short name of the command in ‘#1’, the symbol in ‘#2’ and the exponent in ‘#3’ (empty when there is none). The symbol is followed by the `sub-index`, if present, with its own `\MathMLintent` set to `:silent`, and then by the exponent.

```

7271 \cs_new_protected:Nn \__spintent_stat_typeset:nnn
7272 {
7273     #2
7274     \str_if_empty:cF { l__spintent_ #1 _sub_index_str }
7275     {
7276         \c_math_subscript_token
7277         {
7278             \MathMLintent { :silent }
7279             { \str_use:c { l__spintent_ #1 _sub_index_str } }
7280         }
7281     }
7282     \tl_if_blank:nF {#3}
7283     { \c_math_superscript_token {#3} }
7284 }

```

(End of definition for `\__spintent_stat_typeset:nnn`.)

`\__spintent_stat_print:nnnnnn`

The function `\__spintent_stat_print:nnnnnn` receives the short name of the command in ‘#1’, the symbol in ‘#2’, the exponent in ‘#3’, the name of the concept in ‘#4’, the name of the letter in ‘#5’ (empty when the symbol is not a letter) and the spoken form of the exponent in ‘#6’ (empty when there is none), the last three already written as they go in the intent (with hyphens). The symbol, with its subscript and exponent, is always typeset by `\__spintent_stat_typeset:nnn` inside one native `\MathMLintent`, preceded by the inner ghost `\__spintent_inner_sep:`.

With `silent true`, the intent is `_-`. With `read-arg` given, the intent is `_{read-arg}:nofix` and `sub-index` and `read-sub` are ignored for the spoken content (the subscript, if any, still displays, silenced). Otherwise the spoken content is the concept, followed by the letter and, when there is a `sub-index`, by `de-⟨value⟩` (`sub-⟨value⟩` when `read-sub` is `true`, its default for `\spdesv` and `\spvarz`), and last by the exponent. With no `sub-index` the intent ends in `:nofix`.

```

7285 \cs_new_protected:Nn \__spintent_stat_print:nnnnnn
7286 {
7287   \__spintent_invisible_sep:
7288   \bool_if:cTF { l__spintent_ #1 _silent_bool }
7289   {
7290     \MathMLintent { _- }
7291     {
7292       \__spintent_inner_sep:
7293       \__spintent_stat_typeset:nnn {#1} {#2} {#3}
7294     }
7295   }
7296   {
7297     \tl_if_empty:cTF { l__spintent_ #1 _read_arg_tl }
7298     {
7299       \str_if_empty:cTF { l__spintent_ #1 _sub_index_str }
7300       {
7301         \MathMLintent
7302         {
7303           _ #4
7304           \tl_if_blank:nF {#5} { - #5 }
7305           \tl_if_blank:nF {#6} { - #6 }
7306           :nofix
7307         }
7308         {
7309           \__spintent_inner_sep:
7310           \__spintent_stat_typeset:nnn {#1} {#2} {#3}
7311         }
7312       }
7313       {
7314         \MathMLintent
7315         {
7316           _ #4
7317           \tl_if_blank:nF {#5} { - #5 }
7318           -
7319           \bool_if:cTF { l__spintent_ #1 _read_sub_bool } { sub } { de }
7320           - \str_use:c { l__spintent_ #1 _sub_index_str }
7321           \tl_if_blank:nF {#6} { - #6 }
7322         }
7323         {
7324           \__spintent_inner_sep:
7325           \__spintent_stat_typeset:nnn {#1} {#2} {#3}
7326         }
7327       }
7328     }
7329     {
7330       \MathMLintent { _ \tl_use:c { l__spintent_ #1 _read_arg_tl } :nofix }
7331       {
7332         \__spintent_inner_sep:
7333         \__spintent_stat_typeset:nnn {#1} {#2} {#3}
7334       }
7335     }
7336   }
7337 }

```

(End of definition for `\__spintent_stat_print:nnnnnn`.)

`\spmedia`
`\spmediana`
`\spmoda`

The public commands `\spmedia`, `\spmediana` and `\spmoda` enforce *math mode*, open a local group, set the `⟨keys⟩` and delegate execution to `\__spintent_stat_print:nnnnnn` with the symbols MA, MD and MO and

the concepts media-aritmética, mediana and moda.

```

7338 \NewDocumentCommand \spmedia { 0{} }
7339 {
7340   \mode_if_math:TF
7341   {
7342     \group_begin:
7343       \attribute \l__spintent_scope_attr_int = 1
7344       \keys_set:nn { spintent / spmedia } {#1}
7345       \__spintent_stat_print:nnnnnn { spmedia } { \mathrm{MA} } { } { } { media-aritmética } { } { }
7346     \group_end:
7347   }
7348   { \msg_error:nnn { spintent } { need-math-mode } { \spmedia } }
7349 }
7350 \NewDocumentCommand \spmediana { 0{} }
7351 {
7352   \mode_if_math:TF
7353   {
7354     \group_begin:
7355       \attribute \l__spintent_scope_attr_int = 1
7356       \keys_set:nn { spintent / spmediana } {#1}
7357       \__spintent_stat_print:nnnnnn { spmediana } { \mathrm{MD} } { } { } { mediana } { } { } { }
7358     \group_end:
7359   }
7360   { \msg_error:nnn { spintent } { need-math-mode } { \spmediana } }
7361 }
7362 \NewDocumentCommand \spmoda { 0{} }
7363 {
7364   \mode_if_math:TF
7365   {
7366     \group_begin:
7367       \attribute \l__spintent_scope_attr_int = 1
7368       \keys_set:nn { spintent / spmoda } {#1}
7369       \__spintent_stat_print:nnnnnn { spmoda } { \mathrm{MO} } { } { } { moda } { } { } { }
7370     \group_end:
7371   }
7372   { \msg_error:nnn { spintent } { need-math-mode } { \spmoda } }
7373 }

```

(End of definition for `\spmedia`, `\spmediana`, and `\spmoda`. These functions are documented on page 37.)

`\spdesv` The public commands `\spdesv` and `\spvarz` work the same way. The symbol is s, or `\sigma` when `sigma` is `true`, and its name is part of the spoken content; `\spvarz` adds the exponent 2, spoken al-cuadrado. The concepts are desviación-estándar and varianza.

```

7374 \NewDocumentCommand \spdesv { 0{} }
7375 {
7376   \mode_if_math:TF
7377   {
7378     \group_begin:
7379       \attribute \l__spintent_scope_attr_int = 1
7380       \keys_set:nn { spintent / spdesv } {#1}
7381       \bool_if:NTF \l__spintent_spdesv_sigma_bool
7382       {
7383         \__spintent_stat_print:nnnnnn { spdesv } { \sigma } { } { }
7384         { desviación-estándar } { sigma } { } { }
7385       }
7386       {
7387         \__spintent_stat_print:nnnnnn { spdesv } { s } { } { }
7388         { desviación-estándar } { s } { } { }
7389       }
7390     \group_end:
7391   }
7392   { \msg_error:nnn { spintent } { need-math-mode } { \spdesv } }
7393 }
7394 \NewDocumentCommand \spvarz { 0{} }
7395 {
7396   \mode_if_math:TF
7397   {
7398     \group_begin:
7399       \attribute \l__spintent_scope_attr_int = 1
7400       \keys_set:nn { spintent / spvarz } {#1}
7401       \bool_if:NTF \l__spintent_spvarz_sigma_bool

```

```

7402         {
7403             \__spintent_stat_print:nnnnnn { spvarz } { \sigma } { 2 }
7404             { varianza } { sigma } { al-cuadrado }
7405         }
7406         {
7407             \__spintent_stat_print:nnnnnn { spvarz } { s } { 2 }
7408             { varianza } { s } { al-cuadrado }
7409         }
7410     \group_end:
7411 }
7412 { \msg_error:nnn { spintent } { need-math-mode } { \spvarz } }
7413 }

```

(End of definition for `\spdesv` and `\spvarz`. These functions are documented on page 37.)

`\sprango` The public commands `\sprango`, `\spriq`, `\spmin` and `\spmax` work the same way, with the symbols R, RIQ, $x_{\{\min\}}$ and $x_{\{\max\}}$ and the concepts rango, rango-intercuartil, mínimo and máximo.

```

7414 \NewDocumentCommand \sprango { 0{} }
7415 {
7416     \mode_if_math:TF
7417     {
7418         \group_begin:
7419         \attribute \__spintent_scope_attr_int = 1
7420         \keys_set:nn { spintent / sprango } {#1}
7421         \__spintent_stat_print:nnnnnn { sprango } { R } { } { rango } { } { }
7422         \group_end:
7423     }
7424     { \msg_error:nnn { spintent } { need-math-mode } { \sprango } }
7425 }
7426 \NewDocumentCommand \spriq { 0{} }
7427 {
7428     \mode_if_math:TF
7429     {
7430         \group_begin:
7431         \attribute \__spintent_scope_attr_int = 1
7432         \keys_set:nn { spintent / spriq } {#1}
7433         \__spintent_stat_print:nnnnnn { spriq } { \mathrm{RIQ} } { } {
7434             { rango-intercuartil } { } { }
7435         }
7436         \group_end:
7437     }
7438     { \msg_error:nnn { spintent } { need-math-mode } { \spriq } }
7439 }
7440 \NewDocumentCommand \spmin { 0{} }
7441 {
7442     \mode_if_math:TF
7443     {
7444         \group_begin:
7445         \attribute \__spintent_scope_attr_int = 1
7446         \keys_set:nn { spintent / spmin } {#1}
7447         \__spintent_stat_print:nnnnnn { spmin } { x \c_math_subscript_token { \min } } { } {
7448             { mínimo } { } { }
7449         }
7450         \group_end:
7451     }
7452     { \msg_error:nnn { spintent } { need-math-mode } { \spmin } }
7453 }
7454 \NewDocumentCommand \spmax { 0{} }
7455 {
7456     \mode_if_math:TF
7457     {
7458         \group_begin:
7459         \attribute \__spintent_scope_attr_int = 1
7460         \keys_set:nn { spintent / spmax } {#1}
7461         \__spintent_stat_print:nnnnnn { spmax } { x \c_math_subscript_token { \max } } { } {
7462             { máximo } { } { }
7463         }
7464         \group_end:
7465     }
7466     { \msg_error:nnn { spintent } { need-math-mode } { \spmax } }
7467 }

```

(End of definition for `\sprango` and others. These functions are documented on page 38.)

13.55 The commands of the pos family

The user commands `\spcuartil`, `\spdecil` and `\spPerc` typeset the measures of position $Q_{\{k\}}$ (quartiles), $D_{\{k\}}$ (deciles) and $P_{\{k\}}$ (percentiles). The index k is the $\{\langle argument \rangle\}$: an integer from 1 to 3, 1 to 9 and 1

to **99** respectively. Each one is read with the name of the measure and the index, for example «*cuartil 1*».

13.55.1 Definition of variables for the pos family

```
\__spintent_spcuartil_silent_bool
\__spintent_spcuartil_read_arg_tl
\__spintent_pos_error_bool
\__spintent_pos_value_int
```

Each command of the family has its own copy of the variables `\__spintent_⟨name⟩_silent_bool` and `\__spintent_⟨name⟩_read_arg_tl`, shown here for `\spcuartil`, so each command can be configured separately with `\setspintent`. The boolean variable `\__spintent_pos_error_bool` is set to `true` by `\__spintent_pos_check:nnn` when the index is rejected, so that nothing is typeset. The integer variable `\__spintent_pos_value_int` holds the accepted index without leading zeros.

```
7463 \clist_map_inline:nn { spcuartil , spdecil , spPerc }
7464 {
7465     \bool_new:c { \__spintent_ #1 _silent_bool }
7466     \tl_new:c { \__spintent_ #1 _read_arg_tl }
7467 }
7468 \bool_new:N \__spintent_pos_error_bool
7469 \int_new:N \__spintent_pos_value_int
```

(End of definition for `\__spintent_spcuartil_silent_bool` and others.)

13.55.2 Definition of error messages for the pos family

```
pos-invalid-index
```

The error message `pos-invalid-index` is raised when the index passed to `\spcuartil`, `\spdecil` or `\spPerc` is not an integer without sign between **1** and the maximum of the measure.

```
7470 \msg_new:nnnn { spintent } { pos-invalid-index }
7471 {
7472     Invalid~index~'#2'~for~#1~\msg_line_context:.
7473 }
7474 {
7475     Command~#1 \c_space_tl only~accepts~an~integer~without~sign~between~1~and~#3.\\
7476 }
```

(End of definition for `pos-invalid-index`.)

13.55.3 Definition of keys for the pos family

```
silent
read-arg
unknown
```

We define the `⟨keys⟩` `silent` and `read-arg` for the three commands. As elsewhere, `read-arg` goes through `\__spintent_sanitize_affix:cn`.

```
7477 \clist_map_inline:nn { spcuartil , spdecil , spPerc }
7478 {
7479     \keys_define:nn { spintent / #1 }
7480     {
7481         silent      .bool_set:c = { \__spintent_ #1 _silent_bool },
7482         silent      .initial:n = false,
7483         silent      .value_required:n = true,
7484         read-arg    .code:n =
7485             { \__spintent_sanitize_affix:cn { \__spintent_ #1 _read_arg_tl } {##1} },
7486         read-arg    .initial:n = ,
7487         read-arg    .value_required:n = true,
7488         unknown     .code:n =
7489             {
7490                 \str_set:Nn \__spintent_command_name_str { #1 }
7491                 \__spintent_unknown_keys_cmd:n {##1}
7492             },
7493     }
7494 }
```

(End of definition for `silent`, `read-arg`, and `unknown`.)

13.55.4 Internal functions for the pos family

```
\__spintent_pos_check:nnn
```

The function `\__spintent_pos_check:nnn` receives the short name of the command in ‘**#1**’, the maximum of the index in ‘**#2**’ and the index in ‘**#3**’. It calls `\__spintent_luafun_clean_split_and_set:n` on the index and accepts it only if it is an integer without sign between **1** and ‘**#2**’. Otherwise it raises `pos-invalid-index`, whatever the value of `silent`, and sets `\__spintent_pos_error_bool` to `true`.

```
7495 \cs_new_protected:Nn \__spintent_pos_check:nnn
7496 {
7497     \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#3} }
7498     \bool_set_true:N \__spintent_pos_error_bool
7499     \bool_lazy_and:nnT
7500     { \str_if_eq_p:Vn \__spintent_luaset_has_integer_str { true } }
7501     { \tl_if_empty_p:N \__spintent_luaset_sign_tl }
7502     {
7503         \bool_lazy_and:nnT
7504         { \int_compare_p:nNn { \__spintent_luaset_only_part_int_str } > { 0 } }
7505         { \int_compare_p:nNn { \__spintent_luaset_only_part_int_str } < { #2 + 1 } }
```

```

7506         { \bool_set_false:N \__spintent_pos_error_bool }
7507     }
7508     \bool_if:NT \__spintent_pos_error_bool
7509     {
7510         \msg_error:nneee { spintent } { pos-invalid-index }
7511         { \token_to_str:c {#1} } { \exp_not:n {#3} } {#2}
7512     }
7513 }

```

(End of definition for `\__spintent_pos_check:nnn`.)

`\__spintent_pos_typeset:nn` The function `\__spintent_pos_typeset:nn` typesets the symbol in ‘#1’ with the index in ‘#2’ as a subscript, with its own `\MathMLintent` set to `:silent`.

```

7514 \cs_new_protected:Nn \__spintent_pos_typeset:nn
7515 {
7516     #1
7517     \c_math_subscript_token { \MathMLintent { :silent } {#2} }
7518 }

```

(End of definition for `\__spintent_pos_typeset:nn`.)

`\__spintent_pos_print:nnnn` The function `\__spintent_pos_print:nnnn` receives the short name of the command in ‘#1’, the symbol in ‘#2’, the name of the measure in ‘#3’ and the index in ‘#4’. The symbol with its index is always typeset by `\__spintent_pos_typeset:nn` inside one native `\MathMLintent`, preceded by the inner ghost `\__spintent_inner_sep:`.

With `silent` true, the intent is `_-`. With `read-arg` given, the intent is `_⟨read-arg⟩:nofix`. Otherwise the spoken content is the name of the measure followed by the index, as in `_cuartil-1:nofix`.

```

7519 \cs_new_protected:Nn \__spintent_pos_print:nnnn
7520 {
7521     \__spintent_invisible_sep:
7522     \bool_if:cTF { l__spintent_ #1 _silent_bool }
7523     {
7524         \MathMLintent { _- }
7525         {
7526             \__spintent_inner_sep:
7527             \__spintent_pos_typeset:nn {#2} {#4}
7528         }
7529     }
7530     {
7531         \tl_if_empty:cTF { l__spintent_ #1 _read_arg_tl }
7532         {
7533             \MathMLintent { _ #3 - #4 :nofix }
7534             {
7535                 \__spintent_inner_sep:
7536                 \__spintent_pos_typeset:nn {#2} {#4}
7537             }
7538         }
7539         {
7540             \MathMLintent { _ \tl_use:c { l__spintent_ #1 _read_arg_tl } :nofix }
7541             {
7542                 \__spintent_inner_sep:
7543                 \__spintent_pos_typeset:nn {#2} {#4}
7544             }
7545         }
7546     }
7547 }
7548 \cs_generate_variant:Nn \__spintent_pos_print:nnnn { nnnV }

```

(End of definition for `\__spintent_pos_print:nnnn`.)

`\__spintent_pos_exec:nnnnn` The function `\__spintent_pos_exec:nnnnn` receives the short name of the command in ‘#1’, the symbol in ‘#2’, the name of the measure in ‘#3’, the maximum of the index in ‘#4’ and the index itself in ‘#5’. It validates the index with `\__spintent_pos_check:nnn` and, if it is accepted, stores its value in `\l__spintent_pos_value_int`, so `025` is printed and read as `25`, and prints it through `\__spintent_pos_print:nnnn`.

```

7549 \cs_new_protected:Nn \__spintent_pos_exec:nnnnn
7550 {
7551     \__spintent_pos_check:nnn {#1} {#4} {#5}
7552     \bool_if:NF \__spintent_pos_error_bool
7553     {
7554         \int_set:Nn \l__spintent_pos_value_int { \l__spintent_luaseta_only_part_int_str }
7555         \__spintent_pos_print:nnnV {#1} {#2} {#3} \l__spintent_pos_value_int

```

```

7556     }
7557 }

```

(End of definition for `\__spintent_pos_exec:nnnnn`.)

The public commands `\spcuartil`, `\spdecil` and `\spPerc` enforce *math mode*, open a local group, set the `\keys` and delegate execution to `\__spintent_pos_exec:nnnnn` with the symbols Q, D and P, the measures `cuartil`, `decil` and `percentil` and the maxima 3, 9 and 99.

```

7558 \NewDocumentCommand \spcuartil { 0{} m }
7559 {
7560   \mode_if_math:TF
7561   {
7562     \group_begin:
7563     \attribute \__spintent_scope_attr_int = 1
7564     \keys_set:nn { spintent / spcuartil } {#1}
7565     \__spintent_pos_exec:nnnnn { spcuartil } { Q } { cuartil } { 3 } {#2}
7566     \group_end:
7567   }
7568   { \msg_error:nnn { spintent } { need-math-mode } { \spcuartil } }
7569 }
7570 \NewDocumentCommand \spdecil { 0{} m }
7571 {
7572   \mode_if_math:TF
7573   {
7574     \group_begin:
7575     \attribute \__spintent_scope_attr_int = 1
7576     \keys_set:nn { spintent / spdecil } {#1}
7577     \__spintent_pos_exec:nnnnn { spdecil } { D } { decil } { 9 } {#2}
7578     \group_end:
7579   }
7580   { \msg_error:nnn { spintent } { need-math-mode } { \spdecil } }
7581 }
7582 \NewDocumentCommand \spPerc { 0{} m }
7583 {
7584   \mode_if_math:TF
7585   {
7586     \group_begin:
7587     \attribute \__spintent_scope_attr_int = 1
7588     \keys_set:nn { spintent / spPerc } {#1}
7589     \__spintent_pos_exec:nnnnn { spPerc } { P } { percentil } { 99 } {#2}
7590     \group_end:
7591   }
7592   { \msg_error:nnn { spintent } { need-math-mode } { \spPerc } }
7593 }

```

(End of definition for `\spcuartil`, `\spdecil`, and `\spPerc`. These functions are documented on page 38.)

13.56 The command `\spFrec`

The user command `\spFrec` typesets the usual symbols of the frequencies of a table of data: `f` (absolute), `F` (accumulated absolute), `f_{r}` (relative), `F_{r}` (accumulated relative), `f_{r}\%` (percentage) and `F_{r}\%` (accumulated percentage). The symbol is the `{\argument}`, written exactly as it is printed, and it is read with the name of the frequency, for example «*frecuencia absoluta acumulada*».

13.56.1 Definition of variables for `\spFrec`

The variables of the keys are those of the `stat` family. The string variable `\__spintent_spFrec_symbol_str` holds the `{\argument}` without spaces, and the token list variable `\__spintent_spFrec_concept_tl` the name of the frequency, empty while the symbol is not in the table.

```

7594 \str_new:N \__spintent_spFrec_symbol_str
7595 \tl_new:N \__spintent_spFrec_concept_tl

```

(End of definition for `\__spintent_spFrec_symbol_str` and `\__spintent_spFrec_concept_tl`.)

13.56.2 Definition of error messages for `\spFrec`

The error message `spFrec-invalid-symbol` is raised when the symbol is not in the table and the key `read-arg` was not given. The error message `spFrec-invalid-sub-index` is raised when the key `sub-index` is given with a symbol other than `f` or `F`.

```

7596 \msg_new:nnnn { spintent } { spFrec-invalid-symbol }
7597 {
7598   Invalid~symbol~'#1'~for~\token_to_str:N \spFrec \c_space_tl \msg_line_context:.
7599 }
7600 {

```

```

7601     The~symbol~must~be~one~of~f,~F,~f_{r},~F_{r},~f_{r}\iow_char:N \%-or~
7602     F_{r}\iow_char:N \%,~or~the~key~read~arg~must~be~given.\\
7603 }
7604 \msg_new:nnnn { spintent } { spFrec-invalid-sub-index }
7605 {
7606     The~key~sub-index~is~not~valid~for~'#1'~\msg_line_context:.
7607 }
7608 {
7609     The~key~sub-index~of~\token_to_str:N \spFrec \c_space_tl only~accepts~the~symbols~f~and~F.\\
7610 }

```

(End of definition for spFrec-invalid-symbol and spFrec-invalid-sub-index.)

13.56.3 Internal functions for \spFrec

`\__spinttent_spFrec_exec:n` The function `\__spinttent_spFrec_exec:n` receives the symbol in ‘#1’. It removes the spaces and looks the result up in the table, setting `\__spinttent_spFrec_concept_tl` to frecuencia-absoluta, frecuencia-absoluta-acumulada, frecuencia-relativa, frecuencia-relativa-acumulada, frecuencia- or frecuencia-porcentual-acumulada.

If the symbol is not in the table, it is rejected with `spFrec-invalid-symbol`, unless the key `read-arg` was given: then the symbol is printed as it was written and read as `read-arg` says. A `sub-index` is accepted only with the symbols `f` and `F`; with any other symbol, even with `read-arg`, it is rejected with `spFrec-invalid-sub-index`. The `{(argument)}` is rejected whatever the value of the key `silent`. When it is accepted, the function delegates the printing to `\__spinttent_stat_print:nnnnnn`.

```

7611 \cs_new_protected:Nn \__spinttent_spFrec_exec:n
7612 {
7613     \str_set:Nc \__spinttent_spFrec_symbol_str { \tl_to_str:n {#1} }
7614     \str_remove_all:Nn \__spinttent_spFrec_symbol_str { ~ }
7615     \tl_clear:N \__spinttent_spFrec_concept_tl
7616     \str_case:Vn \__spinttent_spFrec_symbol_str
7617     {
7618         { f }
7619         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-absoluta } }
7620         { F }
7621         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-absoluta-acumulada } }
7622         { f_{r} }
7623         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-relativa } }
7624         { F_{r} }
7625         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-relativa-acumulada } }
7626         { f_{r}\% }
7627         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-porcentual } }
7628         { F_{r}\% }
7629         { \tl_set:Nn \__spinttent_spFrec_concept_tl { frecuencia-porcentual-acumulada } }
7630     }
7631     \bool_lazy_and:nnTF
7632     { \tl_if_empty_p:N \__spinttent_spFrec_concept_tl }
7633     { \tl_if_empty_p:N \__spinttent_spFrec_read_arg_tl }
7634     {
7635         \msg_error:nnn { spintent } { spFrec-invalid-symbol } {#1}
7636     }
7637     {
7638         \bool_lazy_and:nnTF
7639         { \bool_not_p:n { \str_if_empty_p:N \__spinttent_spFrec_sub_index_str } }
7640         {
7641             \bool_not_p:n
7642             {
7643                 \bool_lazy_or_p:nn
7644                 { \str_if_eq_p:Vn \__spinttent_spFrec_symbol_str { f } }
7645                 { \str_if_eq_p:Vn \__spinttent_spFrec_symbol_str { F } }
7646             }
7647         }
7648         {
7649             \msg_error:nnn { spintent } { spFrec-invalid-sub-index } {#1}
7650         }
7651         {
7652             \__spinttent_stat_print:nnnnnn { spFrec } {#1} { }
7653             { \__spinttent_spFrec_concept_tl } { } { }
7654         }
7655     }
7656 }

```

(End of definition for __spinttent_spFrec_exec:n)

`\spFrec` The public command `\spFrec` enforces *math mode*, opens a local group, sets the $\langle keys \rangle$ and delegates execution to `\__spintent_spFrec_exec:n`.

```

7657 \NewDocumentCommand \spFrec { O{} m }
7658 {
7659   \mode_if_math:TF
7660   {
7661     \group_begin:
7662     \attribute \__spintent_scope_attr_int = 1
7663     \keys_set:nn { spintent / spFrec } {#1}
7664     \__spintent_spFrec_exec:n {#2}
7665     \group_end:
7666   }
7667   { \msg_error:nnn { spintent } { need-math-mode } { \spFrec } }
7668 }

```

(End of definition for `\spFrec`. This function is documented on page 39.)

13.57 The command `\spcase`

The user command `\spcase` typesets the symbols of the cases of a probability: $C_{\{f\}}$ (favorable), $C_{\{p\}}$ (possible) and $C_{\{t\}}$ (total). The letter is the $\langle argument \rangle$, and the symbol is read with the name of the cases, for example «*casos favorables*».

13.57.1 Definition of variables for `\spcase`

`\l__spintent_spcase_letter_str` The variables of the keys are those of the `stat` family. The string variable `\l__spintent_spcase_letter_str` holds the $\langle argument \rangle$ without spaces, and the token list variable `\l__spintent_spcase_concept_tl` the name of the cases, empty while the letter is not in the table.

```

7669 \str_new:N \l__spintent_spcase_letter_str
7670 \tl_new:N \l__spintent_spcase_concept_tl

```

(End of definition for `\l__spintent_spcase_letter_str` and `\l__spintent_spcase_concept_tl`.)

13.57.2 Definition of error messages for `\spcase`

`spcase-invalid-letter` The error message `spcase-invalid-letter` is raised when the $\langle argument \rangle$ is not one of the letters `f`, `p` or `t`.

```

7671 \msg_new:nnnn { spintent } { spcase-invalid-letter }
7672 {
7673   Invalid~letter~'#1'~for~\token_to_str:N \spcase \c_space_tl \msg_line_context:.
7674 }
7675 {
7676   The~letter~must~be~f,~p~or~t.\\
7677 }

```

(End of definition for `spcase-invalid-letter`.)

13.57.3 Internal functions for `\spcase`

`\__spintent_spcase_exec:n` The function `\__spintent_spcase_exec:n` receives the letter in `'#1'`. It removes the spaces and looks the result up in the table, setting `\l__spintent_spcase_concept_tl` to `casos-favorables`, `casos-possibles` or `casos-totales`. If the letter is not in the table, it is rejected with `spcase-invalid-letter` whatever the value of the keys `silent` and `read-arg`. Otherwise the function delegates the printing of the symbol, with the letter as its subscript, to `\__spintent_stat_print:nnnnnn`.

```

7678 \cs_new_protected:Nn \__spintent_spcase_exec:n
7679 {
7680   \str_set:Ne \l__spintent_spcase_letter_str { \tl_to_str:n {#1} }
7681   \str_remove_all:Nn \l__spintent_spcase_letter_str { ~ }
7682   \tl_clear:N \l__spintent_spcase_concept_tl
7683   \str_case:Vn \l__spintent_spcase_letter_str
7684   {
7685     { f } { \tl_set:Nn \l__spintent_spcase_concept_tl { casos-favorables } }
7686     { p } { \tl_set:Nn \l__spintent_spcase_concept_tl { casos-possibles } }
7687     { t } { \tl_set:Nn \l__spintent_spcase_concept_tl { casos-totales } }
7688   }
7689   \tl_if_empty:NTF \l__spintent_spcase_concept_tl
7690   { \msg_error:nnn { spintent } { spcase-invalid-letter } {#1} }
7691   {
7692     \__spintent_stat_print:nnnnnn { spcase }
7693     { C \c_math_subscript_token { \str_use:N \l__spintent_spcase_letter_str } }
7694     { } { \l__spintent_spcase_concept_tl } { } { }
7695   }
7696 }

```

(End of definition for `\__spintent_spcase_exec:n`.)

`\spcase` The public command `\spcase` enforces *math mode*, opens a local group, sets the *(keys)* and delegates execution to `\__spintent_spcase_exec:n`.

```

7697 \NewDocumentCommand \spcase { O{} m }
7698 {
7699   \mode_if_math:TF
7700   {
7701     \group_begin:
7702       \attribute \__spintent_scope_attr_int = 1
7703       \keys_set:nn { spintent / spcase } {#1}
7704       \__spintent_spcase_exec:n {#2}
7705     \group_end:
7706   }
7707   { \msg_error:nnn { spintent } { need-math-mode } { \spcase } }
7708 }

```

(End of definition for `\spcase`. This function is documented on page 39.)

13.58 The command `\spOmega`

The user command `\spOmega` typesets the symbol Ω of the sample space, or $\#\Omega$ (its cardinality) with the key `card`. It is read «*espacio muestral*» or «*cardinalidad del espacio muestral*».

13.58.1 Definition of variables for `\spOmega`

`\__spintent_spOmega_card_bool` The variables of the keys `silent`, `sub-index` and `read-arg` are those of the `stat` family. The variable `\__spintent_spOmega_read_sub_bool` of that family has no key here and is set to `true`, so the subscript is always spoken with «*sub*». The boolean variable `\__spintent_spOmega_card_bool` stores the key `card`.

```

7709 \bool_new:N \__spintent_spOmega_card_bool

```

(End of definition for `\__spintent_spOmega_card_bool`.)

13.58.2 Definition of keys for `\spOmega`

`silent` We define the *(keys)* `silent`, `sub-index`, `read-arg` and `card`, the last one starting in `false`. As elsewhere, `sub-index` is kept as a string and `read-arg` goes through `\__spintent_sanitizé_affix:cn`.

```

read-arg 7710 \keys_define:nn { spintent / spOmega }
card      7711 {
unknown   7712   silent      .bool_set:N = \__spintent_spOmega_silent_bool,
          7713   silent      .initial:n = false,
          7714   silent      .value_required:n = true,
          7715   sub-index   .code:n =
          7716     { \str_set:Nn \__spintent_spOmega_sub_index_str {#1} },
          7717   sub-index   .initial:n = ,
          7718   sub-index   .value_required:n = true,
          7719   read-arg    .code:n =
          7720     { \__spintent_sanitizé_affix:Nn \__spintent_spOmega_read_arg_tl {#1} },
          7721   read-arg    .initial:n = ,
          7722   read-arg    .value_required:n = true,
          7723   card        .bool_set:N = \__spintent_spOmega_card_bool,
          7724   card        .initial:n = false,
          7725   card        .value_required:n = true,
          7726   unknown    .code:n =
          7727     {
          7728       \str_set:Nn \__spintent_command_name_str { spOmega }
          7729       \__spintent_unknown_keys_cmd:n {#1}
          7730     },
          7731 }

```

(End of definition for `silent` and others.)

13.58.3 Internal functions for `\spOmega`

`\spOmega` The public command `\spOmega` enforces *math mode*, opens a local group, sets the *(keys)* and delegates the printing to `\__spintent_stat_print:nnnnnn`, with the symbol Ω and the concept `espacio-muestral`, or the symbol $\#\Omega$ and the concept `cardinalidad-del-espacio-muestral` when `card` is `true`.

```

7732 \NewDocumentCommand \spOmega { O{} }
7733 {
7734   \mode_if_math:TF
7735   {
7736     \group_begin:
7737       \attribute \__spintent_scope_attr_int = 1
7738       \keys_set:nn { spintent / spOmega } {#1}
7739       \bool_if:NTF \__spintent_spOmega_card_bool

```

```

7740         {
7741             \__spintenc_stat_print:nnnnnn { spOmega } { \# \Omega } { }
7742             { cardinalidad-del-espacio-muestral } { } { }
7743         }
7744         {
7745             \__spintenc_stat_print:nnnnnn { spOmega } { \Omega } { }
7746             { espacio-muestral } { } { }
7747         }
7748     \group_end:
7749 }
7750 { \msg_error:nnn { spintenc } { need-math-mode } { \spOmega } }
7751 }

```

(End of definition for `\spOmega`. This function is documented on page 39.)

13.59 The command `\spPval`

The user command `\spPval` typesets the value of a probability as a fraction a/b (`\dfrac` or `\frac`, per `use-dfrac`), read natively as «*a de b*» — no Lua parsing involved.

13.59.1 Definition of keys for `\spPval`

We define the `\keys` `silent` and `use-dfrac`.

```

7752 \keys_define:nn { spintenc / spPval }
7753 {
7754     silent      .bool_set:N = \__spintenc_spPval_silent_bool,
7755     silent      .initial:n = false,
7756     silent      .value_required:n = true,
7757     use-dfrac    .bool_set:N = \__spintenc_spPval_use_dfrac_bool,
7758     use-dfrac    .initial:n = false,
7759     use-dfrac    .value_required:n = true,
7760     unknown     .code:n =
7761     {
7762         \str_set:Nn \__spintenc_command_name_str { spPval }
7763         \__spintenc_unknown_keys_cmd:n {#1}
7764     },
7765 }

```

(End of definition for `silent`, `use-dfrac`, and `unknown`.)

`\__spintenc_spPval_exec:nnn` The function `\__spintenc_spPval_exec:nnn` receives the optional `\keys` in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent`, the fraction (`\frac` or `\dfrac`, per `use-dfrac`) is silenced as one unit. Otherwise numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside the fraction, and the outer `\MathMLintenc` reads `_de:infix($num,$denom)`.

```

7766 \cs_new_protected:Nn \__spintenc_spPval_exec:nnn
7767 {
7768     \keys_set:nn { spintenc / spPval } { #1 }
7769     \__spintenc_invisible_sep:
7770     \bool_if:NTF \__spintenc_spPval_silent_bool
7771     {
7772         \MathMLintenc { _- }
7773         {
7774             \__spintenc_inner_sep:
7775             \bool_if:NTF \__spintenc_spPval_use_dfrac_bool
7776             { \dfrac { #2 } { #3 } }
7777             { \frac { #2 } { #3 } }
7778         }
7779     }
7780     {
7781         \MathMLintenc { _de:infix($num,$denom) }
7782         {
7783             \__spintenc_inner_sep:
7784             \bool_if:NTF \__spintenc_spPval_use_dfrac_bool
7785             {
7786                 \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
7787             }
7788             { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
7789         }
7790     }
7791 }

```

(End of definition for `\__spintenc_spPval_exec:nnn`.)

```

\spPval The public command \spPval enforces math mode, opens a local group and delegates execution to
  \__spintent_spPval_exec:nnn.
7792 \NewDocumentCommand \spPval { O{} m m }
7793 {
7794   \mode_if_math:TF
7795   {
7796     \group_begin:
7797     \__spintent_spPval_exec:nnn { #1 } { #2 } { #3 }
7798     \group_end:
7799   }
7800   { \msg_error:nnn { spintent } { need-math-mode } { \spPval } }
7801 }

```

(End of definition for \spPval. This function is documented on page 40.)

13.60 The command \spProb

The user command `\spProb` typesets the probability of an event as $P(\langle event \rangle)$. The $\langle argument \rangle$ is written as it is printed: a single event, or two events joined by exactly one of `\cap`, `\cup`, `\mid` or the character ‘|’, which are read «*probabilidad de A*», «*probabilidad de A y B*», «*probabilidad de A o B*» and «*probabilidad de A dado B*».

13.60.1 Definition of variables for \spProb

Variables set by the function `\__spintent_luafun_spProb_parse_and_set:n`. The string variable `\l__spintent_spProb_luaset_error_str` is `true` when the $\langle argument \rangle$ is invalid. The string variable `\l__spintent_spProb_luaset_op_str` is `none`, `cap`, `cup` or `mid`, and `\l__spintent_spProb_luaset_a_str` and `\l__spintent_spProb_luaset_b_str` hold the event before and after the operator (the second one is empty when there is no operator).

```

7802 \str_new:N \l__spintent_spProb_luaset_error_str
7803 \str_new:N \l__spintent_spProb_luaset_op_str
7804 \str_new:N \l__spintent_spProb_luaset_a_str
7805 \str_new:N \l__spintent_spProb_luaset_b_str

```

(End of definition for `\l__spintent_spProb_luaset_error_str` and others.)

The boolean variable `\l__spintent_spProb_silent_bool` stores the key `silent`, and the token list variable `\l__spintent_spProb_read_arg_tl` the key `read-arg`.

```

7806 \bool_new:N \l__spintent_spProb_silent_bool
7807 \tl_new:N \l__spintent_spProb_read_arg_tl

```

(End of definition for `\l__spintent_spProb_silent_bool` and `\l__spintent_spProb_read_arg_tl`.)

13.60.2 Definition of error messages for \spProb

The error message `spProb-invalid-argument` is raised when the $\langle argument \rangle$ is empty, has unbalanced braces or parentheses, has an empty event, or has more than one operator.

```

7808 \msg_new:nnnn { spintent } { spProb-invalid-argument }
7809 {
7810   Invalid-argument~'~#1'~for~\token_to_str:N \spProb \c_space_tl \msg_line_context:.
7811 }
7812 {
7813   The~argument~must~be~an~event,~or~two~events~joined~by~exactly~one~of~
7814   \token_to_str:N \cap,~\token_to_str:N \cup,~\token_to_str:N \mid \c_space_tl or~|,~
7815   outside~any~braces~or~parentheses.\\
7816 }

```

(End of definition for `spProb-invalid-argument`.)

13.60.3 Definition of keys for \spProb

We define the $\langle keys \rangle$ `silent` and `read-arg`. As elsewhere, `read-arg` goes through `\__spintent_sanitize_affix:N`.

```

7817 \keys_define:nn { spintent / spProb }
7818 {
7819   silent      .bool_set:N = \l__spintent_spProb_silent_bool,
7820   silent      .initial:n = false,
7821   silent      .value_required:n = true,
7822   read-arg    .code:n =
7823     { \__spintent_sanitize_affix:Nn \l__spintent_spProb_read_arg_tl {#1} },
7824   read-arg    .initial:n = ,
7825   read-arg    .value_required:n = true,
7826   unknown     .code:n =
7827     {
7828       \str_set:Nn \l__spintent_command_name_str { spProb }
7829       \__spintent_unknown_keys_cmd:n {#1}

```

```

7830     },
7831 }

```

(End of definition for `silent`, `read-arg`, and `unknown`.)

13.60.4 Internal functions for `\spProb`

```

__spintent_spProb_nested_ops:
__spintent_spProb_event:N

```

The function `\__spintent_spProb_nested_ops`: redefines, locally, `\cap`, `\cup` and `\mid` so that each one is wrapped in its own `\MathMLintent` (`_y`, `_o` and `_dado`) while keeping its math class (`\mathbin` or `\mathrel`) and therefore its spacing. The function `\__spintent_spProb_event:N` typesets the event stored in its argument inside a group with these definitions, so the operators nested in braces or parentheses are also read; the operators at the top level never go through it, they are printed by `\__spintent_spProb_print`. The boolean variable `\l__spintent_spProb_nested_bool` makes the redefinition happen only once when the groups are nested (for instance a `\spcomp` inside `\spProb`), so that the saved meanings are the original ones.

```

7832 \bool_new:N \l__spintent_spProb_nested_bool
7833 \cs_new_protected:Nn \__spintent_spProb_nested_ops:
7834 {
7835   \bool_if:NF \l__spintent_spProb_nested_bool
7836   {
7837     \bool_set_true:N \l__spintent_spProb_nested_bool
7838     \cs_set_eq:NN \__spintent_spProb_cap_orig: \cap
7839     \cs_set_eq:NN \__spintent_spProb_cup_orig: \cup
7840     \cs_set_eq:NN \__spintent_spProb_mid_orig: \mid
7841     \cs_set_protected:Npn \cap
7842       { \mathbin { \MathMLintent { _y } { \__spintent_spProb_cap_orig: } } }
7843     \cs_set_protected:Npn \cup
7844       { \mathbin { \MathMLintent { _o } { \__spintent_spProb_cup_orig: } } }
7845     \cs_set_protected:Npn \mid
7846       { \mathrel { \MathMLintent { _dado } { \__spintent_spProb_mid_orig: } } }
7847   }
7848 }
7849 \cs_new_protected:Nn \__spintent_spProb_event:N
7850 {
7851   \group_begin:
7852   \__spintent_spProb_nested_ops:
7853   \str_use:N #1
7854   \group_end:
7855 }

```

(End of definition for `\__spintent_spProb_nested_ops`: and `\__spintent_spProb_event:N`.)

```
__spintent_spProb_content:
```

The function `\__spintent_spProb_content`: typesets the event, or the two events and the operator, without any annotation.

```

7856 \cs_new_protected:Nn \__spintent_spProb_content:
7857 {
7858   \str_use:N \l__spintent_spProb_luaset_a_str
7859   \str_case:Vn \l__spintent_spProb_luaset_op_str
7860   {
7861     { cap } { \cap \str_use:N \l__spintent_spProb_luaset_b_str }
7862     { cup } { \cup \str_use:N \l__spintent_spProb_luaset_b_str }
7863     { mid } { \mid \str_use:N \l__spintent_spProb_luaset_b_str }
7864   }
7865 }

```

(End of definition for `\__spintent_spProb_content`:.)

```
__spintent_spProb_print:
```

The function `\__spintent_spProb_print`: prints $P(\langle content \rangle)$ inside one native `\MathMLintent`, preceded by the outer ghost `\__spintent_invisible_sep`: and by the inner ghost `\__spintent_inner_sep`:. With `silent` true, the intent is `_-`. With `read-arg` given, the intent is `_ $\langle read-arg \rangle$ :no fix`. Otherwise the intent is `_probabilidad-de:prefix($a)` with the single event tagged as `a`; with an operator, the intent is `_probabilidad-de:prefix($p)`, where `p` is the group of the two events, which carries its own intent `_y:infix($a,$b)`, `_o:infix($a,$b)` or `_dado:infix($a,$b)` according to the operator, with the two events tagged as `a` and `b`.

```

7866 \cs_new_protected:Nn \__spintent_spProb_print:
7867 {
7868   \__spintent_invisible_sep:
7869   \bool_if:NTF \l__spintent_spProb_silent_bool
7870   {
7871     \MathMLintent { _- }
7872   }

```

```

7873     \__spintenc_inner_sep:
7874     P ( \__spintenc_spProb_content: )
7875   }
7876 }
7877 {
7878   \tl_if_empty:NTF \__spintenc_spProb_read_arg_tl
7879   {
7880     \str_if_eq:VnTF \__spintenc_spProb_luaset_op_str { none }
7881     {
7882       \MathMLintent { _probabilidad-de:prefix($a) }
7883       {
7884         \__spintenc_inner_sep:
7885         P
7886         (
7887           \MathMLarg { a }
7888           { \__spintenc_inner_sep: \__spintenc_spProb_event:N \__spintenc_spProb_luaset_op_str }
7889         )
7890       }
7891     }
7892     {
7893       \MathMLintent { _probabilidad-de:prefix($p) }
7894       {
7895         \__spintenc_inner_sep:
7896         P
7897         (
7898           \MathMLarg { p }
7899           {
7900             \MathMLintent
7901             {
7902               -
7903               \str_case:Vn \__spintenc_spProb_luaset_op_str
7904               { { cap } { y } { cup } { o } { mid } { dado } }
7905               :infix($a,$b)
7906             }
7907             {
7908               \__spintenc_inner_sep:
7909               \MathMLarg { a }
7910               { \__spintenc_inner_sep: \__spintenc_spProb_event:N \__spintenc_spProb_luaset_op_str }
7911               \str_case:Vn \__spintenc_spProb_luaset_op_str
7912               { { cap } { \cap } { cup } { \cup } { mid } { \mid } }
7913               \MathMLarg { b }
7914               { \__spintenc_inner_sep: \__spintenc_spProb_event:N \__spintenc_spProb_luaset_op_str }
7915             }
7916           }
7917         )
7918       }
7919     }
7920   }
7921   {
7922     \MathMLintent { _ \tl_use:N \__spintenc_spProb_read_arg_tl :nofix }
7923     {
7924       \__spintenc_inner_sep:
7925       P ( \__spintenc_spProb_content: )
7926     }
7927   }
7928 }
7929 }

```

(End of definition for __spintenc_spProb_print:.)

__spintenc_spProb_exec:nn

The function `\__spintenc_spProb_exec:nn` sets the *keys* in ‘#1’ and calls `\__spintenc_luafun_spProb_parse_and_set:n` with the *argument* in ‘#2’. If the Lua module reports an error, it is rejected with `spProb-invalid-argument` whatever the value of the key `silent`, and nothing is printed; otherwise it delegates the printing to `\__spintenc_spProb_print:.`

```

7930 \cs_new_protected:Nn \__spintenc_spProb_exec:nn
7931 {
7932   \keys_set:nn { spintenc / spProb } {#1}
7933   \__spintenc_luafun_spProb_parse_and_set:n { \exp_not:n {#2} }
7934   \str_if_eq:VnTF \__spintenc_spProb_luaset_error_str { true }
7935   { \msg_error:nnn { spintenc } { spProb-invalid-argument } {#2} }
7936   { \__spintenc_spProb_print: }
7937 }

```

(End of definition for `\__spintent_spProb_exec:nn`.)

`\spProb` The public command `\spProb` enforces *math mode*, opens a local group and delegates execution to `\__spintent_spProb_exec:nn`.

```

7938 \NewDocumentCommand \spProb { O{} m }
7939 {
7940   \mode_if_math:TF
7941   {
7942     \group_begin:
7943       \attribute \__spintent_scope_attr_int = 1
7944       \__spintent_spProb_exec:nn { #1 } { #2 }
7945     \group_end:
7946   }
7947   { \msg_error:nnn { spintent } { need-math-mode } { \spProb } }
7948 }

```

(End of definition for `\spProb`. This function is documented on page 40.)

13.61 The command `\spcomp`

The user command `\spcomp` typesets the complement of an event, as $\langle event \rangle^{\{c\}}$ or, with the key `bar`, as $\overline{\langle event \rangle}$, and reads it «*complemento de*» the event.

13.61.1 Definition of variables for `\spcomp`

The boolean variables `\__spintent_spcomp_silent_bool` and `\__spintent_spcomp_bar_bool` store the keys `silent` and `bar`, the token list variable `\__spintent_spcomp_read_arg_tl` the key `read-arg`, and the string variable `\__spintent_spcomp_paren_str` the key `paren`. The boolean variable `\__spintent_spcomp_do_paren_bool` tells whether the event is printed in parentheses once the key and the event have been examined. The boolean variable `\__spintent_spcomp_compound_bool` is true when the event contains `\cap`, `\cup`, `\setminus` or `\mid`, whatever the keys are.

```

7949 \bool_new:N \__spintent_spcomp_silent_bool
7950 \bool_new:N \__spintent_spcomp_bar_bool
7951 \tl_new:N \__spintent_spcomp_read_arg_tl
7952 \str_new:N \__spintent_spcomp_paren_str
7953 \bool_new:N \__spintent_spcomp_do_paren_bool
7954 \bool_new:N \__spintent_spcomp_compound_bool

```

(End of definition for `\__spintent_spcomp_silent_bool` and others.)

13.61.2 Definition of keys for `\spcomp`

`silent` We define the *keys* `silent`, `bar`, `paren` and `read-arg`. As elsewhere, `read-arg` goes through `\__spintent_sanitizate_affix:Nn`.

```

7955 \keys_define:nn { spintent / spcomp }
7956 {
7957   silent      .bool_set:N = \__spintent_spcomp_silent_bool,
7958   silent      .initial:n = false,
7959   silent      .value_required:n = true,
7960   bar         .bool_set:N = \__spintent_spcomp_bar_bool,
7961   bar         .initial:n = false,
7962   bar         .value_required:n = true,
7963   paren       .choices:nn = { auto , true , false }
7964     { \str_set:Nx \__spintent_spcomp_paren_str { \l_keys_choice_tl } },
7965   paren       .initial:n = auto,
7966   paren       .value_required:n = true,
7967   read-arg    .code:n =
7968     { \__spintent_sanitizate_affix:Nn \__spintent_spcomp_read_arg_tl {#1} },
7969   read-arg    .initial:n = ,
7970   read-arg    .value_required:n = true,
7971   unknown     .code:n =
7972     {
7973       \str_set:Nn \__spintent_command_name_str { spcomp }
7974       \__spintent_unknown_keys_cmd:n {#1}
7975     },
7976 }

```

(End of definition for `silent` and others.)

13.61.3 Internal functions for \spcomp

```

\__spintent_spcomp_set_paren:n
\__spintent_spcomp_event:n

```

The function `\__spintent_spcomp_set_paren:n` sets `\l__spintent_spcomp_compound_bool` and `\l__spintent_spcomp_do_paren_bool`. The first one is true when the event contains `\cap`, `\cup`, `\setminus` or `\mid`. The second one follows the key `paren:` with `auto` it is true when the event is compound, and it is always false when the key `bar` is set (the overline already covers the event). The function `\__spintent_spcomp_event:n` typesets the event in a group where the operators nested in it are read as in `\__spintent_spProb_event:N`, and in parentheses when `\l__spintent_spcomp_do_paren_bool` is true. The parentheses belong to the event, so that they are inside the `\MathMLarg` and are read.

```

7977 \cs_new_protected:Nn \__spintent_spcomp_set_paren:n
7978 {
7979   \bool_set_false:N \l__spintent_spcomp_do_paren_bool
7980   \bool_set_false:N \l__spintent_spcomp_compound_bool
7981   \tl_map_inline:nn { \cap \cup \setminus \mid }
7982     {
7983       \tl_if_in:nnT {#1} {##1}
7984       { \bool_set_true:N \l__spintent_spcomp_compound_bool }
7985     }
7986   \bool_if:NF \l__spintent_spcomp_bar_bool
7987   {
7988     \str_case:Vn \l__spintent_spcomp_paren_str
7989     {
7990       { true } { \bool_set_true:N \l__spintent_spcomp_do_paren_bool }
7991       { auto }
7992       {
7993         \bool_if:NT \l__spintent_spcomp_compound_bool
7994         { \bool_set_true:N \l__spintent_spcomp_do_paren_bool }
7995       }
7996     }
7997   }
7998 }
7999 \cs_new_protected:Nn \__spintent_spcomp_event:n
8000 {
8001   \group_begin:
8002     \__spintent_spProb_nested_ops:
8003     \bool_if:NTF \l__spintent_spcomp_do_paren_bool
8004       { ( #1 ) }
8005       { #1 }
8006   \group_end:
8007 }

```

(End of definition for `\__spintent_spcomp_set_paren:n` and `\__spintent_spcomp_event:n`)

```

\__spintent_spcomp_typeset:n

```

The function `\__spintent_spcomp_typeset:n` typesets its argument as the complement of the event, with the exponent `c` or with an overline according to the key `bar`.

```

8008 \cs_new_protected:Nn \__spintent_spcomp_typeset:n
8009 {
8010   \bool_if:NTF \l__spintent_spcomp_bar_bool
8011     { \overline { #1 } }
8012     { { #1 } ^ { c } }
8013 }

```

(End of definition for `\__spintent_spcomp_typeset:n`)

```

\__spintent_spcomp_exec:nn

```

The function `\__spintent_spcomp_exec:nn` sets the `<keys>` in `'#1'` and prints the complement of the event `'#2'` inside one native `\MathMLintent`, preceded by the outer ghost `\__spintent_invisible_sep:` and by the inner ghost `\__spintent_inner_sep:`. With `silent true` the intent is `_-`; with `read-arg` given it is `_<read-arg>`:nofix; otherwise it is `_complemento-de:prefix($a)` with the event tagged as `a`; when the key `bar` is set and the event is compound, the intent is `_complemento-de-la-expresión:prefix($a)`, because the overline is not read and the reading would not tell what the complement covers.

```

8014 \cs_new_protected:Nn \__spintent_spcomp_exec:nn
8015 {
8016   \keys_set:nn { spintent / spcomp } {#1}
8017   \__spintent_spcomp_set_paren:n {#2}
8018   \__spintent_invisible_sep:
8019   \bool_if:NTF \l__spintent_spcomp_silent_bool
8020     {
8021       \MathMLintent { _- }
8022     }
8023     {
8024       \__spintent_inner_sep:
8025       \__spintent_spcomp_typeset:n { \__spintent_spcomp_event:n {#2} }

```

```

8025     }
8026   }
8027   {
8028     \tl_if_empty:NTF \l__spintent_spcomp_read_arg_tl
8029     {
8030       \MathMLintent
8031       {
8032         \bool_lazy_and:nnTF
8033         { \bool_if_p:N \l__spintent_spcomp_bar_bool }
8034         { \bool_if_p:N \l__spintent_spcomp_compound_bool }
8035         { _complemento-de-la-expresión:prefix($a) }
8036         { _complemento-de:prefix($a) }
8037       }
8038     }
8039     \__spintent_inner_sep:
8040     \__spintent_spcomp_typeset:n
8041     { \MathMLarg { a } { \__spintent_inner_sep: \__spintent_spcomp_event:n {#2} } }
8042   }
8043 }
8044 {
8045   \MathMLintent { _ \tl_use:N \l__spintent_spcomp_read_arg_tl :nofix }
8046   {
8047     \__spintent_inner_sep:
8048     \__spintent_spcomp_typeset:n { \__spintent_spcomp_event:n {#2} }
8049   }
8050 }
8051 }
8052 }

```

(End of definition for `\__spintent_spcomp_exec:nn`.)

`\spcomp` The public command `\spcomp` enforces *math mode*, opens a local group and delegates execution to `\__spintent_spcomp_exec:nn`.

```

8053 \NewDocumentCommand \spcomp { O{} m }
8054 {
8055   \mode_if_math:TF
8056   {
8057     \group_begin:
8058     \attribute \l__spintent_scope_attr_int = 1
8059     \__spintent_spcomp_exec:nn { #1 } { #2 }
8060     \group_end:
8061   }
8062   { \msg_error:nnn { spintent } { need-math-mode } { \spcomp } }
8063 }

```

(End of definition for `\spcomp`. This function is documented on page 41.)

13.62 The commands `\spcomb` and `\spfact`

The user command `\spcomb` typesets the counting symbols of combinatorics: combinations, variations, arrangements and permutations, with or without repetition, and circular permutations. The keys `type`, `rep` and `circ` choose the symbol and the reading; the key `style` chooses between the binomial coefficient and the letter form. The user command `\spfact` typesets the factorial.

13.62.1 Definition of variables for `\spcomb`

Variables set by the function `\__spintent_luafun_spcomb_prepare:nnnn`. The string variable `\l__spintent_spcomb_is_true` is `true` when the arguments are invalid. The string variables `\l__spintent_spcomb_luaset_n_str` and `\l__spintent_spcomb_luaset_k_str` hold the arguments without the surrounding spaces. In the `list` mode, `\l__spintent_spcomb_luaset_list_tl` holds the multiplicities, each one in its own braces, and `\l__spintent_spcomb_luaset_count_str` how many they are.

```

8064 \str_new:N \l__spintent_spcomb_luaset_error_str
8065 \str_new:N \l__spintent_spcomb_luaset_n_str
8066 \str_new:N \l__spintent_spcomb_luaset_k_str
8067 \tl_new:N \l__spintent_spcomb_luaset_list_tl
8068 \str_new:N \l__spintent_spcomb_luaset_count_str

```

(End of definition for `\l__spintent_spcomb_luaset_error_str` and others.)

The boolean variables store the keys `silent`, `rep` and `circ`; the token list variable `\l__spintent_spcomb_read_arg_tl` stores the key `read-arg`; and the string variables store the keys `type` and `style`.

```

8069 \bool_new:N \l__spintent_spcomb_silent_bool
8070 \bool_new:N \l__spintent_spcomb_rep_bool

```

©2026 by Pablo González L

```

8071 \bool_new:N \l__spintent_spcomb_circ_bool
8072 \tl_new:N \l__spintent_spcomb_read_arg_tl
8073 \str_new:N \l__spintent_spcomb_type_str
8074 \str_new:N \l__spintent_spcomb_style_str

```

(End of definition for `\l__spintent_spcomb_silent_bool` and others.)

Variables computed from the keys. The token list variable `\l__spintent_spcomb_letter_tl` holds the letters of the symbol (C, V, A or P, followed by R and/or C), and `\l__spintent_spcomb_concept_tl` the name of the concept that is read (before the final -de). The string variable `\l__spintent_spcomb_mode_str` is one, pair or list according to the arguments the symbol takes, and `\l__spintent_spcomb_problem_str` describes an incompatible combination of keys (it is empty when there is none). The boolean variable `\l__spintent_spcomb_use_binom_bool` is true when the symbol is printed with `\binom`. The integer `\l__spintent_spcomb_i_int` and the token list `\l__spintent_spcomb_listargs_tl` are used to number the multiplicities.

```

8075 \tl_new:N \l__spintent_spcomb_letter_tl
8076 \tl_new:N \l__spintent_spcomb_concept_tl
8077 \str_new:N \l__spintent_spcomb_mode_str
8078 \str_new:N \l__spintent_spcomb_problem_str
8079 \bool_new:N \l__spintent_spcomb_use_binom_bool
8080 \int_new:N \l__spintent_spcomb_i_int
8081 \tl_new:N \l__spintent_spcomb_listargs_tl

```

(End of definition for `\l__spintent_spcomb_letter_tl` and others.)

13.62.2 Definition of error messages for `\spcomb`

The message `spcomb-invalid-keys` is raised for a combination of keys that does not exist (`circ` without `type=permutation`, or with `rep`; `style=binom` for anything but combinations without repetition). The message `spcomb-missing-arg` is raised when the second argument is missing and the symbol needs it, and `spcomb-extra-arg` when it is given and the symbol does not take it. The message `spcomb-invalid-argument` is raised when the arguments are empty, when the list of multiplicities has fewer than two elements or an empty one, or when integers are not possible (`k` greater than `n`, or multiplicities that add more than `n`).

```

8082 \msg_new:nnnn { spintent } { spcomb-invalid-keys }
8083 {
8084   Incompatible~keys~for~\token_to_str:N \spcomb \c_colon_str \c_space_tl #1~\msg_line_context:.
8085 }
8086 {
8087   The~key~circ~needs~type=permutation~and~rep=false;~the~value~binom~of~
8088   the~key~style~needs~type=combination~and~rep=false.\\
8089 }
8090 \msg_new:nnnn { spintent } { spcomb-missing-arg }
8091 {
8092   Missing~second~argument~for~\token_to_str:N \spcomb \c_space_tl \msg_line_context:.
8093 }
8094 {
8095   This~kind~of~symbol~needs~a~second~argument:~k,~or~the~list~of~
8096   repetitions.\\
8097 }
8098 \msg_new:nnnn { spintent } { spcomb-extra-arg }
8099 {
8100   Unexpected~second~argument~for~\token_to_str:N \spcomb \c_space_tl \msg_line_context:.
8101 }
8102 {
8103   Circular~permutations~take~only~n.\\
8104 }
8105 \msg_new:nnnn { spintent } { spcomb-invalid-argument }
8106 {
8107   Invalid~arguments~'#1'~and~'#2'~for~\token_to_str:N \spcomb \c_space_tl \msg_line_context:.
8108 }
8109 {
8110   The~arguments~cannot~be~empty;~the~repetitions~must~be~a~list~of~at~
8111   least~two~elements~separated~by~commas;~if~they~are~integers,~k~cannot~
8112   be~greater~than~n~and~the~repetitions~cannot~add~more~than~n.\\
8113 }

```

(End of definition for `spcomb-invalid-keys` and others.)

13.62.3 Definition of keys for \spcomb

We define the *⟨keys⟩* `silent`, `type`, `rep`, `circ`, `style` and `read-arg`. As elsewhere, `read-arg` goes through `\__spintent_sanitize_affix:Nn`. The key `style` is `auto` by default: `binom` for combinations without repetition and `letter` otherwise.

```

8114 \keys_define:nn { spintent / spcomb }
8115 {
8116   silent      .bool_set:N = \l__spintent_spcomb_silent_bool,
8117   silent      .initial:n = false,
8118   silent      .value_required:n = true,
8119   rep         .bool_set:N = \l__spintent_spcomb_rep_bool,
8120   rep         .initial:n = false,
8121   rep         .value_required:n = true,
8122   circ        .bool_set:N = \l__spintent_spcomb_circ_bool,
8123   circ        .initial:n = false,
8124   circ        .value_required:n = true,
8125   type        .choices:nn =
8126     { combination , variation , arrangement , permutation }
8127     { \str_set:Nx \l__spintent_spcomb_type_str { \l_keys_choice_tl } },
8128   type        .initial:n = combination,
8129   type        .value_required:n = true,
8130   style       .choices:nn = { auto , binom , letter }
8131     { \str_set:Nx \l__spintent_spcomb_style_str { \l_keys_choice_tl } },
8132   style       .initial:n = auto,
8133   style       .value_required:n = true,
8134   read-arg    .code:n =
8135     { \__spintent_sanitize_affix:Nn \l__spintent_spcomb_read_arg_tl {#1} },
8136   read-arg    .initial:n = ,
8137   read-arg    .value_required:n = true,
8138   unknown     .code:n =
8139     {
8140       \str_set:Nn \l__spintent_command_name_str { spcomb }
8141       \__spintent_unknown_keys_cmd:n {#1}
8142     },
8143 }
```

(End of definition for `silent` and others.)

13.62.4 Internal functions for \spcomb

`\__spintent_spcomb_setup:` The function `\__spintent_spcomb_setup:` computes the letters and the concept from the keys, decides whether `\binom` is used, and leaves in `\l__spintent_spcomb_problem_str` the description of an incompatible combination of keys, if any.

```

8144 \cs_new_protected:Nn \__spintent_spcomb_setup:
8145 {
8146   \str_clear:N \l__spintent_spcomb_problem_str
8147   \str_case:Vn \l__spintent_spcomb_type_str
8148   {
8149     { combination }
8150     {
8151       \tl_set:Nn \l__spintent_spcomb_letter_tl { C }
8152       \tl_set:Nn \l__spintent_spcomb_concept_tl { combinaciones }
8153     }
8154     { variation }
8155     {
8156       \tl_set:Nn \l__spintent_spcomb_letter_tl { V }
8157       \tl_set:Nn \l__spintent_spcomb_concept_tl { variaciones }
8158     }
8159     { arrangement }
8160     {
8161       \tl_set:Nn \l__spintent_spcomb_letter_tl { A }
8162       \tl_set:Nn \l__spintent_spcomb_concept_tl { arreglos }
8163     }
8164     { permutation }
8165     {
8166       \tl_set:Nn \l__spintent_spcomb_letter_tl { P }
8167       \tl_set:Nn \l__spintent_spcomb_concept_tl { permutaciones }
8168     }
8169   }
8170   \bool_if:NT \l__spintent_spcomb_circ_bool
8171   {
8172     \bool_lazy_and:nnTF
```

```

8173         { \str_if_eq_p:Nn \__spintent_spcomb_type_str { permutation } }
8174         { ! \bool_if_p:N \__spintent_spcomb_rep_bool }
8175         {
8176             \tl_put_right:Nn \__spintent_spcomb_letter_tl { C }
8177             \tl_put_right:Nn \__spintent_spcomb_concept_tl { -circulares }
8178         }
8179         {
8180             \str_set:Nn \__spintent_spcomb_problem_str
8181             { circ~requires~type=permutation~and~rep=false }
8182         }
8183     }
8184     \bool_if:NT \__spintent_spcomb_rep_bool
8185     {
8186         \tl_put_right:Nn \__spintent_spcomb_letter_tl { R }
8187         \tl_put_right:Nn \__spintent_spcomb_concept_tl { -con-repetición }
8188     }
8189     \bool_set_false:N \__spintent_spcomb_use_binom_bool
8190     \bool_lazy_and:nnTF
8191     { \str_if_eq_p:Nn \__spintent_spcomb_type_str { combination } }
8192     { ! \bool_if_p:N \__spintent_spcomb_rep_bool }
8193     {
8194         \str_if_eq:NnF \__spintent_spcomb_style_str { letter }
8195         { \bool_set_true:N \__spintent_spcomb_use_binom_bool }
8196     }
8197     {
8198         \str_if_eq:NnT \__spintent_spcomb_style_str { binom }
8199         {
8200             \str_if_empty:NT \__spintent_spcomb_problem_str
8201             {
8202                 \str_set:Nn \__spintent_spcomb_problem_str
8203                 { style=binom~requires~type=combination~and~rep=false }
8204             }
8205         }
8206     }
8207 }

```

(End of definition for __spintent_spcomb_setup:.)

```

__spintent_spcomb_arg:nn
  \__spintent_spcomb_arg_ann:nn
  \__spintent_spcomb_arg_plain:nn
  \__spintent_spcomb_list_wrap:n
  \__spintent_spcomb_list_wrap_ann:n
  \__spintent_spcomb_list_wrap_plain:n

```

The functions `\__spintent_spcomb_arg:nn` and `\__spintent_spcomb_list_wrap:n` are the two places where the symbol is annotated. They are set locally to the annotated versions (an `\MathMLarg` with the inner ghost, and the intent that joins the multiplicities) or to the plain ones, so that the same function `\__spintent_spcomb_symbol:` prints the symbol in both cases.

```

8208 \cs_new_protected:Nn \__spintent_spcomb_arg_ann:nn
8209 { \MathMLarg {#1} { \__spintent_inner_sep: #2 } }
8210 \cs_new_protected:Nn \__spintent_spcomb_arg_plain:nn { #2 }
8211 \cs_new_eq:NN \__spintent_spcomb_arg:nn \__spintent_spcomb_arg_plain:nn
8212 \cs_new_protected:Nn \__spintent_spcomb_list_wrap_ann:n
8213 {
8214     \MathMLintent { _y:infix ( \__spintent_spcomb_listargs_tl ) }
8215     { \__spintent_inner_sep: #1 }
8216 }
8217 \cs_new_protected:Nn \__spintent_spcomb_list_wrap_plain:n { #1 }
8218 \cs_new_eq:NN \__spintent_spcomb_list_wrap:n \__spintent_spcomb_list_wrap_plain:n

```

(End of definition for __spintent_spcomb_arg:nn and others.)

```

__spintent_spcomb_listarg:n
  \__spintent_spcomb_list:

```

The function `\__spintent_spcomb_listarg:n` expands to the argument reference $\$m\langle i \rangle$, preceded by a comma from the second one on; it builds the argument list of the intent that joins the multiplicities. The function `\__spintent_spcomb_list:` typesets the multiplicities separated by commas, each one tagged as $m\langle i \rangle$.

```

8219 \cs_new:Nn \__spintent_spcomb_listarg:n
8220 {
8221     \int_compare:nT { #1 > 1 } { , }
8222     $ m #1
8223 }
8224 \cs_new_protected:Nn \__spintent_spcomb_list:
8225 {
8226     \int_zero:N \__spintent_spcomb_i_int
8227     \tl_map_inline:Nn \__spintent_spcomb_luaset_list_tl
8228     {
8229         \int_incr:N \__spintent_spcomb_i_int

```

```

8230     \int_compare:nT { \l__spintent_spcomb_i_int > 1 } { , }
8231     \exp_args:Nx \l__spintent_spcomb_arg:nn
8232     { m \int_use:N \l__spintent_spcomb_i_int } {##1}
8233   }
8234 }

```

(End of definition for `\l__spintent_spcomb_listarg:n` and `\l__spintent_spcomb_list:.`)

`\l__spintent_spcomb_symbol:`

The function `\l__spintent_spcomb_symbol:` typesets the symbol: $\binom{n}{k}$ with n and k , or the letters with k as the subscript and n as the superscript. With one argument n is the subscript, and with a list of multiplicities they form the superscript. The letters are grouped so the scripts belong to all of them.

```

8235 \cs_new_protected:Nn \l__spintent_spcomb_symbol:
8236 {
8237   \bool_if:NTF \l__spintent_spcomb_use_binom_bool
8238   {
8239     \int_compare:nTF { \mathstyle < 2 } { \Bigl } { \bigl } \lparen
8240     \genfrac { } { } { } { \opt } { }
8241     { \l__spintent_spcomb_arg:nn { n } { \str_use:N \l__spintent_spcomb_luaset_n_str } }
8242     { \l__spintent_spcomb_arg:nn { k } { \str_use:N \l__spintent_spcomb_luaset_k_str } }
8243     \int_compare:nTF { \mathstyle < 2 } { \Bigr } { \bigr } \rparen
8244   }
8245   {
8246     { \tl_use:N \l__spintent_spcomb_letter_tl }
8247     \str_case:Vn \l__spintent_spcomb_mode_str
8248     {
8249       { pair }
8250       {
8251         \c_math_subscript_token
8252         { \l__spintent_spcomb_arg:nn { k } { \str_use:N \l__spintent_spcomb_luaset_k_str } }
8253         ^
8254         { \l__spintent_spcomb_arg:nn { n } { \str_use:N \l__spintent_spcomb_luaset_n_str } }
8255       }
8256       { one }
8257       {
8258         \c_math_subscript_token
8259         { \l__spintent_spcomb_arg:nn { n } { \str_use:N \l__spintent_spcomb_luaset_n_str } }
8260       }
8261       { list }
8262       {
8263         \c_math_subscript_token
8264         { \l__spintent_spcomb_arg:nn { n } { \str_use:N \l__spintent_spcomb_luaset_n_str } }
8265         ^
8266         {
8267           \l__spintent_spcomb_arg:nn { m }
8268           { \l__spintent_spcomb_list_wrap:n { \l__spintent_spcomb_list: } }
8269         }
8270       }
8271     }
8272   }
8273 }

```

(End of definition for `\l__spintent_spcomb_symbol:.`)

`\l__spintent_spcomb_print:`

The function `\l__spintent_spcomb_print:` prints the symbol inside one native `\MathMLintent`, preceded by the outer ghost `\l__spintent_invisible_sep:` and by the inner ghost `\l__spintent_inner_sep:`.

With `silent true` the intent is `_-`. With `read-arg` given it is `_(read-arg):no fix`. Otherwise the intent of the whole symbol is the concept followed by `-de:` with one argument it is `_(concept)-de:prefix($n)`; with pair it is `_(concept)-de:prefix($c)`, where c is the symbol, which carries its own intent `_en:infix($n,$k)`, except for the binomial coefficient, whose single intent is `_sobre:infix($n,$k)`; with list it is `_(concept)-de:prefix($c)`, where c is the symbol, which carries the intent `_con-repeticiones:infix($n,$m)`, and m is the group of the multiplicities, with the intent `_y:infix($m1,$m2,...)`.

```

8274 \cs_new_protected:Nn \l__spintent_spcomb_print:
8275 {
8276   \l__spintent_invisible_sep:
8277   \bool_if:NTF \l__spintent_spcomb_silent_bool
8278   {
8279     \cs_set_eq:NN \l__spintent_spcomb_arg:nn \l__spintent_spcomb_arg_plain:nn
8280     \cs_set_eq:NN \l__spintent_spcomb_list_wrap:n \l__spintent_spcomb_list_wrap_plain:n
8281     \MathMLintent { _- }
8282     { \l__spintent_inner_sep: \l__spintent_spcomb_symbol: }
8283   }

```

```

8284 {
8285   \tl_if_empty:NTF \l__spintent_spcomb_read_arg_tl
8286   {
8287     \cs_set_eq:NN \__spintent_spcomb_arg:nn \__spintent_spcomb_arg_ann:nn
8288     \cs_set_eq:NN \__spintent_spcomb_list_wrap:n \__spintent_spcomb_list_wrap_ann:n
8289     \str_case:Vn \l__spintent_spcomb_mode_str
8290     {
8291       { one }
8292       {
8293         \MathMLint
8294         { _ \tl_use:N \l__spintent_spcomb_concept_tl -de:prefix($n) }
8295         { \__spintent_inner_sep: \__spintent_spcomb_symbol: }
8296       }
8297       { pair }
8298       {
8299         \bool_if:NTF \l__spintent_spcomb_use_binom_bool
8300         {
8301           \MathMLint { _sobre:infix($n,$k) }
8302           { \__spintent_inner_sep: \__spintent_spcomb_symbol: }
8303         }
8304         {
8305           \MathMLint
8306           { _ \tl_use:N \l__spintent_spcomb_concept_tl -de:prefix($c) }
8307           {
8308             \__spintent_inner_sep:
8309             \MathMLarg { c }
8310             {
8311               \MathMLint { _en:infix($n,$k) }
8312               { \__spintent_inner_sep: \__spintent_spcomb_symbol: }
8313             }
8314           }
8315         }
8316       }
8317       { list }
8318       {
8319         \tl_set:Nx \l__spintent_spcomb_listargs_tl
8320         {
8321           \int_step_function:nN
8322           { \l__spintent_spcomb_luaset_count_str }
8323           \__spintent_spcomb_listarg:n
8324         }
8325         \MathMLint
8326         { _ \tl_use:N \l__spintent_spcomb_concept_tl -de:prefix($c) }
8327         {
8328           \__spintent_inner_sep:
8329           \MathMLarg { c }
8330           {
8331             \MathMLint { _con-repeticiones:infix($n,$m) }
8332             { \__spintent_inner_sep: \__spintent_spcomb_symbol: }
8333           }
8334         }
8335       }
8336     }
8337   }
8338   {
8339     \cs_set_eq:NN \__spintent_spcomb_arg:nn \__spintent_spcomb_arg_plain:nn
8340     \cs_set_eq:NN \__spintent_spcomb_list_wrap:n \__spintent_spcomb_list_wrap_plain:n
8341     \MathMLint { _ \tl_use:N \l__spintent_spcomb_read_arg_tl :nofix }
8342     { \__spintent_inner_sep: \__spintent_spcomb_symbol: }
8343   }
8344 }
8345 }

```

(End of definition for __spintent_spcomb_print:.)

__spintent_spcomb_exec:nnn

The function `\__spintent_spcomb_exec:nnn` sets the *⟨keys⟩* in ‘#1’, rejects an incompatible combination of keys, decides the mode from the keys and from the presence of the second argument ‘#3’, and calls `\__spintent_luafun_spcomb_prepare:nnnn` with the arguments. Whatever the value of `silent`, an invalid case is rejected with its error message and nothing is printed; otherwise the printing is delegated to `\__spintent_spcomb_print:.`

8346 \cs_new_protected:Nn __spintent_spcomb_exec:nnn

```

8347 {
8348   \keys_set:nn { spintent / spcomb } {#1}
8349   \__spintent_spcomb_setup:
8350   \str_if_empty:NTF \l__spintent_spcomb_problem_str
8351   {
8352     \bool_if:NTF \l__spintent_spcomb_circ_bool
8353     {
8354       \tl_if_novalue:NTF {#3}
8355       { \str_set:Nn \l__spintent_spcomb_mode_str { one } }
8356       { \str_set:Nn \l__spintent_spcomb_mode_str { extra } }
8357     }
8358     {
8359       \bool_lazy_and:nnTF
8360       { \str_if_eq_p:Vn \l__spintent_spcomb_type_str { permutation } }
8361       { \bool_if_p:N \l__spintent_spcomb_rep_bool }
8362       {
8363         \tl_if_novalue:NTF {#3}
8364         { \str_set:Nn \l__spintent_spcomb_mode_str { missing } }
8365         { \str_set:Nn \l__spintent_spcomb_mode_str { list } }
8366       }
8367     }
8368     \tl_if_novalue:NTF {#3}
8369     {
8370       \str_if_eq:VnTF \l__spintent_spcomb_type_str { permutation }
8371       { \str_set:Nn \l__spintent_spcomb_mode_str { one } }
8372       { \str_set:Nn \l__spintent_spcomb_mode_str { missing } }
8373     }
8374     { \str_set:Nn \l__spintent_spcomb_mode_str { pair } }
8375   }
8376 }
8377 \str_case:VnF \l__spintent_spcomb_mode_str
8378 {
8379   { extra }
8380   { \msg_error:nn { spintent } { spcomb-extra-arg } }
8381   { missing }
8382   { \msg_error:nn { spintent } { spcomb-missing-arg } }
8383 }
8384 {
8385   \__spintent_luafun_spcomb_prepare:nnnn
8386   { \l__spintent_spcomb_mode_str }
8387   { \exp_not:n {#2} }
8388   { \tl_if_novalue:NF {#3} { \exp_not:n {#3} } }
8389   {
8390     \bool_if:NTF \l__spintent_spcomb_rep_bool
8391     { false } { true }
8392   }
8393   \str_if_eq:VnTF \l__spintent_spcomb_luaset_error_str { true }
8394   {
8395     \tl_if_novalue:NTF {#3}
8396     { \msg_error:nnnn { spintent } { spcomb-invalid-argument } {#2} { } }
8397     { \msg_error:nnnn { spintent } { spcomb-invalid-argument } {#2} {#3} }
8398   }
8399   { \__spintent_spcomb_print: }
8400 }
8401 }
8402 {
8403   \msg_error:nnx { spintent } { spcomb-invalid-keys }
8404   { \l__spintent_spcomb_problem_str }
8405 }
8406 }

```

(End of definition for __spintent_spcomb_exec:nnn.)

```

\__spintent_spcomb_begin:nnn
  \__spintent_spcomb_with_k:nnn

```

The function `\__spintent_spcomb_begin:nnn` opens a local group, activates the scope attribute and delegates execution to `\__spintent_spcomb_exec:nnn`; its variant `nnV` is used when the second argument is absent, passing the marker `\c_novalue_tl` that `\tl_if_novalue:NTF` recognises. The function `\__spintent_spcomb_with_k:nnn` reads the second argument, which is present, and passes it on.

```

8407 \cs_new_protected:Nn \__spintent_spcomb_begin:nnn
8408 {
8409   \group_begin:
8410   \attribute \l__spintent_scope_attr_int = 1

```

```

8411     \__spintent_spcomb_exec:nnn { #1 } { #2 } { #3 }
8412     \group_end:
8413   }
8414   \cs_new_protected:Npn \__spintent_spcomb_with_k:nnn #1#2#3
8415   { \__spintent_spcomb_begin:nnn {#1} {#2} {#3} }
8416   \cs_generate_variant:Nn \__spintent_spcomb_begin:nnn { nnV }

```

(End of definition for __spintent_spcomb_begin:nnn and __spintent_spcomb_with_k:nnn.)

\spcomb The public command `\spcomb` enforces *math mode*. The second argument is optional and it is not delimited by brackets but by its braces: the command looks at the next token and, if it opens a group, reads it as the second argument (k or the list of repetitions); otherwise the second argument is absent, as for the permutations that take only n.

```

8417   \NewDocumentCommand \spcomb { O{} m }
8418   {
8419     \mode_if_math:TF
8420     {
8421       \peek_catcode_ignore_spaces:NTF \c_group_begin_token
8422       { \__spintent_spcomb_with_k:nnn { #1 } { #2 } }
8423       { \__spintent_spcomb_begin:nnV { #1 } { #2 } \c_novalue_tl }
8424     }
8425     { \msg_error:nnn { spintent } { need-math-mode } { \spcomb } }
8426   }

```

(End of definition for \spcomb. This function is documented on page 41.)

13.62.5 The command \spfact

The user command `\spfact` typesets the factorial $n!$ and reads it «*n factorial*». It has the keys `silent` and `read-arg`.

The boolean variable `\l__spintent_spfact_silent_bool` stores the key `silent`, and the token list variable `\l__spintent_spfact_read_arg_tl` the key `read-arg`.

```

8427   \bool_new:N \l__spintent_spfact_silent_bool
8428   \tl_new:N \l__spintent_spfact_read_arg_tl

```

(End of definition for \l__spintent_spfact_silent_bool and \l__spintent_spfact_read_arg_tl.)

```

silent   The <keys> of \spfact.
read-arg
unknown
8429   \keys_define:nn { spintent / spfact }
8430   {
8431     silent      .bool_set:N = \l__spintent_spfact_silent_bool,
8432     silent      .initial:n = false,
8433     silent      .value_required:n = true,
8434     read-arg    .code:n =
8435     { \__spintent_sanitize_affix:Nn \l__spintent_spfact_read_arg_tl {#1} },
8436     read-arg    .initial:n = ,
8437     read-arg    .value_required:n = true,
8438     unknown     .code:n =
8439     {
8440       \str_set:Nn \l__spintent_command_name_str { spfact }
8441       \__spintent_unknown_keys_cmd:n {#1}
8442     },
8443   }

```

(End of definition for silent, read-arg, and unknown.)

`\__spintent_spfact_exec:nn` The function `\__spintent_spfact_exec:nn` sets the <keys> in ‘#1’ and prints the factorial of ‘#2’ inside one native `\MathMLintent`, preceded by the outer ghost `\__spintent_invisible_sep:` and followed by the inner ghost `\__spintent_inner_sep:`. With `silent true` the intent is `_-`; with `read-arg` given it is `_<read-arg>`:nofix; otherwise it is `_factorial:postfix($n)`.

```

8444   \cs_new_protected:Nn \__spintent_spfact_exec:nn
8445   {
8446     \keys_set:nn { spintent / spfact } {#1}
8447     \__spintent_invisible_sep:
8448     \bool_if:NTF \l__spintent_spfact_silent_bool
8449     { \MathMLintent { _- } { \__spintent_inner_sep: #2 ! } }
8450     {
8451       \tl_if_empty:NTF \l__spintent_spfact_read_arg_tl
8452       {
8453         \MathMLintent { _factorial:postfix($n) }
8454         { \__spintent_inner_sep: \MathMLarg { n } { \__spintent_inner_sep: #2 } ! }

```

```

8455         }
8456     {
8457         \MathMLintent { _ \tl_use:N \l__spintent_spfact_read_arg_tl :nofix }
8458         { \__spintent_inner_sep: #2 ! }
8459     }
8460 }
8461 }

```

(End of definition for `\__spintent_spfact_exec:nn`.)

`\spfact` The public command `\spfact` enforces *math mode*, opens a local group and delegates execution to `\__spintent_spfact_exec:nn`.

```

8462 \NewDocumentCommand \spfact { O{} m }
8463 {
8464     \mode_if_math:TF
8465     {
8466         \group_begin:
8467         \attribute \l__spintent_scope_attr_int = 1
8468         \__spintent_spfact_exec:nn { #1 } { #2 }
8469         \group_end:
8470     }
8471     { \msg_error:nnn { spintent } { need-math-mode } { \spfact } }
8472 }

```

(End of definition for `\spfact`. This function is documented on page 42.)

13.63 Finish package

Finish package implementation.

```

8473 \file_input_stop:
8474 \endpackage

```

14 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
<code>\angle</code>	153
<code>autofit</code>	<u>4219</u>
B	
<code>bar</code>	<u>7955</u>
C	
<code>card</code>	<u>7710</u>
<code>cifras-sep</code>	<u>289</u> , <u>574</u> , <u>861</u> , <u>1900</u> , <u>1971</u> , <u>2594</u>
<code>circ</code>	<u>8114</u>
Document class:	
<code>l3doc</code>	49
<code>cmd-key-unknown</code>	<u>208</u>
<code>cmd-key-value-unknown</code>	<u>208</u>
Commands provide by spintent :	
<code>\spmoney</code>	67
<code>\spnum</code>	57
<code>\spsiglo</code>	77, 79
<code>\spunit</code>	62
<code>concept</code> <u>2062</u> , <u>2274</u> , <u>3127</u> , <u>3483</u> , <u>4219</u> , <u>4839</u> , <u>5048</u> , <u>5335</u> , <u>5489</u> , <u>5674</u> , <u>6076</u> , <u>6332</u> , <u>6631</u> , <u>6949</u>	
<code>concept*</code>	<u>5674</u>
<code>currency</code>	<u>861</u>
<code>currency-no-subunits</code>	<u>830</u>
D	
<code>diameter</code>	<u>6332</u>
I	
<code>inline</code>	<u>2456</u>
<code>inline-numeric-denominator</code>	<u>552</u>
<code>invalid-angle-format</code>	<u>2544</u>
<code>invalid-century-format</code>	<u>1256</u>
<code>invalid-currency</code>	<u>830</u>
<code>invalid-date-format</code>	<u>1486</u>
<code>invalid-time-format</code>	<u>1407</u>
J	
<code>join-word</code>	<u>2682</u>
K	
Keys for <code>\spAcirc</code> provide by spintent :	
<code>concept</code>	183
<code>prefix</code>	182, 183
<code>print-sym</code>	182–184
<code>radio-sty</code>	182
<code>read-arg</code>	182, 183
<code>read-cmd</code>	182, 183
<code>read-sty</code>	182, 183
<code>read-sub</code>	183
<code>silent</code>	183
<code>sub-index</code>	182, 183
Keys for <code>\spAfig</code> provide by spintent :	
<code>prefix</code>	164
<code>print-sym</code>	164, 165
<code>read-arg</code>	164–166
<code>read-cmd</code>	164
<code>read-sub</code>	165
Keys for <code>\spangle</code> provide by spintent :	
<code>concept</code>	152
<code>prefix</code>	152
<code>print-m</code>	152
<code>read-cmd</code>	152
<code>recto</code>	152
<code>silent</code>	152
Keys for <code>\sparc</code> provide by spintent :	
<code>concept</code>	158
<code>prefix</code>	157, 158
<code>read-cmd</code>	157, 158
<code>silent</code>	158
Keys for <code>\spcase</code> provide by spintent :	
<code>read-arg</code>	203
<code>silent</code>	203
Keys for <code>\spcirc</code> provide by spintent :	
<code>prefix</code>	176
<code>print-sym</code>	176–178
<code>radio-sty</code>	176, 177
<code>read-arg</code>	177
<code>read-cmd</code>	176
<code>read-sty</code>	176, 177
<code>read-sub</code>	177
<code>sub-index</code>	176, 177
Keys for <code>\spcomb</code> provide by spintent :	
<code>circ</code>	211, 213
<code>read-arg</code>	211, 213
<code>rep</code>	211, 213
<code>silent</code>	211, 213
<code>style</code>	211, 213
<code>type</code>	211, 213
Keys for <code>\spcomp</code> provide by spintent :	
<code>bar</code>	209, 210
<code>paren</code>	209, 210
<code>read-arg</code>	209
<code>silent</code>	209
Keys for <code>\spcoord</code> provide by spintent :	
<code>silent</code>	111
Keys for <code>\spcuartil</code> provide by spintent :	
<code>read-arg</code>	199
<code>silent</code>	199
Keys for <code>\spdecil</code> provide by spintent :	
<code>read-arg</code>	199
<code>silent</code>	199
Keys for <code>\spdesv</code> provide by spintent :	
<code>read-arg</code>	194
<code>read-sub</code>	194
<code>sigma</code>	194
<code>silent</code>	194
<code>sub-index</code>	194
Keys for <code>\spdiam</code> provide by spintent :	
<code>concept</code>	172
<code>read-arg</code>	172
<code>read-cmd</code>	172
<code>read-sub</code>	172
<code>silent</code>	172
<code>sub-index</code>	172
<code>upper-case</code>	172
Keys for <code>\spdiv</code> provide by spintent :	
<code>cifras-sep</code>	90

notation-sym	90	Keys for \spmediana provide by spintent:	
read-parens	90	read-arg	194
read-period-sep	90	read-sub	194
read-sign	90	silent	194
read-sym	90	sub-index	194
set-sym	90	Keys for \spmedia provide by spintent:	
wrap-parens	90	read-arg	194
Keys for \spdom provide by spintent:		read-sub	194
prefix	130	silent	194
silent	130	sub-index	194
Keys for \spdsym provide by spintent:		Keys for \spmmin provide by spintent:	
prefix	87	read-arg	194
read-sym	87	silent	194
set-sym	87	Keys for \spmoda provide by spintent:	
suffix	87	read-arg	194
Keys for \spfact provide by spintent:		read-sub	194
read-arg	218	silent	194
silent	218	sub-index	194
Keys for \spFrec provide by spintent:		Keys for \spmoney provide by spintent:	
read-arg	201, 202	cifras-sep	69
sub-index	201	currency	69
Keys for \spfuncomp provide by spintent:		money-math	69
silent	127	sub-units	69
Keys for \spfun provide by spintent:		Keys for \spmQ provide by spintent:	
concept	122	join-word	105–107
prefix	122	read-parens	106, 107
read-cmd	122	silent	105, 106, 109, 110
silent	122	use-dfrac	106–109
Keys for \spinterval provide by spintent:		use-spnZ	106, 109
concept	116, 118	wrap-parens	105–107
prefix	115, 116, 118	Keys for \spmside provide by spintent:	
read-arg	115, 116, 118	concept	148
read-cmd	115, 116, 120	prefix	148
silent	116, 118, 121	print-m	148
size	115–117	print-sym	172
Keys for \spmangle provide by spintent:		read-arg	148
concept	152	read-cmd	148, 151
prefix	152	read-sty	148
print-m	152	silent	148, 151
read-cmd	152	Keys for \spmsym provide by spintent:	
recto	152	not-print	88
silent	152	prefix	88
Keys for \spmarc provide by spintent:		read-sym	88
concept	158	set-sym	88
prefix	157, 158	suffix	88
read-cmd	157, 158	Keys for \spmul provide by spintent:	
silent	158	cifras-sep	91
Keys for \spmax provide by spintent:		notation-sym	91
read-arg	194	read-parens	91
silent	194	read-period-sep	91
Keys for \spmcD provide by spintent:		read-sign	91
concept	93	read-sym	91
prefix	93	set-sym	91
read-cmd	93	wrap-parens	91
result	93	Keys for \spnD provide by spintent:	
silent	93	concept	97
upper-case	93	prefix	97
Keys for \spmcM provide by spintent:		result-num	97
concept	93	result	97
prefix	93	silent	97
read-cmd	93	Keys for \spnM provide by spintent:	
result	93	concept	97
silent	93	prefix	97
upper-case	93	result-num	97
		result	97

silent	97	Keys for \sprango provide by spintent :	
Keys for \spnum provide by spintent :		read-arg	194
cifras-sep	56–58	read-sub	194
notation-sym	57	silent	194
read-period-sep	57, 59	sub-index	194
read-sign	57	Keys for \sprayo provide by spintent :	
Keys for \spnZ provide by spintent :		concept	145
cifras-sep	103	prefix	145
read-parens	103, 104	read-cmd	145, 146
read-sign	103	read-sty	145
wrap-parens	103, 104	silent	145, 146
Keys for \spOmega provide by spintent :		Keys for \sprazon provide by spintent :	
card	204	inline	100
read-arg	204	silent	100
silent	204	use-dfrac	100
sub-index	204	Keys for \spread provide by spintent :	
Keys for \spPcirc provide by spintent :		read-arg	85
concept	183	silent	85
prefix	182, 183	Keys for \sprecta provide by spintent :	
print-sym	182–184	prefix	141
radio-sty	182	read-cmd	141
read-arg	182, 183	silent	141
read-cmd	182, 183	Keys for \spriq provide by spintent :	
read-sty	182, 183	read-arg	194
read-sub	183	read-sub	194
silent	183	silent	194
sub-index	182, 183	sub-index	194
Keys for \spPerc provide by spintent :		Keys for \spshort provide by spintent :	
read-arg	199	read-arg	74
silent	199	small-caps	74
Keys for \spPfig provide by spintent :		Keys for \spside provide by spintent :	
prefix	164	concept	148
print-sym	164, 165	prefix	148
read-arg	164–166	read-arg	148
read-cmd	164	read-cmd	148, 151
read-sub	165	read-sty	148
Keys for \spPoint provide by spintent :		silent	148, 151
concept	134	Keys for \spsiglo provide by spintent :	
prefix	134	print-era	77, 79
read-cmd	134	Keys for \spunit provide by spintent :	
silent	134	cifras-sep	63
Keys for \spPoly provide by spintent :		notation-sym	63
concept	161	print-cdot	63
prefix	161	read-cdot	63
print-sym	161	read-period-sep	63
read-cmd	161	read-slash	63
silent	161	Keys for \spusep provide by spintent :	
Keys for \spProb provide by spintent :		silent	83
read-arg	206	size	83
silent	206	Keys for \spvarz provide by spintent :	
Keys for \spProm provide by spintent :		read-arg	194
read-arg	193	read-sub	194
silent	193	sigma	194
Keys for \spPval provide by spintent :		silent	194
silent	205	sub-index	194
use-dfrac	205	Keys for \spvec provide by spintent :	
Keys for \sprad provide by spintent :		autofit	137
concept	172	concept	137
read-arg	172	prefix	136, 137
read-cmd	172	read-cmd	136, 137
read-sub	172	silent	137
silent	172	Keys for \spVol provide by spintent :	
sub-index	172	concept	189
upper-case	172	read-arg	189
		read-cmd	189

read-sub	189	size	1534, 3127
silent	189	small-caps	1134
sub-index	189	\spAcirc	31, 182, 6916
M			
mathcal-fallback	80	spAcirc-invalid-arg	6622
\MathMLintent	156	\spAfig	26, 164, 6047
mc-invalid-arguments	2055	spAfig-invalid-arg	5636
mc-too-few-arguments	2300	spAfig-invalid-points	5636
md-invalid-argument	2300	spAfig-read-arg-key	5636
Lua module provide by spintent :			
spintent.lua	49, 50, 52, 55-57, 62, 66-69, 74, 75, 77-81, 93, 97, 102, 105, 110, 134	spAfig-sym-cardinality-conflict	5636
money-math	861	spAfig-sym-letra-conflict	5636
N			
not-print	1825	\spang	24, 102, 2584
notation-sym	289, 574, 1900, 1971	\spangle	23, 152, 5298
P			
Packages :			
expl3	55, 57	spangle-invalid-arg	5038
lua-unicode-math	49	\sparc	25, 157, 5458
luamml	52, 105, 108	sparc-invalid-points	5325
tagpdf	73, 74, 78	\spcase	39, 203, 7697
unicode-math	49, 84	spcase-invalid-letter	7671
paren	7955	\spcirc	30, 176, 6598
pos-invalid-index	7470	spcirc-invalid-arg	6325
prefix	1754, 1825, 2062, 2274, 3127, 3483, 3899, 4107, 4219, 4462, 4699, 4839, 5048, 5335, 5489, 5674, 6332, 6631	\spcod	36, 130, 3966
print-era	1264	\spcomb	41, 211, 8417
print-m	4839, 5048	spcomb-extra-arg	8082
print-sym	5489, 5674, 6076, 6332, 6631	spcomb-invalid-argument	8082
R			
radio-sty	6332	spcomb-invalid-keys	8082
read-arg	1134, 1674, 3127, 4839, 5674, 6076, 6332, 6631, 6949, 7149, 7212, 7477, 7710, 7817, 7955, 8114, 8429	spcomb-missing-arg	8082
read-cdot	574	\spcomp	41, 209, 8053
read-cmd	2062, 3127, 3483, 4107, 4219, 4462, 4699, 4839, 5048, 5335, 5489, 5674, 6076, 6332, 6631, 6949	\spcoord	19, 111, 3074
read-parens	1900, 1971, 2594, 2682	spcoord-invalid-arg	2928
read-period-sep	289, 574, 1900, 1971	\spcuartil	38, 7558
read-sign	289, 1900, 1971, 2594	\spdate	11, 81, 1510
read-slash	574	\spdecil	38, 7558
read-sty	4699, 4839, 6332, 6631	\spdesv	37, 7374
read-sub	5674, 6076, 6332, 6631, 6949, 7212	\spdiam	29, 172, 6292
read-sym	1754, 1825, 1900, 1971	\spdiv	13, 89, 1961
recto	5048	spdiv-empty-argument	1920
rep	8114	\spdom	36, 130, 3966
result	2062, 2274	spdom-invalid-arg	3895
result-num	2274	\spdsym	12, 87, 1801
right-angle-sqr-fallback	80	\spfact	42, 211, 8462
\rightangle	153	\spFrec	39, 201, 7657
S			
seq commands:			
\seq_set_from_clist:Nn	62	spFrec-invalid-sub-index	7596
set-sym	1754, 1825, 1900, 1971	spFrec-invalid-symbol	7596
\setspintent	3, 53, 196	\spfun	35, 121, 3697
sigma	7212	spfun-invalid-arg	3479
silent	1534, 1674, 2062, 2274, 2456, 2682, 2932, 3127, 3483, 3722, 3899, 4107, 4219, 4462, 4699, 4839, 5048, 5335, 5489, 5674, 6076, 6332, 6631, 6949, 7149, 7212, 7477, 7710, 7752, 7817, 7955, 8114, 8429	\spfuncomp	36, 126, 3877
		spfuncomp-invalid-arg	3718
		spintent internal commands:	
		__spintent_collapse_hyphens:N	128
		\l__spintent_command_name_str	208
		\l__spintent_geo_circfig_mode_str	6621
		\l__spintent_geo_final_print_tl	5628
		\l__spintent_geo_luaset_angulo_case_str	2900
		\l__spintent_geo_luaset_circ_radio_raw_str	6314
		\l__spintent_geo_luaset_circ_radio_type_str	6314
		\l__spintent_geo_luaset_concept_str	2900
		\l__spintent_geo_luaset_error_str	2900
		\l__spintent_geo_luaset_intent_str	2900
		\l__spintent_geo_luaset_letra_str	2900
		\l__spintent_geo_luaset_letra_word_str	2900

<code>__spintent_spangle_sym:n</code>	5088	<code>__spintent_spcomb_list_wrap:n</code>	8208
<code>__spintent_sparc_exec:nnn</code>	5453	<code>__spintent_spcomb_list_wrap_ann:n</code> ...	8208
<code>\l__spintent_sparc_intent_arg_tl</code>	5379	<code>__spintent_spcomb_list_wrap_plain:n</code> ..	8208
<code>\l__spintent_sparc_intent_tl</code>	5379	<code>__spintent_spcomb_listarg:n</code>	8219
<code>__spintent_sparc_mathml:n</code>	5397	<code>\l__spintent_spcomb_listargs_tl</code>	8075
<code>\l__spintent_sparc_mode_str</code>	5320	<code>\l__spintent_spcomb_luaset_count_str</code> ..	8064
<code>\l__spintent_sparc_prefix_tl</code>	5320	<code>\l__spintent_spcomb_luaset_error_str</code> ..	8064
<code>__spintent_sparc_print_arc:n</code>	5373	<code>\l__spintent_spcomb_luaset_k_str</code>	8064
<code>__spintent_sparc_process:nn</code>	5384	<code>\l__spintent_spcomb_luaset_list_tl</code> ...	8064
<code>\l__spintent_sparc_pt_idx_int</code>	5379	<code>\l__spintent_spcomb_luaset_n_str</code>	8064
<code>\l__spintent_sparc_pt_seq</code>	5379	<code>\l__spintent_spcomb_mode_str</code>	8075
<code>\l__spintent_sparc_refs_tl</code>	5379	<code>__spintent_spcomb_print:</code>	8274
<code>__spintent_sparc_wide_arc:n</code>	5369	<code>\l__spintent_spcomb_problem_str</code>	8075
<code>\l__spintent_spcase_concept_tl</code>	203, 7669	<code>\l__spintent_spcomb_read_arg_tl</code>	8069
<code>__spintent_spcase_exec:n</code>	7678	<code>\l__spintent_spcomb_rep_bool</code>	8069
<code>\l__spintent_spcase_letter_str</code>	7669	<code>__spintent_spcomb_setup:</code>	8144
<code>\l__spintent_spcirc_articulo_str</code>	6316	<code>\l__spintent_spcomb_silent_bool</code>	8069
<code>__spintent_spcirc_build_concept:</code>	6470	<code>\l__spintent_spcomb_style_str</code>	8069
<code>\l__spintent_spcirc_connector_arg_tl</code> ..	6418	<code>__spintent_spcomb_symbol:</code>	8235
<code>\l__spintent_spcirc_connector_tl</code>	6418	<code>\l__spintent_spcomb_type_str</code>	8069
<code>__spintent_spcirc_exec:nn</code>	6593	<code>\l__spintent_spcomb_use_binom_bool</code> ...	8075
<code>\l__spintent_spcirc_has_arg_bool</code>	6418	<code>__spintent_spcomb_with_k:nnn</code>	8407
<code>\l__spintent_spcirc_intent_arg_tl</code>	6418	<code>\l__spintent_spcomp_bar_bool</code>	7949
<code>\l__spintent_spcirc_intent_tl</code>	6418	<code>\l__spintent_spcomp_compound_bool</code>	7949
<code>__spintent_spcirc_mathml:</code>	6540	<code>\l__spintent_spcomp_do_paren_bool</code>	7949
<code>\l__spintent_spcirc_mode_str</code>	6316	<code>__spintent_spcomp_event:n</code>	7977
<code>\l__spintent_spcirc_prefix_tl</code>	6316	<code>__spintent_spcomp_exec:nn</code>	8014
<code>\l__spintent_spcirc_print_sym_str</code>	6316	<code>\l__spintent_spcomp_paren_str</code>	7949
<code>__spintent_spcirc_process:n</code>	6524	<code>\l__spintent_spcomp_read_arg_tl</code>	7949
<code>\l__spintent_spcirc_radio_arg_tl</code>	6418	<code>__spintent_spcomp_set_paren:n</code>	7977
<code>__spintent_spcirc_radio_print:</code>	6455	<code>\l__spintent_spcomp_silent_bool</code>	7949
<code>\l__spintent_spcirc_radio_sty_str</code>	6316	<code>__spintent_spcomp_typeset:n</code>	8008
<code>\l__spintent_spcirc_read_sty_str</code>	6316	<code>__spintent_spcoord_exec:nn</code>	3057
<code>\l__spintent_spcirc_sub_index_str</code>	6316	<code>__spintent_spcoord_inner_sep:n</code>	3022
<code>__spintent_spcirc_sym:</code>	6424	<code>__spintent_spcoord_intent_infix:</code>	3028
<code>__spintent_spcirc_word_and_article:</code> ..	6434	<code>\l__spintent_spcoord_intent_str</code>	2924
<code>\l__spintent_spcirc_word_tl</code>	6324	<code>\l__spintent_spcoord_luaset_error_str</code> .	2914
<code>__spintent_spcircfig_build_concept:nn</code>	6778	<code>\l__spintent_spcoord_luaset_sep_str</code> ...	2914
<code>__spintent_spcircfig_build_short:n</code> ...	6737	<code>\l__spintent_spcoord_luaset_x_has_frac_-</code> <code>str</code>	2914
<code>__spintent_spcircfig_del_word:n</code>	6726	<code>\l__spintent_spcoord_luaset_x_sign_str</code>	2914
<code>__spintent_spcircfig_exec:nnn</code>	6911	<code>\l__spintent_spcoord_luaset_x_type_str</code>	2914
<code>__spintent_spcircfig_finish_intent:nn</code>	6762	<code>\l__spintent_spcoord_luaset_x_value_-</code> <code>str</code>	2914
<code>\l__spintent_spcircfig_intent_arg_tl</code> ..	6697	<code>\l__spintent_spcoord_luaset_y_has_frac_-</code> <code>str</code>	2914
<code>\l__spintent_spcircfig_intent_tl</code>	6697	<code>\l__spintent_spcoord_luaset_y_sign_str</code>	2914
<code>__spintent_spcircfig_mathml:nn</code>	6870	<code>\l__spintent_spcoord_luaset_y_type_str</code>	2914
<code>\l__spintent_spcircfig_op_tl</code>	6697	<code>\l__spintent_spcoord_luaset_y_value_-</code> <code>str</code>	2914
<code>__spintent_spcircfig_process:nn</code>	6825	<code>__spintent_spcoord_mathml:</code>	3040
<code>\l__spintent_spcircfig_short_bool</code>	6697	<code>__spintent_spcoord_print:n</code>	2999
<code>__spintent_spcircfig_sym:n</code>	6702	<code>\l__spintent_spcoord_raw_arg_tl</code>	2924
<code>__spintent_spcircfig_word_and_-</code> <code>article:n</code>	6708	<code>__spintent_spcoord_set_wrap_parens_-</code> <code>size:</code>	2943
<code>__spintent_spcomb_arg:nn</code>	8208	<code>__spintent_spcoord_value:n</code>	2990
<code>__spintent_spcomb_arg_ann:nn</code>	8208	<code>\l__spintent_spcoord_wrap_parens_l_tl</code> .	2924
<code>__spintent_spcomb_arg_plain:nn</code>	8208	<code>\l__spintent_spcoord_wrap_parens_r_tl</code> .	2924
<code>__spintent_spcomb_begin:nnn</code>	8407	<code>\l__spintent_spcuartil_read_arg_tl</code> ...	7463
<code>\l__spintent_spcomb_circ_bool</code>	8069	<code>\l__spintent_spcuartil_silent_bool</code> ...	7463
<code>\l__spintent_spcomb_concept_tl</code>	8075	<code>\l__spintent_spdate_luaset_error_str</code> ..	1484
<code>__spintent_spcomb_exec:nnn</code>	8346	<code>\l__spintent_spdate_luaset_output_str</code> .	1484
<code>\l__spintent_spcomb_i_int</code>	8075		
<code>\l__spintent_spcomb_letter_tl</code>	8075		
<code>__spintent_spcomb_list:</code>	8219		

<code>__spintent_spdate_process:n</code>	82, 1495
<code>\l__spintent_spdesv_sigma_bool</code>	7198
<code>__spintent_spdiv_process:nnn</code>	1946
<code>__spintent_spdiv_render_op:nn</code>	1925
<code>__spintent_spdiv_render_parens:nn</code> . . .	1934
<code>\l__spintent_spdom_cmd_str</code>	3888
<code>__spintent_spdom_exec:nnnnn</code>	3952
<code>\l__spintent_spdom_intent_arg_tl</code>	3888
<code>\l__spintent_spdom_luaset_error_str</code> . . .	3888
<code>__spintent_spdom_mathml:</code>	3923
<code>__spintent_spdom_name:</code>	3914
<code>\l__spintent_spdom_op_tl</code>	3888
<code>\l__spintent_spdom_prefix_tl</code>	3888
<code>\l__spintent_spdom_word_tl</code>	3888
<code>\l__spintent_spdsym_prefix_tl</code>	1750
<code>\l__spintent_spdsym_read_sym_tl</code>	1750
<code>\l__spintent_spdsym_suffix_tl</code>	1750
<code>\l__spintent_spdsym_symbol_tl</code>	1750
<code>__spintent_spfact_exec:nn</code>	8444
<code>\l__spintent_spfact_read_arg_tl</code>	8427
<code>\l__spintent_spfact_silent_bool</code>	8427
<code>__spintent_spfig_build_concept_- vertices:n</code>	5918
<code>__spintent_spfig_build_refs:</code>	5905
<code>__spintent_spfig_build_simple:n</code>	5778
<code>__spintent_spfig_exec:nnn</code>	6042
<code>\l__spintent_spfig_intent_arg_tl</code>	5744
<code>\l__spintent_spfig_intent_tl</code>	5744
<code>__spintent_spfig_mathml:n</code>	6002
<code>\l__spintent_spfig_op_tl</code>	5744
<code>__spintent_spfig_points_loop:n</code>	5990
<code>__spintent_spfig_process:nn</code>	5754
<code>\l__spintent_spfig_pt_idx_int</code>	5744
<code>\l__spintent_spfig_pt_seq</code>	5744
<code>\l__spintent_spfig_refs_tl</code>	5744
<code>__spintent_spfig_sym:</code>	5736
<code>\l__spintent_spfig_symbol_tl</code>	5744
<code>\l__spintent_spFrec_concept_tl</code>	202, 7594
<code>__spintent_spFrec_exec:n</code>	7611
<code>\l__spintent_spFrec_symbol_str</code>	7594
<code>__spintent_spfun_aname:n</code>	3514
<code>__spintent_spfun_arg:n</code>	3570
<code>__spintent_spfun_args:</code>	3591
<code>__spintent_spfun_args_plain:</code>	3600
<code>__spintent_spfun_body:</code>	3638
<code>__spintent_spfun_exec:nn</code>	3683
<code>__spintent_spfun_fname:</code>	3514
<code>\l__spintent_spfun_intent_arg_tl</code>	3465
<code>\l__spintent_spfun_kv_tl</code>	3465
<code>\l__spintent_spfun_luaset_error_str</code> . . .	3465
<code>\l__spintent_spfun_luaset_nargs_str</code> . . .	3465
<code>__spintent_spfun_mathml:</code>	3633
<code>\l__spintent_spfun_mode_str</code>	3465
<code>__spintent_spfun_name_mathcat:</code>	3562
<code>__spintent_spfun_name_plain:</code>	3527
<code>__spintent_spfun_name_school:</code>	3539
<code>\l__spintent_spfun_nargs_int</code>	3465
<code>__spintent_spfun_nested:n</code>	3672
<code>\l__spintent_spfun_prefix_tl</code>	3465
<code>\l__spintent_spfun_refs_tl</code>	3465
<code>__spintent_spfun_set_intent:</code>	3608
<code>\l__spintent_spfun_tmp_str</code>	3465
<code>__spintent_spfuncomp_exec:nn</code>	3863
<code>__spintent_spfuncomp_fun:n</code>	3733
<code>__spintent_spfuncomp_group:</code>	3769
<code>\l__spintent_spfuncomp_intent_tl</code>	3708
<code>\l__spintent_spfuncomp_luaset_apply_- str</code>	3708
<code>\l__spintent_spfuncomp_luaset_error_- str</code>	3708
<code>\l__spintent_spfuncomp_luaset_n_str</code> . . .	3708
<code>\l__spintent_spfuncomp_luaset_paren_- str</code>	3708
<code>__spintent_spfuncomp_mathml:</code>	3807
<code>\l__spintent_spfuncomp_n_int</code>	3708
<code>__spintent_spfuncomp_plain:</code>	3793
<code>\l__spintent_spfuncomp_refs_tl</code>	3708
<code>\l__spintent_spinterval_delim_l_tl</code> . . .	3103
<code>\l__spintent_spinterval_delim_r_tl</code> . . .	3103
<code>__spintent_spinterval_exec:nn</code>	3436
<code>__spintent_spinterval_inner_sep:n</code> . . .	3353
<code>\l__spintent_spinterval_intent_arg_tl</code> . .	3103
<code>\l__spintent_spinterval_intent_group_- tl</code>	3103
<code>\l__spintent_spinterval_luaset_#1_- composite_str</code>	3094
<code>\l__spintent_spinterval_luaset_#1_- inf_-str</code>	3094
<code>\l__spintent_spinterval_luaset_#1_- sign_-str</code>	3094
<code>\l__spintent_spinterval_luaset_#1_- tall_-str</code>	3094
<code>\l__spintent_spinterval_luaset_#1_- type_-str</code>	3094
<code>\l__spintent_spinterval_luaset_#1_- value_str</code>	3094
<code>\l__spintent_spinterval_luaset_error_- code_str</code>	3085
<code>\l__spintent_spinterval_luaset_error_- str</code>	3085
<code>\l__spintent_spinterval_luaset_left_open_- str</code>	3085
<code>\l__spintent_spinterval_luaset_left_sym_- str</code>	3085
<code>\l__spintent_spinterval_luaset_right_- open_str</code>	3085
<code>\l__spintent_spinterval_luaset_right_sym_- str</code>	3085
<code>\l__spintent_spinterval_luaset_sep_str</code> . .	3085
<code>\l__spintent_spinterval_luaset_tall_- str</code>	3085
<code>\l__spintent_spinterval_luaset_type_- str</code>	3085
<code>__spintent_spinterval_mathml:</code>	3416
<code>\l__spintent_spinterval_mode_str</code>	3103
<code>__spintent_spinterval_plain_number:n</code> . .	3359
<code>\l__spintent_spinterval_prefix_tl</code>	3103
<code>__spintent_spinterval_print:n</code>	3384
<code>\l__spintent_spinterval_raw_arg_tl</code> . . .	3103
<code>\l__spintent_spinterval_read_arg_tl</code> . . .	3103
<code>\l__spintent_spinterval_reason_str</code> . . .	3103
<code>__spintent_spinterval_set_delims:</code> . . .	3215
<code>__spintent_spinterval_set_intents:</code> . . .	3303
<code>__spintent_spinterval_set_reason:</code> . . .	3179

<code>\l__spintent_spinterval_size_l_tl</code>	3103
<code>\l__spintent_spinterval_size_r_tl</code>	3103
<code>\l__spintent_spinterval_size_str</code>	3103
<code>\l__spintent_spinterval_sym_l_tl</code>	3103
<code>\l__spintent_spinterval_sym_r_tl</code>	3103
<code>\l__spintent_spinterval_tmp_tl</code>	3103
<code>__spintent_spinterval_value:n</code>	3367
<code>\l__spintent_spinterval_word_g_tl</code>	3103
<code>\l__spintent_spinterval_word_w_tl</code>	3103
<code>\l__spintent_spmangle_mode_str</code>	5033
<code>\l__spintent_spmangle_prefix_tl</code>	5033
<code>\l__spintent_spmarc_mode_str</code>	5320
<code>__spintent_spmc_process:nnnn</code>	2159
<code>\l__spintent_spmcd_prefix_tl</code>	2042
<code>\l__spintent_spmcd_print_upper_bool</code> ...	2042
<code>\l__spintent_spmcd_read_terse_bool</code> ...	2042
<code>\l__spintent_spmcd_suffix_tl</code>	2042
<code>\l__spintent_spmcm_prefix_tl</code>	2042
<code>\l__spintent_spmcm_print_result_bool</code> ..	2042
<code>\l__spintent_spmcm_print_upper_bool</code> ...	2042
<code>\l__spintent_spmcm_read_terse_bool</code> ...	2042
<code>\l__spintent_spmcm_spmcd_luaset_error_- str</code>	2042
<code>\l__spintent_spmcm_suffix_tl</code>	2042
<code>\l__spintent_spmmedia_read_arg_tl</code>	7198
<code>\l__spintent_spmmedia_read_sub_bool</code> ...	7198
<code>\l__spintent_spmmedia_silent_bool</code>	7198
<code>\l__spintent_spmmedia_sub_index_str</code> ...	7198
<code>__spintent_spmoney_build_tree:</code>	1046
<code>__spintent_spmoney_check_sub_units:</code> ..	997
<code>\l__spintent_spmoney_extracted_curr_str</code> ..	822
<code>\l__spintent_spmoney_extracted_num_tl</code> .	822
<code>__spintent_spmoney_intent_int:</code>	962
<code>\l__spintent_spmoney_intent_money_str</code> .	822
<code>\l__spintent_spmoney_intent_sign_str</code> ..	822
<code>\l__spintent_spmoney_intent_sub_unit_- str</code>	822
<code>\l__spintent_spmoney_luaset_conde_str</code> .	812
<code>\l__spintent_spmoney_luaset_currency_arg_- str</code>	812
<code>\l__spintent_spmoney_luaset_plur_str</code> ..	812
<code>\l__spintent_spmoney_luaset_position_- str</code>	812
<code>\l__spintent_spmoney_luaset_print_iso_- str</code>	812
<code>\l__spintent_spmoney_luaset_sing_str</code> ..	812
<code>\l__spintent_spmoney_luaset_status_str</code> ..	812
<code>\l__spintent_spmoney_luaset_subplur_str</code> ..	812
<code>\l__spintent_spmoney_luaset_subsing_str</code> ..	812
<code>\l__spintent_spmoney_luaset_symbol_str</code> ..	812
<code>__spintent_spmoney_normalize_key:n</code> ...	847
<code>__spintent_spmoney_parse_and_route:n</code> .	1087
<code>__spintent_spmoney_print_dec_sub_- unit:</code>	1046
<code>__spintent_spmoney_print_int:</code>	933
<code>__spintent_spmoney_print_sep_con:</code> ...	1046
<code>__spintent_spmoney_print_symbol:</code>	895
<code>__spintent_spmoney_render_layout:</code> ...	912
<code>__spintent_spmoney_render_number_node:</code> ..	924
<code>__spintent_spmoney_render_only_integer_- node:</code>	924
<code>\l__spintent_spmoney_res_main_voice_str</code> ..	822
<code>__spintent_spmoney_set_and_print_sub_- units:</code>	997
<code>__spintent_spmoney_set_currency:n</code> ...	847
<code>\l__spintent_spmoney_sub_units_bool</code> ...	822
<code>\l__spintent_spmoney_subnum_int</code>	822
<code>__spintent_spmoney_validate_money:</code> ...	879
<code>__spintent_spmQ_check_part:nn</code>	2841
<code>\l__spintent_spmQ_error_bool</code>	109, 2653
<code>__spintent_spmQ_exec:nnnn</code>	2869
<code>\l__spintent_spmQ_intent_read_sign_str</code> ..	2653
<code>\l__spintent_spmQ_join_word_str</code> ...	107, 2653
<code>__spintent_spmQ_print:nnn</code>	2833
<code>__spintent_spmQ_print_scale_int:</code>	2784
<code>__spintent_spmQ_print_scale_int:n</code> ...	2793
<code>__spintent_spmQ_print_spnZ_dfrac:nn</code> ..	2821
<code>\l__spintent_spmQ_read_parens_bool</code> ...	107
<code>__spintent_spmQ_set_join_word:</code>	2775
<code>__spintent_spmQ_set_wrap_parens_size:</code> ..	2720
<code>\l__spintent_spmQ_silent_bool</code>	2653
<code>__spintent_spmQ_start_wrap_parens:</code> ...	2755
<code>__spintent_spmQ_stop_wrap_parens:</code> ...	2755
<code>\l__spintent_spmQ_use_dfrac_bool</code>	109
<code>\l__spintent_spmQ_use_spnZ_bool</code>	109
<code>\l__spintent_spmQ_wrap_parens_l_tl</code>	106, 107, 2653
<code>\l__spintent_spmQ_wrap_parens_r_tl</code>	106, 107, 2653
<code>\l__spintent_spm_side_mode_str</code>	4822
<code>\l__spintent_spm_side_prefix_tl</code>	4822
<code>\l__spintent_spm_side_read_arg_tl</code>	4822
<code>\l__spintent_spm_side_read_sty_str</code>	4822
<code>\l__spintent_spm_sym_prefix_tl</code>	1821
<code>\l__spintent_spm_sym_read_sym_tl</code>	1821
<code>\l__spintent_spm_sym_suffix_tl</code>	1821
<code>\l__spintent_spm_sym_symbol_tl</code>	1821
<code>__spintent_spmul_process:nnn</code>	2017
<code>__spintent_spmul_render_op:nn</code>	1996
<code>__spintent_spmul_render_parens:nn</code> ...	2005
<code>\l__spintent_spnD_luaset_is_truncated_- str</code>	2265
<code>\l__spintent_spnD_luaset_result_tl</code> ...	2265
<code>\l__spintent_spnD_prefix_tl</code>	2265
<code>\l__spintent_spnD_print_result_num_tl</code> ..	2265
<code>\l__spintent_spnM_luaset_is_truncated_- str</code>	2265
<code>\l__spintent_spnM_luaset_result_tl</code> ...	2265
<code>\l__spintent_spnM_prefix_tl</code>	2265
<code>\l__spintent_spnM_print_result_num_tl</code> ..	2265
<code>__spintent_spnM_spnD_exec:nnnn</code>	2371
<code>__spintent_spnM_spnD_intent:nn</code>	2319
<code>\l__spintent_spnM_spnD_luaset_error_- str</code>	2265
<code>__spintent_spnM_spnD_process:nnnnn</code> ...	2314
<code>__spintent_spnM_spnD_result:nn</code>	2336
<code>__spintent_spnM_spnD_silent:nn</code>	2328
<code>\l__spintent_spnmd_luaset_is_number_- str</code>	2265
<code>\l__spintent_spnum_notation_sym_tl</code>	57
<code>\l__spintent_spnum_intent_read_dec_sep_- str</code>	57, 59, 284
<code>\l__spintent_spnum_intent_read_sign_str</code> ..	57, 284
<code>\l__spintent_spnum_notation_sym_tl</code> ...	284

<code>__spintent_snum_print_integer:</code>	451	<code>\l__spintent_spProm_read_arg_tl</code>	7146
<code>__spintent_snum_print_num_start: . . .</code>	451	<code>\l__spintent_spProm_silent_bool</code>	7146
<code>__spintent_snum_print_number_core: . .</code>	451	<code>__spintent_spPval_exec:nnn</code>	7766
<code>__spintent_snum_print_part_dec:</code>	430	<code>__spintent_sprayo_exec:nn</code>	4806
<code>__spintent_snum_print_part_period: . .</code>	430	<code>\l__spintent_sprayo_intent_arg_tl</code>	4743
<code>__spintent_snum_print_period_bar: . . .</code>	359	<code>__spintent_sprayo_mathml:</code>	4756
<code>\l__spintent_snum_read_period_sep_str . 57,</code>	284	<code>\l__spintent_sprayo_mode_str</code>	4689
<code>\l__spintent_snum_read_terse_bool . 57,</code>	284	<code>\l__spintent_sprayo_prefix_tl</code>	4689
<code>__spintent_snum_render_dec:</code>	342	<code>__spintent_sprayo_print:n</code>	4747
<code>__spintent_snum_render_decimal_sep: . .</code>	396	<code>\l__spintent_sprayo_pt_idx_int</code>	4743
<code>__spintent_snum_render_int:</code>	342	<code>\l__spintent_sprayo_pt_seq</code>	4743
<code>__spintent_snum_render_only_period: . .</code>	359	<code>\l__spintent_sprayo_read_sty_str</code>	4689
<code>__spintent_snum_render_part:n</code>	342	<code>\l__spintent_sprayo_refs_tl</code>	4743
<code>__spintent_snum_render_period_sep: . .</code>	359	<code>__spintent_sprazon_exec:nnn</code>	2473
<code>__spintent_spnz_process:nn</code>	2627	<code>__spintent_spread_exec:nn</code>	1691
<code>__spintent_spnz_render_parens:</code>	2615	<code>\l__spintent_spread_luaset_reading_str</code>	1670
<code>\l__spintent_spOmega_card_bool</code>	7709	<code>\l__spintent_spread_luaset_status_str .</code>	1670
<code>\l__spintent_spPcirc_mode_str</code>	6611	<code>\l__spintent_spread_read_arg_tl</code>	1670
<code>\l__spintent_spPcirc_prefix_tl</code>	6611	<code>\l__spintent_spread_read_tmp_tl</code>	1670
<code>\l__spintent_spPcirc_print_sym_str . . .</code>	6611	<code>__spintent_sprecta_exec:nn</code>	4673
<code>\l__spintent_spPcirc_radio_sty_str . . .</code>	6611	<code>\l__spintent_sprecta_intent_arg_tl . . .</code>	4448
<code>\l__spintent_spPcirc_read_arg_tl</code>	6611	<code>\l__spintent_sprecta_kv_tl</code>	4448
<code>\l__spintent_spPcirc_read_sty_str</code>	6611	<code>__spintent_sprecta_mathml:n</code>	4502
<code>\l__spintent_spPcirc_sub_index_str . . .</code>	6611	<code>\l__spintent_sprecta_mode_str</code>	4448
<code>\l__spintent_spPfig_mode_str</code>	5629	<code>\l__spintent_sprecta_prefix_tl</code>	4448
<code>\l__spintent_spPfig_prefix_tl</code>	5629	<code>__spintent_sprecta_print:n</code>	4493
<code>\l__spintent_spPfig_print_sym_str</code>	5629	<code>\l__spintent_sprecta_pt_idx_int</code>	4448
<code>\l__spintent_spPfig_read_arg_tl</code>	5629	<code>\l__spintent_sprecta_pt_seq</code>	4448
<code>__spintent_spPoint_exec:nnn</code>	4180	<code>\l__spintent_sprecta_refs_tl</code>	4448
<code>\l__spintent_spPoint_intent_arg_tl . . .</code>	4096	<code>__spintent_spsshort_execute_layout: . .</code>	1195
<code>__spintent_spPoint_mathml:n</code>	4171	<code>\l__spintent_spsshort_luaset_base_str . .</code>	1112
<code>\l__spintent_spPoint_mode_str</code>	4096	<code>\l__spintent_spsshort_luaset_expand_str</code>	1112
<code>\l__spintent_spPoint_prefix_tl</code>	4096	<code>\l__spintent_spsshort_luaset_layout_str</code>	1112
<code>__spintent_spPoint_print:n</code>	4138	<code>\l__spintent_spsshort_luaset_output_str</code>	1112
<code>__spintent_spPoly_build_concept:</code>	5548	<code>\l__spintent_spsshort_luaset_status_str</code>	1112
<code>__spintent_spPoly_build_refs:</code>	5535	<code>\l__spintent_spsshort_luaset_suffix_str</code>	1112
<code>__spintent_spPoly_exec:nn</code>	5612	<code>__spintent_spsshort_print:nn</code>	1148
<code>\l__spintent_spPoly_intent_arg_tl</code>	5523	<code>__spintent_spsshort_process_and_-</code>	
<code>__spintent_spPoly_mathml:</code>	5592	<code>render:n</code>	1160
<code>\l__spintent_spPoly_mode_str</code>	5480	<code>__spintent_spsshort_render_bypass:n . .</code>	1171
<code>__spintent_spPoly_points_loop:</code>	5570	<code>__spintent_spsshort_render_error:n . . .</code>	1171
<code>\l__spintent_spPoly_prefix_tl</code>	5480	<code>__spintent_spsshort_render_fallback: . .</code>	1171
<code>__spintent_spPoly_process:n</code>	5582	<code>__spintent_spsshort_render_success: . . .</code>	1171
<code>\l__spintent_spPoly_pt_idx_int</code>	5523	<code>__spintent_spsshort_tag_span:nn</code>	1121
<code>\l__spintent_spPoly_pt_seq</code>	5523	<code>__spintent_spside_exec:nnn</code>	5006
<code>\l__spintent_spPoly_refs_tl</code>	5523	<code>\l__spintent_spside_intent_arg_tl</code>	4896
<code>__spintent_spPoly_sym:</code>	5527	<code>\l__spintent_spside_intent_tl</code>	4896
<code>__spintent_spProb_content:</code>	7856	<code>__spintent_spside_mathml:n</code>	4929
<code>__spintent_spProb_event:N</code>	7832	<code>\l__spintent_spside_mode_str</code>	4822
<code>__spintent_spProb_exec:nn</code>	7930	<code>__spintent_spside_points_loop:n</code>	4994
<code>\l__spintent_spProb_luaset_a_str</code>	7802	<code>\l__spintent_spside_prefix_tl</code>	4822
<code>\l__spintent_spProb_luaset_b_str</code>	7802	<code>__spintent_spside_process:nn</code>	4901
<code>\l__spintent_spProb_luaset_error_str . .</code>	7802	<code>\l__spintent_spside_pt_idx_int</code>	4896
<code>\l__spintent_spProb_luaset_op_str</code>	7802	<code>\l__spintent_spside_pt_seq</code>	4896
<code>__spintent_spProb_nested_ops:</code>	7832	<code>\l__spintent_spside_read_arg_tl</code>	4822
<code>__spintent_spProb_print:</code>	7866	<code>\l__spintent_spside_read_sty_str</code>	4822
<code>\l__spintent_spProb_read_arg_tl</code>	7806	<code>\l__spintent_spside_refs_tl</code>	4896
<code>\l__spintent_spProb_silent_bool</code>	7806	<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1252
<code>__spintent_spProm_exec:nn</code>	7164	<code>\l__spintent_spsiglo_luaset_error_str .</code>	1252
<code>\l__spintent_spProm_intent_tl</code>	7146	<code>\l__spintent_spsiglo_luaset_roman_min_-</code>	
		<code>str</code>	1252
		<code>\l__spintent_spsiglo_print_era_str . . .</code>	1252

<code>__spintent_spsiglo_print_math:n</code>	1336	<code>\l__spintent_spVol_suffix_tl</code>	6942
<code>__spintent_spsiglo_print_text:n</code>	1336	<code>__spintent_spVol_symbol_print:</code>	7036
<code>__spintent_spsiglo_render_simple:nn</code> . .	1287	<code>__spintent_spVol_verbal:</code>	7015
<code>__spintent_spsiglo_render_with_era:nnn</code>	1287	<code>\l__spintent_spVol_word_tl</code>	6942
<code>__spintent_spsiglo_tag_span:nn</code>	1323	<code>__spintent_stat_print:nnnnnn</code>	7285
<code>__spintent_sptime_build_seconds:nn</code> . .	1416	<code>__spintent_stat_typeset:nnn</code>	7271
<code>__spintent_sptime_build_tree:</code>	81, 1434	<code>__spintent_unknown_keys:n</code>	235
<code>\l__spintent_sptime_luaset_base_str</code> . .	1402	<code>__spintent_unknown_keys:nn</code>	235
<code>\l__spintent_sptime_luaset_error_str</code> . .	1402	<code>spintent.lua</code>	70
<code>\l__spintent_sptime_luaset_has_seconds_str</code>	1402	<code>\spinterval</code>	34, 114, 3454
<code>\l__spintent_sptime_luaset_is_fraction_str</code>	1402	<code>spinterval-invalid-arg</code>	3120
<code>\l__spintent_sptime_luaset_seconds_str</code> .	1402	<code>\spmangle</code>	24, 152, 5298
<code>\l__spintent_spunit_exp_seq</code>	545	<code>spmangle-invalid-arg</code>	5038
<code>\l__spintent_spunit_exponent_tl</code>	545	<code>\spmarc</code>	25, 157, 5458
<code>__spintent_spunit_format_canonical:</code> . .	608	<code>spmarc-invalid-points</code>	5325
<code>\l__spintent_spunit_is_mult_bool</code>	545	<code>\spmax</code>	38, 7414
<code>\l__spintent_spunit_luaset_canonical_str</code>	538	<code>\spmc</code>	15, 93, 2245
<code>\l__spintent_spunit_luaset_compact_exp_str</code>	538	<code>\spmcn</code>	16, 93, 2245
<code>\l__spintent_spunit_luaset_is_compact_str</code>	538	<code>\spmedia</code>	37, 7338
<code>\l__spintent_spunit_luaset_is_sexagesimal_str</code>	538	<code>\spmediana</code>	37, 7338
<code>\l__spintent_spunit_luaset_lookup_status_str</code>	538	<code>\spmin</code>	38, 7414
<code>\l__spintent_spunit_luaset_read_str</code> . .	538	<code>\spmoda</code>	37, 7338
<code>\l__spintent_spunit_luaset_status_str</code> .	538	<code>\spmoney</code>	6, 67, 1101
<code>\l__spintent_spunit_mult_seq</code>	545	<code>\spmQ</code>	17, 105, 109, 2890
<code>__spintent_spunit_process_base_exp:n</code> .	616	<code>spmQ-not-integer</code>	2660
<code>__spintent_spunit_process_mult:n</code>	668	<code>spmQ-signed-part</code>	2660
<code>\l__spintent_spunit_read_cdot_bool</code> . .	545	<code>spmQ-zero-denominator</code>	2660
<code>\l__spintent_spunit_read_slash_str</code> . .	545	<code>\spmside</code>	22, 148, 5011
<code>__spintent_spunit_safe_exec:n</code>	687	<code>spmside</code>	6332
<code>\l__spintent_spunit_unit_name_tl</code>	545	<code>spmside-invalid-points</code>	4829
<code>\l__spintent_spusep_arg_tl</code>	83, 84, 1531	<code>\spmsym</code>	13, 88, 1874
<code>__spintent_spusep_convert:n</code>	1593	<code>\spmul</code>	14, 91, 2032
<code>\l__spintent_spusep_left_tl</code>	83, 84, 1531	<code>spmul-empty-argument</code>	1991
<code>\l__spintent_spusep_right_tl</code>	83, 84, 1531	<code>\spnD</code>	14, 97, 2436
<code>__spintent_spusep_set_parens:nn</code>	1582	<code>\spnewread</code>	11, 1730
<code>\l__spintent_spusep_silent_bool</code>	84	<code>spnewread-duplicate</code>	1642
<code>__spintent_spvec_exec:nnn</code>	4417	<code>spnewread-empty-args</code>	1642
<code>\l__spintent_spvec_intent_arg_tl</code>	4208	<code>\spnewunit</code>	6, 66, 780
<code>\l__spintent_spvec_kv_tl</code>	4208	<code>spnewunit-duplicate</code>	759
<code>__spintent_spvec_mathml:n</code>	4270	<code>\spnM</code>	15, 97, 2436
<code>\l__spintent_spvec_mode_str</code>	4208	<code>\spnum</code>	3, 57, 58, 520
<code>\l__spintent_spvec_prefix_tl</code>	4208	<code>spnum-has-units</code>	506
<code>__spintent_spvec_print:n</code>	4253	<code>spnum-no-digits</code>	506
<code>\l__spintent_spvec_pt_idx_int</code>	4208	<code>\spnZ</code>	17, 103, 2643
<code>\l__spintent_spvec_pt_seq</code>	4208	<code>spnz-not-integer</code>	2610
<code>\l__spintent_spvec_refs_tl</code>	4208	<code>\spOmega</code>	39, 204, 7732
<code>__spintent_spVol_attach_subscript:</code> . .	7074	<code>\spPcirc</code>	32, 182, 6916
<code>\l__spintent_spVol_mode_str</code>	6942	<code>spPcirc-invalid-arg</code>	6622
<code>\l__spintent_spVol_native_bool</code>	6942	<code>\spPerc</code>	38, 7558
<code>__spintent_spVol_print:</code>	6987	<code>\spPfig</code>	28, 164, 6047
<code>\l__spintent_spVol_read_arg_full_tl</code> . .	6942	<code>spPfig-invalid-arg</code>	5636
<code>__spintent_spVol_read_arg_print:</code> . . .	7106	<code>spPfig-invalid-points</code>	5636
<code>\l__spintent_spVol_read_arg_tl</code>	6942	<code>spPfig-read-arg-key</code>	5636
<code>__spintent_spVol_silent:</code>	6994	<code>spPfig-sym-cardinality-conflict</code>	5636
<code>\l__spintent_spVol_sub_index_str</code>	6942	<code>spPfig-sym-letra-conflict</code>	5636
		<code>\spPoint</code>	19, 134, 4197
		<code>spPoint-invalid-point</code>	4099
		<code>\spPoly</code>	26, 160, 5617
		<code>spPoly-invalid-points</code>	5482
		<code>\spProb</code>	40, 206, 7938
		<code>spProb-invalid-argument</code>	7808

[\spProm](#) [37](#), [193](#), [7187](#)
[\spProp](#) [18](#), [102](#), [2524](#)
[\spPval](#) [40](#), [205](#), [7792](#)
[\sprad](#) [29](#), [172](#), [6292](#)
[\sprango](#) [38](#), [7414](#)
[\sprayo](#) [21](#), [145](#), [4811](#)
[sprayo-invalid-points](#) [4692](#)
[\sprazon](#) [18](#), [100](#), [2514](#)
[\spread](#) [12](#), [53](#), [85](#), [1720](#)
[spread-unknown-sym](#) [1642](#)
[\sprect](#) [36](#), [130](#), [3966](#)
[\sprecta](#) [20](#), [140](#), [4678](#)
[sprecta-invalid-points](#) [4455](#)
[\spriq](#) [38](#), [7414](#)
[\spshort](#) [8](#), [73](#), [1239](#)
[\spside](#) [22](#), [148](#), [5011](#)
[spside](#) [6332](#)
[spside-invalid-points](#) [4829](#)
[\spsiglo](#) [10](#), [77](#), [1389](#)
[\sptime](#) [11](#), [80](#), [1466](#)
[\spunit](#) [4](#), [62](#), [63](#), [66](#), [747](#)
[spunit-invalid-unit](#) [552](#)
[spunit-no-units](#) [552](#)
[\spunitalias](#) [6](#), [66](#), [794](#)
[spunitalias-duplicate](#) [759](#)
[spunitalias-invalid-expr](#) [759](#)
[\spusep](#) [12](#), [83](#), [1631](#)
[spusep-empty-arg](#) [1520](#)
[spusep-invalid-arg](#) [1520](#)

[\spvarz](#) [37](#), [7374](#)
[\spvec](#) [20](#), [136](#), [4437](#)
[spvec-invalid-arg](#) [4215](#)
[\spVol](#) [34](#), [189](#), [7134](#)
str commands:
 [\str_if_eq:nnTF](#) [62](#)
 [\str_lowercase:n](#) [62](#)
 [\str_uppercase:n](#) [62](#)
style [8114](#)
sub-index [6076](#), [6332](#), [6631](#), [6949](#), [7212](#), [7710](#)
sub-units [861](#)
suffix [1754](#), [1825](#)

T

type [8114](#)

U

unknown [1534](#), [1674](#), [1754](#), [1825](#), [2062](#), [2274](#), [2456](#), [2682](#), [3127](#),
 [3483](#), [3722](#), [3899](#), [4107](#), [4219](#), [4462](#), [4699](#), [4839](#), [5048](#),
 [5335](#), [5489](#), [5674](#), [6076](#), [6332](#), [6631](#), [6949](#), [7149](#), [7212](#),
 [7477](#), [7710](#), [7752](#), [7817](#), [7955](#), [8114](#), [8429](#)
unknown-abbreviation [1118](#)
unknown-choice [208](#)
unknown-command-setup [159](#)
upper-case [2062](#), [6076](#)
use-dfrac [2456](#), [2682](#), [7752](#)
use-spN [2682](#)

W

wrap-parens [1900](#), [1971](#), [2594](#), [2682](#)